Fiches de Cours

Master IMA

Axel PIGEON
axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier
Années universitaires 2025–2027

4 novembre 2026

Table des matières

I Simulation Aléatoire **5**

1 Simulation de variables aléatoires **6**
1.1 Simulation de variables aléatoires discrètes 7
1.2 Simulation de variables aléatoires à densité 10
1.3 Méthodes ad-hoc 14

2 Simulation de chaînes de Markov **21**
2.1 Généralités 21
2.2 Comportement asymptotique d'une chaîne de Markov 24

3 Processus de Markov en temps continu **29**
3.1 Lois exponentielles et lemme des réveils 30
3.2 Processus de Markov en temps continu 33

4 Algorithmes de Metropolis-Hasting et Recuit Simulé **38**
4.1 Cas d'un espace d'états fini 38
4.2 Cas d'un espace d'états continu 41
4.3 Mesures de Gibbs sur un espace d'états fini 42
4.4 Algorithme du recuit simulé 43

5 Algorithme de Robbins Monro **45**
5.1 Cadre Général 45
5.2 Convergence de l'algorithme 46

II Statistiques **51**

1 Théorie de l'Information **52**
1.1 Cadre 52
1.2 Informations Qualitatives 57
1.3 Information de Kullback-Leibler 59
1.4 Informations Quantitatives 59

2 Estimation et Risque **63**
2.1 Risque 63
2.2 Estimateurs 64
2.3 Risque Quadratique 67
2.4 Bornes inférieures de variance et Efficacité 69

III Algorithmique 71

1 Flot maximal dans un graphe 72
1.1 Rappels sur les graphes . . . 72
1.2 Cycle et flot . . . 74
1.3 Flot et coupure . . . 75
1.4 Flot maximal : principe de Ford-Fulkerson . . . 77

2 Programmation Linéaire 79
2.1 Programmation Linéaire . . . 79
2.2 Programmation Linéaire en Nombres Entiers . . . 88

3 Méta-heuristiques, CSP et SAT 92

4 Théorie de la complexité 93
4.1 Généralités . . . 93
4.2 Problèmes de décisions, classes P, NP et NP-complet . . . 94

IV Apprentissage Automatique 96

1 Fondements, Réseaux Denses et Optimisation 97
1.1 Généralités . . . 97
1.2 Du Perceptron au MLP . . . 99
1.3 Fonctions d'Activation et Classification Multiclasse . . . 101
1.4 Fonctions de Perte (*Loss Functions*) . . . 102
1.5 Rétropropagation du Gradient et Dérivation Automatique . . . 102

2 Réseaux Convolutifs 105
2.1 L'Opérateur de Convolution en Vision Artificielle . . . 105
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul . . . 106
2.3 Techniques de Normalisation et Réseaux Résiduels . . . 107

3 Réseaux Récurrents 109
3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN) . . . 109
3.2 Rétropropagation à travers le Temps ($BPTT$) et Pathologies . . . 110
3.3 Cellules à Portes de Contrôle : LSTM et GRU . . . 110
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention . . . 111

4 Modèles de Markov Caché 113
4.1 Modèles de Langage et Hypothèses Markoviennes . . . 113
4.2 Formalisme des Modèles de Markov Cachés (HMM) . . . 115
4.3 Résolution Algorithmique des Trois Problèmes de Rabiner . . . 116

V Mathématiques du Machine Learning 122

1 Introduction à l'Apprentissage Supervisé 123
1.1 Des données et des prédictions . . . 123
1.2 Formalisation mathématique . . . 124
1.3 Risque et prédicteur de Bayes . . . 125
1.4 Paradigmes classiques d'apprentissage . . . 126

2 Régression Linéaire et Régression Ridge **130**
2.1 Moindres Carrés . . . 130
2.2 Analyse statistique – design fixe . . . 131
2.3 Régression Ridge – design fixe . . . 133

3 Méthodes à Noyaux **135**
3.1 De la similarité au Théorème du Représentant . . . 135
3.2 Noyaux définis positifs, Théorème de Bochner, et Théorème d'Aronszajn . . . 137
3.3 Séparateurs à Vaste Marge (SVM) . . . 141

4 Théorie de l'Apprentissage et Minimisation du Risque Empirique **144**
4.1 Convexification du risque et substituts convexes . . . 145
4.2 Décomposition du Risque et Classes Finies . . . 146
4.3 Complexité de Rademacher . . . 150
4.4 Théorèmes d'excès de risque pour les modèles linéaires . . . 156

A Inégalités de concentration et régime sous-gaussien **158**
A.1 La méthode exponentielle de Cramér-Chernoff . . . 158
A.2 Régime Sous-Gaussien . . . 161
A.3 Bilan : la division du travail en apprentissage statistique . . . 166

VI Informatique Graphique **167**

A Modélisation Géométrique **168**
A.1 Cadre mathématique et prérequis géométriques . . . 168
A.2 Représentations polynomiales . . . 170

B Fondements physiques et modélisation du transport radiatif **175**
B.1 Grandeurs radiométriques fondamentales . . . 175
B.2 Équation des transferts radiatif & Équation du rendu . . . 179
B.3 Modélisation de la réflexion . . . 185

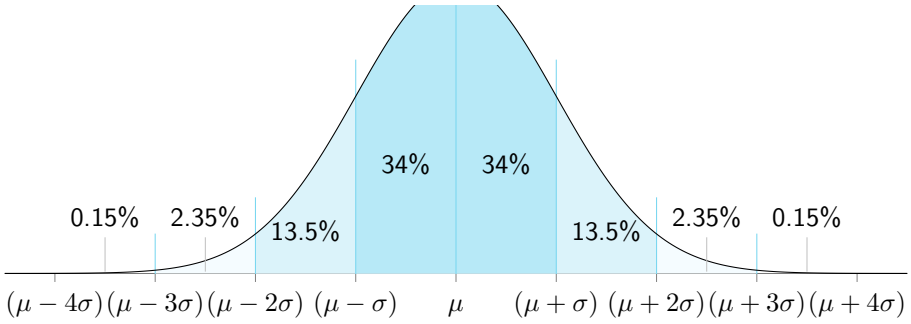
VII Traitement du signal **190**

VIII Planification **191**

1 Formalisation de la Planification Classique **192**
1.1 Le cadre classique . . . 192
1.2 Algorithmes et Langages de représentation . . . 195

2 Le Planificateur GRAPHPLAN **198**
2.1 Définitions et fonctionnement . . . 198
2.2 Améliorations de Graphplan . . . 205
««««< HEAD

Simulation Aléatoire



Chapitre 1

Simulation de variables aléatoires

Contents

1.0.1 Introduction 6
1.1 Simulation de variables aléatoires discrètes 7
1.1.1 Exemples Introductifs 7
1.1.2 Méthode Générale - Inverse de la fonction de masse 8
1.1.3 Méthode du rejet (Cas discret) 9
1.2 Simulation de variables aléatoires à densité 10
1.2.1 Méthode de l'inverse généralisée 10
1.2.2 Méthode du rejet (Cas continu) 12
1.3 Méthodes ad-hoc 14
1.3.1 Loi Normale - Méthode de Box-Muller 15
1.3.2 Lois de Poisson 16
1.3.3 Mélange de lois 18
1.3.4 Méthode de Monte-Carlo 18

1.0.1 Introduction

La simulation peut se définir comme la prédiction de l'évolution d'un système en résolvant des équations venues de domaines appliqués. La notion d'aléatoire vient du fait que l'on simule des phénomènes dont nous n'aurons que des informations imparfaites tels que des phénomènes de désintégration nucléaire ou de prévention des risques. Les utilisations possibles de la simulation sont :

- L'étude d'un phénomène (ex : la trajectoire d'un avion dans des turbulences)
- L'estimation de caractéristiques d'un système (ex : déterminer les caractéristiques d'une simulation météo)
- Tester des hypothèses (ex : en médecine, on peut utiliser la simulation aléatoire pour déterminer à l'avance les effets d'un médicament sur un patient.)
- La prise de décision dans un environement incertain.
- Réaliser des expériences coûteuses voire impossibles (ex : effectuer des essais nucléaires informatiquement en fonction des essais passés ou simuler une propagation virale dans une population).

La simulation aléatoire est donc un outil rapide économique et contrôlé.

Définition (**Simuler une variable aléatoire**) . Simuler une variable aléatoire de loi μ consiste à produire un algorithme dont les réalisations donnent des copies indépendantes d'une variable de loi μ.

La simulation aléatoire est impossible sans l'accès à des *données aléatoires*. En pratique, on se contentera de données *pseudo-aléatoires* tel que le nombre de secondes écoulées depuis le 01/01/1070, la température d'un processeur ou les dernières touches entrées avant l'exécution du programme. Ceci est obtenu en `Python` avec la fonction `rand()`. Dans ce cours, on considère que chaque appel à cette fonction nous renvoie une variable iid de loi Uniforme sur $[0,1]$.
Simuler une variable aléatoire consiste donc à construire un algorithme qui transforme un nombre fini de variables aléatoires idd de loi $\mathcal{U}([0,1])$ en une variable aléatoire de loi μ :

$$\mathcal{U}([0,1]) \overset{\text{simulation}}{\longrightarrow} \mu$$

1.1 Simulation de variables aléatoires discrètes

1.1.1 Exemples Introductifs

Exemple (Simuler une variable aléatoire de loi $\mathcal{B}(p)$) Soit $X \sim \mathcal{U}([0,1])$ avec $p \in [0,1]$. Alors la variable $Y = \mathbb{1}_{X \leqslant p}$ suit un loi de Bernouilli de paramètre p.

$$\mathbb{P}(Y=1) = \mathbb{P}(\mathbb{1}_{X\leqslant p} = 1) = \mathbb{P}(X \leqslant p) = p$$

$$\mathbb{P}(Y=0) = 1-p$$

On peut donc développer l'algorithme suivant :

```
def bernouilli(p):
    x = rand()
    if u <= p:
        return 1
    return 0

```

Exemple (Simuler une loi binomiale $\mathcal{B}(n,p)$) Soient $(X_1,\dots,X_n)$ une suite de variables aléatoires iid de loi $\mathcal{B}(n,p)$. La variable $Y := \sum_{n=1}^{n} X_i \sim \mathcal{B}(n,p)$ telle que :

$$\mathbb{P}(Y=k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Il nous faut donc seulement générer la somme de n variables aléatoires suivant une loi de Bernouilli. On a donc :

```
def binom(n,p):
    s = 0
    for i in range(n):
        s += 1 * (rand() <= p)
    return s

```

On cherchera à développer des programmes qui terminent en temps fini. On préferera deux qui terminent en temps borné (comme `binom(n,p)`).

1.1.2 Méthode Générale - Inverse de la fonction de masse

Proposition Soit $(p_n)_{n\geqslant 1}$ une suite réelle positive telle que :

$$\sum_{k=1}^{\infty} p_k = 1$$

et (x_n) une suite d'éléments d'un ensemble E. On définit la loi μ par :

$$\mu = \sum_{k=1}^{\infty} p_k \delta_{x_k}$$

si une variable aléatoire X est de loi μ on aura alors $\mathbb{P}(X = x_k) = p_k$. On pose $f_0 = 0$ et $s_k = p_1 + \cdots + p_k$. Soit U une variable aléatoire de loi $\mathcal{U}([0,1])$. Alors la variable :

$$Y = \sum_{k=1}^{\infty} x_n \times \mathbb{1}_{s_{n-1}\leqslant U\leqslant s_k}$$

suit la loi μ. L'algorithme suivant simule une variable aléatoire de loi μ.

```
def simulation_mu([p1, ..., pk],[x1, .., xk]):
    u = rand()
    s = p1
    while u > s :
        i++
        s += pi
    return xi

```

Démonstration Pour tout $n \in \mathbb{N}$, on a :

$$\mathbb{P}(s_{n-1} \leqslant U \leqslant s_n) = s_n - s_{n-1} = p_n$$

donc $\mathbb{P}(Y = x_n) = \mathbb{P}(s_{n-1} \leqslant U \leqslant s_n) = p_n$.

Exemple (Simuler une loi de Poisson $\mathcal{P}(\lambda)$) Pour rappel, si $X \sim \mathcal{P}(\lambda)$ alors :

$$\forall k \in \mathbb{N}, \quad \mathbb{P}(X = k) = e^{-\lambda}\frac{\lambda^k}{k!}$$

Posons $p_k := e^{-\lambda}\frac{\lambda^k}{k!}$. On remarque que :

$$p_{k+1} = e^{-\lambda}\frac{\lambda^{k+1}}{(k+1)!} = p_k \frac{\lambda}{k+1}$$

On en déduis donc l'algorithme suivant :

```
def poisson(lambda):
    x = 0
    p = exp(-\lamdba)
    s = p
    u = rand()
    while u > s :
        x += 1
        p = p * lambda / x
        s += p
    return x

```

Exemple (Loi uniforme) Simulons une loi uniforme sur $\{1, \dots, n\}$. On a donc l'algorithme suivant :

```
def uniforme(n):
    x = 1
    s = x/n
    u = rand()
    while u > s :
        x += 1
        s += 1/n
    return x

def uniforme(n):
    return ceil(n * rand(u))

```

1.1.3 Méthode du rejet (Cas discret)

Proposition (Algorithme du Rejet) Soit $(E, \mathcal{E}, \mu)$ un espace probabilisé et $A \subset E$ un évènement tel que $\mu(A) > 0$. La loi μ conditionnellement à A, notée μ_A, est définie par :

$$\mu_A(B) = \frac{\mu(A \cap B)}{\mu(A)}, \qquad B \in \mathcal{E}.$$

Soit $(X_n)_{n \geqslant 1}$ une suite de variables aléatoires i.i.d. de loi μ. On pose :

$$T := \inf\{n \geqslant 1 \mid X_n \in A\},$$

c'est-à-dire le rang du premier tirage appartenant à A. Alors :

1. $T \sim \mathcal{G}(\mu(A))$, c'est-à-dire
$$\mathbb{P}(T = k) = (1 - \mu(A))^{k-1}\mu(A), \qquad k = 1, 2, \dots$$
2. $X_T \sim \mu_A$ (loi conditionnelle sur A)
3. X_T et T sont indépendantes.

Démonstration Soit $B \in \mathcal{E}$ et $k \geqslant 1$. On calcule :

$$\begin{aligned}
\mathbb{P}(X_T \in B, T = k) &= \mathbb{P}(X_k \in B, T = k) \\
&= \mathbb{P}(X_1 \notin A, \dots, X_{k-1} \notin A, X_k \in A, X_k \in B) \\
&= \mathbb{P}(X_1 \notin A)^{k-1} \times \mathbb{P}(X_1 \in A \cap B) \\
&= (1 - \mu(A))^{k-1}\mu(A)\frac{\mu(A \cap B)}{\mu(A)} \\
&= \underbrace{(1 - \mu(A))^{k-1}\mu(A)}_{\mathbb{P}(T=k)} \underbrace{\frac{\mu(A \cap B)}{\mu(A)}}_{\mu_A(B)}.
\end{aligned}$$

On en déduit :

- $\mathbb{P}(T = k) = (1 - \mu(A))^{k-1}\mu(A)$ (loi géométrique)
- $\mathbb{P}(X_T \in B) = \mu_A(B)$
- $\mathbb{P}(X_T \in B, T = k) = \mathbb{P}(T = k)\mathbb{P}(X_T \in B)$, donc T et X_T sont indépendantes.

L'algorithme du rejet décrit la procédure suivante :

1. Tirer $X_1, X_2, \dots$ de manière indépendante selon la loi μ.

2. Rejeter tout tirage $X_n \notin A$ et recommencer.
3. Accepter le premier $X_T \in A$: sa loi est exactement μ_A.

Ainsi :

- T représente le nombre de tirages nécessaires avant d'obtenir un succès.
- X_T est la valeur de ce premier succès, distribuée comme μ conditionnellement à A.
- T et X_T sont indépendants : le temps d'attente n'influence pas la valeur obtenue.

Exemple Si E est l'ensemble des notes d'étudiants, μ la distribution des notes et $A = \{\text{note} > 15\}$, alors :

- T = nombre d'étudiants observés avant d'en trouver un avec une note > 15,
- X_T = note de ce premier étudiant (loi conditionnelle sur A).

La loi de T est géométrique et la loi de X_T est la loi des notes sachant qu'elles sont > 15.

Exemple On veut simuler une variable aléatoire U de loi uniforme sur les nombres premiers inférieurs à n. D'après la proposition précédente, on peut donc utiliser l'algorithme suivant :

```
def unif_prem(n):
    x = uniforme(n)
    while not premier(x):
        x = uniforme(n)
    return x

```

Ainsi le premier nombre premier sur lequel on tombe est uniformément distribué sur l'ensemble des nombres premiers inférieurs ou égaux à n.

1.2 Simulation de variables aléatoires à densité

Dans cette section, nous allons voir comment simuler des variables aléatoires continues. Nous découvrirons la méthode d'inversion de la fonction de répartition, une autre version de la méthode du rejet ainsi que d'autres exemples.

Rappel (Fonction de répartition) Soit μ une mesure de probabilité sur $\mathbb{R}$. La fonction de répartition de μ est la fonction :

$$F : \begin{cases} \mathbb{R} \longrightarrow \mathbb{R} \\ x \longmapsto \mu(]-\infty, x[) \end{cases}$$

Proposition La fonction de répartition F d'une loi admet plusieurs propriétés :

- F est croissante
- F est continue à droite, elle tend vers 1 en $+\infty$ et vers 0 en $-\infty$.

De plus, si μ admet une densité f par rapport à la mesure de Lebesgue, alors F et dérivable presque partout et $\forall x \in \mathbb{R}, F'(x) = f(x)$.

1.2.1 Méthode de l'inverse généralisée

La fonction de répartition F d'une variable aléatoire X nous donne la probabilité que X soit inférieure à $x \in \mathbb{R}$ notée $\mathbb{P}(X < x) \in [0, 1]$. Puisque l'on sait facilement générer un nombre aléatoire entre 0 et 1 avec la fonction `rand()`, il serait intéressant de voir si on ne peut pas, à partir de celui-ci, déterminer une valeur aléatoire de la variable X. C'est ce que fait la méthode de l'inverse généralisée.

Définition (**Inverse Généralisée**) . Soit $F : U \longrightarrow V$ une fonction croissante définie à droite. L'inverse généralisée de F est la fonction :

$$F^{-1} : \begin{cases} V \longrightarrow U \\ y \longmapsto \inf\{x \in U | F(x) > y\} \end{cases}$$

Proposition Si $F : U \longrightarrow V$ est inversible sur $I \subset U$ alors :

$$\forall y \in F(I), \quad F^{-1}(y) = (F_{|I})^{-1}(y)$$

Exemple Soit F la fonction de répartition de la loi uniforme $\mathcal{U}([a,b])$. On sait que F est une bijection de $[a,b]$ sur $[0,1]$ comme fonction continue strictement croissante.

$$F_{|[a,b]} = \varphi : x \longmapsto \frac{x-a}{b-a}$$

d'inverse φ^{-1} telle que $\forall y \in [0,1]$, on ait :

$$\begin{aligned} \varphi^{-1}(y) = x &\iff \varphi(x) = y \\ &\iff y = \frac{x-a}{b-a} \\ &\iff x = (b-a)y + a \end{aligned}$$

D'où $\forall y \in [0,1]$, $\varphi^{-1}(y) = (b-a)y + a$.

Théorème (**Inverse de la fonction de répartition**) . Soit μ une mesure de probabilité sur $\mathbb{R}$. On note F sa fonction de répartition et U une variable aléatoire de loi Uniforme sur $[0,1]$. On a alors :

$$F^{-1}(U) \sim \mu$$

Démonstration Soit F la fonction de répartition d'une loi μ et $U \sim \mathcal{U}([0,1])$. Montrons que $F^{-1}(U) \sim \mu$. Soit $x \in \mathbb{R}$, on a :

$$\mathbb{P}(F^{-1}(U) < y) = \mathbb{P}(U < F(y)) = F(y)$$

Ce théorème nous assure que la méthode de l'inverse de la fonction de répartition nous permet de générer des valeurs d'une loi voulue, en appliquant simplement la fonction de répartition au résultat de la fonction `rand()`. Pour les variables aléatoires discrètes, cette méthode revient au même que le premier algorithme vu dans ce cours.

Proposition Soit X une variable aléatoire de densité f.

1. Calculer $F(x) = \int_{-\infty}^{x} f(t)\,dt$ la fonction de répartition de X.
2. Calculer F^{-1} l'inverse généralisé de F sur un invervalle bien choisi.
3. Écrire l'algorithme :

```
def simulerX():
    return invF(rand())

```

Exemple Simulons une variable aléatoire X dont la densité est :

$$f_X(x) = \frac{1}{\pi}\frac{1}{\sqrt{1-x^2}}, \quad \forall x \in]-1,1[$$

Calculons sa fonction de répartition :

$$\begin{aligned} F_X(x) &= \int_{-\infty}^{x} \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}}\, dt \\ &= \int_{-1}^{x} \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}}\, dt \\ &= \frac{1}{\pi} \left[\arcsin t\right]_{-1}^{x} \\ &= \frac{1}{\pi} arcsinx - \frac{1}{2} \end{aligned}$$

or F est strictement croissante et continue sur $]-1,1[$ donc elle réalise une bijection de $[-1,1]$ dans $[0,1]$. Soient $x \in [-1,1]$ et $y \in [0,1]$ tels que :

$$\begin{aligned} F(x) = y &\iff \frac{1}{\pi} arcsinx + \frac{1}{2} = y \\ &\iff x = \sin\left(\pi\left(y + \frac{1}{2}\right)\right) \end{aligned}$$

Donc F est bijective d'inverse :

$$F^{-1} = x \longmapsto \sin\left(\pi\left(y + \frac{1}{2}\right)\right)$$

1.2.2 Méthode du rejet (Cas continu)

On souhaite simuler une variable aléatoire X de fonction de densité f que l'on ne sait pas bien simuler. On va donc utiliser une autre fonction auxiliaire g que l'on sait bien simuler.

Théorème (**Méthode du rejet**) . Soient f, g deux fonctions de densité telles qu'il existe $c > 0$ vérifiant :

$$f \leqslant cg$$

Soient $(Y_i)_{i\geqslant 1}$ des variables iid de densité g et $(U_i)_{i\geqslant 1}$ des variables iid de loi $\mathcal{U}([0,1])$. Posons $T = \inf\left\{n \geqslant 1 | U_n \leqslant \frac{f(Y_n)}{cg(Y_n)}\right\}$. Alors :

1. Y_T a pour densité f
2. T est de loi géométrique $\mathcal{G}(1/c)$
3. T et Y_T sont indépendantes.

Proposition On peut donc ensuite simuler la variable aléatoire de densité f grâce à la densité de g avec l'algorithme suivant :

```
def simul_f():
    y = simul_g()
    u = rand()
    while u > f(y) / (c * g(y)):
        y = simul_g()
        u = rand()
    return y

```

Démonstration Soient φ, f deux fonctions continues et bornées sur $\mathbb{R}$. Montrons que Y_T est une variable aléatoire de densité f :

$$\begin{aligned}
\mathbb{E}(\varphi(Y_T)) &= \mathbb{E}\left(\sum_{n=1}^{\infty} \mathbb{1}_{T=n}\varphi(Y_T)\right) \\
&= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_T) \times \mathbb{1}_{T=n}\right) \\
&= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{T=n}\right) \\
&= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leqslant \frac{f(Y_n)}{cg(Y_n)}} \times \mathbb{1}_{U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}} \times \cdots \times \mathbb{1}_{U_1 > \frac{f(Y_1)}{cg(Y_1)}}\right) \\
&= \sum_{n=1}^{\infty} \left(\mathbb{E}\left(\varphi(Y_n)\mathbb{1}_{U_n \leqslant \frac{f(Y_n)}{cg(Y_n)}}\right) \mathbb{P}\left(U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}\right) \ldots \mathbb{P}\left(U_1 > \frac{f(U_1)}{cg(U_1)}\right)\right)
\end{aligned}$$

or on a :

$$\begin{aligned}
\mathbb{P}(U_1 > \frac{f(Y_1)}{cg(Y_1)}) &= \mathbb{E}\left(\mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)} \mid Y_1\right)\right) \\
&= \mathbb{E}\left(1 - \frac{f(Y_1)}{cg(Y_1)}\right) = 1 - \mathbb{E}\left(\frac{f(Y_1)}{cg(Y_1)}\right) \\
&= 1 - \int_{\mathbb{R}} \frac{f(y)}{cg(y)} \times g(y)\, dy = 1 - \frac{1}{c}
\end{aligned}$$

de plus :

$$\begin{aligned}
&\mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leqslant \frac{f(Y_n)}{cg(Y_n)}}\right) \\
=&\mathbb{E}\left(\varphi(Y_n)\frac{f(Y_n)}{cg(Y_n)}\right) = \int_{\mathbb{R}} g(y)\varphi(y)\frac{f(y)}{cg(y)}\, dy \\
=&\frac{1}{c}\int_{\mathbb{R}} \varphi(y)f(y)\, dy
\end{aligned}$$

enfin :

$$\begin{aligned}
\mathbb{E}(\varphi(y)) &= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leqslant \frac{f(Y_n)}{cg(Y_n)}}\right) \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \left(\int_{\mathbb{R}} \varphi(y)f(y)\, dy\right) \times \frac{1}{c} \times \left(\sum_{n=1}^{\infty} \left(1 - \frac{1}{c}\right)^{n-1}\right) = \int_{\mathbb{R}} \varphi(y)f(y)\, dy
\end{aligned}$$

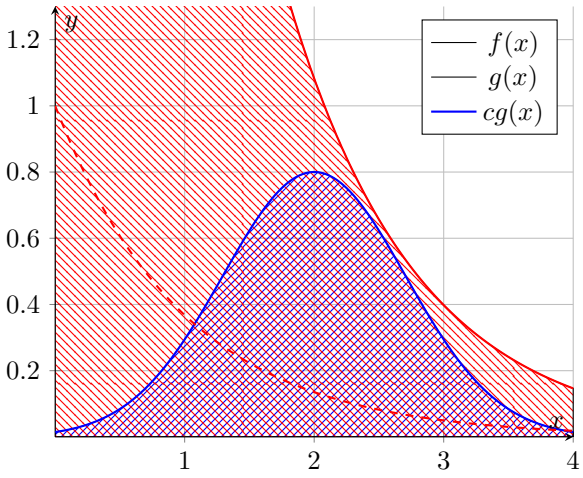
Donc Y_n est une variable aléatoire de densité f. Calculons la loi de T :

$$\begin{aligned}
\mathbb{P}(T > n) &= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}, \ldots, U_n > \frac{f(Y_n)}{cg(Y_n)}\right) \\
&= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}\right)^n = \left(1 - \frac{1}{c}\right)^n
\end{aligned}$$

d'où $T \sim \mathcal{G}(1/c)$. Pour finir, montrons que T et Y_T sont indépendantes :

$$\begin{aligned}
\mathbb{E}\left(\varphi(Y_T) \times \mathbb{1}_{T=n}\right) &= \mathbb{E}\left(\varphi(Y_T)\right) \times \frac{1}{c} \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \mathbb{E}(\varphi(Y_T)) \times \mathbb{P}(T = n)
\end{aligned}$$

Proposition (Méthode du rejet : Principe) Comme dit précédement, la méthode du rejet nous permet de simuler une densité f à partir d'une autre densité g que l'on sait simuler. Pour cela, on supposera qu'il existe $c > 0$ tel que $f \leqslant cg$.



Soit $X \sim g$ et $U \sim \mathcal{U}([0,1])$. L'idée est donc de tirer un point aléatoirement selon la densité g, $(X, U \times cg(X))$ dans la zone rouge. Si la valeur $U \times cg(X)$ est sous la courbe de f (i.e dans la zone bleu), on accepte le point, sinon on recommence. La probabilité d'acceptation d'un point est donc de $1/c$.

Exemple Soit f la fonction suivante :

$$f : \begin{cases} \mathbb{R}_+^* \longrightarrow \mathbb{R} \\ x \longmapsto \frac{\sqrt{2}}{\pi} e^{x^2/2} \end{cases}$$

et g la fonction $g : x \longmapsto -e^x$ définie sur $\mathbb{R}_+^*$. Pour tout $x \in \mathbb{R}_+^*$, on a :

$$f(x) \leqslant cg(x) \quad \text{où} \quad c = \frac{\sqrt{2}}{\pi} e$$

On peut donc simuler f via l'algorithme suivant :

```
def simul_f():
    u = rand()
    y = - log(1 - rand())
    while u > exp(-y**2/2) / exp(1 - y):
        u = rand()
        y = - log(1 - rand())
    return y

```

1.3 Méthodes ad-hoc

Dans cette section, nous allons détailler un ensemble de méthodes destinées à des usages précis. Nous verrons comment simuler efficacement une loi normale, une loi de poisson, ainsi qu'un mélange de lois. Nous finirons par aborder la méthode de Monte-Carlo pour estimer des quantités qui s'expriment à partir d'intégrales.

1.3.1 Loi Normale - Méthode de Box-Muller

Rappel (Loi Normale) Une variable aléatoire X suit une *loi normale* de moyenne μ et de variance σ^2 si la densité de la loi sur $\mathbb{R}$ est :

$$x \longmapsto \frac{1}{\sqrt{2\pi\sigma^2}} e^{\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)}$$

On note alors $X \sim \mathcal{N}(\mu, \sigma^2)$ et on a :

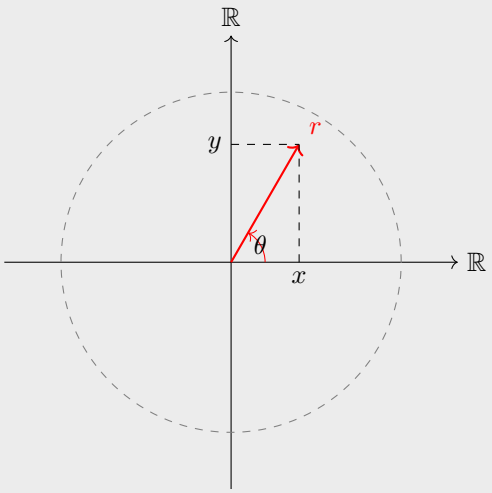
$$\mathbb{E}(X) = \mu \quad \text{et} \quad \mathbb{V}(X) = \sigma^2$$

Propriété (Loi centrée réduite.) . Si $Y \sim \mathcal{N}(0,1)$, alors :

$$X = \mu + \sigma Y \sim \mathcal{N}(\mu, \sigma^2)$$

On dit alors que Y suit une *loi normale centrée réduite*.

Théorème (Méthode de Box-Muller) . Soit (X, Y) un couple de variables aléatoires iid de loi $\mathcal{N}(0,1)$. Alors $X^2 + Y^2$ est une variable aléatoire de loi $\mathcal{E}(1/2)$ indépendante de $\arctan(Y/X) \sim \mathcal{U}\left(\left[-\frac{\pi}{2}; \frac{\pi}{2}\right]\right)$.



Le théorème nous dit que θ est indépendant de r.

Démonstration Soient (X, Y) deux variables aléatoires indépendantes de loi $\mathcal{N}(0,1)$. Calculons l'espérance du couple :

$$\mathbb{E}(g(X,Y)) = \frac{1}{2\pi} \int_{\mathbb{R}^2} g(x,y) e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}} \, dxdy$$

Pour cela, utilisons la formule de changement de variable. Posons :

$$\Phi : \begin{cases} U = R_+ \times [0, 2\pi[\longrightarrow V = \mathbb{R}^2 \\ (r, \theta) \longmapsto (x, y) = (r\cos\theta, r\sin\theta) \end{cases}$$

Φ est bijective de $R_+^* \times [0, 2\pi[$ dans $\mathbb{R}^2/\{(0,0)\}$ donc c'est un $\mathcal{C}^1$ difféomorphisme. On peut donc appliquer la formule de changement de variable :

$$\int_V f(x,y) \, dxdy = \int_U f(\Phi(r,\theta)) \times \mid \det J_\Phi(r,\theta) \mid \, drd\theta$$

On a donc :

$$\Phi(r,\theta) = \begin{pmatrix} r\cos\theta \\ r\sin\theta \end{pmatrix} \quad \text{d'où} \quad J_\Phi(r,\theta) = \begin{pmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{pmatrix}$$

Avec $D_\Phi(r,\theta) = \det J_\Phi(r,\theta) = r$. D'où :

$$\begin{aligned} \mathbb{E}(g(X,Y)) &= \frac{1}{2\pi}\int_{\mathbb{R}^2} g(x,y)e^{-\frac{x^2}{2}}e^{-\frac{y^2}{2}}\,dxdy \\ &= \frac{1}{2\pi}\int_0^{2\pi}\int_0^{\infty} g(r\cos\theta, r\sin\theta)\times e^{-\frac{r^2}{2}}\times r\,drd\theta \end{aligned}$$

Posons donc $z = r^2$, on a donc $dz = 2r\,dr$ d'où :

$$\begin{aligned} \mathbb{E}(g(X,Y)) &= \frac{1}{4\pi}\int_0^{2\pi}\int_0^{\infty} g(\sqrt{z}\cos\theta, \sqrt{z}\sin\theta)e^{-z/2}\,dzd\theta \\ &= \int_0^{2\pi}\int_0^{\infty} g(\sqrt{z}\cos\theta, \sqrt{z}\sin\theta)f_z(z)f_\theta(\theta)\,dzd\theta \end{aligned}$$

où :

$$\begin{cases} f_z(z) = \frac{1}{2}e^{-z/2} \\ f_\theta(\theta) = \frac{1}{2\pi} \end{cases} \implies \begin{cases} Z \sim \mathcal{E}(1/2) \\ \Theta \sim \mathcal{U}([0,2\pi]) \end{cases}$$

On en déduit donc que $\Theta \perp Z$.

Proposition On peut donc simuler deux lois normales à partir d'une loi exponentielle et d'une loi uniforme via l'algorithme suivant :

```
def simuler_N():
    u = 2 * pi * rand()
    e = -2 * log(rand())
    return sqrt(e) * cos(u)

```

Ou plus généralement :

```
def simuler_N(mu,sigma):
    u = 2 * pi * rand()
    e = -2 * log(rand())
    return sqrt(sigma**2 * e) * cos(u + mu)

```

1.3.2 Lois de Poisson

Rappel (Loi de Poisson) Une variable aléatoire X suit une loi de Poisson de paramètre λ si $\forall k \in \mathbb{N}$:

$$\mathbb{P}(X = k) = e^{-\lambda}\frac{\lambda^k}{k!}$$

Propriété (**Loi de Poisson et loi Exponentielle**) . Soit $(E_n)_{n\geqslant 1}$ une suite de variables aléatoires iid de loi $\mathcal{E}(1)$. On note $S_n = \sum_{k=1}^n E_k$. Posons :

$$N = \max\{k \in \mathbb{N} \mid S_k < \lambda\}$$

Alors N suit une loi de Poisson de paramètre λ.

Démonstration Démontrons ce résultat par réccurence sur $\mathbb{N}$:

- Initialisation : On observe que :

$$\mathbb{P}(N=0) = \mathbb{P}(E_1 > \lambda) = e^{-\lambda}$$

- Hérédité : Soit $k \geqslant 1$, on suppose que :

$$\mathbb{P}(N=k-1) = \frac{\lambda^{k-1}}{(k-1)!}e^{-\lambda} \quad \forall \lambda \in \mathbb{R}_+$$

En utilisant le fait que pour tout évènement A et pour toute partie mesurable $\mathcal{F}$, on a :

$$\mathbb{P}(A) = \mathbb{E}(\mathbb{1}_A) = \mathbb{E}(\mathbb{E}(\mathbb{1}_A|\mathcal{F})) = \mathbb{E}(\mathbb{P}(A|\mathcal{F}))$$

alors :

$$\mathbb{P}(N=k) = \mathbb{P}(S_k < k < S_{k+1}) = \mathbb{E}(\mathbb{P}(S_k < \lambda < S_{k+1}|E_1))$$

or :

$$\begin{aligned}\mathbb{P}(S_k < \lambda < S_{k+1}) &= \mathbb{P}(E_1 + \cdots + E_k < \lambda < E_1 + \cdots + E_{k+1}|E_1)\\ &= \mathbb{P}(E_2 + \cdots + E_k < \lambda - E_1 < E_2 + \cdots + E_{k+1}|E_1)\\ &= \mathbb{P}(S'_{k-1} < \lambda - E_1 < S4_k|E_1)\\ &= \mathbb{P}(N' = k-1)\end{aligned}$$

où S' est une copie indépendante de S et indépendante de E_1, d'où :

$$\mathbb{P}(N'=k-1) = \frac{(\lambda - E_1)^{k-1}}{(k-1)!}e^{-(\lambda - E_1)}$$

On a donc :

$$\begin{aligned}\mathbb{P}(N=k) &= \mathbb{E}\left(\frac{(\lambda-E_1)^{k-1}}{(k-1)!}e^{-(\lambda-E_1)} \times \mathbb{1}_{E_1 \leqslant \lambda}\right)\\ &= \int_0^\lambda \frac{(\lambda-e)^{k-1}}{(k-1)!}\underbrace{e^{-(\lambda-e)}e^{-e}}_{e^{-\lambda}}\,de\\ &= \frac{e^{-\lambda}}{(k-1)!}\int_0^\lambda (\lambda-e)^{k-1}\,de\\ &= \frac{e^{-\lambda}}{(k-1)!} \times \frac{\lambda^k}{k} = \frac{\lambda^k}{k!}e^{-\lambda}\end{aligned}$$

D'où $N \sim \mathcal{P}(\lambda)$.

Proposition Pour simuler une loi de poisson de pramètre λ, on a donc l'algorithme suivant :

```
def simuler_poisson(lambda):
    s - 1 * log(rand())
    k = 0
    while s <= lambda :
        k += 1
        s = -log(rand())
    return k

```

Algorithme dont la complexité dépend linéairement de λ.

Remarque Quand λ revient très grand, on peut approcher une loi de Poisson de paramètre λ par une loi normale $\mathcal{N}(\lambda, \lambda)$.

1.3.3 Mélange de lois

On cherche ici à déterminer la loi d'une combinaison linéaire de loi μ_1 et μ_2 de la forme :

$$\mu = t\mu_1 + (1-t)\mu_2$$

Pour simuler ce genre de variables aléatoires, on utilise le résultat suivant.

Lemme Soit $X \sim \mu_1$ et $Y \sim \mu_2$ et $B \sim \mathcal{B}(t)$ avec $B \perp (X, Y)$. On a alors :

$$\boxed{BX + (1-B)Y \sim t\mu_1 + (1-t)\mu_2}$$

Attention, ici X et Y ne sont pas forcément indépendantes.

Démonstration Calculons la loi de $BX + (1-B)Y$. Soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_\mathbb{R})$ on a :

$$\begin{aligned}
\mathbb{E}(g(BX + (1-B)Y)) &= \mathbb{E}\left[\mathbb{E}(g(BX + (1-B)Y)) \mid B\right] \\
&= \mathbb{E}\left[g(X) \times \mathbb{1}_{B=1} + g(Y) \times \mathbb{1}_{B=0}\right] \\
&= t\mathbb{E}(g(X)) + (1-t)\mathbb{E}(g(Y)) \\
&= t\int g \, d_{\mu_1} + (1-t)\int g \, d_{\mu_2} \\
&= \int g \, d_{t\mu_1 + (1-t)\mu_2}
\end{aligned}$$

d'où $BX + (1-B)Y \sim t\mu_1 + (1-t)\mu_2$.

Proposition On a donc l'algorithme suivant :

```
def simuler_mu(t):
    if rand() < t :
        return simuler_mu1()
    else :
        return simuler_mu2()

```

1.3.4 Méthode de Monte-Carlo

Pour la fin de ce chapitre, on souhaite simuler des variables aléatoires pour estimer des quantités déterministes pouvant s'écrire sous forme d'intégrale.

Proposition Soit $I = \int_a^b f(x)\, dx$ une intégrale que l'on ne sait pas calculer algébriquement, mais que l'on souhaite estimer. On peut alors réécrire :

$$I = \int_a^b \frac{f(x)}{g(x)} g(x)\, dx$$

où g est une densité de probabilité d'une variable que l'on sait simuler. Or, d'après la loi des grands nombres, on a :

$$\hat{I}_n = \frac{1}{n}\sum_{i=1}^{n} \frac{f(X_i)}{g(X_i)} \xrightarrow[n\to\infty]{\text{p.s.}} \mathbb{E}\left(\frac{f(X)}{g(X)}\right)$$

où $(X_1, \ldots, X_n)$ sont des variables aléatoires iid de densité g. Ainsi, on va pouvoir estimer toute intégrale de la forme :

$$I \simeq \mathbb{E}\left(\frac{f(X_1)}{g(X_1)}\right)$$

par un estimateur convergent $\hat{I}_n$.

Théorème (**Rappel - Loi des grands nombres**) . Soit (X_n) une suite de variables aléatoires iid et intégrables. On a alors :

$$\boxed{\frac{1}{n}\sum_{i=1}^{n} X_i \underset{n\to\infty}{\longrightarrow} \mathbb{E}(X_1)}$$

Remarque On peut aussi approcher I par une méthode de Riemann :

$$I = \frac{1}{n}\sum_{i=1}^{n} f\left(a + \frac{\tau}{n}\right)$$

or elle ne nous permet d'estimer l'intégrale que pour des fonctions Riemann-intégrables, alors que la Loi des grands nombres (LGN) nous permet d'élargir notre méthode aux fonctions intégrables au sens de Lebesgue. De plus, les méthodes déterministes sont très lentes en plusieurs dimensions comparé aux méthodes stochastiques.

Théorème (**Rappel - Théorème Central Limite**) . Soit (X_n) une suite de variables iid de carré intégrable. On a alors :

$$\boxed{\sqrt{n}\left(\frac{1}{n}\sum_{i=1}^{n} X_i - \mathbb{E}(X_1)\right) \underset{n\to\infty}{\overset{\text{Loi}}{\longrightarrow}} \mathcal{N}(0, \mathbb{V}(X_1))}$$

Autrement dit :

$$\lim_{n\to\infty} \mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i - \mathbb{E}(X_1)\right| > \frac{t}{\sqrt{n}}\right) = \int_t^{\infty} \frac{e^{\frac{z^2}{z\mathbb{V}(X_1)}}}{\sqrt{2\pi\mathbb{V}(X_1)}}\, dy$$

Rappel Soit (X_n) une suite de variables iid de carré intégrable. On note $\overline{X_n} = \frac{1}{n}\sum_{i=1}^{n} X_i$ et on pose :

$$S_n^2 := \frac{1}{n}\sum_{i=0}^{n} (X_i - \overline{X_n})^2$$

D'après la loi des grands nombres, on a :

$$\overline{X_n} \underset{n\to\infty}{\longrightarrow} \mathbb{E}(X_1)$$

D'après le théorème central limite, on a :

$$\sqrt{n}\frac{\overline{X_n} - \mathbb{E}(X_1)}{\sqrt{S_n^2}} \underset{n\to\infty}{\longrightarrow} \mathcal{N}(0,1)$$

avec $\mathbb{P}(I \in IC_n) \underset{n\to\infty}{\longrightarrow} \alpha$.

Proposition On peut donc utiliser le TCL pour proposer des intervalles de confiance asymptotiques pour la valeur de I. On a donc, pour $I = \mathbb{E}(X_1)$, l'intervalle asymptotique de niveau α :

$$\boxed{IC_n = \left[\overline{X_n} \pm \frac{q_{1-\alpha/2}}{\sqrt{n}}\sqrt{S_n^2}\right]}$$

Exemple On souhaite calculer :

$$I = \int_{[0,1]^3} \mathbb{1}_{\substack{x_1 \leqslant \cos x_2 \sin x_3 \\ x_2 \leqslant x_3^2}} \, dx_1 dx_2 dx_3$$

D'après la méthode de Monte-Carlo, soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_\mathbb{R})$ telle que :

$$I = \mathbb{E}(f(Z))$$

où $f(Z) = \mathbb{1}_{\substack{z_1 \leqslant \cos z_2 \sin z_3 \\ z_2 \leqslant z_3^2}}$ et $Z = (z_1, z_2, z_3) \sim \mathcal{U}([0,1]^3)$. Autrement dit :

$$I = \mathbb{P}(z_1 \leqslant \cos z_2 \sin z_3, z_2 \leqslant z_3^2)$$

On peut donc approcher I par :

$$\frac{1}{n} \sum_{i=1}^{n} \mathbb{1}_{\substack{z_1 \leqslant \cos z_2 \sin z_3 \\ z_2 \leqslant z_3^2}} = \hat{I}_n$$

D'après le TCL, on a :

$$\lim_{n \to \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{I(1-I)}} = \mathcal{N}(0,1)$$

$$\text{i.e} \quad \lim_{n \to \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{\hat{I}(1-\hat{I})}} = \mathcal{N}(0,1)$$

On peut donc construire l'intervalle de confiance de niveau α :

$$\left[\hat{I}_n \pm \frac{\sqrt{\hat{I}_n(1-\hat{I}_n)}}{\sqrt{n}} q_\alpha \right] \subseteq \left[\hat{I}_n \pm \frac{q_\alpha}{2\sqrt{n}} \right]$$

Chapitre 2

Simulation de chaînes de Markov

Contents

2.1 Généralités 21

2.1.1 Définition et exemples 21

2.1.2 La simulation en pratique 22

2.2 Comportement asymptotique d'une chaîne de Markov 24

2.2.1 Méthode MCMC 26

Dans ce chapitre, nous nous intéresserons à la simulation de processus de Markov grâce à l'étude de celles-ci. Pour rappel, une chaîne de Markov décrit l'évolution d'une quantité soumise à des fluctuations aléatoires. C'est une notion plus générale que les variables i.i.d.

2.1 Généralités

2.1.1 Définition et exemples

Définition (**Chaîne de Markov**) . Une chaîne de Markov est un processus aléatoire $(X_n)_{n\geqslant 1}$ défini sur un espace E tel qu'il existe $f \in \mathcal{M}(E \times [0,1], E)$ et $(u_n)_{n\geqslant 0}$ une suite idd de loi $\mathcal{U}([0,1])$ telle que :

$$\boxed{\forall n \geqslant 1, \quad X_{n+1} = f(X_n, u_{n+1})}$$

avec :

$$X_0 \perp (u_n)_{n\geqslant 0}$$

Exemple Soit $P \in \mathcal{M}_n(\mathbb{R})$ une matrice stochastique. Soit une chaîne de Markov sur un espace $E = [\![1, n]\!]$ de matrice de transition P. Alors c'est bien une chaîne de Markov au sens de la définition précédente. En effet, on pose :

$$\forall i, j \in E, \quad f(i,x) = j \quad \text{avec} \quad x \in \left[\sum_{k=1}^{j-1} P(i,j); \sum_{k=1}^{j} P(i,k)\right]$$

On aura donc :

$$\begin{aligned}
&\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) \\
&= \mathbb{P}(X_0 = x_0, f(x_0, u_1) = x_1, \dots, f(x_{n-1}, u_n) = x_n) \\
&= \mathbb{P}(X_0 = x_0)\mathbb{P}(f(x_0, u_1) = x_1) \dots \mathbb{P}(f(x_{n-1}, u_n) = x_n) \\
&= \mathbb{P}(X_0 = x_0)P(x_0, x_1) \times \dots \times P(x_{n-1}, x_n)
\end{aligned}$$

Exemple Soit le processus $(X_n)_{n\in\mathbb{N}}$ décrivant l'évolution de la température locale au cours du temps défini par :

$$X_0 = 25 \quad X_{n+1} = \frac{X_n + 25}{2} + Z_{n+1}$$

où $(Z_n)_{n\in\mathbb{N}}$ est une suite de variables aléatoires i.i.d de loi $\mathcal{N}(0, 10)$. Alors ce processus est une chaîne de Markov avec :

$$f(x, u) = \frac{x + 25}{2} + \Phi^{-1}(u)$$

avec $N \sim \mathcal{N}(0, 10)$ et :

$$\Phi(u) = \mathbb{P}(N \leqslant u) = \left(\int_{-\infty}^{u} e^{-x^2/20}\, dx \right) \times \frac{1}{\sqrt{20\pi}}$$

Remarque Cet exemple nous pose une problématique. En effet, nous avons défini notre chaîne de Markov avec seulement une suite de variables i.i.d de loi uniforme et chaque terme de la chaîne ne dépend que d'une seule réalisation de cette loi uniforme. Pour nous permettre de faire dépendre un X_n d'une loi normale, nous devrions pouvoir utiliser deux variables uniformes pour la simuler (par la méthode de Box-Muller).

Le lemme suivant nous l'assure donc en affirmant que l'on peut produire autant de loi uniformes souhaité à partir d'une seule.

2.1.2 La simulation en pratique

Lemme (Doublement de la loi uniforme) Il existe $g : [0, 1] \longrightarrow [0, 1]^2$ mesurable telle que si $U \sim \mathcal{U}([0, 1])$ et en posant $(V, W) = g(U)$ alors :

$$V \perp W \quad \text{et} \quad W, V \sim \mathcal{U}([0, 1])$$

Corollaire (**Génération de lois uniformes**) . Il existe une fonction mesurable $h : [0, 1] \longrightarrow [0, 1]^{\mathbb{N}}$ telle que $h(U)$ est une suite de variables idd de loi $\mathcal{U}([0, 1])$.

Remarque Ces deux derniers résultats nous permettent d'écrire :

$$X_{n+1} = f(X_n, Z_{n+1,1}, \dots, Z_{n+1,k})$$

avec $(Z_{n,k})_{\substack{k\geqslant 1 \\ n\geqslant 1}}$ une matrice de suites i.i.d. On a alors que $(X_n)_{n\in\mathbb{N}}$ est une chaîne de Markov.

Exemple (Processus de Galton-Watson) Soit $(X_{n,k})_{n\geqslant 1, k\geqslant 1}$ un tableau de variables aléatoires i.i.d à valeurs dans $\mathbb{N}$. Le processus défini par :

$$Z_0 = 1 \quad \text{et} \quad Z_{n+1} = \sum_{j=1}^{Z_n} X_{n+1,j}$$

est une chaîne de Markov. C'est un exemple de processus représentant l'évolution d'une population au cours du temps. Ici, Z_n représente le nombre d'individus à la génération n et $X_{n+1,j}$ le nombre de descendants de j-ème individu de cette génération. Z_{n+1} est donc le nombre d'individus créés à la génération $n + 1$.

Soit $(X_n)_{n\geqslant 1}$ une chaîne de Markov. Pour tout $x \in E$, on note $\mathbb{P}_x$ la loi de $(X_n)_{n\geqslant 1}$ conditionnellement à $X_0 = x$. On a donc :

Propriété (Markov faible) . Soit $(X_n)_{n\geqslant 1}$ une chaîne de Markov et $N \in \mathbb{N}$, alors :

$$\mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0) = \mathbb{E}_{X_N}(h(X_k, \dots, X_2, X_1))$$

Autrement dit, à partir du moment où l'on connaît X_N, les évènements $X_{N+1}, \dots, X_{N+k}$ ne dépendent plus du passé.

Démonstration Cette propriété est juste une reformulation de la propriété de Markov vue en probabilités. En effet, d'après la propriété de Markov faible, on sait que, étudier la chaîne $X_{N+k+1}, \dots, X_{N+1}$ sachant que l'on est passé par $X_n, \dots, X_0$ revient à étudier la chaîne décalée $X_k, \dots, X_1$ sachant que l'on est parti de X_n. Plus formellement :

$$\begin{aligned}&\mathbb{P}(X_{N+k} = i_{N+k}, \dots, X_{N+1} = i_{N+1} \mid X_N = i_N, \dots, X_0 = i_0)\\ &= \mathbb{P}(X_k = i_k, \dots, X_1 = i_1 \mid X_N = i_N)\end{aligned}$$

Donc en terme d'espérance :

$$\begin{aligned}&\mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0)\\ &= \sum_{i_1,\dots,i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_{N+k} = i_k, \dots, X_{N+1} = i_1 \mid X_N, \dots, X_0)\\ &= \sum_{i_1,\dots,i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_k = i_k, \dots, X_i = i_1 \mid X_N)\\ &= \mathbb{E}(h(X_k, \dots, X_1) \mid X_N)\\ &= \mathbb{E}_{X_N}(h(X_k, \dots, X_1))\end{aligned}$$

Exemple On définit par récurrence :

$$F_0 = 1, F_1 = 1 \quad \text{et} \quad F_{n+1} = F_n + F_{n-1} + \varepsilon_{n+1}$$

avec $(\varepsilon_n)_{n\in\mathbb{N}}$ une suite de variables aléatoires idd de loi $\mathcal{U}(\{-1, 0, 1\})$. La suite $(F_n)_{n\in\mathbb{N}}$ représente la suite de Fibonacci pour laquelle on ferait des erreurs de calculs. Cette suite n'est pas une chaîne de Markov car :

$$\mathbb{P}(F_4 = 2 \mid F_3 = 1) \neq \mathbb{P}(F_4 = 2 \mid F_3 = 1, F_2 = 1)$$

elle ne satisfait pas la propriété de Markov. Par contre, le couple $(F_n, F_{n-1})_{n\geqslant 1}$ est une chaîne de Markov avec :

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = f\left(\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}, \varepsilon_{n+1}\right)$$

avec :

$$f : \begin{cases} \mathbb{R}^2 \times \mathbb{R} \longrightarrow \mathbb{R}^2 \\ (x, y, \varepsilon) \longmapsto (x + y + \varepsilon, x) \end{cases}$$

Remarque Soit $(Z_n)_{n\geqslant 1}$ une suite de variables aléatoires idd et :

$$S_n = Z_1 + \cdots + Z_n$$

alors :

- S_n est une chaîne de Markov
- $\max\{S_k, k \leqslant n\}$ n'est pas une chaîne de Markov.

- $(S_n, \max\{S_k, k \leqslant n\})$ est une chaîne de Markov.

Proposition (Algorithme de simulation) Comme dit précédemment, une chaîne de Markov est définie telle que :

$$\begin{cases} X_0 = x_0 \\ X_{n+1} = f(X_n, u_{n+1}) \end{cases}$$

On peut donc utiliser l'algorithme suivant pour les simuler :

```
def simul_chaine_Markov(n,x0,f):
    traj = [X0]
    for i in range(n):
        traj.append(f(traj[-1],rand()))
    return traj
```

Remarque La principale difficulté est donc de définir la fonction f qui caractérise notre chaîne de Markov.

2.2 Comportement asymptotique d'une chaîne de Markov

L'utilisation des chaînes de Markov en simulation est liée au théorème suivant.

Théorème (Ergodique de Birkhoff) . Soit $(X_n)_{n\geqslant 1}$ une chaîne de Markov. Si cette chaîne admet une mesure de probabilités π invariante et ergodique alors $\forall f \in L^1(\pi)$, on a :

$$\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} f(X_i) = \int f \, d_\pi \quad \text{p.s}$$

Par conséquent, pour calculer $I = \int f \, d_\pi$ avec π une mesure coûteuse à simuler mais une mesure invariante et ergodique d'une chaîne de Markov, on peut approcher I par $\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} f(X_i)$. Évidemment, puisque les variables X_i ne sont pas i.i.d, la convergence a lieu a un rythme plus lent. Sous certaines conditions, on peut avoir :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^{n} f(X_i) - \int f \, d_\pi \right) \underset{n\to\infty}{\longrightarrow} \mathcal{N}(0,1)$$

Ce théorème peut être vu comme une généralisation de la loi des grands nombres aux chaînes de Markov.

Définition (Mesure Invariante) . Soient un espace mesurable $(E, \mathcal{F})$ et $(X_n)_{n\in\mathbb{N}}$ une chaîne de Markov et $u \sim \mathcal{U}([0,1])$. Soit π une mesure sur $(E, \mathcal{F})$. On dit que π est *invariante* si pour tout $A \in \mathcal{F}$ on a :

$$\int_E \mathbb{P}_x(X_1 \in A) \, d_{\pi(x)} = \pi(A)$$

$$\iff \int_E \mathbb{P}(f(x, u_1) \in A) \, d_{\pi(x)} = \pi(A)$$

Si π est une *mesure de probabilité*, cette définition est équivalente à :

$$X_0 \sim \pi \quad \text{et} \quad U_1 \sim \mathcal{U}([0,1]) \Longrightarrow X_1 = f(X_0, U_1) \sim \pi$$

Exemple Reprenons notre exemple de simulation de l'évolution de la température :

$$X_{n+1} = \frac{X_n + 25}{2} + Z_n \quad \text{avec} \quad Z_n \sim \mathcal{N}(0,1)$$

Si $X_0 \sim \mathcal{N}(\mu, \sigma^2)$ alors :

$$X_1 = \frac{X_0 + 25}{2} + Z_1 \sim \mathcal{N}$$

comme combinaison de gaussiennes indépendantes. De plus on a :

$$\mathbb{E}(X_1) = \frac{25+\mu}{2} \quad \text{et} \quad \mathbb{V}(X_1) = \frac{\sigma^2}{4} + 10$$

On a donc $X_1 \sim \mathcal{N}\left(\frac{25+\mu}{2}, \frac{\sigma^2}{4} + 10\right)$. La loi $\mathcal{N}(\mu, \sigma^2)$ est donc invariante si :

$$\mu = \frac{25+\mu}{2} \quad \text{et} \quad \sigma^2 = \frac{\sigma^2}{4} + 10$$

$$\iff \mu = 25,\ \sigma^2 = \frac{40}{3}$$

Remarque Attention, si $X_0 \sim \pi$ avec π une mesure invariante, alors $\forall n \in \mathbb{N}^*, X_n \sim \pi$ mais $X_n \not\perp X_{n-1}$ en général.

Définition (**Mesure Ergodique**) . Une mesure π sur un espace mesurable $(E, \mathcal{F})$ est dite *ergodique* pour une chaîne de Markov $(X_n)_{n\in\mathbb{N}}$ si pour tout $A \in \mathcal{F}$ on a :

$$\boxed{\forall x \in E, \quad \mathbb{P}_x(X_1 \in A) = \mathbb{1}_{x\in A} \Longrightarrow \pi(A) = 0 \text{ ou } \pi(\overline{A}) = 0}$$

Une mesure ergodique est une mesure sur E qui ne peut pas "partager" sa masse entre différents sous-ensembles invariants pour la chaîne. Autrement dit, pour tout $A \subset E$ tel que, partant de A, la chaîne reste dans A et partant de $\overline{A}$, la chaîne reste dans $\overline{A}$, la mesure ergodique charge soit A, soit $\overline{A}$, mais pas les deux.
Pour une chaîne de Markov dans E, cela implique que la chaîne ne peut pas rester bloquée entre plusieurs sous-ensembles invariants : elle explore tous les états accessibles dans l'ensemble invariant où elle a démarré, et finit par "oublier" son point de départ.

Remarque Si une chaîne de Markov n'est pas ergodique, cela signifie qu'il existe au moins un sous-ensemble invariant non trivial $A \subset E$ tel que :

- partant de A, la chaîne reste dans A ;
- partant de A^c, la chaîne reste dans A^c.

Une mesure non invariante peut alors mettre du poids à la fois sur A et sur A^c. En pratique, cela signifie que la chaîne peut rester confinée dans différents "blocs" selon l'état initial, et donc qu'elle ne se mélange pas complètement.

Propriété (**Ergodicité et Irréductibilité**) . Pour les chaînes de Markov sur des espaces d'états discrets, être *ergodique* est équivalent à être *irréductible*.

Corollaire (**Sur les espaces d'états discrets**) . Si E est un espace d'états dicret, alors toute chaîne de Markov irréductible est μ-ergodique pour toute mesure de probabilité μ.

Théorème (**Birkhoff - Variante**) . Soit $(X_n)_{n\in\mathbb{N}}$ une chaîne de Markov de mesure invariante et ergodique π. Soient $f, g \in L^1(\pi)$ on a alors :

$$\lim_{n\to\infty} \frac{\sum_{i=1}^{n} f(X_i)}{\sum_{i=1}^{n} g(X_i)} = \frac{\int f \, d\pi}{\int g \, d\pi} \quad \text{p.s}$$

Intuitivement, puisque qu'une chaîne ergodique se "mélange bien" dans son espace d'état, alors les fréquences empiriques (la moyenne des trajectoires) reflètent bien les fréquences théoriques (la mesure π) de la chaîne de Markov.

Exemple Soit $(S_n)_{n\geqslant 1}$ la marche aléatoire simple sur $\mathbb{Z}$. Cette chaîne de Markov n'admet pas de mesure de probabilité invariante. En revanche, la mesure de comptage sur $\mathbb{Z}$ est invariante et ergodique. Par conséquent :

$$\forall A, B \subset \mathbb{Z}, \quad \lim_{n\to\infty} \frac{\sum_{k=1}^{n} \mathbb{1}_{S_k\in A}}{\sum_{k=1}^{n} \mathbb{1}_{S_k\in B}} = \frac{|A|}{|B|}$$

alors que :

$$\frac{1}{n}\sum \mathbb{1}_{S_k} \in A = 0$$

2.2.1 Méthode MCMC

Les méthodes *Monte-Carlo Markov Chain* sont des méthodes qui généralisent la méthode de Monte-Carlo classique pour calculer des intégrales en utilisant le théorème ergodique. Si on souhaite calculer $I = \int f \, d\pi$ avec π une mesure difficile à simuler, mais qui est une mesure invariante et ergodique d'une chaîne de Markov, on peut alors estimer :

$$\hat{I}_n = \frac{1}{n}\sum_{i=1}^{n} f(X_i)$$

Exemple (Sac à dos) On souhaite charger un sac à dos avec des objets de poids w_i. On définit :

$$C = \{u \in \{0,1\}^n \mid \langle u, w\rangle \leqslant 25\}$$

l'ensemble des remplissages possibles du sac, et π la loi uniforme sur C :

$$\forall x \in C, \quad \pi(x) = \frac{1}{|C|}$$

Si $|C|$ est très grand, le dénombrement direct devient impossible, et la méthode du rejet est inefficace si $|C| \ll 2^n$. On peut alors définir une chaîne de Markov $(X_n)_{n\in\mathbb{N}}$ sur C telle que :

- Pour tout $x \in C$, on choisit uniformément un indice $i \in \{1, \dots, n\}$ et on propose de "flipper" x_i pour obtenir y :

$$y_j = \begin{cases} x_j & j \neq i \\ 1 - x_i & j = i \end{cases}$$

- Si $y \in C$, alors on accepte la transition $X_{n+1} = y$, sinon on reste en X_n.

Cette chaîne de Markov est *ergodique* (on peut atteindre tout état depuis n'importe quel autre) et π est sa *mesure invariante* :

$$\pi(x) = \sum_{y \in C} \pi(y) P(y \to x), \quad \forall x \in C$$

où $\mathbb{P}(y \to x)$ est la probabilité de transition de y vers x. Ainsi, pour n grand, la loi de X_n est approximativement π, et on peut estimer des quantités d'intérêt :

$$\mathbb{E}_\pi[f(X)] \approx \frac{1}{N} \sum_{n=1}^{N} f(X_n).$$

Ce procédé permet également d'estimer $|C|$ de façon approchée par des techniques de comptage MCMC.

Théorème (**Vitesse de convergence du théorème ergodique**) . Sous certaines hypothèses de régularité (que l'on ne détaillera pas ici), si $(X_n)_{n \geqslant 1}$ est une chaîne de Markov de mesure de probabilité π invariante et ergodique, on a :

$$\boxed{\lim_{n \to \infty} \sqrt{n} \left(\frac{1}{n} \sum_{i=1}^{n} f(X_i) - \int f \, d\pi \right) = \mathcal{N}(0, \sigma_f^2) \quad \text{en loi}}$$

où :

- $\sigma_f^2 = \mathbb{V}_\pi \left(f(X_0) \right) + 2 \sum_{k \geqslant 1} \operatorname{cov}_\pi(f(X_0), f(X_k))$
- $f \in L^2(\pi)$.

Remarque Supposons que $X_0 \sim \pi$, posons $S_n = \frac{1}{n} \sum_{i=1}^{n} f(X_i)$, alors :

$$\begin{aligned} \mathbb{E}(S_n) = \mathbb{E} \left(\frac{1}{n} \sum_{i=0}^{n} f(X_i) \right) &= \frac{1}{n} \sum_{i=0}^{n} \mathbb{E}(f(X_i)) \\ = \frac{1}{n} \sum_{i=0}^{n} \left(\sum_{x \in E} f(x) \, d_\pi \right) &= \int f \, d_\pi \end{aligned}$$

$$
\begin{aligned}
\mathbb{V}(S_n) &= \mathbb{E}(S_n^2) - \mathbb{E}(S_n)^2 \\
&= \mathbb{E}\left[\left(\frac{1}{n}\sum_{i=0}^{n} f(X_i)\right)^2\right] - \left(\mathbb{E}\left[\frac{1}{n}\sum_{i=0}^{n} f(X_i)\right]\right)^2 \\
&= \frac{1}{n^2}\left(\sum_{i=0}^{n}\sum_{j=0}^{n}\mathbb{E}(f(X_i)f(X_j)) - \sum_{i=0}^{n}\sum_{j=0}^{n}\mathbb{E}(f(X_i))\mathbb{E}(f(X_j))\right) \\
&= \frac{1}{n^2}\sum_{i=0}^{n}\sum_{j=0}^{n}\mathbb{E}(f(X_i)f(X_j)) - \mathbb{E}(f(X_i))\mathbb{E}(f(X_j)) \\
&= \frac{1}{n^2}\sum_{i=0}^{n}\sum_{j=0}^{n}\ \mathrm{Cov}(f(X_i), f(X_j)) \\
&= \frac{1}{n^2}\left(2\sum_{\substack{i,j=0 \\ i\neq j}}^{n}\ \mathrm{Cov}(f(X_i), f(X_j)) + \sum_{i=0}^{n}\mathbb{V}(f(X_i))\right) \\
&= \frac{1}{n^2}\sum_{i=0}^{n}\mathbb{V}(f(X_i)) + \frac{2}{n^2}\sum_{i=0}^{n-1}\ \mathrm{Cov}(f(X_i), f(X_{i+1})) \\
&\quad \frac{2}{n^2}\sum_{i=0}^{n-2}\ \mathrm{Cov}(f(X_i), f(X_{i+2})) + \cdots + \frac{2}{n^2}\ \mathrm{Cov}(f(X_1), f(X_n)) \\
&= \frac{1}{n^2}\left(n\mathbb{V}(X_0) + 2(n-1)\mathrm{Cov}(f(X_0), f(X_1)) + \cdots +\ \mathrm{Cov}(f(X_0), f(X_n))\right)
\end{aligned}
$$

Un obstacle à l'utilisation de méthodes MCMC est l'estimation de σ_f^2 par construire des intervalles de confiance. Plusieurs méthodes existent pour s'en affranchir.

Chapitre 3

Processus de Markov en temps continu

Contents

3.1 Lois exponentielles et lemme des réveils 30
3.1.1 Rappels et Abscence de mémoire 30
3.1.2 Lemme des réveils et exemples 31
3.2 Processus de Markov en temps continu 33
3.2.1 Définition, semi-groupe et générateur 33
3.2.2 Construction 34
3.2.3 Simulation 37

Pour certains modèles, la description d'un processus étape par étape à l'aide d'une chaîne de Markov peut masquer des informations importantes, comme le temps passé dans chaque état. Illustrons cela par un exemple.

Exemple (Désintégration radioactive) Considérons la désintégration radioactive de l'uranium 238, représentée par le processus suivant :

$$^{238}\mathrm{U} \underset{10^9 \text{ ans}}{\longrightarrow} {}^{234}\mathrm{Th} \underset{20 \text{ jours}}{\longrightarrow} {}^{234}\mathrm{Pr} \underset{8 \text{ h}}{\longrightarrow} {}^{234}\mathrm{U} \longrightarrow ... \longrightarrow \mathrm{Pb}.$$

Si l'on modélisait ce processus par une chaîne de Markov classique, on ne connaîtrait que la succession des atomes traversés avant la formation du plomb, mais pas le temps passé dans chaque état, pourtant souvent l'information la plus pertinente.
Nous allons donc construire un processus de Markov en temps continu $(X_t)_{t\in\mathbb{R}_+}$ qui nous donnera cette information.

Dans la suite de ce cours, on cherchera donc à construire des processus de Markov $(X_t)_(t \in \mathbb{R}_+)$ sur des espaces d'états finis ou dénombrables qui vérifient une propriété de Markov. Autrement dit, tels que la variable :

$$T = \inf\{t > 0 \mid X_t \neq X_0\}$$

satisfasse une propriété d'abscence de mémoire. On veut donc un équivalent à la propriété de Markov mais pour les processus à temps continu.

3.1 Lois exponentielles et lemme des réveils

3.1.1 Rappels et Abscence de mémoire

Rappel (Loi exponentielle) Une variable aléatoire E est de loi exponentielle de paramètre $\lambda > 0$, noté $X \sim \mathcal{E}(\lambda)$ si pour tout $x \geqslant 0$, on ait :

$$\mathbb{P}(X \geqslant x) = e^{-\lambda x} \iff \mathbb{P}(X \leqslant x) = 1 - e^{-\lambda x}$$

Cette variable aléatoire a donc pour densité la fonction :

$$x \longmapsto \lambda e^{-\lambda x} \mathbb{1}_{x \geqslant 0}$$

par rapport à la mesure de Lebesgue.

Propriété (**Loi exponentielle**) . Soit $\lambda > 0$ et $X \sim \mathcal{E}(1)$ alors $X/\lambda \sim \mathcal{E}(\lambda)$. De plus, si $U \sim \mathcal{U}[0,1]$ alors $-\log U \sim \mathcal{E}(1)$. On a aussi, pour $X \sim \mathcal{E}(\lambda)$:

$$\mathbb{E}(X) = \frac{1}{\lambda} \quad \text{et} \quad \mathbb{V}(X) = \frac{1}{\lambda^2}$$

Propriété (**Abscence de Mémoire**) . On dit qu'une variable aléatoire T satisfait la propriété d'abscence de mémoire si :

$$\boxed{\forall t, s > 0, \quad \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s)}$$

Les seules variables satisfaisant cette propriété sont les variables aléatoires de loi exponentielle.

Démonstration Soit T une variable aléatoire satisfaisant la propriété d'abscence de mémoire. Montrons que T est de loi exponentielle. Posons :

$$f : \begin{cases} \mathbb{R}_+ \longrightarrow [0,1] \\ t \longmapsto \mathbb{P}(T > t) \end{cases}$$

On a donc $\forall t, s\, 0$:

$$\begin{aligned} & \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s, T > t)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{f(t+s)}{f(t)} = f(s) \\ \iff & f(t+s) = f(t)f(s) \end{aligned}$$

En particulier, pour tout $n \in \mathbb{N}$, on a :

$$f(n) = f(n - 1 + 1) = f(1)f(n-1) = f(1)^n$$

Soient $p, q \in \mathbb{N}$, on a alors :

$$f(p) = f\underbrace{\left(\frac{p}{q} + \cdots + \frac{p}{q}\right)}_{q \text{ fois}} < = f\left(\frac{p}{q}\right)^q$$

or $f(p) = f(1)^p$ donc pour tout $q \in \mathbb{Q}_+$, on a :

$$f(q) = f(1)^q = e^{q \log(f(1))}$$

Soit $x \in \mathbb{R}_+$, par décroissante de f, pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} & f\left(\frac{\lceil nx \rceil + 1}{n}\right) \leqslant f(x) \leqslant f\left(\frac{\lceil nx \rceil}{n}\right) \\ \iff & f(x) \in \left[\exp\left(\frac{\lceil nx \rceil + 1}{n} \log f(1)\right); \exp\left(\frac{\lceil nx \rceil}{n} \log f(1)\right)\right] \end{aligned}$$

d'où $f(x) = \exp(x \log f(1))$. Par conséquent $T \sim \mathcal{E}(-\log f(1))$.

3.1.2 Lemme des réveils et exemples

Lemme (Réveils) Soient $T_1, \ldots, T_n$ des variables aléatoires indépendantes de lois respectives $\mathcal{E}(\lambda_1), \ldots, \mathcal{E}(\lambda_n)$. On pose :

$$T = \min_{1 \leqslant i \leqslant n} T_i \quad \text{et} \quad I \in [\![1, n]\!] \text{ tel que } T_I = T$$

On a donc :

$$\{I = k\} = \{T_k < T_1, T_k < T_2, \ldots, T_k < T_n\}$$

Alors :

1. $T \sim \mathcal{E}(\lambda_1 + \ldots, \lambda_n)$
2. $\mathbb{P}(I = k) = \frac{\lambda_k}{\lambda_1 + \cdots + \lambda_k}$
3. $T \perp I$.

Démonstration Soient $T_1, \ldots, T_n$ des variables aléatoires indépendantes de loi $\mathcal{E}(\lambda_1), \ldots, \mathcal{E}_{\lambda_n}$. Posons :

$$T = \min_{1 \leqslant i \leqslant n} T_i \quad I \in [\![1, n]\!], T_I = T$$

Soient $x \geqslant 0$ et $k \in [\![1, n]\!]$, on a :

$$\mathbb{P}(T > x, I = k) = \mathbb{P}(x \leqslant T_k \leqslant \underbrace{\min\{T_j \mid j \leqslant n, j \neq k\}}_{z_k})$$

on a donc $z_k = \min\{T_1, T_2, \ldots, T_{k-1}, T_{k+1}, \ldots, T_n\}$ de plus :

$$\begin{aligned} \mathbb{P}(z_k \geqslant y) &= \prod_{\substack{k=1 \\ j \neq k}}^{n} \mathbb{P}(T_j \geqslant y) = \prod_{\substack{k=1 \\ j \neq k}}^{n} e^{\lambda_j y} \\ &= e^{-((\lambda_1 + \ldots \lambda_n) - \lambda_k) y} \end{aligned}$$

Par suite :

$$\begin{aligned}
&\mathbb{P}(T > x, I = k) \\
&= \mathbb{E}\left(\mathbb{P}(x \leqslant T_k \leqslant z_k \mid T_k)\right) \\
&= \mathbb{E}\left(\mathbb{1}_{T_k \geqslant x} \times e^{-((\lambda_1+\dots\lambda_n)-\lambda_k)T_k}\right) \\
&= \int_x^\infty e^{-((\lambda_1+\dots\lambda_n)-\lambda_k)t} \times f_{T_k}(t)\, dt \\
&= \lambda_k \int_x^\infty e^{-\lambda_k t} e^{-((\lambda_1+\dots\lambda_n)-\lambda_k)t}\, dt \\
&= \lambda_k \int_x^\infty e^{-(\lambda_1+\dots+\lambda_n)t}\, dt \\
&= \lambda_k \left[-\frac{e^{-(\lambda_1+\dots+\lambda_n)t}}{\lambda_1+\dots+\lambda_n}\right]_x^\infty \\
&= \lambda_k \frac{e^{-(\lambda_1+\dots+\lambda_n)}}{\lambda_1+\dots+\lambda_n}
\end{aligned}$$

En particulier, on a :

$$\mathbb{P}(I = k) = \mathbb{P}(I = k, T > 0) = \frac{\lambda_k}{\lambda_1+\dots+\lambda_n}$$

Par indépendance, on a donc :

$$\begin{aligned}
\mathbb{P}(T \geqslant x) &= \prod_{i=1}^n \mathbb{P}(T_i \geqslant x) \\
&= \prod_{i=1}^n e^{-\lambda_i x} = e^{-(\lambda_1+\dots+\lambda_n)x}
\end{aligned}$$

Enfin, on a donc :

$$\mathbb{P}(T \geqslant x, I = k) = \mathbb{P}(T \geqslant x)\mathbb{P}(I = k)$$

Remarque Si on souhaite tirer un entier x proportionnellement à un poinds w_x sans calculer $\sum_{i=1}^n w_i$, le lemme des réveils nous donne une solution. On peut donc utiliser l'algorithme suivant :

```
def tirer(poids):
    x = -1
    val = infty
    for k in range(len(poids)):
        e = - log(rand())/poids[k]
        if e < val :
            val = e
            x = k
    return x

```

Exemple (Sélection sans remplacement) Soient $w_1, \dots, w_n$ des poids positifs associés aux entiers $[\![1, n]\!]$. On souhaite tirer sans replacement k entiers avec des probabilités proportionnelles à w_k. On procédera donc comme suit :

1. On sélection I_1 avec :
$$\mathbb{P}(I_1 = j)\frac{w_j}{w_1+\dots+w_n}$$

2. Conditionnellement à I_1, on sélectionne I_2 avec :

$$\mathbb{P}(I_2 = j \mid I_1) = \frac{w_j \mathbb{1}_{I_1 \neq j}}{\sum_{i=1}^{n} w_i - w_{I_1}}$$

Par généralisation, du lemme des réveils, les variables $I_1, \dots, I_n$ sont de même loi que les variables construites comme suit. Soient $T_1, \dots, T_n$ des variables de loi $\mathcal{E}(w_k)$ on pose I_j l'indice correspondant à la jième plus petite valeur de $\{T_1, \dots, T_n\}$.

$$\text{i.e} \quad \left|\left\{k \in [\![1, n]\!] \mid T_k \leqslant T_{I_j}\right\}\right|$$

Exemple Soient $w_1 = w_2 = \cdots = w_{n-1} = 1$ et $w_n = a < 1$. Alors :

$$\begin{aligned}
&\mathbb{P}(\text{ dernier choisi soit } n) = \mathbb{P}(I_n = n) \\
&= \mathbb{P}(T_n \geqslant \max(T_1, \dots, T_{n-1})) \\
&= \int_0^\infty a e^{-at} \mathbb{P}(\max(T_1, \dots, T_{n-1}) \leqslant t)\, dt \\
&= a \int_0^\infty e^{-at} (1 - e^{-t})^{n-1}\, dt
\end{aligned}$$

3.2 Processus de Markov en temps continu

3.2.1 Définition, semi-groupe et générateur

Nous considérons ici que E est un espace d'états fini ou dénombrable.

Définition (**Processus de Markov en temps continu**) . Un processus $(X_t)_{t \geqslant 0}$ sur E est un processus de Markov en temps continue si :

$$\forall j \in E, \forall t, s \geqslant 0, \quad \mathbb{P}(X_{t+s} = j \mid \mathcal{F}_t) = \mathbb{P}_s(X_t, j)$$

où $\mathcal{F}_t$ est la tribu engendrée par les évènements $\{X_r = x_r\}, 0 \leqslant r \leqslant t$. En particulier, pour tout $i, j \in E$ et $\forall t \geqslant 0$, on a :

$$P_t(i, j) = \mathbb{P}(X_t = j \mid X_0 = i)$$

où (P_t) est le *semi-groupe* du processus de Markov.

Propriété (**Semi-groupe**) . $(P_t)_{t \geqslant 0}$ est un *semi-groupe* si :

$$\forall t, s \in E, \quad P_{t+s} = P_t P_s$$

si E est fini, alors $(P_t)_{t \geqslant 0}$ est une matrice et $P_t P_s$ est un produit matriciel. Dans un cas plus général, on a :

$$\begin{aligned}
P_t P_s(i, j) &= \sum_{k \in E} P_t(i, k) P_s(k, j) = P_{t+s}(i, j) \\
&= \sum_{k \in E} \mathbb{P}(X_t = k \mid X_0 = i) \mathbb{P}(X_{t+s} = j | X_t = k) \\
&= \sum_{k \in E} \mathbb{P}(X_t = k, X_{t+s} = j \mid X_0 = i) \\
&= \mathbb{P}(X_{t+s} = j \mid X_0 = i)
\end{aligned}$$

Donc le semi-groupe caractérise la loi du processus de Markov. De même, grâce à la cette propriété, il n'est pas nécessaire de connaître tout le semi-groupe.

Propriété (**Génération du semi-groupe**) . En particulier, $(P_t)_{t\geqslant 0}$ est uniquement déterminé par $(P_s)_{0\leqslant s\leqslant a}$ avec a arbitrairement petit.

Définition (**Générateur Infinitésimal**) . Le *générateur infinitésimal* d'un processus de Markov $(X_t)_{t\geqslant 0}$ est défini par :

$$\begin{aligned} Q_{i,j} &= \lim_{t\to 0} \frac{P_t(i,j) - \mathbb{1}_{i=j}}{t} \\ &= \lim_{t\to 0} \frac{\mathbb{P}_i(X_t = j) - \mathbb{1}_{i=j}}{t} \end{aligned}$$

Il correspond à la dérivée de $P_t(i,j)$ par rapport à t en 0.

Lemme Soit $(X_t)_{t\geqslant 0}$ un processus de Markov sur E tel que $X_0 = i$ p.s. On pose :

$$T = \inf\{t > 0 \mid X_t \neq i\} \quad \text{et} \quad Y = X_T$$

La variable T suit une loi exponentielle et est indépendante de Y.

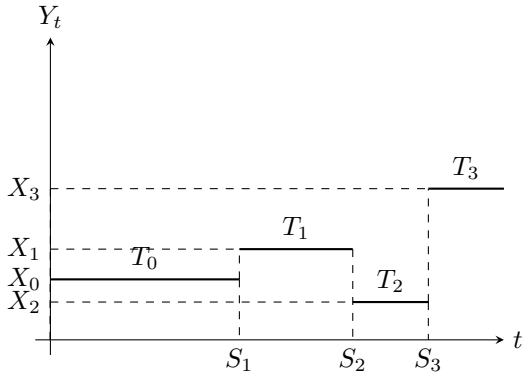
Intuitivement, cela signifie que le temps que prend le processus pour sortir d'un état suit une loi exponentielle et ne dépend pas de l'état suivant.

Remarque Le générateur infinitésimal caractérise la loi du processus de Markov. Donc $-Q_{i,i}$ donne le paramètre de la loi exponentielle correspondant au temps de sortie de l'état i par X. De même, $\frac{Q_{i,j}}{Q_{i,i}}$ donne la probabilité de finir en j après un saut hors de i.

3.2.2 Construction

Proposition Un processus de Markov de générateur Q peut donc être construit comme suit :

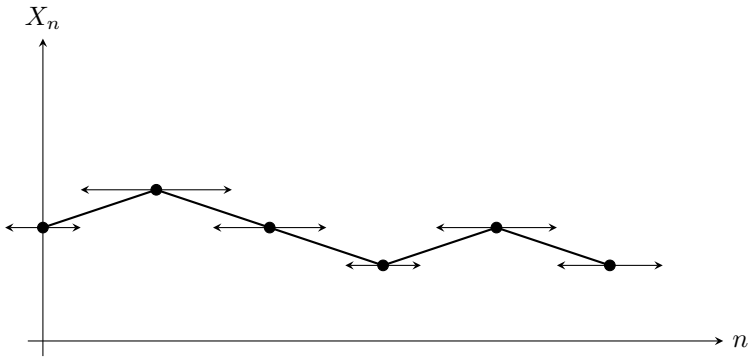
- On fixe X_0.
- On génère T_0 une variable aléatoire de loi $\mathcal{E}(-Q_{X_0,X_0})$ ainsi que X_1 indépendant de X_0 tel que :
$$\mathbb{P}(X_1 = j) = -\frac{Q_{X_0,X_1}}{Q_{X_0,X_0}}$$
- Par récurrence, X_n étant donné, on génère T_{n+1} de loi $\mathcal{E}(-Q_{X_n,X_n})$ et X_{n+1} de loi :
$$\mathbb{P}(X_{n+1} = j \mid X_n) = -\frac{Q_{X_n,j}}{Q_{X_n,X_n}}$$



Un processus de Markov en temps continu est donc un processus qui reste un temps exponentiel dans un état avant de sauter dans un autre. On peut alors poser :

$$S_k = T_0 + T_1 + \cdots + T_{k-1} \quad \text{et} \quad \forall t \in [S_k, S_{k+1}], Y_t = X_k$$

Les variables $S_1, S_2, \ldots$ définissent les temps de saut du processus Y et $(X_n)_{n\geqslant 0}$ est une chaîne de Markov (à temps discret) appelée le *squelette* du processus de Markov.



Un processus de Markov en temps continu est donc une chaîne de Markov en temps discret auquel on a rajouté des durées aléatoires (exponentielles) entre le passage d'un état à l'autre. On peut observer une limite au processus. En effet, le processus de Markov est définit jusqu'au temps $\overline{S} = \lim_{k\to\infty} S_k$. Ainsi un processus tel que $\mathbb{P}(\overline{S} < \infty) > 0$ est dit *explosif*. Sinon il est dit *non explosif*.

Formalisons maintenant toutes ces nouvelles notions.

Définition (**Squelette d'un processus de Markov**) . Soit $(Y_t)_{t\geqslant 0}$ un processus de Markov à temps continu, et $(S_k)_{k\geqslant 0}$ la suite croissante de ses temps de saut, définie par :

$$S_0 = 0, \quad S_{k+1} = S_k + T_k$$

où les T_k sont les durées passées dans chaque état avant le saut suivant. On appelle *squelette* du processus (Y_t) la suite $(X_n)_{n\geqslant 0}$ définie par :

$$X_n = Y_{S_n}, \quad \text{pour tout } n \geqslant 0$$

Le processus $(X_n)_{n\geqslant 0}$ est alors une *chaîne de Markov à temps discret*, dont la matrice de transition P est donnée par :

$$P_{ij} = \mathbb{P}(X_{n+1} = j \mid X_n = i) = -\frac{Q_{ij}}{Q_{ii}}, \quad i \neq j.$$

Définition (**Processus explosif**) . Soit $(Y_t)_{t\geqslant 0}$ un processus de Markov à temps continu et $(S_k)_{k\geqslant 0}$ la suite de ses temps de saut. On définit :

$$\overline{S} = \lim_{k\to\infty} S_k$$

- Le processus (Y_t) est dit *explosif* si

$$\mathbb{P}(\overline{S} < \infty) > 0$$

c'est-à-dire qu'il effectue une infinité de sauts en un temps fini.

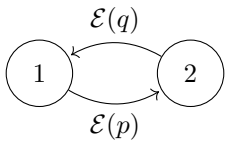
- Il est dit *non explosif* si
$$\mathbb{P}(\overline{S} = \infty) = 1$$

Vocabulaire. On appelle $Q_{i,j}$ le temps de saut de l'état i vers j. On a aussi $Q_{i,j} = \lambda_i \mathbb{P}_i(Y = j)$.

Exemple (Espace d'états fini) Soit $E = \{1, 2\}$ et $(X_t)_{t\geqslant 0}$ un processus de Markov sur E donné par le générateur infinitésimal :

$$Q = \begin{pmatrix} -p & p \\ q & -q \end{pmatrix}$$

On a donc le graphe suivant pour cette chaîne de Markov :



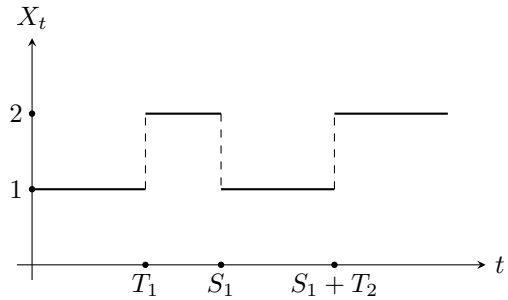
La durée de séjour du processus dans l'état 1 suit donc une loi $\mathcal{E}(p)$ et celle dans l'état 2 suit $\mathcal{E}(q)$.
La probabilité de passer d'un état à l'autre suit une loi exponentielle. Soient $(T_k)_{k\in\mathbb{N}}$ des variables iid de loi $\mathcal{E}(p)$ et $(Q_k)_{k\in\mathbb{N}}$ des variables iid de loi $\mathcal{E}(q)$. Posons :

$$S_0 = 0, \quad \text{et} \quad S_n = \sum_{k=1}^{n} T_k + Q_k$$

On a donc :

$$\begin{cases} X_t = 1 & \text{si } t \in \bigcup\limits_{n\in\mathbb{N}} [S_n, S_n + T_{n+1}[\\ X_t = 2 & \text{si } t \in \bigcup\limits_{n\in\mathbb{N}} [S_n + T_{n+1}, S_{n+1}[\end{cases}$$

On peut donc représenter la trajectoire de ce processus par le graphe suivant :



Exemple (Espace d'états infini) Posons maintenant $E = \mathbb{N}^*$ et soit $(X_t)_{t\geqslant 0}$ un processus de Markov sur E de générateur :

$$Q_{n,n} = -n^2, \quad \text{et} \quad Q_{n,n+1} = n^2$$

Le processus passe donc de l'état n à $n+1$ à taux n^2. Le squelette de la chaîne est donc le processus

$$(1, 2, 3, 4, \dots)$$

avec $S_n = T_1 + T_2 + \cdots + T_n$ où $(T_n)_{n\in\mathbb{N}}$ sont indépendantes et de loi $\mathcal{E}(k^2)$. On peut donc en déduire que :

$$S_n = E_1 + \frac{E_2}{2^2} + \cdots + \frac{E_n}{n^2}$$

où les $(E_i)_{i\in\mathbb{N}}$ sont iid de loi $\mathcal{E}(1)$. On a alors :

$$\overline{S} = \lim_{n\to\infty} S_n = \sum_{k=1}^{\infty} \frac{E_k}{k^2}$$

$$\mathbb{E}(\overline{S}) = \sum_{k=1}^{\infty} \frac{\mathbb{E}(E_k)}{k^2} = \sum_{k=1}^{\infty} \frac{1}{k^2} < \infty$$

d'où $\overline{S} < \infty$ presque sûrement. Cette chaîne est donc *explosive*.

3.2.3 Simulation

Proposition La construction des processus de Markov donné ici est parfois appelé *processus de Gillepsie*. On peut définir l'algorithme suivant pour les simuler :

```
def procMarkov(t,x,Q):
    tau = - 1 * log(rand()) / (- Q(x,x))
    if tau > t :
        return x
    # On tire x selon la loi -Qx,. / -Qx,x
    return procMarkov(t - tau, y, Q)
```

Exemple (Processus de naissance et de mort) Soit $(X_t)_{t\geqslant 0}$ un processus de Markov sur $\mathbb{N}^*$ et pour tout $i \geqslant 0$ soit b_i le taux de naissance en i et d_i le taux de mort en i.

$$\text{i.e} \quad \begin{cases} X_t \text{ saute de } i \text{ en } i+1 \text{ à taux } b_i \\ X_t \text{ saute de } i \text{ en } i-1 \text{ à taux } d_i \end{cases}$$

On ne simule évidemment pas ce processus à partir de son générateur, mais plutôt par un tirage successif des durées de séjour exponentielles et des transitions, comme dans le code ci-dessous :

```
def simulMortNaissance(x,t,b,d):
    etat = x
    tps = - log(rand()) / (b[etat] + d[etat])
    while tps < t :
        if rand() < b[etat] / (b[etat] + d[etat]):
            etat += 1
        else :
            etat -= 1
        tps += - log(rand()) / (b[etat] + d[etat])
    return etat
```

Proposition (Construction par le lemme des réveils) Il existe une deuxième construction des processus de Markov, fondée sur le *lemme des réveils*.
Lorsque le processus (X_t) est en état $x \in E$, on associe à chaque $y \in E \setminus \{x\}$ une variable aléatoire indépendante T_y de loi exponentielle $\mathcal{E}(Q_{x,y})$. On pose alors :

$$T = \min_{y\neq x} T_y, \qquad Y = \arg\min_{y\neq x} T_y.$$

Le processus reste dans l'état x pendant une durée T, puis saute en l'état Y.
Ainsi, la durée de séjour en x suit une loi exponentielle de paramètre $-Q_{x,x} = \sum_{y\neq x} Q_{x,y}$, et la probabilité de transition vérifie :

$$\mathbb{P}(X_{t+T} = y \mid X_t = x) = -\frac{Q_{x,y}}{Q_{x,x}}.$$

Chapitre 4

Algorithmes de Metropolis-Hasting et Recuit Simulé

Contents

4.1 Cas d'un espace d'états fini 38
4.2 Cas d'un espace d'états continu 41
4.3 Mesures de Gibbs sur un espace d'états fini 42
4.4 Algorithme du recuit simulé 43

L'algorithme de Metropolis-Hasting est une procédure permettant de transformer une chaîne de Markov par le rejet sélectif de certains de ses pas en une nouvelle chaîne admettant une mesure invariante fixée.

Proposition (Algorithme de Metropolis-Hastings) Soit $(U_n)_{n\geq 1}$ et $(V_n)_{n\geq 0}$ deux suites de variables aléatoires indépendantes et identiquement distribuées, telles que $U_n \sim \mathcal{U}([0,1])$ et $V_n \sim \mathcal{U}([0,1])$.
Soit $f : E \times [0,1] \longrightarrow E$ une fonction qui génère un candidat $y = f(x,U)$ à partir d'un état $x \in E$ et d'une variable uniforme U.
On définit la chaîne $(Y_n)_{n\geqslant 0}$ de Metropolis-Hastings par :

$$Y_0 \in E, \quad Y_{n+1} = \begin{cases} y = f(Y_n, U_{n+1}), & \text{si } V_n < \alpha(Y_n, y), \\ Y_n, & \text{sinon,} \end{cases}$$

où la probabilité d'acceptation est

$$\alpha(x,y) = \min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right)$$

et

$$Q(x,A) = \mathbb{P}(f(x,U) \in A), \quad A \subset E.$$

4.1 Cas d'un espace d'états fini

Proposition (Matrice de transition) Soit $(X_n)_{n\in\mathbb{N}}$ une chaîne de Markov sur un espace d'états fini E. On note :

$$\forall x, y \in E, \quad Q(x,y) = \mathbb{P}(X_1 = y \mid X_0 = x)$$

Supposons que $(X_n)_{n\in\mathbb{N}}$ est irréductible. Alors l'algorithme de Metropolis-Hasting transforme la chaîne $(X_n)_{n\in\mathbb{N}}$ en une chaîne $(Y_n)_{n\in\mathbb{N}}$ de matrice de transition :

$$P(x,y) = \begin{cases} Q(x,y) \times \min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) & \text{si} \quad n \neq y \\ 1 - \sum_{z\neq x} P(x,z) & \text{si} \quad x = y \end{cases}$$

Démonstration Soient $x, y \in E$ tels que $x \neq y$. Calculons $\mathbb{P}(Y_1 = y \mid Y_0 = x)$. On a :

$$\begin{aligned} \mathbb{P}(Y_1 = y \mid Y_0 = x) &= \mathbb{P}_x\left(f(x, U_1) = y, V_1 < \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) \\ &= \mathbb{P}_x(X_1 = y)\mathbb{P}\left(V_1 < \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) \\ &= Q(x,y)\min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) \end{aligned}$$

De même :

$$\begin{aligned} \mathbb{P}(Y_1 =\mid Y_0 = x) &= 1 - \sum_{y\neq x} \mathbb{P}(Y_1 = y \mid Y_0 = x) \\ &= 1 - \sum_{y\neq x} P(x,y) \end{aligned}$$

Proposition Si pour tout $x, y \in E$, on a :

$$Q(x,y) > 0 \iff Q(y,x) > 0$$

alors la mesure π est réversible pour la chaîne $(Y_n)_{n\in\mathbb{N}}$.

Démonstration Soient $X, y \in E$. On souhaite vérifier que :

$$\pi(x)P(x,y) = \pi(y)P(y,x)$$

Distinguons deux cas :

- si $x = y$ alors on a évidement $\pi(x)P(x,x) = \pi(x)P(x,x)$.
- si $x \neq y$ alors :

  $$\pi(x)P(x,y) = \pi(x)Q(x,y)\min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right)$$

  de même :

  $$\pi(y)P(y,x) = \pi(y)Q(y,x)\min\left(1, \frac{\pi(x)Q(x,y)}{\pi(y)Q(y,x)}\right)$$

  Supposons que $\pi(x)Q(x,y) \leqslant \pi(y)Q(y,x)$ alors :

  $$\pi(x)P(x,y) = \pi(x)Q(x,y)$$

  et

  $$\begin{aligned} \pi(y)P(y,x) &= \pi(y)Q(y,x)\frac{\pi(x)Q(x,y)}{\pi(y)Q(y,x)} \\ &= \pi(x)P(x,y) \end{aligned}$$

  Si $\pi(x)Q(x,y) > \pi(y)Q(y,x)$, le raisonnement est le même.

Exemple Soit $(X_n)_{n\in\mathbb{N}}$ la marche aléatoire simple sur $\mathbb{Z}$. Soit π une mesure de probabilité sur $\mathbb{Z}$. On cherche à construire une chaîne de Markov $(Y_n)_{n\in\mathbb{N}}$ sur $\mathbb{Z}$ avec l'algorithme de Metropolis-Hasting de mesure réversible π. On a donc les algorithmes suivants :

```
def etapeMa(x):
    if rand() < 1/2:
        return x+1
    return x-1

```

```
def algoMetHast(x):
    y = etapeMa(x)
    if rand() < pi(y) * (1/2) / pi(x
    ) * (1/2):
        return y
    return x

```

On a donc :

$$Q(x,x+1) = Q(x,x-1) = \frac{1}{2}$$

$$\forall x,y \in E,\ |x-y| \neq 1, \quad Q(x,y) = 0$$

Et :

$$P(x,x+1) = \frac{1}{2}\min\left(1, \frac{\pi(x+1)}{\pi(x)}\right) \quad \text{et} \quad P(x,x-1) = \frac{1}{2}\min\left(1, \frac{\pi(x-1)}{\pi(x)}\right)$$

Propriété (**Irréductibilité et apériodicité**) . Soit Q une matrice de transition irréductible sur un espace d'états E tel que

$$\forall x,y \in E, \quad Q(x,y) > 0 \iff Q(y,x) > 0,$$

et soit π une distribution cible strictement positive sur E. Soit P la matrice de transition de la chaîne de Markov construite par l'algorithme de Metropolis-Hastings à partir de Q et π.
Alors :

i) La chaîne de transition P est irréductible.

ii) Si $P(x,x) > 0$ pour au moins un $x \in E$, alors P est apériodique.

Démonstration i) **Irréductibilité.** Soient $x,y \in E$. Comme Q est irréductible, il existe un chemin fini

$$x = z_1, z_2, \ldots, z_k = y \in E$$

tel que $Q(z_i, z_{i+1}) > 0$ pour tout $i = 1,\ldots,k-1$. Or par définition de Metropolis-Hastings :

$$P(z_i,z_{i+1}) = Q(z_i,z_{i+1})\min\left(1, \frac{\pi(z_{i+1})Q(z_{i+1},z_i)}{\pi(z_i)Q(z_i,z_{i+1})}\right) > 0,$$

donc $z_1,\ldots,z_k$ constitue un chemin admissible pour P. Ainsi, P est irréductible.

ii) **Apériodicité.** Pour tout état $x \in E$, la probabilité de rester en x est

$$P(x,x) = 1 - \sum_{y\neq x} P(x,y) \geq 0.$$

Si $P(x,x) > 0$ pour au moins un x, alors la chaîne peut rester en x avec une probabilité strictement positive, ce qui assure l'apériodicité de P.

Remarque (Cas des chaînes symétriques) Si Q est une matrice symétrique.

$$\text{i.e} \quad \forall x,y \in E, \quad Q(x,y) = Q(y,x)$$

et si elle est irréductible, alors l'algorithme de Metropolis-Hasting s'exprime de façon simplifiée :

$$P(x,y) = Q(x,y)\min\left(1, \frac{\pi(y)}{\pi(x)}\right) \quad \forall x, y \in E$$

et :

$$P(x,y) = 1 - \sum_{z \neq x} P(x,z) \quad \text{si } x = y$$

Dans ce cas, la probabilité d'acceptation de la transition $x \to y$ est $1 - \min(1, \pi(y)/\pi(x))$.

4.2 Cas d'un espace d'états continu

Proposition Dans le cas où E est un espace continu, tout fonctionne de manière analogue. On note $Q(x,y)$ la densité de la variable X_1 sachant $X_0 = x$. Alors :

$$\mathbb{E}(\varphi(X_1) \mid X_0 = x) = \int_E \varphi(y) Q(x,y)\, dy, \quad \forall \varphi \text{ mesurable.}$$

Pour tout petit intervalle $[y, y + \Delta_y]$, on a :

$$\mathbb{P}(X_1 \in [y, y+\Delta_y] \mid X_0 = x) \simeq Q(x,y)\Delta_y.$$

On note $\pi(x)$ la densité de la mesure cible et on définit la transition de la chaîne $(Y_n)_{n\in\mathbb{N}}$ par :

$$P(x, dy) = Q(x,y)\min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) dy + \left[1 - \int_E Q(x,y)\min\left(1, \frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}\right) dy\right] \delta_x(dy),$$

où δ_x est la mesure de Dirac en x. On a alors :

$$\int_E P(x, dy) = 1.$$

L'algorithme reste lui aussi inchangé. On génère $y = X_1$ à, partir de la chaîne Q. On accepte la transition avec probabilité $\frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)}$. Si on refuse la transition, on reste sur place. À nouveau, la chaîne de Markov sera, sous de bonnes hypothèses, irréductible et la mesure π sera réversible pour cette chaîne.

Exemple (Marche aléatoire à pas gaussiens) Soit $(Z_n)_{n\in\mathbb{N}}$ une suite de variables iid de loi $\mathcal{N}(0,1)$. On définit le processus $(X_n)_{n\in\mathbb{N}}$ par :

$$X_0 \in \mathbb{R}, \quad \forall n \geqslant 1, X_{n+1} = X_n + Z_{n+1}$$

On aura donc :

$$\forall x, y \in \mathbb{R}, \quad Q(x,y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y-x)^2}{2}\right)$$

Donc si π est une densité cible quelconque, alors :

$$\frac{\pi(y)Q(y,x)}{\pi(x)Q(x,y)} = \frac{\pi(y)}{\pi(x)} \quad \text{car} \quad Q(x,y) = Q(x,y)$$

On peut alors définir la nouvelle chaîne $(Y_n)_{n\in\mathbb{N}}$ par :

$$\forall n \in \mathbb{N}, Y_{n+1} = Y_n + Z_{n+1}\mathbb{1}_{\frac{\pi(Y_n+Z_{n+1})}{\pi(Y_n)} < V_{n+1}}$$

avec $(V_n)_{n\in\mathbb{N}}$ une suite idd de variables de loi $\mathcal{U}([0,1])$. Cette nouvelle chaîne de Markov admet alors π pour mesure invariante, réversible et ergodique. De plus :

$$Y_n \xrightarrow[n\to\infty]{\text{L}} \pi$$

L'algorithme de Metropolis-Hasting permet donc d'approcher n'importe quelle densité sur $\mathbb{R}$. Or il peut être très lent dans certains cas.

4.3 Mesures de Gibbs sur un espace d'états fini

Soit E un espace d'états fini. On définit :

$$H : E \longrightarrow \mathbb{R} \tag{4.3.1}$$

une fonction d'énergie sur E. En physique statistique, elle représenterait l'énergie d'un état du système.

Définition (**Mesure de Gibbs**) . La mesure de Gibbs sur E de température inverse β est la mesure de probabilité μ_β définie par :

$$\mu_\beta(x) = \frac{1}{z_\beta} e^{\beta H(x)} \tag{4.3.2}$$

où $z_\beta = \sum_{x \in E} e^{\beta H(x)}$.

Proposition (Fondamentale) Par construction de la mesure de Gibbs, lorsque β est petit (le système a beaucoup d'énergie), la mesure a tendence à s'uniformiser sur E. Or, lorsque β est petit (i.e le système a peu d'énergie), alors μ_β ne charge plus que l'ensemble $\mathcal{M} = \{x \in E \mid \mu_\beta(x) = \mu_0\}$ où μ_0 est le maximum de μ_β.

L'algorithme du recuit simulé, que nous verrons plus tard, cherche donc à déterminer de telles solutions en simulant une marche aléatoire sur E de loi μ_β. En reffroidissant le système petit à petit (i.e en augmentant β), la marche se stabilisera sur un maximum. Mais pour cela, nous devons savoir simuler μ_β. Pour cela, on utilisera l'algorithme de Metropolis-Hastings.

Or lorsque E est grand, simuler une variable aléatoire de loi μ_β est difficile pour plusieurs raisons :

- z_β devient très grand : lors de l'approximation numérique, on peut donc avoir des valeurs de $e^{\beta H(x)}/z_\beta$ encodées comme 0.
- z_β devient difficile à calculer du fait de la taille de E.

Les méthodes directes de simulation de mesure de Gibbs fonctionnent très mal. Heureusement, on utilise alors l'algorithme de Metropolis-Hasting.

Remarque (Adaptation de l'algorithme) Soit Q la matrice de transition d'une chaîne de Markov irréductible sur E telle que $Q(x,y) > 0 \iff Q(y,x) > 0$ pour tout $x, y \in E$. Pour simuler la loi μ_β, on utilise la matrice de transition définie pour tout $x, y \in E$ par :

$$P_\beta(x,y) = Q(x,y) \min\left(1, \frac{\mu_\beta(y)Q(y,x)}{\mu_\beta(x)Q(x,y)}\right) \tag{4.3.3}$$

$$= Q(x,y) \min\left(1, \frac{\frac{e^{\beta H(y)}}{z_\beta} Q(y,x)}{\frac{e^{\beta H(x)}}{z_\beta} Q(x,y)}\right) \tag{4.3.4}$$

$$= Q(x,y) \min\left(1, e^{\beta(H(y)-H(x))} \frac{Q(y,x)}{Q(x,y)}\right) \tag{4.3.5}$$

On pourra donc appliquer l'algorithme sans problème d'approximation de très petites valeurs, car z_β n'apparaît plus.

Exemple (Depinning Transition) Soit l'espace d'états suivant :

$$E = \{\text{chemins au plus proche voisin sur } \mathbb{Z} \text{ de longueur } n\}$$

Un élément de E est donc une marche aléatoire sur E. On définit la fonctionnelle d'énergie H sur E telle que :

$$\forall s \in E, \quad H(s) = \text{Card}(\{0 \leqslant i \leqslant n \mid s_i = 0\})$$

On souhaite simuler un élément de E sous la loi μ_β. On peut proposer différentes transformations pour un chemin de E :

1. Si $s \in E$, on tire $i \in [\![1, n]\!]$ on remplace ensuite le saut en i par le saut opposé. *(C'est une marche réversible).*
2. Si $s \in E$, on tire $i \in [\![1, n]\!]$ et on remplace un schéma $\bigvee$ en i par un $\bigwedge$ en i. *(Celle-ci n'est pas réversible.).*

D'après 4.3.5, on peut donc définir l'algorithme suivant :

```
def etapeP(x,b):
    y = etapeQ(x)
    if rand() < (Q(y,x) / Q(x,y)) * exp(b (H(y) - H(x))):
        return y
    return x

```

Le comportement de la chaîne ainsi produite sera donc déterminé par le paramètre β. Si β sera faible (i.e température basse), la chaîne aura un comportement de marche aléatoire symétrique. En revanche, autour d'une certaine valeur de β appelée β_c, dit critique, la chaîne orbitera autour de l'axe des abscisses. C'est ce que l'on appelle une *transition de phase*.

Remarque Si Q est une matrice *réversible et symétrique* alors

$$\forall x, y \in E, P_\beta(x, y) = Q(x, y) \min\left(1, e^{\beta(H(y)-H(y))}\right)$$

Dans ce cas, plutôt que de tester `rand() < (Q(y,x) / Q(x,y)) * exp(b (H(y) - H(x)))`, on préfère tester `log(rand()) < b (H(y) - H(x))`. Qui sera moins sujet à provoquer des erreurs d'approximation (de valeurs infinies).

4.4 Algorithme du recuit simulé

On se place toujours dans un espace d'états E fini et $H : E \longrightarrow \mathbb{R}$ une fonction. On souhaite déterminer :

$$\begin{cases} \max_E H \\ \text{argmax}_E H \end{cases}$$

La méthode du recuit simulé permet de déterminer un estimateur de ces quantités pour un coût relativement faible par rapport à une exploration complète de l'espace d'états. Elle est basée sur la remarque suivante.

Lemme Soit E^* l'ensemble définit par :

$$E^* := \{e \in E \mid H(e) = \max_E H\} \tag{4.4.1}$$

On a $\lim_{\beta\to\infty} \mu_\beta \sim \mathcal{U}(E^*)$.

Démonstration Soit $\delta > 0$ tel que $H(e) < \max H - \delta$ pour tout $e \notin E^*$. On a donc :

$$\sum_{e\in E^*} e^{\beta H(e)} \leqslant z_\beta = \sum_{e\in E} e^{\beta H(e)} = \sum_{e\in E^*} e^{\beta H(e)} + \sum_{e\in E/E^*} e^{\beta H(e)}$$

$$\iff |E^*|e^{\beta \max H} \leqslant z_\beta \leqslant |E^*|e^{\beta \max H} + |E|e^{\beta(\max H-\delta)}$$

Donc pour $e \in E^*$, on a :

$$\mu_\beta(e) = \frac{e^{\beta \max H}}{z_\beta}$$
$$\iff \frac{e^{\beta \max H}}{e^{\beta \max H}|E^*| + |E|e^{\beta(\max H - \delta)}} \leqslant \mu_\beta(e) \leqslant \frac{e^{\beta \max H}}{e^{\beta \max H}|E^*|}$$

donc $\frac{1}{|E^*|+|E|e^{-\beta\delta}} \leqslant \mu_\beta(e) \leqslant \frac{1}{|E^*|}$. D'où $\lim_{\beta\to\infty} \mu_\beta(e) = 1/|E^*|$.
De même pour $e \in E/E^*$, on a :

$$\mu_\beta(e) \leqslant \frac{e^{\beta(\max H - \delta)}}{|E^*|e^{\beta \max H}} \leqslant \frac{e^{-\beta\delta}}{|E^*|} \underset{\beta\to\infty}{\longrightarrow} 0$$

d'où $\mu_\beta(e) \underset{\beta\to\infty}{\longrightarrow} 1/|E^*|\mathbb{1}_{e\in E^*}$.

L'algorithme du recuit simulé consite donc à utiliser à chaque étape l'algorithme de Metropolis-Hasting pour simuler μ_β avec $\beta_n \underset{n\to\infty}{\longrightarrow} \infty$ assez lentement.

Proposition (Algorithme du recuit simulé) On se donne `etape Q`, et une suite $(\beta_n)_{n\geqslant 0}$. On définit ensuite :

```
1
2 def recuitSimule(x,n):
3     y = etapeQ(x)
4         if rand() < exp(beta[n] (H(y) - H(x))) * (Q(y,x) / Q(x,y)):
5             return y
6         return x
7
```

La différence fondamentale de cet algorithme est que la probabilité de transition dépend maintenant de n.

Théorème (**Convergence de l'algorithme du recuit simulé**) . On note $(z_n)_{n\geqslant 0}$ la chaîne de Markov inhomogène (la chaîne dépend de l'état précédent et de la posittion dans le temps n) dans le temps de recuit simulé. Il existe $c_* > 0$ tel que pour tout $c < c_*$, en posant $\beta_n := c \log n$, alors :

$$\boxed{\lim_{n\to\infty} z_n \in E^*} \quad \text{p.s} \tag{4.4.2}$$

Ici, la suite $(\beta_n)_{n\geqslant 0}$ est la suite des températures inverses que nous utilisons pour simuler le reffroidissement du système lors de l'exécution de l'algorithme. Ce théorème nous assure donc qu'il existe une courbe de reffroidissement optimale qui permet la convergence de l'algorithme.

Remarque Ce théorème nous assure que si l'on refroidit petit à petit le système (i.e $n \to \infty$), alors la chaîne tend à s'arrêter sur un maximum de H. Dans cette situation, $\hat{M}_n = \max_{k\leqslant n} H(z_k)$ et $\hat{E}_n = \operatorname{argmax}_{z\in\{z_k|k\in\mathbb{N}\}} H(z)$ sont des estimateurs fortement consistants de $\max H$ et $\operatorname{argmax} H$.

En pratique ces estimateurs convergent très lentement pour un tel choix de β. On préfèrera donc faire tendre β vers ∞ plus rapidement et simuler plusieurs chaînes partant de points différents de l'espace d'état.

Cet algorithme est très performant en plusieurs dimensions puisque l'algorithme ne dépend pas de la nature des données sur lequelles sont calculées H. En revanche, il est peu performant pour des fonctions H trop périodiques.

Chapitre 5

Algorithme de Robbins Monro

Contents

5.1 Cadre Général 45
5.2 Convergence de l'algorithme 46
5.2.1 Théorème de Robbins-Sigmund 46
5.2.2 Convergence de l'algorithme de Robbins-Monro 48

5.1 Cadre Général

Dans ce chapitre, nous allons présenter un algorithme permettant de déterminer les solutions de l'équation :

$$h(\theta) = 0$$

pour une fonction h lorsqu'on peut écrire $h(\theta) = \mathbb{E}(g(\theta, Y))$. Par exemple, chercher θ tel que :

$$\begin{aligned} & \int_{\mathbb{R}_+} (x-\theta)e^{-x^3}\, dx = 0 \\ \iff & \int_{\mathbb{R}_+} xe^{-x^3}\, dx - \theta \int_{\mathbb{R}_+} e^{-x^3}\, dx = 0 \\ \iff & \theta = \frac{\int_{\mathbb{R}_+} xe^{-x^3}\, dx}{\int_{\mathbb{R}_+} e^{-x^3}\, dx} \end{aligned}$$

Proposition (Algorithme) On se fixe une suite $(\gamma_n)_{n\in\mathbb{N}} \in (\mathbb{R}_+^*)^{\mathbb{N}}$ telle que $\sum_{n\geqslant 0} \gamma_n = \infty$ mais $\sum_{n\geqslant 0} \gamma_n^2 < \infty$. On définit par récurrence :

$$\boxed{\theta_{n+1} = \theta_n - \gamma_n g(\theta_n, Y_{n+1})} \tag{5.1.1}$$

où les $(Y_n)_{n\in\mathbb{N}}$ sont des variables aléatoires idd de même loi que Y. Sous de bonnes conditions, $(\theta_n)_{n\in\mathbb{N}}$ converge vers une solution θ de $h(\theta) = 0$.

Exemple Supposons que $g(\theta, Y) = \theta - Y$ et $\gamma_n = \frac{1}{n+1}$. La solution de $h(\theta) = 0$ est exactement $\theta = \mathbb{E}(Y)$. On remarque que :

$$\begin{aligned} \theta_{n+1} &= \theta_n - \frac{1}{n+1}(\theta_n - Y_{n+1}) \\ &= \frac{n}{n+1}\theta_n + \frac{1}{n+1}Y_{n+1} \end{aligned}$$

Donc $(n+1)\theta_{n+1} - n\theta_n = Y_{n+1}$, d'où $n\theta_n = Y_1 + \cdots + Y_n$. Soit $\theta_n = \frac{1}{n}\sum_{i=1}^{n} Y_i$. On a bien $\lim_{n\to\infty} \theta_n = \mathbb{E}(Y)$ d'après la loi des grands nombres.

Dans le cas déterministe, on suppose ggg connue et calculable, ce qui permet d'utiliser des méthodes comme Newton. Cependant, en pratique, ggg est souvent inconnue (définie par une espérance) ou bruitée (observations aléatoires). L'algorithme de Robbins-Monro résout ce problème en remplaçant $g(\theta)$ par une estimation $H(\theta, Y)$, tout en garantissant la convergence vers la solution de $h(\theta) = 0$ sous des conditions sur les pas (α_n).

5.2 Convergence de l'algorithme

Toute cette section est consacrée à la démonstration de la convergence de l'algorithme. Pour cela, nous allons introduire un théorème très important dans l'étude des algorithmes stochastiques : le théorème de Robbins-Sigmund.

5.2.1 Théorème de Robbins-Sigmund

Nous allons étudier ce théorème sous la forme suivante :

Théorème (**Robbins-Sigmund**) . Soient quatre suites $(z_n)_{n\in\mathbb{N}}, (\alpha_n)_{n\in\mathbb{N}}, (\beta_n)_{n\in\mathbb{N}}$ et $(\gamma_n)_{n\in\mathbb{N}}$ adaptées pour une filtration $(\mathcal{F}_n)_{n\in\mathbb{N}}$. Supposons qu'elles vérifient :

i) $\sum_{n\geqslant 0} \alpha_n = \infty$ p.s

ii) $\sum_{n\geqslant 0} \beta_n < \infty$ p.s

iii) $\sum_{n\geqslant 0} \gamma_n < \infty$ p.s

iv) $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1 - \alpha_n + \beta_n) z_n + \gamma_n$

Alors :

$$\lim_{n\to\infty} z_n = 0 \text{ ps} \tag{5.2.1}$$

Dans notre situation, nous pouvons considérer que $\beta_n = 0$ pour tout $n \in \mathbb{N}$. En effet, nous cherchons à modifier l'hypothèse $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1 - \alpha_n + \beta_n) z_n + \gamma_n$ en :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1 - \alpha_n) z_n + \gamma_n$$

Lemme Réduisons le théorème au cas $\beta_n = 0$.

Démonstration Supposons donc que :

i $\sum_{n\geqslant 0} \alpha_n = \infty$ p.s

ii $\sum_{n\geqslant 0} \beta_n < \infty$ p.s

iii $\sum_{n\geqslant 0} \gamma_n < \infty$ p.s

iv $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1 - \alpha_n) z_n + \gamma_n$

Montrons que si $\beta_n = 0$, alors $z_n \underset{n\to\infty}{\longrightarrow} 0$ aussi. Pour cela, posons, pour tout $n \in \mathbb{N}$:

$$Y_n = \frac{z_n}{\prod_{j=1}^{n}(1+\beta_j)}$$

On a les égalités suivantes :

$$\begin{aligned}\mathbb{E}\left(\frac{z_{n+1}}{\prod_{j=1}^{n+1}(1+\beta_j)} \mid \mathcal{F}_n\right) &= \frac{\mathbb{E}(z_{n+1} \mid \mathcal{F}_n)}{\prod_{j=1}^{n+1}(1+b_j)} \\ &\leqslant \frac{(1-\alpha_n+\beta_n)z_n}{\prod_{j=1}^{n+1}(1+\beta_j)} + \frac{\gamma_n}{\prod_{j=1}^{n+1}(1+\beta_j)} \\ &= \frac{1-\alpha_n+\beta_n}{1+\beta_{n+1}} \frac{z_n}{\prod_{j=1}^{n}(1+\beta_j)} + \frac{\gamma_n}{\prod_{j=1}^{n+1}(1+\beta_j)}\end{aligned}$$

Or, pour n grand, on a :

$$\frac{1-\alpha_n+\beta_n}{1+\beta_n} \sim_\infty 1-\alpha_n$$

D'où :

$$\mathbb{E}\left(Y_{n+1} \mid \mathcal{F}_n\right) \leqslant (1-\alpha'_n)Y_n + \gamma'_n$$

Avec :

$$Y_n = \frac{z_n}{\prod_{j=1}^{n}(1+\beta_j)}, \quad \alpha'_n = \frac{\alpha_n}{1+\beta_n} \quad \text{et} \quad \gamma'_n = \frac{\gamma_n}{\prod_{j=1}^{n}(1+\beta_j)}$$

On souhaite maintenant appliquer le theorème de Robbins-Sigmund à la suite $(Y_n)_{n\geqslant 0}$. Pour cela, vérifions les hypothèses du théorème :

i Pour α'_n, on a que $\alpha'_n = \frac{\alpha_n}{1+\beta_n}$ donc $\alpha'_n \sim_\infty \alpha_n$. Par comparaison :

$$\sum_{n\geqslant 0} \alpha_n < \infty \text{ p.s} \quad \Longrightarrow \quad \sum_{n\geqslant 0} \alpha'_n < \infty \text{ p.s}$$

ii Pour γ'_n, on a :

$$\sum_{n\geqslant 0} \gamma'_n = \sum_{n\geqslant 0} \frac{\gamma_n}{\prod_{j=1}^{n}(1+\beta_j)} < \sum_{n\geqslant 0} \gamma_n < \infty$$

On peut donc appliquer le théorème d'où :

$$\lim_{n\to\infty} Y_n = 0 \text{ p.s}$$

De plus, par construction de Y_n, on a :

$$\lim_{n\to\infty} z_n = \lim_{n\to\infty} Y_n \underbrace{\prod_{j=1}^{n}(1+\beta_j)}_{<\infty} \quad \Longrightarrow \quad \lim_{n\to\infty} z_n = 0$$

Le théorème est donc valable avec ce nouveau jeu d'hypothèses. □

Nous pouvons désormais passer à la démonstration du théorème de Robbins-Sigmund.

Démonstration La démonstration de ce résultat est basée sur la construction d'une sur-martingale et de l'utilisation du théorème de convergence des martingales pour conclure. Nous commençons donc avec les hypothèses suivantes :

i) $\sum_{n\geqslant 0} \alpha_n = \infty$ p.s
ii) $\sum_{n\geqslant 0} \beta_n < \infty$ p.s
iii) $\sum_{n\geqslant 0} \gamma_n < \infty$ p.s
iv) $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1-\alpha_n)z_n + \gamma_n$

Montrons que $z_n \underset{n\to\infty}{\longrightarrow} 0$.

1. Construction de la sur-martingale. Posons :

$$\forall n \in \mathbb{N}, \quad S_n = z_n + \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k$$

Montrons que $(S_n)_{n\geqslant 0}$ est une sur-martingale.

$$\begin{aligned}
\mathbb{E}(S_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(z_{n+1} \mid \mathcal{F}_n) + \sum_{k=0}^{n} \mathbb{E}(\alpha_k z_k \mid \mathcal{F}_n) - \sum_{k=0}^{n} \mathbb{E}(\gamma_k \mid \mathcal{F}_n) \\
&= \mathbb{E}(z_{n+1} \mid \mathcal{F}_n) + \sum_{k=0}^{n} \alpha_k z_k - \sum_{k=0}^{n} \gamma_k \\
&\leqslant (1-\alpha_n) z_n + \gamma_n + \sum_{k=0}^{n} \alpha_k z_k - \sum_{k=0}^{n} \gamma_k \\
&= \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k + z_n = S_n
\end{aligned}$$

Par définition, $(S_n)_{n\geqslant 0}$ est bien une sur-martingale pour la filtration $(\mathcal{F}_n)_{n\geqslant 0}$.

2. Convergence de la sur-martingale. Soit τ_c le temps d'arrêt suivant :

$$\tau_c = \inf \left\{ n \in \mathbb{N} \mid \sum_{k=0}^{n} \gamma_k \geqslant \right\}$$

Soit la martingale arrêtée par τ_C $(S_{n\wedge\tau_c})_{n\geqslant 0}$, elle est bornée inférieurement par $-C$ car :

$$S_{n\wedge\tau_C} \geqslant -\sum_{k=0}^{\tau_C - 1} \gamma_k \geqslant -C$$

Par théorème, elle converge donc presque sûrement vers une limite finie W_C. Puisque $\sum_{k\geqslant 1} \gamma_k < \infty$ p.s, il existe $C > \sum_{k\geqslant 1} \gamma_k$ tel que $\tau_c = \infty$ p.s. Donc sur l'évènnement $\{\tau_c = \infty\}$, on a $S_n = S_{n\wedge\tau_C}$ pour tout n, donc :

$$\lim_{n\to\infty} S_n = W_c \text{ p.s}$$

Or $S_n = z_n + \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k$, et comme $\sum_{k\geqslant 1} \gamma_k$ converge, la convergence de S_n implique celle de :

$$z_n + \sum_{k=0}^{n-1} \alpha_k z_k$$

mais $\sum_{k\geqslant 1} \alpha_k = \infty$ donc nécessairement $\sum_{k\geqslant 1} \alpha_k z_k < \infty$. Finalement :

$$z_n \underset{n\to\infty}{\longrightarrow} 0 \text{ p.s}$$

□

5.2.2 Convergence de l'algorithme de Robbins-Monro

Maintenant que nous avons tous les outils nécessaires, nous pouvons passer à l'étude de la convergence de cet algorithme. Pour rappel, on souhaite déterminer les solutions de l'équation $h(\theta) = 0$ où h peut s'écrire sous la forme :

$$h(\theta) = \mathbb{E}[H(Y, \theta)],$$

où Y est une variable aléatoire quelconque que l'on sait simuler. L'algorithme de Robbins-Monro approche la solution θ^* par les itérés suivants :

$$\forall n \geq 0, \quad \theta_{n+1} = \theta_n - \gamma_n H(Y_{n+1}, \theta_n),$$

où $(Y_n)_{n\geq 0}$ est une suite de variables i.i.d. de même loi que Y.
Le théorème suivant nous assure sa convergence sous de bonnes hypothèses :

Théorème (**Conditions suffisantes de convergence**) . Avec les mêmes notations que précédemment :

i) (Coercivité) Il existe θ^* tel que $h(\theta)(\theta - \theta^*) \geq \mu\|\theta - \theta^*\|^2$ pour un certain $\mu > 0$,

ii) (Croissance de h) Il existe $A, B > 0$ tels que $\|h(\theta)\|^2 \leq A + B\|\theta\|^2$,

iii) (Contrôle du bruit) $\mathbb{E}(\|h(\theta) - H(\theta, Y)\|^2) \leq K$ pour un certain $K > 0$.

Alors, si $\sum_{n\geq 0} \gamma_n = \infty$ et $\sum_{n\geq 0} \gamma_n^2 < \infty$, on a :

$$\lim_{n\to\infty} \theta_n = \theta^* \quad \text{p.s.}$$

Démonstration La démonstration de cette convergence utilise la construction récursive de l'algorithme ainsi que le théorème de Robbins-Siegmund. Soit $\mathcal{F}_n = \sigma(\theta_0, \ldots, \theta_n, Y_0, \ldots, Y_n)$ la filtration associée au n-ième itéré de l'algorithme. L'idée est de séparer l'incrément de l'algorithme en deux parties :

$$\Delta M_n = h(\theta_n) - H(\theta_n, Y_{n+1}),$$

appelée incrément de martingale. On peut donc réécrire l'algorithme comme suit :

$$\theta_{n+1} = \theta_n - \gamma_n h(\theta_n) + \gamma_n \Delta M_n.$$

Montrons que $(M_n)_{n\geq 0}$ est une martingale :

$$\begin{aligned}
\mathbb{E}(\Delta M_n \mid \mathcal{F}_n) &= \mathbb{E}(h(\theta_n) - H(\theta_n, Y_{n+1}) \mid \mathcal{F}_n) \\
&= h(\theta_n) - \mathbb{E}(H(\theta_n, Y_{n+1}) \mid \mathcal{F}_n) \\
&= h(\theta_n) - h(\theta_n) = 0.
\end{aligned}$$

Pour étudier la convergence, nous considérons la distance quadratique des itérés :

$$z_n = \|\theta_n - \theta^*\|^2.$$

ainsi que l'égalité suivante :

$$\|a + b + c\|^2 = \|a\|^2 + \|b\|^2 + \|c\|^2 + 2\langle a, b\rangle + 2\langle a, c\rangle + 2\langle b, c\rangle.$$

On a donc :

$$\begin{aligned}
\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(\|\theta_{n+1} - \theta^*\|^2 \mid \mathcal{F}_n) \\
&= \mathbb{E}(\|\theta_n - \gamma_n h(\theta_n) + \gamma_n \Delta M_n - \theta^*\|^2 \mid \mathcal{F}_n) \\
&= \|\theta_n - \theta^*\|^2 + \gamma_n^2 \|h(\theta_n)\|^2 + \gamma_n^2 \mathbb{E}(\|\Delta M_n\|^2 \mid \mathcal{F}_n) \\
&\quad - 2\gamma_n \langle \theta_n - \theta^*, h(\theta_n)\rangle + 2\gamma_n \langle \theta_n - \theta^*, \Delta M_n\rangle \\
&\quad - 2\gamma_n^2 \langle h(\theta_n), \Delta M_n\rangle.
\end{aligned}$$

Comme $\mathbb{E}(\Delta M_n \mid \mathcal{F}_n) = 0$, les termes croisés $\langle \theta_n - \theta^*, \Delta M_n\rangle$ et $\langle h(\theta_n), \Delta M_n\rangle$ disparaissent en espérance conditionnelle. On obtient donc :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant z_n + \gamma_n^2 \|h(\theta_n)\|^2 + \gamma_n^2 \mathbb{E}(\|\Delta M_n\|^2 \mid \mathcal{F}_n) - 2\gamma_n \langle \theta_n - \theta^*, h(\theta_n)\rangle.$$

En utilisant les hypothèses du théorème :

$$\begin{aligned}\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) &\leqslant z_n + \gamma_n^2(A + B\|\theta_n\|^2) + \gamma_n^2 K - 2\gamma_n\mu\|\theta_n - \theta^*\|^2 \\ &\leqslant z_n(1 - 2\gamma_n\mu) + \gamma_n^2(A + K + B\|\theta_n\|^2).\end{aligned}$$

Comme $\|\theta_n\|^2 \leqslant 2\|\theta_n - \theta^*\|^2 + 2\|\theta^*\|^2$, on peut écrire :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant z_n(1 - 2\gamma_n\mu + 2B\gamma_n^2) + \gamma_n^2(A + K + 2B\|\theta^*\|^2).$$

En posant $\alpha_n = 2\gamma_n\mu - 2B\gamma_n^2$ et $\beta_n = \gamma_n^2(A + K + 2B\|\theta^*\|^2)$, on obtient :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leqslant (1 - \alpha_n)z_n + \beta_n.$$

Sous les hypothèses $\sum \gamma_n = \infty$ et $\sum \gamma_n^2 < \infty$, on a $\sum \alpha_n = \infty$ et $\sum \beta_n < \infty$. On peut donc appliquer le théorème de Robbins-Siegmund pour conclure que :

$$\lim_{n\to\infty} z_n = 0 \quad \text{p.s.},$$

c'est-à-dire $\lim_{n\to\infty} \theta_n = \theta^*$ p.s. □

Statistiques

Chapitre 1

Théorie de l'Information

Contents

1.1 Cadre 52
1.1.1 Modèle 52
1.1.2 Les variables aléatoires d'un modèle 53
1.1.3 Domination et Densités 54
1.1.4 Hypothèses de régularité 54
1.1.5 Vraissemblance 55
1.1.6 Maximum de Vraissemblance 56
1.2 Informations Qualitatives 57
1.2.1 Exhaustivité 57
1.2.2 Minimalité et complétude 58
1.2.3 Statistiques Libres 58
1.3 Information de Kullback-Leibler 59
1.4 Informations Quantitatives 59
1.4.1 Vecteur Score 59
1.4.2 Modèles exponentiels 60
1.4.3 Information de Fisher 61
1.4.4 Propriétés de l'information de Fisher 62

1.1 Cadre

Dans cette section, nous posons le cadre de l'étude des statistiques ainsi que l'ensemble des notations et objets utilisés.

1.1.1 Modèle

L'étude des statistiques se résume à l'étude de la loi de variables aléatoires ou de suites de variables aléatoires. Nous devons donc définir un cadre dans lequel ces différents objets vivrons. On considère ainsi des variables aléatoires X définies telles que :

$$X : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) \longrightarrow (\mathcal{X}, \mathcal{A}, \mathbb{P}^X) \\ \omega \longmapsto X(\omega) \end{cases} \tag{1.1.1}$$

Dans notre contexte, nous n'avons accès qu'à des *réalisations* $X(\omega)$ de la variable aléatoire X et nous souhaitons déterminer sa loi $\mathbb{P}^X$. Pour cela, nous faisons une hypothèse appelée *hypothèse*

paramétrique forte qui suppose que la loi de X ne dépend que d'un paramètre $\theta \in \Theta$, où $\Theta \subset \mathbb{R}^d$. Ainsi, la recherche de la loi $\mathbb{P}^X$ se réduit à la détermination d'un paramètre θ_* tel que $\mathbb{P}^X = \mathbb{P}_{\theta_*}$.

Définition (**Modèle Paramétrique**) . On appellera *modèle paramétrique* l'espace probabilisé $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ tel que $\mathcal{P}_\Theta = \{\mathbb{P}_\theta \mid \theta \in \Theta\}$ où Θ est un ouvert convexe de $\mathbb{R}^d$.

Or cet espace ne nous permet de considérer que de simples variables aléatoires. Dans un cadre plus général, nous souhaiterions étudier des *échantillons* $(X_1, \dots, X_n)$ i.i.d d'une variable aléatoire X. Nous définissons pour cela le *modèle d'échantillonnage*.

Définition (**Modèle d'échantillonnage**) . Soit $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ un modèle paramétrique. Un *échantillon* $(X_1(\omega), \dots, X_n(\omega))$ de ce modèle est un n-uplet de réalisations indépendantes d'une variable aléatoire X de loi $\mathbb{P}_\theta \in \mathcal{P}_\Theta$. On note $\mathbb{P}^{\{X_1,\dots,X_n\}} = \mathbb{P}_{\theta_*}^{\otimes n}$ la loi de cet échantillon. Le modèle paramétrique de cet échantillon est donc :

$$(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\otimes n} = (\mathcal{X}^n, \mathcal{A}^n, \mathcal{P}_\Theta^{\otimes n})$$

On a donc :

$$(X_1, \dots, X_n) : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) & \longrightarrow (\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\otimes n} \\ \omega & \longmapsto (X_1(\omega), \dots, X_n(\omega)) \end{cases}$$

Enfin, pour l'étude plus théorique de tels objets, il serait intéressant de pouvoir faire tendre vers l'infini la taille de ces échantillons $(X_1, \dots, X_n)$. On définit donc un modèle dit *asymptotique* :

$$(X_i)_{i\in\mathbb{N}^*} : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) & \longrightarrow (\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\infty} \\ \omega & \longmapsto \{X_i(\omega) \mid i \in \mathbb{N}_*\} \end{cases}$$

1.1.2 Les variables aléatoires d'un modèle

L'objet principal que nous allons manipuler permet d'effectuer des opérations sur des échantillons d'une variable aléatoire. On les appelle les *statistiques*, notées S.

Définition (**Statistique**) . Une fonction mesurable S est une *statistique* si elle ne dépend pas de θ. On a donc :

$$S(X_1, \dots, X_n) : \begin{cases} (\mathcal{X}, \mathcal{A})^n \longrightarrow (\mathcal{Y}, \mathcal{B}) \\ (X_1, \dots, X_n) \longmapsto S(X_1, \dots, X_n) \end{cases}$$

On notera aussi $\mathbb{P}_\theta^S = (\mathbb{P}_\theta^{\oplus n})^S$ la mesure image de $\mathbb{P}_\theta^{\oplus n}$ par S.

Cette translation de nos observations $(X_1, \dots, X_n)$ implique la définition d'un *modèle image* nous permettant de manipuler ce nouvel objet $S(X_1, \dots, X_n)$.

Définition (**Modèle Image**) . Le modèle image de $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\oplus n}$ par S_n est donc $(\mathcal{Y}, \mathcal{B}, \mathcal{P}_\Theta^{S_n})$, avec :

$$\mathcal{P}_\Theta^{S_n} := \{\mathbb{P}_\theta \circ S_n^{-1} \mid \theta \in \Theta\} \tag{1.1.2}$$

Pour rappel, l'objectif des statistiques est de déterminer le paramètre $\theta_* \in \Theta$. Pour cela, il faut que la donnée utilisée ($(X_1, \dots, X_n)$, X, ou $S(X_1, \dots, X_n)$) contienne le plus d'information possible sur θ. Il serait donc préférable que la translation S ne réduise pas cette quantité. De telles statistiques seront donc dites *exhaustives*. Elles seront caractérisées par la loi conditionnelle de l'échantillon.

Définition (**Loi conditionnelle d'une statistique**) . Soit T une statistique. La loi conditionnelle de $(X_1, \ldots, X_n)$ sachant $T(X_1, \ldots, X_n)$ est définie par :

$$\mathbb{P}_\theta^{(X_1,\ldots,X_n)|T}(A) = \mathbb{P}((X_1, \ldots, X_n) \in A \mid T) = \mathbb{E}_\theta(\mathbb{1}_{(X_1,\ldots,X_n)\in A} \mid T) \tag{1.1.3}$$

pour tout $A \in \mathcal{A}^n$.

1.1.3 Domination et Densités

Définition (**Domination**) . On dira qu'une mesure μ positive *domine* $\mathbb{P}_\theta$ sur $(\mathcal{X}, \mathcal{A})$ si :

$$\boxed{\forall A \in \mathcal{A}, \quad \mu(A) = 0 \Longrightarrow \mathbb{P}_\theta(A) = 0} \tag{1.1.4}$$

On notera alors $\mathbb{P}_\theta \ll \mu$. Si c'est le cas pour tout $\theta \in \Theta$, on dira alors que *le modèle est dominé par* μ.

Plus formellement, $\mathbb{P}_\theta \ll \mu$ ssi

$$\forall \varepsilon > 0, \exists \delta > 0 \quad \text{tel que} \quad \forall A \in \mathcal{A}, \mu(A) < \delta \Longrightarrow \mathbb{P}_\theta(A) < \varepsilon \tag{1.1.5}$$

Théorème (**Radon-Nikodym**) . Soit μ σ-finie. On a $\mathbb{P}_\theta \ll \mu$ ssi

$$\forall \theta \in \Theta, \exists f \in L^1_+(\mu) \quad \text{telle que} \quad \forall A \in \mathcal{A}, \quad \mathbb{P}_\theta(A) = \int_A f \, d\mu \tag{1.1.6}$$

La densite f de $\mathbb{P}_\theta$ est unique à égalité μ-presque partout. Elle est notée $f = d\mathbb{P}_\theta/d\mu$ comme dérivée de Radon-Nicodym.

Remarque (Cas discret) Le cas des lois discrètes peut paraître contre-intuitif, mais s'explique bien grâce à ce théorème. Soit $\mathcal{X}$ fini ou dénombrable et $\mathcal{A} = \mathcal{P}(\mathcal{X})$. Soit m_c la mesure de comptage sur $\mathcal{A}$. Par définition de l'intégrale de Lebesgue, pour toute fonction $f : \mathcal{X} \longrightarrow \mathbb{R}$ et $A \in \mathcal{A}$, on a :

$$\int_A f d\, m_c = \sum_{x \in A} f(x) m_c(x) = \sum_{x \in A} f(x)$$

Soit $\mathcal{P}_\Theta$ une famille de lois sur $\mathcal{X}$ et $n = 1$. On a $\mathcal{P}_\theta \ll m_c$ et les densités $f = d\mathbb{P}_\theta/dm_c$ existent et vérifient :

$$\forall A = x \in \mathcal{A}, \quad \mathbb{P}_\theta(\{x\}) = \int_{\{x\}} f \, dm_c = f(x)$$

On a donc bien $f(x) = \mathbb{P}_\theta(X = x)$.

1.1.4 Hypothèses de régularité

Pour tous les raisonnements suivants sur notre modèle, nous devons définir des *hypothèses*. Celles-ci nous assureront certaines propriétés nécessaires à l'application des théorèmes suivants.

Définition (**Modèle Identifiable**) . S'il existe μ σ-finie telle que $\mathcal{P}_\Theta \ll \mu$ et $\mu(\{f_{\theta_0} \neq f_{\theta_1}\}) > 0$ pour tout $\theta_0 \neq \theta_1$, alors le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est dit *identifiable*.

Un modèle est identifiable si θ caractérise de façon unique ue loi. Dans le cas contraire, un même valeur de θ pourrait correspondre à plusieurs lois initiales. L'estimation du paramètre n'aurait donc plus aucun sens.

Définition (**Modèle Homogène**) . Le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est dit *homogène* si $\mathcal{P}_\Theta \ll \mathbb{P}_\theta$ pour tout $\theta \in \Theta$.

L'homogénéité est une propriété fondamentale d'un modèle et elle intervient sur plusieurs aspects. Un modèle homogène garantit que tous les $\mathbb{P}_\theta$ ont les mêmes ensembles négligeables. D'autre part, l'homogénéité garantit que le support des variables aléatoires ne dépend pas de θ. Sans cela, les inversion de dérivées/intégrales (voir ci-dessous) ne seraient pas possibles.

Proposition Le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est homogène ssi, il existe μ σ-finie telle que :

i) $\mathcal{P}_\Theta \ll \mu$

ii) $\{f_\theta > 0\}$ μ-presque partout pour tout $\theta \in \Theta$.

En conséquence, pour tout $\theta \in \Theta$, $\mathcal{P}_\Theta \ll \mathbb{P}_\theta$. Donc les $\mathbb{P}_\theta$ ont tous les mêmes négligeables et la densité de $\mathbb{P}_{\theta_2}$ par rapport à $\mathbb{P}_{\theta_1}$ s'écrit :

$$\frac{d\mathbb{P}_{\theta_2}}{d\mathbb{P}_{\theta_1}} = \frac{f_{\theta_2}}{f_{\theta_1}} = \frac{d\mathbb{P}_{\theta_2}/d\mu}{d\mathbb{P}_{\theta_1}/d\mu}$$

En effet, on a pour tout $A \in \mathcal{A}$:

$$\begin{aligned}\mathbb{P}_{\theta_2}(A) &= \int_A \frac{f_{\theta_2}}{f_{\theta_1}}\, d\mathbb{P}_{\theta_1} = \int_A \frac{f_{\theta_2}}{f_{\theta_1}} f_{\theta_1}\, d\mu \\ &= \int_A f_{\theta_2}\, d\mu = \mathbb{P}_{\theta_2}(A).\end{aligned}$$

Définition (**Hypothèses de régularité**) . Définissons donc ces hypothèses :

(H_1) : Le modèle est **identifiable** et **dominé**.

$$i.e \quad \mathcal{P}_\Theta << \mu, \quad \text{et} \quad \forall \theta \in \Theta, f_\theta = \frac{d\mathbb{P}_\theta}{d\mu}, \quad \text{et} \quad \forall \theta_1 \neq \theta_2,\ \mu(\{f_{\theta_1} = f_{\theta_2}\}) = 0$$

(H_2) : Le modèle est **homogène**.

$$i.e \quad \forall \theta \in \Theta,\ \nabla f_\theta \in L^2(\mu)$$

(H_3) : Pour tout $A \in \mathcal{A}$, la fonction $\theta \in \Theta \longmapsto \mathbb{P}_\theta(A)$, admet des dérivées partielles telles que :

$$\frac{\partial}{\partial \theta_k}\mathbb{P}_\theta(A) = \frac{\partial}{\partial \theta_k}\int_A f_\theta\, d\mu = \int_A \left(\frac{\partial}{\partial \theta_k} f_\theta\right)\, d\mu$$

(H_4) : Pour tout $A \in \mathcal{A}$, la fonction $\theta \in \Theta \longmapsto \mathbb{P}_\theta(A)$, admet des dérivées partielles d'ordre 2 telles que :

$$\frac{\partial^2}{\partial \theta_j \partial \theta_k}\mathbb{P}_\theta(A) = \frac{\partial^2}{\partial \theta_j \partial \theta_k}\int_A f_\theta\, d\mu = \int_A \left(\frac{\partial^2}{\partial \theta_j \partial \theta_k} f_\theta\right)\, d\mu$$

1.1.5 Vraissemblance

Soit un échantillon $X = (X_1, \ldots, X_n)$ d'un modèle vérifiant $H_{0,1}$.

Définition (**Fonction de vraissemblance**) . On appelle *fonction de vraissemblance* l'application L définie par :

$$L(\theta; x) = \prod_{i=1}^{n} f_\theta(x_i)$$

Pour un x observé, L est vue comme une fonction du paramètre θ. Il est crucial d'inclure les indicatrices de support $\mathbb{1}_{A(\theta)}(x_i)$ dans l'expression de f_θ.

Remarque (Double interprétation de la vraisemblance) La fonction $L(\theta; x)$ est un pivot entre deux mondes :

- A *priori* (Modélisation) : Pour θ fixé, $x \mapsto L(\theta; x)$ est la *densité* de l'échantillon. Elle décrit la distribution des observations possibles.
- A *posteriori* (Inférence) : Pour x observé et fixé, $\theta \mapsto L(\theta; x)$ est la *vraisemblance*. Elle mesure la *plausibilité* de chaque paramètre au vu des données.

Attention : La vraisemblance n'est pas une mesure de probabilité sur Θ. Elle permet de comparer deux paramètres entre eux via le rapport $\frac{L(\theta_1;x)}{L(\theta_0;x)}$, mais sa valeur absolue n'a pas de sens probabiliste intrinsèque.

Remarque (Log-vraisemblance) En pratique, on utilise la log-vraisemblance notée $\ell_n(\theta)$:

$$\ell_n(\theta) = \log L(\theta; x) = \sum_{i=1}^{n} \log f_\theta(x_i).$$

Elle facilite la recherche du Maximum de Vraisemblance (EMV) et le calcul du Score.

Théorème (**Critère de factorisation de Fisher-Neyman**) . T est une statistique exhaustive ssi il existe deux fonctions mesurables positives h et g_θ telles que :

$$\boxed{L(\theta; x) = h(x) \cdot g_\theta(T(x))}$$

Où h est indépendante de θ et g_θ ne dépend de x qu'à travers $T(x)$.

1.1.6 Maximum de Vraissemblance

Voyons à présent la vraissemblance comme fonction de θ. Elle traduit alors la plausibilité que θ soit le paramètre inconnu vis-à-vis de l'échantillon observé. On souhaiterait donc en déterminer le maximum pour trouver le θ "le plus probable". Pour cela, on utilise la méthode du *maximum de vraissemblance*.

Proposition Soit $X = (X_1, \ldots, X_n)$ un n-échantillon d'une variable aléatoire de loi $\mathbb{P}_{\theta_*}$ et de densité f_θ. On souhaite déterminer le paramètre θ, pour cela, on considère la vraissemblance de l'échantillon définie par :

$$\forall \theta \in \Theta, \quad L(\theta, X_1, \ldots, X_n) = \prod_{i=1}^{n} f_\theta(X_i)$$

dont on cherche le maximum. Or le produit n'est pas pratique à utiliser, notamment lors du calcul de la dérivée. On utilise donc la fonction log. Par sa croissance, la log-vraissemblance atteindra le même maximum de la vraissemblance. On calcule donc :

$$\ell_n(\theta) = \log\left(\prod_{i=1}^{n} f_\theta(X_i)\right) = \sum_{i=1}^{n} \log f_\theta(X_i).$$

Après quelques vérifications, on calculera sa dérivée à annuler pour trouver une valeur de θ candidate, noté $\hat{\theta}_n$, que l'on conservera après avoir confirmé la concavité de ℓ_n en étudiant le signe de sa dérivée seconde. Le $\hat{\theta}_n$ sera appelé un *estimateur* et sera une fonction de $(X_1, \dots, X_n)$.

Définition (**Estimateur**) . Dans un modèle $(\mathcal{X}, \mathcal{A}, \mathbb{P}_\theta)$ dominé par μ, un *estimateur du maximum de vraisemblance* est une statistique $\hat{\theta}$ telle que :

$$L(\hat{\theta}, X_1, \dots, X_n) = \max_{\theta \in \Theta} L(\theta, X_1, \dots, X_n) \tag{1.1.7}$$

pour n'importe quel échantillon $(X_1, \dots, X_n)$ de loi $\mathbb{P}_{\theta_*}$.

1.2 Informations Qualitatives

1.2.1 Exhaustivité

Une statistique T est exhaustive si elle "conserve" toute l'information utile sur θ. Mathématiquement, cela signifie qu'une fois T connue, la loi de l'échantillon ne dépend plus de θ.

Définition (**Statistique Exhaustive**) . T est exhaustive pour θ si la loi conditionnelle de $(X_1, \dots, X_n)$ sachant T est indépendante de θ.

$$\forall \theta \in \Theta, \quad \mathcal{L}_\theta(X_1, \dots, X_n \mid T) = \mathcal{L}(X_1, \dots, X_n \mid T)$$

Remarque En pratique, on utilise presque exclusivement le théorème de factorisation de Fisher-Neyman pour prouver l'exhaustivité.

Proposition T est une *statistique exhaustive* ssi pour toute statistique S réelle intégrable, on a :

$$\mathbb{E}_\theta(S \mid \sigma(T)) = \varphi(T) \tag{1.2.1}$$

(i.e si cela ne dépend pas de θ.)

Exemple (Statistique exhaustive) Soit $(X_1, \dots, X_n)$ un échantillon d'une variable aléatoire $X \sim \mathcal{B}(1, \theta)$. Posons $S_n = \sum_{i=1}^n X_i \sim \mathcal{B}(n, \theta)$. On a ici, $\mathcal{X} = \{0, 1\}^n$. On cherche à estimer θ. Montrons que S est une statistique exhaustive. Soit $k \in [\![1, n]\!]$, on :

$$\begin{aligned}
&\mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n) \mid S_n = k) \\
&= \frac{\mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n) \cap S_n = k)}{\mathbb{P}_\theta(S_n = k)} \\
&= \mathbb{1}_{\{\sum_{i=1}^n x_i = k\}} \times \frac{\mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n))}{\mathbb{P}_\theta(S_n = k)} \\
&= \mathbb{1}_{\{\sum_{i=1}^n x_i = k\}} \times \prod_{i=1}^n \frac{\mathbb{P}_\theta(X_i = x_i)}{\binom{n}{k}\theta^k(1-\theta)^{n-k}} \\
&= \frac{1}{\binom{n}{k}}
\end{aligned}$$

qui est indépendant de θ donc S_n est bien une statistique exhaustive en θ.

1.2.2 Minimalité et complétude

On souhaite définir des statistiques dites minimales, contenant uniquement l'information nécessaire sur θ à partir de l'échantillon. Pour cela, on pose :

$$\mathcal{A}^* := \bigcap_{T \text{ exhaustives}} \sigma(T)$$

appelée la *tribu exhaustive minimale*. Elle contient donc toute l'information apportée par $(X_1, \dots, X_n)$ sur θ.

Définition (**Statistique exhaustive minimale**) . On dira d'une statistique exhaustive S^* qu'elle est *minimale* si $\sigma(S^*) = \mathcal{A}^*$.

Proposition (Caractérisation de $\mathcal{A}$) Sous H_1, pour $\theta_0 \in \Theta$ fixé, on a :

$$\mathcal{A} = \sigma\left\{ \frac{L_\theta}{L_{\theta_0}}(X_1, \dots, X_n) \mid \theta \in \Theta \right\}$$

Proposition (Statistiques exhaustives minimales) Soit S une statistique exhaustive telle que :

$$S(x_1, \dots, x_n) = S(y_1, \dots, y_n) \quad \iff \frac{L_\theta(x_1, \dots, x_n)}{L_\theta(y_1, \dots, y_n)} \text{ indépendants de } \theta \tag{1.2.2}$$

alors S est *minimale*.

Exemple (Loi exponentielle) Soit la densité de la loi exponentielle $f_\theta(x) = \theta e^{-\theta x}$. On cherche à déterminer S^*. Posons $S_n = \sum_{i=1}^n X_i$. On a :

$$L_\theta(X_1, \dots, X_n) = \prod_{i=1}^n f_\theta(X_i) = \theta^n e^{-\theta S_n}$$

Pour tout $\theta_0 \neq \theta_1$, on a :

$$\frac{L_{\theta_1}}{L_{\theta_2}}(X_1, \dots, X_n) = \exp((\theta_0 - \theta_1)S_n + n(\log\theta_1 - \log\theta_0))$$

Donc S est fonction mesurable du rapport $L_{\theta_1}/L_{\theta_2}$, elle est donc minimale.

Définition (**Statistique Complète (ou Totale)**) . Une statistique T est dite *complète* si pour toute fonction mesurable h telle que $\mathbb{E}_\theta[h(T)]$ existe :

$$\forall \theta \in \Theta, \quad \mathbb{E}_\theta[h(T)] = 0 \implies h(T) = 0 \quad \mathbb{P}_\theta\text{-p.s.}$$

Théorème (**Lien entre Complétude et Minimalité**) . Si une statistique T est *exhaustive* et *complète*, alors elle est *exhaustive minimale*.

Remarque Dans le cas des *modèles exponentiels* (voir plus loin), la statistique naturelle $T(X)$ est toujours exhaustive et complète (si l'espace des paramètres Θ est d'intérieur non vide). C'est le moyen le plus rapide en exercice pour obtenir une minimale.

1.2.3 Statistiques Libres

Définition (**Statistique Libre**) . Une statistique S est dite *libre* si sa loi ne dépend pas de θ.

$$\forall \theta, \theta' \in \Theta, \quad \mathbb{P}_\theta^S = \mathbb{P}_{\theta'}^S$$

Cette notion est fondamentale pour isoler le "bruit" du "signal" (θ). Le lien avec l'exhaustivité est donné par le théorème suivant, très utile pour montrer l'indépendance de deux variables ou pour prouver qu'une statistique n'est pas exhaustive.

Théorème (**Théorème de Basu**) . Si T est une statistique *exhaustive complète* et S est une statistique *libre*, alors T et S sont *indépendantes* sous toute loi $\mathbb{P}_\theta$.

Exemple Dans un modèle gaussien $\mathcal{N}(\mu, \sigma^2)$ avec σ^2 connu, la moyenne empirique $\bar{X}_n$ est exhaustive complète. La variance empirique S_n^2 (ou toute valeur centrée $X_i - \bar{X}_n$) est libre. Par le théorème de Basu, $\bar{X}_n$ et S_n^2 sont indépendantes.

1.3 Information de Kullback-Leibler

Définition (**Information de Kullback-Leibler**) . L'information de *Kullback-Leibler* (ou entropie relative) entre deux lois du modèle est définie par :

$$K(\theta_0 \mid \theta_1) = \mathbb{E}_{\theta_0}\left[\log \frac{f_{\theta_0}(X)}{f_{\theta_1}(X)}\right] = \int_{\mathcal{X}} f_{\theta_0} \log\left(\frac{f_{\theta_0}}{f_{\theta_1}}\right) \, d\mu \tag{1.3.1}$$

On parle de *divergence* ou de *contraste* plutôt que de distance, car cette application n'est pas symétrique ($K(\theta_0|\theta_1) \neq K(\theta_1|\theta_0)$) et ne respecte pas l'inégalité triangulaire.

Proposition (Propriétés fondamentales) Soit $\mathcal{P}_\Theta$ un modèle dominé.

- *Indépendance :* $K(\theta_0 \mid \theta_1)$ ne dépend pas du choix de la mesure dominante μ.
- *Positivité (Inégalité de Gibbs) :* $K(\theta_0 \mid \theta_1) \geqslant 0$ (découle de l'inégalité de Jensen).
- *Séparation :* $K(\theta_0 \mid \theta_1) = 0 \iff \mathbb{P}_{\theta_0} = \mathbb{P}_{\theta_1}$.
- *Lien Identifiabilité :* Si le modèle est identifiable, $K(\theta_0 \mid \theta_1) = 0 \iff \theta_0 = \theta_1$.

1.4 Informations Quantitatives

1.4.1 Vecteur Score

Durant toute cette section, nous serons amenés à calculer le gradient de log-vraissemblance. Pour simplifier les calculs et étudier leurs propriétés, nous allons les définir sous la forme de *vecteurs scores*.

Définition (**Vecteur Score**) . Soit $(H_{1,2})$, le vecteur aléatoire score d'un échantillon X est défini par le gradient de la log-vraissemblance de l'échantillon. On a donc :

$$\boxed{S_\theta(X) = \nabla_\theta \log f_\theta(X) = \frac{1}{f_\theta(X)} \nabla f_\theta(X).} \tag{1.4.1}$$

C'est une *variable aléatoire*.

Propriété (**Espérance**) . Sous $(H_{1,2,3})$, les vecteurs scores donc centrés.

$$i.e \quad (H_{1,2,3}) \Longrightarrow \forall \theta \in \Theta,\ \mathbb{E}_\theta(S_\theta(X)) = 0_p$$

Démonstration Supposons $(H_{1,2,3})$, on a alors que $\int_{\mathcal{X}} f_\theta \, d\mu = 1$ car c'est une densité. Donc par $(H_{1,2,3})$, on a :

$$\begin{aligned}
&\int_{\mathcal{X}} \frac{\partial}{\partial \theta_k} f_\theta \, d\mu = 0 \\
\iff &\int_{\mathcal{X}} \frac{1}{f_\theta(x)} \frac{\partial}{\partial \theta_k} f_\theta(x) \times \underbrace{f_\theta(x) \, d\mu(x)}_{d\mathbb{P}_\theta(x)} = 0 \\
\iff &\int_{\mathcal{X}} \frac{\partial}{\partial \theta_k} \log f_\theta \, d\mathbb{P}_\theta = 0 \\
\iff &\mathbb{E}_\theta \left(\frac{\partial}{\partial \theta_k} \log f_\theta \right) = 0
\end{aligned}$$

1.4.2 Modèles exponentiels

Voyons à présent un modèle régulier très utilisé : le *modèle exponentiel*.

Définition (**Modèle Exponentiel**) . Un modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ sera dit *exponentiel* si :

$$\boxed{\forall \theta \in \Theta, \quad f_\theta(x) = k_\theta h(x) \exp\left(\langle T(x), \gamma(\theta) \rangle\right).}$$

Avec $\gamma(\Theta) \subset \mathbb{R}^q$, $T : \mathcal{X} \longrightarrow \mathbb{R}^q$ et $h : \mathcal{X} \longrightarrow \mathbb{R}$ la fonction de changement de mesure.
Il sera qualifié *de plein rang* si $p = q$. On dira aussi qu'il est *canonique* si $p = q$ et γ est l'identité. On appellera enfin $T(X)$ la *statistique privilégiée du modèle*. Par soucis de clarté, on notera souvent $\kappa_\theta = \log k_\theta$.

Théorème (**Régularité**) . Un modèle exponentiel canonique de plein rang vérifie $(H_{1,2,3,4})$ et $k_\theta \in \mathcal{C}^\infty$.

Proposition Les modèles canoniques sont très pratiques à manipuler. On a :

$$f_\theta(x) = k_\theta h(x) \exp\left(\langle T(x), \theta \rangle\right)$$

Après calcul, on obtient que $S_\theta(X) = \nabla_\theta \log k_\theta + T(X) \in \mathbb{R}^p$. Or, on sait que $\mathbb{E}_\theta(S_\theta(X)) = 0$, donc :

$$\mathbb{E}_\theta(T(X)) = -\nabla_\theta \kappa_\theta.$$

Propriété (**Cas général**) . Dans un cadre plus général, un modèle exponentiel de plein rang admet des scores de la forme :

$$S_\theta(X) = J_\gamma^t \times T(X) + \nabla_\theta \kappa_\theta$$

avec $J_\gamma = \left(\left(\frac{\partial \gamma_i}{\partial \theta_j}(\theta) \right)_{i,j} \right)$ ma matrice Jacobienne $p \times p$ de $\gamma : \Theta \longrightarrow \mathbb{R}^p$. Si J_γ est définie positive, alors :

$$\mathbb{E}_\theta(T(X)) = -\left(J_\gamma^{-1}\right)^t \times \nabla_\theta \kappa_\theta$$

Propriété (**Statistique privilégiée**) . Dans le cas d'un modèle exponentiel, la statistique T est *exhaustive, complète* et *minimale*.

1.4.3 Information de Fisher

Passons à la définition principale de cette partie : l'information de Fisher. Elle quantifie l'information contenue sur θ dans l'acte d'observer X.

Définition (**Information de Fisher**) . Sous $(H_{1,2})$, la *quantité d'information de Fisher* en θ apportée par l'acte d'observer une réalisation $X(\omega)$ sous $\mathbb{P}_\theta$ est :

$$\boxed{\mathcal{I}_\theta(X) = \mathbb{E}_\theta(S_\theta(X) \times S_\theta^t(X))} \tag{1.4.2}$$

Si (H_4) est vérifiée, alors $\mathcal{I}_\theta(X) = \mathbb{V}_\theta(S_\theta(X))$, la matrice de covariance du vecteur score.

Propriété (**Information de Fisher**) .

- Sous $(H_{1,2})$, $\mathcal{I}_\theta(X)$ est intrinsèque au modèle (i.e elle ne dépend plus de la mesure dominante μ).
- Sous $(H_{1,2,3,4})$, on a :

$$\boxed{\mathcal{I}_\theta(X) = -\mathbb{E}_\theta(J_{S_\theta(X)}) = -\mathbb{E}_\theta\left(\left(\frac{\partial^2 \log f_\theta(X)}{\partial\theta_i\partial\theta_j}\right)_{i,j}\right)}$$

  avec $J_{S_\theta(X)}$ la Jacobienne du score.

Ces propriétés seront donc privilégiées pour le calcul pratique de l'information de Fisher.

Propriété (**Cas du modèle exponentiel canonique**) . Dans le cas d'un modèle exponentiel canonique, on a :

$$\mathcal{I}_\theta(X) = \mathbb{V}_\theta(T(X)) = -H_{\kappa_\theta}$$

où H_{κ_θ} est la hessienne de $\kappa_\theta = \log k_\theta$. Donc si $p = 1$, alors $\mathcal{I}_\theta(X) = -\frac{\partial^2}{\partial\theta^2}\log k_\theta$.

Proposition Comme dit précédement, les modèles exponentiels de plein rang sont très utiles. Ils permettent de calculer simplement l'information de Fisher à partir des fonctions définissant la densité. On a donc :

$$\begin{aligned}\mathcal{I}_\theta(X) &\underset{\text{def}}{=} J_\gamma^t \times \mathbb{V}_\theta(T(X)) \times J_\gamma(\theta)\\ &\underset{\text{prop}}{=} -H_{\kappa_\theta} - H_\gamma\left(J_\gamma^{-1}\right)^t \nabla_\theta\kappa_\theta\end{aligned}$$

Si $p = 1$,

$$\mathcal{I}_\theta(X) = (\gamma'(\theta))^2 \times V_\theta(T(X)) = \frac{-k'_\theta}{k_\theta}\left(\frac{k''_\theta}{k'_\theta} + \frac{\gamma''_\theta}{\gamma'_\theta} - \frac{k'_\theta}{k_\theta}\right)$$

Exemple Soit le modèle $X \sim \mathcal{N}(\theta_1, \theta_2^2)$, avec $\begin{cases}\theta_1 = \mathbb{E}_\theta(X)\\ \theta_2 = \sqrt{\mathbb{V}_\theta(X)}\end{cases}$. On a donc :

$$f_\theta(x) = \frac{1}{2\pi}\exp\left(\langle T(x), \gamma(\theta)\rangle + \kappa_\theta\right)$$

où $p = q = 2$ et :

$$T(x) = (x, x^2), \quad \kappa_\theta = -\frac{\theta_1^2}{2\theta_2^2} - \log\theta_2, \quad \gamma(\theta) = \left(\frac{\theta_1}{\theta_2}, -\frac{1}{2\theta_2^2}\right)$$

On peut donc ensuite appliquer les formules :

$$J_\gamma = \begin{pmatrix} \frac{1}{\theta_2^2} & -\frac{2\theta_1}{\theta_2^2} \\ 0 & \frac{1}{\theta_2^2} \end{pmatrix} \quad \mathbb{V}_\theta(T(X)) = \begin{pmatrix} \theta_2^2 & Cov(X, X^2) \\ Cov(X, X^2) & \mathbb{V}_\theta(X^2) \end{pmatrix}$$

On a donc :

$$\begin{aligned} \mathcal{I}_\theta(X) &= Y_\gamma^t \times V_\theta(T(X)) \times J_\gamma \\ &= -\mathbb{E}_\theta\left(\frac{\partial^2}{\partial\theta_i\partial\theta_j}\left(-\frac{(X-\theta_1)^2}{2\theta_2^2} - \log\theta_2\right)\right)_{i,j} = \begin{pmatrix} \frac{1}{\theta_2^2} & 0 \\ 0 & \frac{2}{\theta_2^2} \end{pmatrix} \end{aligned}$$

1.4.4 Propriétés de l'information de Fisher

L'information de Fisher possède des propriétés de structure fondamentales qui découlent de la linéarité du score et de la géométrie du modèle.

1. **Additivité :** Soient S et T deux statistiques $\mathcal{P}_\Theta$-indépendantes. Si les modèles associés vérifient $(H_{1,2,3})$, l'information jointe est la somme des informations :

   $$\mathcal{I}_\theta(S, T) = \mathcal{I}_\theta(S) + \mathcal{I}_\theta(T)$$

   Conséquence : Dans le modèle d'échantillonnage i.i.d. $\mathcal{P}_\Theta^{\otimes n}$, on a :

   $$\boxed{\mathcal{I}_\theta(X_1, \ldots, X_n) = n\mathcal{I}_\theta(X_1)}$$

2. **Décroissance par transfert (Information image) :** Soit S une statistique. Si le modèle image vérifie $(H_{1,2,3})$, le score image est $S_\theta(S) = \mathbb{E}_\theta[S_\theta(X) \mid S]$. Par l'inégalité de Jensen (ou la formule de décomposition de la variance), on a :

   $$\mathcal{I}_\theta(S) = \mathbb{V}_\theta(\mathbb{E}_\theta[S_\theta(X) \mid S]) \preceq \mathbb{V}_\theta(S_\theta(X)) = \mathcal{I}_\theta(X)$$

   Ainsi, $\mathcal{I}_\theta(X) - \mathcal{I}_\theta(S)$ est une matrice semi-définie positive. On perd de l'information en condensant les données, sauf si la statistique est exhaustive.

3. **Conservation par exhaustivité :** Sous $(H_{1,2,3})$, une statistique S est exhaustive si et seulement s'il n'y a pas de perte d'information par transfert :

   $$\boxed{S \text{ est exhaustive } \iff \mathcal{I}_\theta(S(X_1, \ldots, X_n)) = n\mathcal{I}_\theta(X_1)}$$

4. **Lien avec le contraste de Kullback :** Sous $(H_{1,2,3,4})$, pour tout $\theta_0 \in \Theta$ fixé, l'information de Fisher est la matrice Hessienne du contraste de Kullback-Leibler (courbure locale de la divergence) :

   $$\mathcal{I}_{\theta_0}(X) = \mathrm{Hess}_\theta\left(K(\theta_0 \mid \theta)\right)\Big|_{\theta=\theta_0} = \left(\left(\frac{\partial^2 K(\theta_0 \mid \theta)}{\partial\theta_i\partial\theta_j}\right)_{i,j}\right)_{\theta=\theta_0}$$

Chapitre 2

Estimation et Risque

Contents

2.1 Risque 63
2.2 Estimateurs 64
2.2.1 Définition 64
2.2.2 Comparaisons 64
2.2.3 Comparaison par le risque 64
2.2.4 Optimalité 66
2.2.5 Biais 66
2.3 Risque Quadratique 67
2.3.1 Cas $q = 1$ 67
2.3.2 Cas $q > 1$ 68
2.3.3 Amélioration de Rao-Blackwell 68
2.3.4 Optimalité de Lehman-Scheffe 69
2.4 Bornes inférieures de variance et Efficacité 69
2.4.1 Inégalité de Fréchet-Darmois-Cramér-Rao (FDCR) 70
2.4.2 La réciproque de Koopman (Cas d'égalité) 70

En statistiques paramétriques, on cherche à déterminer la valeur d'un paramètre θ^* inconnu. Nous posons ici une autré demarche que celle des estimateurs vue précédement.

2.1 Risque

Définition (**Règle de décision**) . On appelle *règle de décision* δ une statistique à valeurs dans un espace $(\mathbb{D}, \mathcal{D})$ telle que :

$$\delta : (\mathcal{X}, \mathcal{A})^{\oplus n} \longrightarrow (\mathbb{D}, \mathcal{D}).$$

Remarque Une règle de décision est donc une fonction mesurable et indépendante d'un paramètre θ puis c'est une statistique.

Proposition Il existe différents types de règles de décisions :

- **Estimation** : On choisit un paramètre θ par un estimateur $\delta(X_1, \ldots, X_n)$ à partir d'informations contenues dans un échantillon. On a donc $\mathbb{D} = \Theta$ et on peut définir *l'erreur quadratique moyenne* :
$$R_Q(\delta, \theta) = \mathbb{E}_\theta(\|\delta - \theta\|_p^2)$$

sous $\mathbb{P}_\theta$. Ici, on aura $\mathbb{D} = \mathcal{P}(\Theta)$.

- **Localisation** : Ici, on choisit plutôt une région pouvant contenir θ. On veut donc une notion de *confiance* telle que :

$$\forall \theta \in \Theta, \quad \mathbb{P}_\theta(\theta \in \delta(X_1, \dots, X_n)) \geqslant 1 - \alpha.$$

- **Test** : On a ici $\mathbb{D} = \{0, 1\}$ ou $\mathbb{D} = [0, 1]$. On cherche à vérifier si une affirmation sur θ est vraie plutôt qu'une autre. Elles seront définies par deux types d'hypothèses : *l'hypothèse nulle* $\mathcal{H}_0$ et *l'hypothèse alternative* $\mathcal{H}_1$. On souhaite donc que :
  - $\mathbb{P}(\delta(X_1, \dots, X_n) = 0)$ soit proche de 1 si $\theta \in \Theta_0$.
  - $\mathbb{P}(\delta(X_1, \dots, X_n) = 1)$ soit proche de 1 si $\theta \notin \Theta_0$.

2.2 Estimateurs

2.2.1 Définition

On cherche ici à estimer une transformation $g(\theta)$ d'un paramètre θ connu. Pour cela, on dispose de :

$$g : \begin{cases} \Theta \subseteq \mathbb{R}^p \longrightarrow \mathbb{R}^q \\ \theta \longmapsto g(\theta) \end{cases}$$

Définition (**Estimateur**) . On appelle *estimateur* de $g(\theta)$ toute fonction mesurable :

$$\delta : (\mathcal{X}, \mathcal{A})^{\oplus n} \longrightarrow (g(\Theta), \mathcal{D})$$

qui ne dépend pas de θ. On note Δ l'ensemble des estimateurs de θ.

Remarque Si $\hat{\theta}_n$ est un "bon" estimateur de θ, alors $g(\hat{\theta}_n)$ n'est pas nécessairement un aussi bon estimateur pour $g(\theta)$. La composition doit s'étudier au cas par cas.

2.2.2 Comparaisons

On souhaite qu'un estimateur se comporte comme une boîte noire pointant toujours sur θ peu importe sa valeur.

Définition (**Estimateur consistant**) . Une suite d'estimateurs $\delta_n \in \Delta^{\mathbb{N}}$ sera dite *consistante* si :

$$\forall \theta \in \Theta, \quad \lim_{n \to \infty} \delta_n(X_1, \dots, X_n) = g(\theta) \quad \mathbb{P}_\theta\text{- p.s.}$$

Définition (**Estimateur Asymptotiquement Gaussien**) . Un suite d'estimateurs $\delta_n \in \Delta^{\mathbb{N}}$ sera dite *asymptotiquement gaussienne* si pour tout $\theta \in \Theta$, il existe une matrice V_θ semi-définie positive telle que sous P_θ^∞, on ait :

$$\sqrt{n}(\delta_n(X_1, \dots, X_n) - g(\theta)) \xrightarrow[n \to \infty]{\mathcal{N}} \mathcal{N}(0_p, \mathbb{V}_\theta).$$

2.2.3 Comparaison par le risque

Un autre façon de comparer deux estimateurs est d'introduire une notion de *perte* qui quantifie le degré d'erreur d'estimation de $g(\theta)$. Pour cela, on définit les fonctions de perte ou *loss* en anglais.

Définition (**Fonction de perte**) . Une fonction de perte ℓ est une fonction mesurable positive :

$$\ell : \Theta \times g(\Theta) \longrightarrow (\mathbb{R}_+, \mathcal{B}_{\mathbb{R}})$$

telle que :

$$\boxed{\forall (\theta, d) \in \Theta \times g(\Theta), \quad l(\theta, d) = 0 \iff d = g(\theta)}.$$

Comme dit précédemment, on souhaite que nos estimateurs se comportent comme des boîtes noires. Il nous faut cela minimiser la perte moyenne d'une décision pour toutes les valeurs de $g(\theta)$ possibles. Cette notion sera donc définie comme le *risque*.

Définition (**Risque**) . On définit la *fonction de risque* associée à une décision $\delta \in \Delta$, comme l'application :

$$\theta \longmapsto R(\theta, \delta) = \mathbb{E}_\theta(\ell(\theta, \delta)).$$

Ainsi, un estimateur δ sera dit *préférable* à un autre $\delta' \in \Delta$, ssi :

$$\forall \theta \in \Theta, \quad R(\theta, \delta') \leqslant R(\theta, \delta).$$

On notera alors $\delta \succeq \delta'$.

Remarque Se pose alors un problème fondamental dans la recherche de l'estimateur optimal. Admettons que vous ayez un ami complotiste qui croit les extraterrestres sont parmi nous. Dans tous les mondes qu'il pourrait exister, la plupart ne sont pas dirigés par une secte martienne. Or, il suffit qu'un seul monde le soit pour que vous ne soyez pas meilleur que votre ami pour une valeur spécifique de θ.
C'est exactement le problème que posent les estimateurs constants. On va donc chercher à les exclure en créant des sous-classes $\Delta_i \subset \Delta$.

Démonstration Supposons qu'il existe $\delta^* \in \Delta$ un estimateur parfait tel que :

$$\forall \theta \in \Theta, \quad \forall \delta \in \Delta, \quad R(\theta, \delta^*) \leqslant R(\theta, \delta).$$

alors $\delta = \delta_{\theta_0}$ qui répond θ_0 presque-partout quelque soit l'échantillon proposé (un estimateur constant). D'où :

$$R(\theta, \delta_{\theta_0}) \begin{cases} = 0 & \text{si } \theta = \theta_0 \\ > 0 & \text{si } \theta \neq \theta_0. \end{cases}$$

Donc nécessairement $\forall \theta \in \Theta, R(\theta, \delta^*) = 0$ pour faire mieux que chacun des estimateurs constants.

Définition (**Estimateurs admissibles**) . On définit l'ensemble des estimateurs *admissibles*, noté Δ_0, comme l'ensemble des estimateurs "meilleurs en moyenne". Plus formellement,

$$\Delta_0 = \{\delta \in \Delta_0 \mid \nexists \delta' \in \Delta_0, \delta' \succeq \delta\}.$$

Proposition Supposons que Δ_0 est stable par combinaison convexe et que ℓ est strictement convexe en la décision δ. Alors si dd_0 est admissible dans Δ_0 et si δ_1 admet la même fonction de risque, alors $\delta_1 = \delta_0$ $\mathbb{P}_\theta$-p.p pour tout $\theta \in \Theta$.

Exemple Soit Δ_0 la classe des estimateurs sans biais moyen :

$$\Delta_0 := \{\delta \in \Delta \mid \mathbb{E}_\theta(\delta(X_1, \ldots, X_n)) = g(\theta)\}.$$

Soit $\delta_0, \delta_1 \in \Delta_0$, alors :

$$\forall \alpha \in [0, 1], \quad \alpha\delta_0 + (1 - \alpha)\delta_1 \in \Delta_0$$

par linéarité de $\mathbb{E}_\theta(\cdot)$. Donc Δ_0 est bien stable et :

$$\delta \longmapsto \ell(\theta, \delta) = \|g(\theta) - \delta\|_2^2$$

est convexe. Donc dans la classe des estimateurs sans biais, s'il existe un estimateur de plus petit risque, alors il est unique.

2.2.4 Optimalité

On cherche donc à déterminer des estimateurs optimaux (i.e minimisant la fonction de risque).

Définition (**Estimateur Optimal**) . On dira que δ_{opt} est ℓ-optimal dans Δ_0 si :

$$\forall \theta \in \Theta, \quad R(\theta, \delta_{opt}) = \min_{\delta \in \Delta_0} R(\theta, \delta).$$

Définition (**Estimateur Asymptotiquement Optimal**) . On dit qu'une suite $\delta_n(X_1, \ldots, X_n)$ est asymptotiquement ℓ-optimale relativement à une suite Δ_n de classes d'estimateurs si :

$$\forall \theta \in \Theta, \quad \lim_{n \to \infty} \frac{R(\theta, \delta_n(X_1, \ldots, X_n))}{\inf\limits_{\delta \in \Delta_n} R(\theta, \delta)} = 1.$$

2.2.5 Biais

La notion de biais permet de quantifier la façon dont un estimateur δ approche $g(\theta)$ en moyenne. Pour cela, il existe différents types de biais.

Définition (**Biais Moyen**) . On dira qu'un estimateyr $\delta \in \Delta$ est *sans biais moyen* pour $g(\theta)$ si :

$$\mathbb{E}_\theta(\delta(X_1, \ldots, X_n)) = g(\theta).$$

On peut ainsi définir la *fonction de biais* b d'un estimateur :

$$b : \theta \in \Theta \longmapsto \mathbb{E}_\theta(\delta) - g(\theta).$$

De même que pour l'optimalité, il est souhaitable que le biais d'un estimateur diminue lorsque la taille de l'échantillon augmente. On parle alors de *biais moyen asymptotique*.

Définition (**Biais moyen asymptotique**) . Un estimateur δ est dit *sans biais moyen asymptotique* si :

$$\forall \theta \in \Theta, \quad \lim_{n \to \infty} b(\delta_n) = 0.$$

Ce qui revient à $\mathbb{E}_\theta(\delta_n) \underset{n \to \infty}{\longrightarrow} g(\theta)$.

Remarque Un estimateur centré n'est pas nécessairement sans biais moyen. La notion de centrage exprime seulement le fait que $\mathbb{E}_\theta(\delta(X_1, \ldots, X_n)) = 0_q$ pour tout $\theta \in \Theta$. Or, si $g(\theta) \neq 0_q$ pour tout θ, alors l'estimateur est biaisé.

Définition (**Biais Général**) . Un estimateur δ est dit *sans biais général* pour une fonction de perte ℓ si :

$$\forall \theta, \theta' \in \Theta, \quad R(\theta, \delta) = \mathbb{E}_\theta(\ell(\theta, \delta)) \leqslant \mathbb{E}_\theta(\ell(\theta', \delta)).$$

On notera Δ_{sb} la classe des estimateurs sans biais.

Définition (**Biais Médian**) . Si $q = 1$, un estimateur $\delta \in \Delta$ est *sans biais médian* si :

$$\forall \theta \in \Theta, \quad \begin{cases} \mathbb{P}_\theta(\delta(X_1, \dots, X_n) > g(\theta)) \leqslant 1/2 \\ \mathbb{P}_\theta(\delta(X_1, \dots, X_n) < g(\theta)) \leqslant 1/2 \end{cases} .$$

La notion de biais médian signifie qu'un estimateur a autant de chances de surestimer $g(\theta)$ que de le sous-estimer. Elle traduit de façon plus fine la dispersion possible de l'estimateur autout de $g(\theta)$.

2.3 Risque Quadratique

La notion de biais vue précédement traduit le fait qu'un estimateur soit en moyenne "précis" ou pas pour estimer $g(\theta)$, mais elle ne donne aucune notion sur la dispersion d'un estimateur autout du paramètre. Or, il serait parfois préférable de travailler avec un estimateur moins "précis" (biaisé) mais moins dispersé.

2.3.1 Cas $q = 1$

On cherche donc une mesure permettant d'additionner la notion de biais et de variance d'un estimateur. En effet, un estimateur sans biais peut être complètement divergent pour n grand (non consistant) et inversement.
Au lieu d'utiliser une métrique différente, on pourrait adapter celles déjà introduites telles que la *perte* et le *risque moyen*. Une bonne fonction de perte pourrait être la *perte quadratique* :

$$\ell_Q(\theta, \delta) = (\delta - g(\theta))^2$$

possédant de bonnes propriétés de convexité et de différentiabilité. De plus elle pénalise de façon égale les erreurs négatives ou positives tout en pénalisant fortement les grosses erreurs.

Définition (**Risque Quadratique Moyen**) . On définit le *risque quadratique moyen* R_Q d'un estimateur δ comme le risque de cet estimateur appliqué à la fonction de perte quedratique. D'où :

$$\forall \theta \in \Theta, \quad R_Q(\theta, \delta) = \mathbb{E}_\theta(\ell_Q(\theta, \delta)) = \mathbb{E}_\theta((\delta - g(\theta))^2).$$

Propriété (**Décomposition**) . Comme souhaité, le risque quadratique moyen vérifiera :

$$\forall \delta \in \Delta, \forall \theta \in \Theta, \quad \boxed{R_Q(\theta, \delta) = \mathbb{V}_\theta(\delta) + b(\theta, \delta)^2}.$$

Démonstration Soient $\delta \in \Delta$ et $\theta \in \Theta$, on a :

$$\begin{aligned} R_Q(\theta, \delta) &= \mathbb{E}_\theta((\delta - g(\theta))^2) \\ &= \mathbb{E}_\theta((\delta - \mathbb{E}_\theta(\delta) + \mathbb{E}_\theta(\delta) - g(\theta))^2) \\ &= \mathbb{E}_\theta((\delta - \mathbb{E}_\theta(\delta))^2) + (\mathbb{E}_\theta(\delta) - g(\theta))^2 + 2\mathbb{E}_\theta(\underbrace{(\delta - \mathbb{E}_\theta(\delta))}_{\text{d'esp. nulle}})(\mathbb{E}_\theta(\delta) - g(\theta)) \\ &= V_\theta(\delta) + b(\theta, \delta). \end{aligned}$$

Proposition Ainsi, sur Δ_{sb}, on aura $\delta \succeq \delta'$ si $R_Q(\theta, \delta) \leqslant R_Q(\theta, \delta')$ pour tout $\theta \in \Theta$. Ce qui équivaut donc à $V_\theta(\delta) \leqslant \mathbb{V}_\theta(\delta')$. Dire que δ_{opt} est ℓ_Q-optimal signifie que δ_{opt} est *sans biais de variance minimale* pour tout $\theta \in \Theta$.

2.3.2 Cas $q > 1$

De même que précedement, on cherche à quantifier la pertience d'un estimateur en termes de dispersion et de précision par rapport au paramètre. On souhaite ici à estimer plusieurs paramètres $\theta = (\theta_1, \dots, \theta_n)$. On notera alors un estimateur $\delta(X_1, \dots, X_n) \in \mathbb{R}^q$ simplement δ de coordonnées $\delta^t = (\delta_1, \dots, \delta_q)$. De même, on aura :

$$\mathbb{E}_\theta(\delta) = (\mathbb{E}_\theta(\delta_1), \dots, \mathbb{E}_\theta(\delta_q))$$

$$\mathbb{V}_\theta(\delta) = \mathbb{E}_\theta\left[(\delta - \mathbb{E}_\theta(\delta))(\delta - \mathbb{E}_\theta(\delta)^t)\right] = \Big((\mathrm{cov}_\theta(\delta_i, \delta_j)_{i,j})\Big).$$

Définition (**Perte, Risque, Comparaison - Cas Multivarié**) . Pour toute matrice M semi-définie positive, on définit la perte ℓ d'un estimateur par rapport à un vecteur paramètre θ comme l'application telle que :

$$\ell_M(\theta, \delta) = (\delta - g(\theta))^t M (\delta - g(\theta)).$$

Aussi le *risque quadratique associé* à M sera :

$$\mathbb{R}_M(\theta, \delta) = \mathbb{E}_\theta(\ell_M(\theta, \delta)).$$

Si $M = Id$, on retrouve alors le risque quadratique moyen. On dira ainsi que $\delta \succeq_Q \delta'$ (i.e il est Q-préférable à δ') si :

$$R_M(\theta, \delta) \leqslant R_M(\theta, \delta') \quad \forall M.$$

Théorème (**Caractérisation**) . On dira $\delta \succeq_Q \delta'$ ssi la matrice de $\mathcal{M}_q(\mathbb{R})$:

$$\boxed{\mathbb{E}_\theta((\delta' - g(\theta))(\delta' - g(\theta))^t) - \mathbb{E}_\theta((\delta - g(\theta))(\delta - g(\theta))^t)}$$

est semi-définie positive.

2.3.3 Amélioration de Rao-Blackwell

Théorème (**Rao-Blackwell**) . Soit $\delta \in L^2(\mathcal{P}_\Theta)$ sans biais (moyen) pour $g(\theta)$ et S une statistique exhaustive en θ. Alors, l'estimateur δ dit *amélioré par* S, tel que $\delta_S = \mathbb{E}_\theta(\delta \mid S)$ est Q-préférable à δ.

Donc pour réduire une variance dans L^2 sans perte d'information, il suffit de conditionner une statistique exhaustive.

Démonstration δ_S est un estimateur, car S est exhaustive dont $\mathbb{E}_\theta(\delta \mid S)$ ne dépend pas de θ. D'autre part, δ_S est sans biais, puisque :

$$\mathbb{E}_\theta(\mathbb{E}_\theta(\delta \mid S)) = \mathbb{E}_\theta(\delta) = g(\theta).$$

Enfin, δ_S est de moindre variance, comme $\mathbb{E}_\theta(\delta \mid S)$ est la projection dans $L^2(\mathbb{P}_\theta)$.

Remarque Dans le cas $q = 1$, pour $\Psi(x) = x^2$ convexe, on a :

$$\begin{aligned}\mathbb{E}_\theta((\delta - g(\theta))^2 \mid S) &\geqslant [\mathbb{E}_\theta(\delta - g(\theta)) \mid S]^2 \\ &= [\delta_S - g(\theta)]^2\end{aligned}$$

par l'inégalité de Jensen conditionnelle. Donc en prenant $\mathbb{E}_\theta(.)$ à gauche et à droite, on a :

$$\mathbb{E}_\theta((\delta - g(\theta))^2) \geqslant \mathbb{E}_\theta(\delta_S - g(\theta))^2$$
$$R_Q(\theta, \delta) \geqslant R_Q(\theta, \delta_S).$$

Donc conditionner par une statistique exhaustive améliore toujours la fonction de risque.

2.3.4 Optimalité de Lehman-Scheffe

Théorème (**Lehman-Scheffe**) . Si δ est sans biais pour $g(\theta)$ et S exhaustive complète, alors $\delta_S = \mathbb{E}_\theta(\delta \mid S)$ est l'unique estimateur sans biais de variance minimale (i.e l'optimal δ_Q^{opt}).

Démonstration Montrons que cet estimateur ne peut plus être amélioré. Soit $\delta' \in \Delta_{sb}$, posons $\delta'_S = \mathbb{E}_\theta(\delta' \mid S)$. On a :

$$\mathbb{E}_\theta(\delta') = \mathbb{E}_\theta(\delta) = g(\theta)$$

Donc, pour tout $\theta \in \Theta$:

$$\mathbb{E}_\theta(\underbrace{\delta'_S - \delta_S}_{=\varphi(\delta)}) = \mathbb{E}_\theta(\delta - \delta') = 0.$$

Avec $\mathbb{E}_\theta(\varphi(S)) = 0$ pour tout θ. Or S est complète donc $\varphi(S) = 0$. D'où $\delta'_S = \delta_S$.

Proposition Pour trouver le δ_Q^{opt}, il faut donc :

- Trouver S complète : on cherche $\varphi(S)$ telle que $\mathbb{E}_\theta(\varphi(S)) = g(\theta)$ et la solution est bien l'estimateur sans biais de variance minimale.
- On cherche $\delta \in \Delta_{sb}$ simple.
- Puis, on conditionne $\mathbb{E}_\theta(\delta \mid S)$. Si S est complète, alors c'est gagné.

2.4 Bornes inférieures de variance et Efficacité

Nous avons précédemment défini Δ_{sb} comme la classe des estimateurs sans biais. Par la suite, nous avons introduit le risque quadratique pour quantifier la pertinence d'un estimateur en fonction de son biais et de sa variance.
Bien que nous sachions construire des estimateurs sans biais, l'enjeu est désormais d'étudier leur variance afin de réduire leur dispersion autour du paramètre cible. Une question fondamentale se pose alors : au sein de Δ_{sb}, existe-t-il un estimateur dont la variance est minimale de manière uniforme ? Si oui, comment peut-on le caractériser ?
Pour répondre à cette question, nous nous plaçons dans le cadre des modèles réguliers ($H_{1,2,3,4}$). L'utilisation de l'information de Fisher comme mesure de précision nous impose toutefois d'ajouter des conditions sur la finitude de l'information (H_5) et sur la régularité de l'estimateur δ (H_6) pour permettre l'interversion intégrale-dérivée :

(H_5) : L'information de Fisher est finie et non nulle :

$$\forall \theta \in \Theta, \quad \mathcal{I}_\theta(X) \in]0, +\infty[$$

(H_6) : δ est de carré intégrable et permet la dérivation sous le signe somme :

$$\delta \in L^2(\mathcal{P}_\Theta), \forall \theta \in \Theta, \quad \mathbb{E}_\theta(\delta(X)) = g(\theta) \quad \text{et} \quad \frac{\partial f_\theta}{\partial \theta}\delta \in L^1(\mu)$$

Tels que pour tout $A \in \mathcal{A}$ et $\theta \in \Theta$:

$$\frac{\partial}{\partial \theta}\left(\int_A \delta f_\theta \, d\mu\right) = \int_A \frac{\partial f_\theta}{\partial \theta}\delta \, d\mu$$

2.4.1 Inégalité de Fréchet-Darmois-Cramér-Rao (FDCR)

Théorème (**Borne de Cramér-Rao**) . Sous les hypothèses de régularité usuelles ($H_{1...5}$) et pour un estimateur δ_n sans biais de $g(\theta)$, on a :

$$\boxed{\mathbb{V}_\theta(\delta_n) \geqslant \frac{(g'(\theta))^2}{n\mathcal{I}_\theta(X)}} \tag{2.4.1}$$

Plus l'information de Fisher est grande, plus la variance minimale possible est petite (le modèle est "précis"). Il convient alors de définir proprement les estimateurs atteignant cette borne.

Définition (**Estimateur Efficace**) . Un estimateur δ_n est dit *efficace* s'il est sans biais et qu'il *atteint la borne* de Cramér-Rao :

$$\boxed{\mathbb{V}_\theta(\delta_n) = \frac{(g'(\theta))^2}{n\mathcal{I}_\theta(X)}}$$

2.4.2 La réciproque de Koopman (Cas d'égalité)

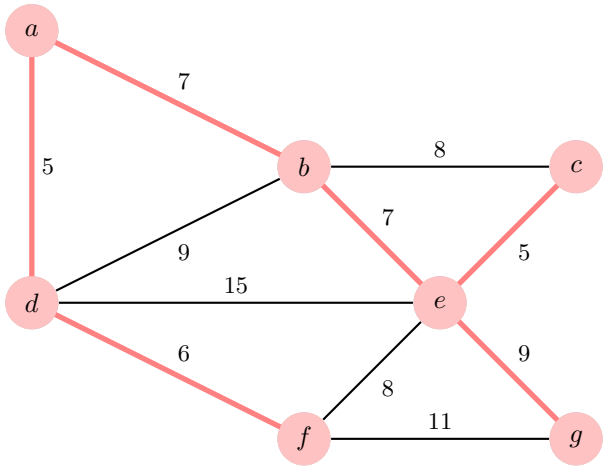
Propriété (**Lien avec les modèles exponentiels**) . La borne de Cramér-Rao est atteinte (cas d'égalité) ssi le modèle est exponentiel de la forme :

$$f_\theta(x) = h(x)k_\theta \exp(\gamma(\theta)\delta(x))$$

Dans ce cas, $\delta(X)$ est l'unique estimateur efficace pour son espérance $g(\theta) = -\frac{\kappa'_\theta}{\gamma'(\theta)}$.

Exemple (Application TD) Pour une loi de Poisson $\mathcal{P}(\theta)$, on montre que $\bar{X}_n$ est efficace car le modèle est exponentiel. Pour une loi Uniforme $\mathcal{U}[0,\theta]$, la borne ne s'applique pas (support dépendant de θ) : l'estimateur peut avoir une variance *plus petite* que la borne de Fisher.

Algorithmique



Chapitre 1

Flot maximal dans un graphe

Contents

1.1 Rappels sur les graphes 72
1.2 Cycle et flot 74
1.3 Flot et coupure 75
1.3.1 Flot maximal dans un réseau 75
1.3.2 Théorème de Coupure minimale 76
1.4 Flot maximal : principe de Ford-Fulkerson 77
1.4.1 Recherche de Flot Maximal 77
1.4.2 Recherche de flot maximal de coût minimal 78

L'objectif de ce chapitre est d'être capable d'organiser efficacement un flot d'objects dans un réseau. Par exemple, on souhaite amener le maxium de camions d'un point A à un point B en passant par certaines routes. De même, on souhaiterai savoir comment router efficacement le plus de paquets dans un réseau informatique.
Tout ces problèmes se généralisent en ce que l'on appelle la *recherche de flot maximal dans un graphe orienté*. Nous allons ici définir proprement ce genre de problème, donner des conditions d'existence de flot maximal et proposer des algorithmes pour les construire.

1.1 Rappels sur les graphes

Pour représenter une structure de réseau (routier, informatique, etc...), on utilise les graphes. Ce cours fait donc suite à celui sur la théorie des graphes enseigné en licence. Effectuons quelques rappels.

Définition (**Graphe Orienté**) . On appelle graphe orienté un couple d'ensembles finis $G = (X, E)$ où $X = \{1, \dots, n\}$ représente les sommets du graphe et $E = \{(x_i, y_j), \dots\}$, $x_i, y_j \in X$ l'ensemble ordonné des arrêtes du graphe. Pour un graphe orienté, les notions de graphe simple et multiraphe sont les même que pour le cas non orienté.

Exemple (Graphes et leur représentation gravarphique) Soient G_1 et G_2 deux graphes, en voici une représentation :

Définition (**Voisinnage**) . Le voisinnage d'un sommet x de G est l'ensemble des sommets

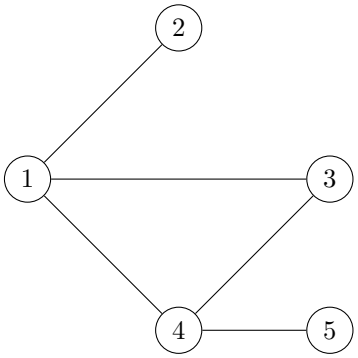


FIGURE 1.1 – Graphe non orienté

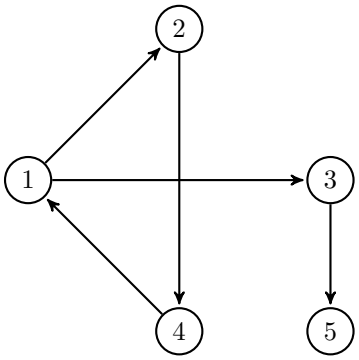


FIGURE 1.2 – Graphe orienté

y de G tels qu'il existe une arrête entre x et y dans G. On le note $V(x)$.

Remarque Si x est dans le voisinnage de y, on dira que x est adjascent à y et inversement.

Définition (**Degré**) . Le degré d'un sommet x de G est le cardinal du voisinnage x. On le note $d(x)$. Un sommet de voisinnage nul est dit *isolé* et un sommet de voisinnage égal à 1 est dit *pendant*.

Définition (**Voisinnage entrant et sortant**) . Soit G un graphe orienté et x un sommet de G. On définit deux types de voisinnages :

- *Voisinnage entrant :* noté $V^-(X)$ est l'ensemble des prédécesseurs de x.
- *Voisinnage sortant :* noté $V^+(X)$ est l'ensemble des successeurs de x.

On définira comme précédemment le degré sortant et le degré entrant d'un sommet x.

Définition (**Chaine**) . On appelle chaine de $G = (X, E)$ de longueur n toute suite alternée de sommets et d'arrêtes de G telle que :

$$c = (x_0, a_1, x_1, \ldots, a_n, x_n), \text{ telle que } \forall i \in [\![1, n]\!], a_i = (x_{i-1}, x_i)$$

Ici, n représente le nombre d'arrêtes de la chaine. Dans le cas d'un graphe simple, on notera les chaines de la façon suivante :

$$c = (x_0, \ldots, x_n) \quad \text{ou} \quad c = x_0 - x_1 - \cdots - x_n$$

Dans le cas des graphes orientés, on notera une chaîne simplement avec les arrêtes dont elle est constituée :

$$c = (u_1, \ldots, u_n), \forall i \in [\![1, n]\!] u_i \in E$$

Définition (**Accessibilité**) . Soit $G = (X, E)$ un graphe. On a :

- Soient x et y deux sommets de G. On dit que y est accessible à partir de x s'il existe une chaine joignant x et y dans G.
- G est dit *connexe* ssi $\forall x \in X, \forall y \in X$, y est accessible à partir de x.

- L'accessibilité est un relation d'équivalence entre les sommets.
  Ses classes d'équivalences sont les composantes connexes de G.

Définition (**Chaîne Simple**) . Une chaine est *simple* si elle ne passe pas deux fois par la même *arrête* et elle est dite *élémentaire* si elle ne passe pas deux fois pas le même *sommet*. On remarquera facilement qu'une chaîne élémentaire est simple.

Définition (**Cycle**) . Un cycle de G est une chaine simple dont le départ et l'arrivée sont le même sommet. Un cycle est donc de la forme :

$$c = x_0 - x_1 - \cdots - x_{n-1} - x0$$

Théorème (**Propriétés des cycles et chaînes**) . Soit G un graphe à $n \in \mathbb{N}$ sommets.

- Toute chaine élémentaire a une longueur inférieure à $n-1$
- Toute cycle élémentaire a une longueur inférieure à n
- De toute chaîne, on peut en extraire une chaîne élémentaire.

1.2 Cycle et flot

Définissons maintenant la notion de flot dans un cas général.

Définition (**Vecteur de chaîne**) . Soit $c = (u_1, \ldots, u_n)$ une *chaîne élémentaire* dans un graphe orienté $G = (X, E)$ tel que $|E| = n$ et d'arcs u_i. On appelle *vecteur de chaîne* associé à c l'application φ associée à c telle que :

$$\varphi : \begin{cases} E \longrightarrow \mathbb{Z} \\ u_i \longmapsto \begin{cases} 1 \text{ si } u_i \text{ est dans le sens de } c \\ -1 \text{ si } u_i \text{ n'est pas dans le sens de } c \\ 0 \text{ si } u_i \text{ n'est pas dans } c \end{cases} \end{cases}$$

On la note en général $\varphi = (\varphi(u_1), \varphi(u_2), \ldots, \varphi(u_n)) \in \mathbb{Z}^n$.

Venons en maintenant à la définition phare de ce chapitre :

Définition (**Flot**) . Soit $G = (X, E, c)$ un graphe orienté. On appelle *flot* dans G tout vecteur de cycle φ de G qui vérifie la loi de Kirchoff :

$$\boxed{\forall x \in X, \quad \sum_{u_i \in \omega^-(\{X\})} \varphi(u_i) = \sum_{u_i \in \omega^+(\{X\})} \varphi(u_i)}$$

Et on note $\varphi^i = \varphi(u_i)$ le *flot* de l'arrête u_i.

Remarque Un flot est donc un chaîne élémentaire d'un graphe orienté dont, pour chaque noeud, la somme des flots sortants est égale à la somme des flots entrants.

Propriété (**Flots**) . Soit $G = (X, E, c)$ un graphe tel que $|E| = n$, alors :

- Le vecteur null de $\mathbb{Z}^n$ est un flot de G.
- Toute combinaison linéaire de flots de G est aussi un flot de G.
- Tout vecteur cyclique de G est un flot de G.

Exemple Soit G le graphe suivant : Le vecteur $\varphi_1 = (2, -2, 1, 3, 4, -1, 1)$ est un flot de G. Le

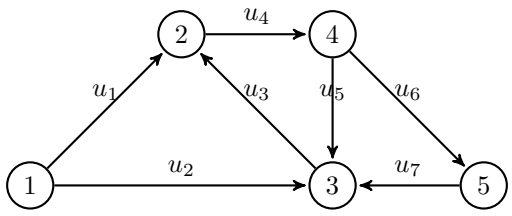


FIGURE 1.3 – flot dans un graphe

vecteur $\varphi_2 = (0, 0, 0, 0, 1, -1, 1)$ est aussi un flot. Soit $v = 2\varphi_1 + 3\varphi_2$, c'est aussi un flot de G.

Proposition (Loi de Kirchoff généralisée) Soit $G = (X, E, c)$ et $A \subset E$ non vide. Un chaîne φ est un flot de G ssi :

$$\sum_{u \in \omega^-(A)} \varphi(u) = \sum_{u \in \omega^+(A)} \varphi(u)$$

1.3 Flot et coupure

1.3.1 Flot maximal dans un réseau

Dans cette section, nous revenons à notre problème de départ en définissant la notion de graphe de transport. Graphes dans lesquels nous chercherons à construire des flots maximaux.

Définition (**Réseau de Transport**) . Un *réseau de transport* est un graphe orienté pondéré $R = (X, E, c)$ qui contient :

- un sommet sans prédécesseur appelé *source* (noté s).
- un sommet sans fils appelé *puit* (noté t).
- c, une application, qui à chaque arc, lui associe sa capacité notée c :

$$c : \begin{cases} E \longrightarrow \mathbb{R} \cup \{\infty\} \\ u = (x, y) \longmapsto c(u) \end{cases}$$

- un arc fictif de t à s doté d'une capacité infinie, appelé *arc retour*.

Définition (**Flot compatible**) . Un *flot compatible* sur un réseau $R = (X, E, c)$ tel que $|E| = n$ est un vecteur $\varphi \in \mathbb{Z}^n$ qui vérifie :

- (**Conservation de Kirchhoff**) pour tout $x \in X \setminus \{s, t\}$ (hors source s et puits t), on a :

$$\sum_{u \in \omega^+(x)} \varphi(u) = \sum_{u \in \omega^-(x)} \varphi(u)$$

où $\omega^+(x)$ (resp. $\omega^-(x)$) est l'ensemble des arcs sortants (resp. entrants) de x.

- (**Compatibilité avec les capacités**) :

$$\forall u \in E, \quad 0 \leqslant \varphi(u) \leqslant c(u).$$

La *valeur* du flot, notée $v(\varphi)$, est définie par :

$$v(\varphi) := \sum_{u \in \omega^+(s)} \varphi(u) - \sum_{u \in \omega^-(s)} \varphi(u),$$

c'est-à-dire le flux net sortant de la source (équivalent au flux net entrant dans le puits). Enfin, on dit qu'un arc u est *saturé* si $\varphi(u) = c(u)$.

Exemple Voici un exemple de réseau de transport :

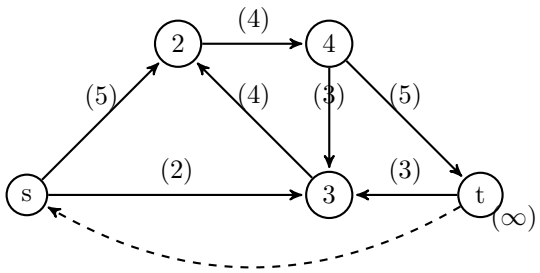


FIGURE 1.4 – Réseau de Transport

Définition (**Flot maximal**) . Un *flot maximal* d'un réseau est un flot compatible qui maximise la valeur $v(\varphi)$ parmi tous les flots compatibles du réseau.

1.3.2 Théorème de Coupure minimale

Pour construire un flot maximal dans un réseau, nous avons besoin d'un peu de théorie.

Définition (**Coupure et cocycle**) . Soit $R = (X, E, c)$ un réseau, où X est l'ensemble des sommets, E l'ensemble des arêtes et $c : E \to \mathbb{R}^+$ la fonction de capacité.
Une *coupure ct* séparant s de t est un couple $(A, X \setminus A)$ tel que :

$$A \subset E, \quad s \in A, \quad t \notin A.$$

L'ensemble des arcs de la coupure, noté arcs(ct) est l'ensemble :

$$\text{arcs}(ct) = \omega^+(A) := \{u = (i, j) \in E \mid i \in A, j \in X \setminus A\}.$$

Qui correspond à l'ensemble des arcs qui, si on les enlève, coupe tout les chemins entre s et t.

Définition (**Capacité d'une coupure**) . La *capacité* d'une coupure ct est la somme des

capacités de ses arêtes :

$$\text{capa}(ct) := \sum_{u \in \text{arcs}(ct)} c(u).$$

Théorème (**Flot et coupure minimale**) . Pour tout flot compatible φ et pour toute coupure ct d'un réseau R, on a :

$$v(\varphi) \leqslant \text{capa}(ct)$$

1.4 Flot maximal : principe de Ford-Fulkerson

L'objectif de l'algorithme de Ford-Fulkerson est de construire de manière itérative un flot maximal. Pour cela, on part du flot nul dans le réseau. Et petit-à petit, on cherche des chemins non saturés de la source s vers le puit t pour lequels on peut augmenter leur flot. Une fois que l'on n'est plus capable de trouver de chemin, alors l'algorithme se termine. La définition ci-dessous détaille le processus de constructiond d'un tel chemin.

1.4.1 Recherche de Flot Maximal

Définition (**Principe de Ford-Fulkerson**) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R. L'algorithme de construction d'un chemin non saturé est :

1. **Initialisation :** On marque s
2. **Itération :** Pour tout sommet marqué x, $y \in X$ peut peut être marqué si :
   - y n'est pas encore marqué
   - $\begin{cases} \exists u = (x, y) \in E, \varphi(u) < c(u) & \text{(marquage direct)} \\ \exists u = (y, x) \in E, \varphi(u) < c(u) & \text{(marquage indirect)} \end{cases}$

L'algorithme se termine si on arrive à marquer le puit t. Dans ce cas, la chaîne produite est alors appelée *chaîne augmentante*, notée ch. On note ch^+ les sommets de ch marqués directement et ch^- les sommets marqués indirectement.

Propriété (**Calcul d'un nouveau flot**) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R. Soit ch une chaîne augmentante de R. Il est alors possible d'augmenter la valeur du flot φ de $k \in \mathbb{R}$ en ajoutant à chaque arcs le flot définit par :

$$\boxed{k = \min\left(\min_{u \in ch^+}\{c(u) - \varphi(u)\}, \min_{u \in ch^-}\{\varphi(u)\}\right)}$$

Le nouveau flot est obtenu en aumgmentant de k le flow des arsc de ch^+ et en diminuant le flot de k des arcs de ch^-.

Théorème (**Coupure Minimale - Flot Maximal**) . Soit $R = (X, E, c)$ un réseau. Soit F l'ensemble des flots possibles sur R et K l'ensemble des coupures possibles de R. Alors la valeur du flot maximal est égale à la capacité de la coupure minimale.

$$\boxed{\text{i.e} \quad \max_{\varphi \in F}\{v(\varphi)\} = \min_{ct \in K}\{\text{capa}(ct)\}}$$

Ce théorème nous permet donc de trouver dicrectement la valeur du flot maximal que l'on peut faire passer dans notre réseau sans avoir à en construire un.

Remarque La dernière itération de l'algorithme de Ford-Fulkerson nous donne une coupure minimale. Les sommets marqués correspondent aux sommets de l'ensemble A de la coupure $ct = (AX/A)$.

1.4.2 Recherche de flot maximal de coût minimal

Rajoutons maintetant une contrainte dans le recherche de notre flot maximal. On souhiate trouver un flot maximal dans un réseau pour lequel le coût du flot sera le plus faible. On cherche dont à faire passser le plus de marchandise dans notre réseau par les routes qui "coûtent le moins cher".
Nous devons donc définir un nouveau type de réseau intégrant cette notion de coût.

Définition (**Réseau à coût**) . Soit $R = (X, E, c)$ un réseau de noeuds E, d'arcs E et de fonction de capacité c. On appelle *réseau à coût* le réseau R auquel on a rajouté une fonction de coût γ, qui à chaque arrête lui associe un coût :

$$\gamma : \begin{cases} E \longrightarrow \mathbb{R} \\ (x, y) \longmapsto \gamma(x, y) \end{cases}$$

Exemple On représentera donc un réseau à coût comme un réseau normal mais en rajoutant une valeur numérique sur chaque arrête correspondant à son coût :

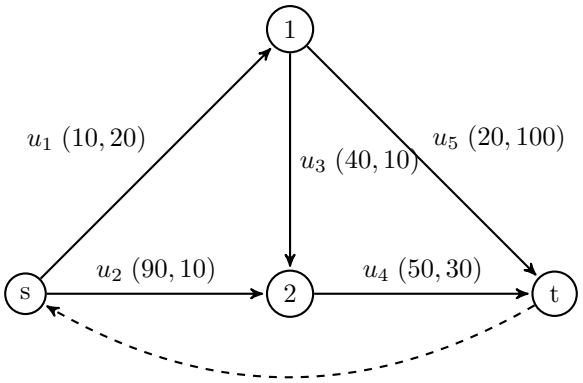


FIGURE 1.5 – Réseau de transport à coût

Remarque Chercher un flot maximal de coût minimal dans R revient donc à construire un flot via l'algorithme de Ford-Fulkerson en choisissant une chaîne augmentante de coût minimal à chaque itération.

Pour trouver cette chaîne augmentante de coût minimal, nous devons définir le graphe résiduel d'un réseau.

Définition (**Graphe Résiduel**) . Soit $R = (X, E, c, \gamma)$ un réseau et φ un flot sur R, on définit le *graphe résiduel* $G_R(\varphi) = (X, U_\varphi, r_\varphi)$ associé à R et φ comme le graphe tel que pour tout arc $(x, y) \in E$, on ait :

- un *arc direct* $(x, y) \in U_\varphi$ si $\varphi(x, y) < c(x, y)$ avec $r_\varphi = c(x, y) - \varphi(x, y)$
- un *arc indirect* $(y, x) \in U_\varphi$ si $\varphi(y, x) > 0$ avec $r_\varphi = \varphi(y, x)$.

Chapitre 2

Programmation Linéaire

Contents

2.1 Programmation Linéaire 79
2.1.1 Généralités 79
2.1.2 Solution Géométrique 82
2.1.3 Algorithme du Simplexe 83
2.1.4 Dualité 86
2.2 Programmation Linéaire en Nombres Entiers 88
2.2.1 Définition et relaxation 88
2.2.2 Algorithme "Branch and Bound" 90

Intéressons nous maintenant aux problèmes de programmation linéaire. Même si le nom peut prêter à confusion, il s'agit en réalité de problèmes d'optimisation d'une *fonction objectif* en respectant certaines *contraintes*. Le caractère linéaire de tels problèmes vient du fait que nous ne traiterons que des fonctions objectifs et des contraintes linéaires.
Ces problèmes de programmation linéaires peuvent être résolus en temps polynomial, notament grâce à l'algorithme du simple. Les problème de flots maximaux, vus précédement, peuvent s'exprimer comme des problèmes de programmation linéaire.

2.1 Programmation Linéaire

2.1.1 Généralités

Définition (**Programme linéaire**) . Un *programme linéaire* ou *problème de programmation linéaire* est un problème d'optimisation de $n \in \mathbb{N}$ variables et $m \in \mathbb{N}$ contraintes de la forme :

$$\begin{aligned} \text{minimize} \quad & z = \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j \preceq b_i \quad \forall i \in [\![1, m]\!], \\ & x \succeq 0 \quad \forall j \in [\![1, n]\!] \end{aligned}$$

où $\preceq \in \{\leqslant, \geqslant, =\}$. Un programme linéaire est donc un simple problème d'opmisation sur un polyèdre convexe. Dans le cas définit plus haut, on dit que le programme est sous forme générale. Différentes formes d'un même problème existent.

Exemple Voyons dès à présent un exemple de programme linéaire.

Un barman possède 30 litres de jus d'orange, 24 litres de pamplemousse et 18 litres de jus de framboise. Il peut préparer 2 types de cocktails :

- *Le "sunrise", vendu 80 euros le baril de 10 litres composé de 5 litres de de jus d'orange, de 2 litres de jus de pamplemouse, d'un litre de jus de framboise et de 2 litres d'eau.*
- *Le "swing", vendu 60 euros le baril et composé de 3 litres de jus de pamplemouse, 3 litres de jus d'orange, 3 litres de jus de framboise et d'un libre d'eau.*

Il souhaite préparer la plus importante quantité des deux cocktails pour maximiser son profit.

Pour résoudre ce problème, nous pouvons construire un programme linéaire de deux variables x et y représentant chacunes le nombre de litres d'un des deux cocktails à produire. La fonction objectif, décrivant le prix de revente de la marchandise est donc :

$$z = 80x + 60y$$

Et nous avons les contraintes suivantes :

$$\begin{aligned} 5x + 3y &\leqslant 30 \quad \text{(stock de jus d'orange)} \\ 2x + 3y &\leqslant 24 \quad \text{(stock de jus de pamplemouse)} \\ x + 3y &\leqslant 18 \quad \text{(stock de jus de framboise)} \\ x, z &\geqslant 0 \end{aligned}$$

On peut donc formuler le problème suivant :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leqslant 30, \\ & 2x + 3y \leqslant 24, \\ & x + 3y \leqslant 18, \\ & x, y \geqslant 0 \end{aligned}$$

Proposition Comme dit dans la définition d'un programme linéaire, il peut exister différentes formes d'un même problème en fonction de la méthode de résolution utilisée. Le caractère linéaire de ces programmes nous permet de passer facilement d'une forme à une autre. Nous définissons la forme canonique d'un programme linéaire comme suit :

$$\begin{aligned} \text{maximize} \quad & z = \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j = b_i, \quad \forall i \in [\![1, m]\!], \\ & x_j \succeq 0, \quad \forall j \in [\![1, n]\!] \end{aligned}$$

FIGURE 2.2 – Formulation standard d'un programme linéaire de maximisation

Nous pouvons ensuite passer à la formulation dite « standard » où les contraintes d'ordre sont remplacées par des contraintes d'égalités. Il faut donc rajouter un certain nombre de variables à notre programme.

$$\begin{aligned} \text{maximize} \quad & z = \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j \preceq b_i, \quad \forall i \in [\![1, m]\!], \\ & x_j \succeq 0, \quad \forall j \in [\![1, n]\!] \end{aligned}$$

FIGURE 2.1 – Formulation canonique d'un programme linéaire

$$\begin{aligned} \text{maximize} \quad & z = CX \\ \text{subject to} \quad & AX = B, \\ & X \geqslant 0_{\mathbb{R}^n} \end{aligned}$$

FIGURE 2.3 – Formulation matricielle d'un programme linéaire de maximisation

Enfin, nous avons la forme matricielle, qui nous permettra d'utiliser des algorithmes itératifs pour résoudre ce genre de problèmes :
où $C \in \mathbb{R}^n, A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathbb{R}^m$.

Exemple Pour le problème de notre barman, nous pouvons ainsi représenter les différentes formes de notre problème :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leqslant 30, \\ & 2x + 3y \leqslant 24, \\ & x + 3y \leqslant 18, \\ & x, y \geqslant 0 \end{aligned}$$

FIGURE 2.4 – Forme canonique du problème du barman

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y + e_1 = 30, \\ & 2x + 3y + e_2 = 24, \\ & x + 3y + e_3 = 18, \\ & x, y, e_1, e_2, e_3 \geqslant 0 \end{aligned}$$

FIGURE 2.5 – Forme standard du problème du barman

Et enfin :

$$\begin{aligned} \text{maximize} \quad & z = \underbrace{(80, 60, 0, 0, 0)}_{C} \times \overbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}^{X} \\ \text{subject to} \quad & \underbrace{\begin{pmatrix} 5 & 3 & 1 & 0 & 0 \\ 2 & 3 & 0 & 1 & 0 \\ 1 & 3 & 0 & 0 & 1 \end{pmatrix}}_{A} \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_{X} = \underbrace{\begin{pmatrix} 30 \\ 24 \\ 18 \end{pmatrix}}_{B} \end{aligned}$$

Maintenant que nous avons défini formellement un programme linéaire, passons à l'étude de ses solutions.

Définition (**Solutions d'un programme linéaire**) . Soit P un programme linéaire sous forme matricielle standard. Le vecteur $X = (x_1, \ldots, x_n)$ est :

- *une solution de* P si $A\,{}^tX = B$
- *une solution possible de* P si $A\,{}^tX = B$ et $X \geqslant 0_{\mathbb{R}^n}$
- *une solution optimale de* P si c'est une solution possible et qu'elle maximise z parmi l'ensemble des solutions possibles.

Proposition Comme énoncé en introduction, on peut représenter un problème de maximisation de flot dans un réseau par un programme linéaire. Soit $R = (X, E, c)$ un réseau tel que $|E| = n$, avec $s, t \in X$ la source et le puits. On note $c(x, y)$ la capacité de l'arête entre x et y dans le réseau. On cherche une application

$$\varphi : E \longrightarrow \mathbb{R}, \quad (x, y) \longmapsto f(x, y)$$

qui maximise le flot entre la source s et le puits t.
Plus formellement, si l'on ordonne les arêtes E sous la forme $E = \{e_1, \ldots, e_n\}$ et que l'on note $\varphi(E) = (f_1, \ldots, f_n) \in \mathbb{R}^n$, alors on cherche à maximiser

$$z = \sum_{(x,y)\in v^-(s)} \varphi(x, y)$$

sous les contraintes :

$$0 \leqslant f(x, y) \leqslant c(x, y), \quad \forall (x, y) \in E$$

ainsi que les contraintes de conservation du flot pour tout sommet $v \in X \setminus \{s, t\}$:

$$\sum_{(u,v)\in v^-(v)} f(u, v) = \sum_{(v,w)\in v^+(v)} f(v, w).$$

2.1.2 Solution Géométrique

La recherche d'une solution d'un programme linéaire revient à trouver une droite de la forme $z = \sum_{i=1}^{n} x_i c_i$ pour laquelle z est maximal tout en respectant les contraintes du programme. En petite dimension (2 variables ou moins) il est possible de tracer ce problème et de le résoudre géométriquement. Détaillons cela autour d'un exemple.

Exemple (Solution Géométrique) Reprenons le problème de notre barman énoncé comme suit :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leqslant 30, \\ & 2x + 3y \leqslant 24, \\ & x + 3y \leqslant 18, \\ & x, y \geqslant 0 \end{aligned}$$

On peut tracer les courbes des contraintes pour identifier la zone contenant les solutions possibles :

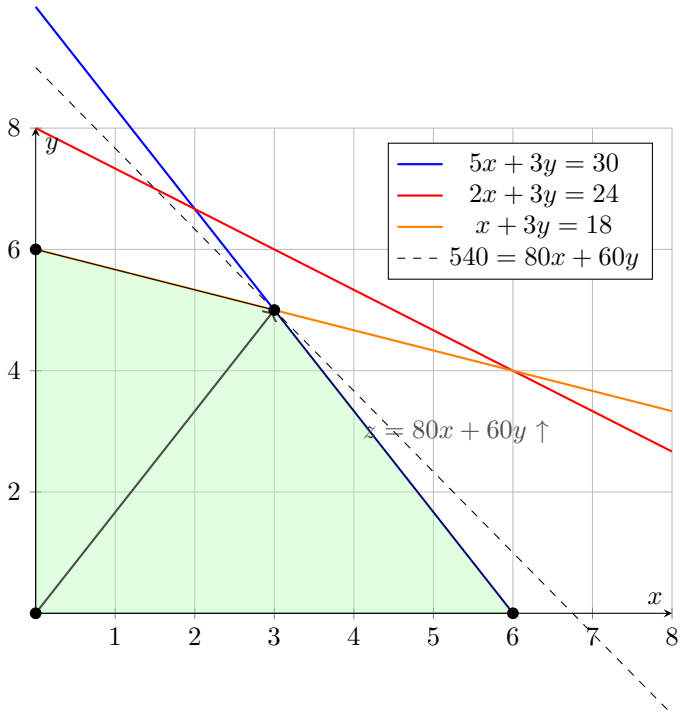


FIGURE 2.6 – Espace des solutions réalisables du programme linéaire
Le barman peut donc produire 80 litres de "sunrise" et 3 litres de "swing" pour un profit total de 540 euros.

Remarque En pratique, il sera très rare de pouvoir résoudre un tel problème graphiquement car le nombre de variable sera bien supérieur à 2.

2.1.3 Algorithme du Simplexe

Comme dit précédement, le caractère linéaire des contraintes d'un programme linéaire nous permettent d'en déduire que l'espace des solutions possible est un polyèdre. D'où le théorème suivant.

Théorème (**Solutions Possibles**) . L'ensemble S des solutions possibels d'un programme linéaire est convexe et est un polyèdre non borné ou un polytope. Si S est non vide et borné, alors il existe au moins un vecteur de S pour lequel le maximum de la fonction objectif est atteint.

Remarque Ce théorème nous permet de distinguer plusieurs cas pour l'ensemble des solutions :

- Le programme linéaire a au moins une solution :
  - soit elle est unique
  - soit il en existe une infinité (i.e le maximum est atteint en plusieurs vecteurs) on parle alors de cas dégénéré.
- Le programme linéaire n'a pas de solution, lorsque les contraintes ne sont pas compatibles, l'ensemble des solutions est donc vide.
- Le programme linéaire n'a pas d'optimum (i.e l'ensemble des solutions n'est pas borné).

Proposition (Résolution d'un programme linéaire par le simplexe) Nous avons précédemment résolu nos programmes linéaires graphiquement. Nous allons voir que nous pouvons

aussi les résoudre *algébriquement*, en utilisant l'*algorithme du simplexe*. Soit le programme suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leqslant 300, \\ & 5x \leqslant 310, \\ & 2x + 4y \leqslant 200, \\ & 3y \leqslant 100, \\ & x, y \geqslant 0 \end{aligned}$$

On passe ce programme sous *forme standard* en introduisant des variables d'écart $e_1, e_2, e_3, e_4 \geqslant 0$:

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y + e_1 = 300, \\ & 5x + e_2 = 310, \\ & 2x + 4y + e_3 = 200, \\ & 3y + e_4 = 100, \\ & x, y, e_1, e_2, e_3, e_4 \geqslant 0 \end{aligned}$$

Le vecteur $(x, y, e_1, e_2, e_3, e_4) = (0, 0, 300, 310, 200, 100)$ est une *solution de base réalisable* : toutes les contraintes sont vérifiées et les variables sont positives. L'idée du simplexe est de :

1. Choisir une *base initiale* (ici, les variables d'écart e_i).
2. Exprimer les variables hors base (x, y) en fonction des variables de base.
3. *Augmenter la valeur de la fonction objectif* z en remplaçant une variable de base par une variable hors base qui améliore z.
4. Répéter jusqu'à ce qu'aucune amélioration ne soit possible : la solution est alors optimale.

Formellement, le simplexe cherche à écrire le système sous la forme :

$$\begin{cases} z = C + \sum_i c_i x_i \\ x_B = b - A x_N \end{cases} \qquad \text{où } x_B \geqslant 0,\ x_N \geqslant 0$$

À chaque itération :

- une variable hors base entre dans la base (celle qui augmente le plus z),
- une variable de base quitte la base (celle qui devient nulle en premier).

Lorsque tous les coefficients $c_i \leqslant 0$, aucune amélioration n'est possible et la solution actuelle est *optimale* : la valeur optimale est alors $z = C$.

Exemple (Résolution algébrique du programme par le simplexe) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leqslant 300, \\ & 5x \leqslant 310, \\ & 2x + 4y \leqslant 200, \\ & 3y \leqslant 100, \\ & x, y \geqslant 0 \end{aligned}$$

1. **Mise sous forme standard :** On introduit une variable d'écart pour chaque contrainte :

$$\begin{cases} 4x + 2y + e_1 = 300 \\ 5x + e_2 = 310 \\ 2x + 4y + e_3 = 200 \\ 3y + e_4 = 100 \end{cases} \quad \text{avec } e_i \geqslant 0.$$

La fonction objectif s'écrit :

$$z = x + 2y.$$

Une solution de base réalisable évidente est obtenue en prenant $x = y = 0$, ce qui donne :

$$(e_1, e_2, e_3, e_4) = (300, 310, 200, 100), \quad z = 0.$$

C'est notre solution de base initiale.

2. **Recherche d'une direction d'amélioration :** Comme $z = x + 2y$, augmenter y est plus rentable que x (le coefficient 2 est supérieur à 1). On va donc chercher à **augmenter** y tant que c'est possible sans violer les contraintes. On part du système :

$$\begin{cases} e_1 = 300 - 2y - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4y \\ e_4 = 100 - 3y \end{cases}$$

Comme on garde $x = 0$ au départ, ces équations deviennent :

$$\begin{cases} e_1 = 300 - 2y \\ e_2 = 310 \\ e_3 = 200 - 4y \\ e_4 = 100 - 3y \end{cases}$$

Pour rester faisable, il faut $e_i \geq 0$. On obtient donc :

$$y \leq 150, \quad y \leq 50, \quad y \leq 25, \quad y \leq 33{,}3.$$

La contrainte la plus restrictive est $y \leq 25$ (provenant de $e_3 = 200 - 4y$). On choisit donc $y = 25$ et $e_3 = 0$. Cela définit une nouvelle base : $\{e_1, e_2, e_4, y\}$.

3. **Amélioration suivante :** Pour cette valeur $y = 25$, les variables d'écart valent :

$$e_1 = 300 - 2(25) = 250, \quad e_2 = 310, \quad e_3 = 0, \quad e_4 = 100 - 3(25) = 25.$$

La fonction objectif vaut alors :

$$z = x + 2y = 2 \times 25 = 50.$$

On peut encore améliorer z en augmentant x, tant que toutes les contraintes restent valides. En remplaçant $y = 25$, on obtient :

$$\begin{cases} e_1 = 300 - 2(25) - 4x = 250 - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4(25) = 100 - 2x \\ e_4 = 100 - 3(25) = 25 \end{cases}$$

Les contraintes donnent :

$$x \leq 62{,}5, \quad x \leq 62, \quad x \leq 50.$$

Le plus restrictif est $x \leq 50$ (provenant de $e_3 \geq 0$). On prend donc $x = 50$ et $e_3 = 0$.

4. **Calcul de la solution optimale :** On obtient :

$$\begin{cases} x = 50, \quad y = 25, \\ e_1 = 300 - 4(50) - 2(25) = 100, \\ e_2 = 310 - 5(50) = 60, \\ e_3 = 0, \\ e_4 = 100 - 3(25) = 25. \end{cases}$$

La fonction objectif vaut :

$$z = x + 2y = 50 + 2 \times 25 = 100.$$

Si on essaye d'augmenter y ou x davantage, on viole immédiatement une contrainte. La solution est donc **optimale**.

5. **Conclusion :** Le programme atteint son maximum en :

$$\boxed{x^* = 50, \quad y^* = 25, \quad z_{\max} = 100.}$$

Cette résolution illustre la logique du simplexe : on part d'une solution réalisable triviale, puis on améliore la fonction objectif pas à pas en déplaçant la base vers les sommets adjacents du polytope faisable, jusqu'à atteindre un sommet où toute augmentation violerait une contrainte.

2.1.4 Dualité

Définition (**Programme Primal**) . Le *problème primal* (ou primaire) est la forme standard d'un programme linéaire :

$$\begin{aligned} \text{maximize} \quad & z = {}^t cx \\ \text{subject to} \quad & Ax \leqslant b, \\ & x \geqslant 0 \end{aligned}$$

où :

- $x \in \mathbb{R}^n$ est le vecteur des variables de décision ;
- $c \in \mathbb{R}^n$ est le vecteur des coefficients de la fonction objectif ;
- $A \in \mathbb{R}^{m \times n}$ est la matrice des coefficients des contraintes ;
- $b \in \mathbb{R}^m$ est le vecteur des bornes du système d'inégalités.

Définition (**Problème Dual**) . À tout problème primal est associé un *problème dual* défini par :

$$\begin{aligned} \text{minimize} \quad & w = {}^t by \\ \text{subject to} \quad & {}^t Ay \geqslant c, \\ & y \geqslant 0 \end{aligned}$$

où :

- $y \in \mathbb{R}^m$ est le vecteur des variables duales ;

- chaque contrainte du primal correspond à une variable du dual ;
- chaque variable du primal correspond à une contrainte du dual.

Remarque (Interprétation) Le problème *primal* cherche à *maximiser un profit* ou une valeur objective en respectant des contraintes de ressources. Le problème *dual* cherche à *minimiser le coût* de ces ressources tout en garantissant que la valeur de chaque ressource couvre la contribution de chaque variable du primal. Autrement dit :

> *Le primal cherche ce qu'on peut obtenir au maximum avec des ressources limitées, tandis que le dual cherche le coût minimal que doivent avoir ces ressources pour que le système reste équilibré.*

Exemple En reprenant l'exemple de notre barman, le programme primal cherche à maximiser le profit du barman en fixant le prix des barrils c_j de cocktails j qu'il produit en utilisant au plus b_i litres de jus i. Les variables x_j représentent alors le nombre de barrils de cocktail j.
Ici, le programme dual cherche à minimiser le profit total obtenu grâce aux stocks b_i de chaque jus i si le barman vent chaque cocktail j pour un prix minimal de c_j. Les variables y_i représentent donc le profit que le barman réalise par litre de jus i.
Voici les expressions mathématiques de ces deux programmes :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leqslant 30, \\ & 2x + 3y \leqslant 24, \\ & x + 3y \leqslant 18, \\ & x, y \geqslant 0 \end{aligned}$$

FIGURE 2.7 – Programme primal du barman

$$\begin{aligned} \text{minimize} \quad & w = 20y_1 + 24y_2 + 18y_3 \\ \text{subject to} \quad & 5y_1 + 2y_2 + y_2 \geqslant 80, \\ & 3y_1 + 3y_2 + 3y_3 \geqslant 60, \\ & y_1, y_2, y_3 \geqslant 0 \end{aligned}$$

FIGURE 2.8 – Programme dual du barman

La solution optimale du primal est $(x_1, x_2) = (3, 5)$ pour un profit maximal de 540. En revanche la solution du dual est $(y_1, y_2, y_3) = (15, 0, 5)$ pour un profit de 540 aussi.

Il existe un lien direct entres les solutions du dual et celles du primal. Elles sont décrites par les théorèmes de dualité.

Théorème (**Dualité Faible**) . La solution optimale du programme primal est inférieure ou égale à celle du dual.

Théorème (**Dualité Forte**) . Nous avons les deux implications suivantes :

1. Si l'un des deux programmes (primal ou dual) admet une solution optimale finie, alors l'autre aussi. De plus, la valeur optimale pour des deux programmes est la même.
2. Si l'ensemble des solutions de l'un des deux programmes est non borné , alors celui de l'autre est vide.

Remarque Le théorème de coupure minimale/flot maximal dans un réseau est l'équivalent du théorème de dualité forte ici. En effet, le problème de coupure minimale dans un réseau est l'exact dual du problème de flot maximal dans le même réseau.

Exemple (Utilité de la dualité) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & x + 3x + 6y + z \\ \text{subject to} \quad & w + x + 5y + 6z \leqslant 20, \\ & w + 2x + y + 3z \leqslant 30, \\ & x, y, w, z \geqslant 0 \end{aligned}$$

Ce programme ne peut pas être résolu graphiquement du fait du trop grand nombre de variables. En passant au dual, nous aurons un programme linéaire à deux variables et 4 contraintes que l'on pourra résoudre graphiquement. Attention, la résolution du dual nous donnera seulement une idée de la valeur de la solution maximale.

Remarque Ainsi, passer au problème dual peut être avantageux dans plusieurs cas :

- Le dual peut être plus simple à résoudre (moins de variables ou moins de contraintes).
- Il peut exister une solution optimale évidente pour le dual mais pas pour le primal.
- La valeur de n'importe quelle solution du dual nous donne une borne supérieure pour pour la solution du primal.
- La solution optimale du dual nous donne des informations sur la solution optimale du primal.

2.2 Programmation Linéaire en Nombres Entiers

Dans cette section, nous allons traiter un nouveau type de problèmes : les *programes linéaires en nombres entiers*. Vous l'aurez compris, il s'agit maintenant de considérer des programmes linéaires à variables entières. Nous allons donc une méthode pour les résoudre ainsi que quelques techniques de modélisation. Commençons tout d'abord par un exemple introductif.

Exemple Un commerçant souhaite remplir son sac à dos d'au plus n objets de poids et valeurs respectifs $p_i \in \mathbb{N}$ et $v_i \in \mathbb{N}$. Or le sac du commerçant ne peut contenir qu'un poids maximal $C \in \mathbb{N}$. Le commerçant veut choisir les objets à prendre de manière à maximiser la valeur emportée. On modélisera donc le problème de la façon suivante :

$$\begin{aligned} \text{maximize} \quad & V = \sum_{i=1}^{n} v_i \times x_i \\ \text{subject to} \quad & \sum_{i=0}^{n} p_i x_i \leqslant C, \\ & \forall i \in [\![1, n]\!],\ x_i \in \mathbb{N} \end{aligned}$$

Ici le vecteur $(x_1, \ldots, x_n) \in \{0, 1\}^n$ représentera les objets que l'on prend. En effet, $x_i = 0$ si le commerçant ne prend pas l'objet i et 1 sinon.
Nous avons donc affaire à un programme linéaire. Or sa résolution pourrait non conduire à une solution comportant des valeurs $x_i \notin \{0, 1\}^n$ alors que l'on considère que l'on ne peut pas prendre une partie d'un objet (ex : 3/4 de l'object i). Nous devons donc chercher une nouvelle méthode pour résoudre ce genre de programmes.

2.2.1 Définition et relaxation

Définition (**Programme linéaire en nombres entiers**) . Un *programme linéaire en nombres entiers* est un programme linéaire comme définit précédement où un certain nombre de variables sont à valeurs entières. Plus formellement, la forme strandard d'un tel programme est de la forme :

$$\begin{aligned} \text{maximize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j = b_i \quad \forall i \in [\![1, m]\!], \\ & x_j \geqslant 0 \quad \forall j \in [\![1, n]\!], \\ & x_j \in \mathbb{N} \quad \forall j \in [\![1, n]\!] \end{aligned}$$

où les constantes c_j, a_{ij} et b_i sont entières.

Définition (**Relaxation linéaire**) . Soit un programme linéaire en nombres entiers (PLNE) :

$$\begin{aligned} \text{maximize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j = b_i \quad \forall i \in [\![1, m]\!], \\ & x_j \in \mathbb{N} \quad \forall j \in [\![1, n]\!] \end{aligned}$$

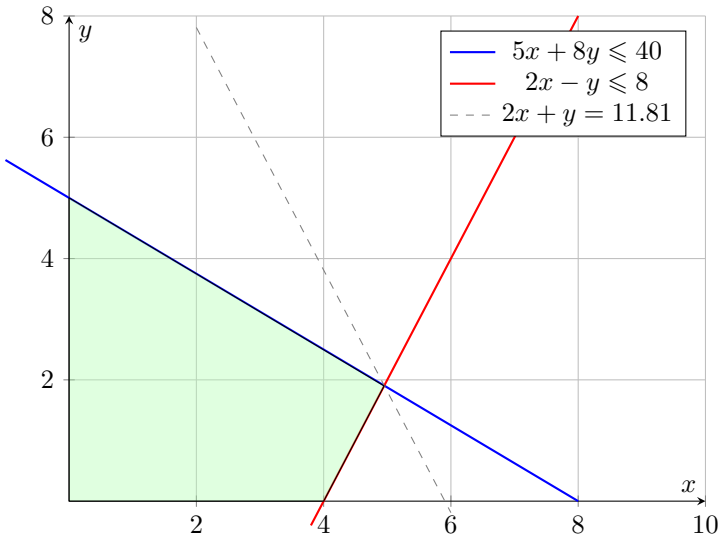
La *relaxation linéaire* de ce programme consiste à supprimer les contraintes de la forme :

$$x_j \in \mathbb{N} \quad \forall j \in [\![1, n]\!]$$

On obtient ainsi un programme linéaire classique, plus facile à résoudre.

Exemple Considérons le programme linéaire suivant que nous allons résoudre graphiquement :

$$\begin{aligned} \text{maximize} \quad & z = 2x + 3y \\ \text{subject to} \quad & 5x + 8y \leqslant 40, \\ & 2x - y \leqslant 8, \\ & x, y \geqslant 0 \end{aligned}$$



Graphiquement, nous trouvons une solution optimale de valeur 11.81.

2.2.2 Algorithme "Branch and Bound"

Théorème (**Lien entre PLNE et relaxation**) . Soit un programme linéaire en nombres entiers (PLNE) et sa relaxation linéaire correspondante. Notons z^*_{int} la valeur optimale du PLNE et z^*_{relax} celle de la relaxation. Alors :

$$z^*_{\text{relax}} \geqslant z^*_{\text{int}} \quad \text{(en cas de maximisation)}, \qquad z^*_{\text{relax}} \leqslant z^*_{\text{int}} \quad \text{(en cas de minimisation)}.$$

Autrement dit, la solution optimale de la relaxation fournit une *borne supérieure* (ou *borne inférieure*) pour le PLNE initial.

La méthode *Branch and Bound* est une technique de résolution des programmes linéaires en nombres entiers (PLNE) qui repose sur l'utilisation des relaxations linéaires pour obtenir des bornes sur la solution optimale. Cet algorithme construit un arbre de recherche binaire dont la racine est le problème initial Π. L'algorithme s'arrête lorsque tous les nœuds feuilles de l'arbre sont *fathomés* (c'est-à-dire qu'ils n'ont plus besoin d'être explorés).

Définition (**Nœud fathomé**) . Un nœud est dit *fathomé* si au moins une des conditions suivantes est vérifiée :

1. La relaxation linéaire est *infaisable*, c'est-à-dire qu'aucune solution réelle ne satisfait les contraintes.
2. La valeur optimale de la relaxation linéaire est inférieure ou égale à la meilleure solution entière connue.
3. La solution optimale de la relaxation linéaire est entière, et est donc également solution optimale du programme entier.

Algorithme 1 : Branch and Bound pour un programme linéaire en nombres entiers

Résoudre la relaxation linéaire du problème Π
if *le nœud est fathomé (élagué)* **then**
    Arrêter l'exploration de ce nœud
else
    Identifier une variable x_i fractionnaire dans la solution x^*
    Créer deux sous-problèmes en ajoutant respectivement les contraintes :
        $x_i \leqslant \lfloor x_i^* \rfloor \quad \text{et} \quad x_i \geqslant \lceil x_i^* \rceil$
    Appeler **Branch and Bound** récursivement sur ces sous-problèmes

Remarque L'algorithme explore donc l'arbre de recherche en combinant deux techniques :

- Le *branching*, qui divise le problème en sous-problèmes plus simples.
- Le *bounding*, qui utilise la relaxation linéaire pour élaguer les sous-problèmes ne pouvant pas améliorer la meilleure solution entière connue.

Chapitre 3

Méta-heuristiques, CSP et SAT

Chapitre 4

Théorie de la complexité

Contents

4.1 Généralités 93
4.2 Problèmes de décisions, classes P, NP et NP-complet 94
4.2.1 Complexité temporelle 94
4.2.2 Classe P 94
4.2.3 Vérifieurs et classe NP 95

Dans ce chapitre, nous nous intéressons aux problèmes d'optimisation combinatoire "difficiles" pour lesquels nous n'avons pas de méthode explicite pour déterminer un solution. Nous essayerons de "trier" ces problèmes en classes de complexité et nous verrons que certains problèmes sont étrangement similaires les uns des autres.

4.1 Généralités

Un problème d'optimisation combinatoire est définit par plusieurs éléments. On définit ainsi $\mathcal{X}$ un ensemble de *variables, d"éléments* ou *d'objets*. Il représente ainsi l'ensemble des paramètres. La résolution de ce problème vise à trouver un *groupement* ou des affectations de $\mathcal{X}$ dont on $\mathcal{E}$ l'ensemble des solutions possibles.
Notre problème peut aussi être définit par un certain nombre de *contraintes* d'où l'existence d'un ensemble $\mathcal{S}$ regroupant toutes les solutions *faisables* (i.e respectant les contraintes).

Remarque On a donc en général $\mathcal{S} \subset \mathcal{E}$. $\mathcal{E}$, énumérer $\mathcal{S}$ peut être impossible d'où la difficulté du problème.

Enfin, nous pouvons aussi définir une fonction de coût $v : \mathcal{E} \longrightarrow \mathbb{R}$ définissant une valeurs ou un coût pour chaque affectation de $\mathcal{X}$ permettant ainsi d'ordonner les solutions et d'en définir un *optimale*.

Définition (**Instance**) . Un *instance I* d'un problème d'optimisation combinatoire est donc un cas particulier de ces problèmes dans un contexte donnée. Elle est caractérisée par un ensemble d'objets/de variables $\mathcal{X}$ et des paramètres.

Définition (**Problème d'optimisation combinatoire**) . Ainsi, un *problème d'optimisation combinatoire* noté Π est définit par :

- un ensemble d'instances $\mathcal{I}$.

- pour chaque $I \in \mathcal{I}$, un ensemble de solutions faisables $\mathcal{S}(I)$.
- une fonction de coût/valeur.

Remarque La différence fondamentale entre une instance et un problème est qu'une instance s'applique dans un contexte donné et a une (ou plusieurs) solution(s) bien spécifique(s). Un problème est plutôt la représentation théorique de ce genre de problèmes.

Pour un problème donné Π il est intéressant de connaître sa classe de complexité pour savoir comment le résoudre et si c'est possible.
Les classes de complexité nous permettront donc de réunir ensemble des problèmes d'optimisation combinatoire ayant la même complexité temporelle (dans le pire des cas).

4.2 Problèmes de décisions, classes P, NP et NP-complet

4.2.1 Complexité temporelle

Définition (**Problème de décision**) . Un *problème de décision* est un problème dans lequel on souhaite répondre par oui ou par non pour une instance d'un problème donné.

On dit qu'une instance d'un problème est *positive* si la réponse au problème de décision est "oui".

Exemple Le problème les cliques dans un graphe est un problème de décision. En effet, on souhaite répondre par oui ou non à la question "Un graphe G contient-il un sous-graphe complet d'ordre $k \in \mathbb{N}$?

Pour différencier la difficulté de tels problèmes, nous nous intéressons à leur *complexité temporelle*.

Définition (**Complexité temporelle**) . Pour un algorithme donné A, on définit sa *complexité temporelle* notée $Time_A$ où $Time_A(x)$ est le nombre d'instructions exécutées par A pour une entrée x.

Proposition On souhaiterai ainsi définir la complexité d'un problème Π par la complexité temporelle de l'algorithme le plus efficace pour résoudre Π. Or, il est impossible de dire qu'un algorithme est optimal et ne peut être amélioré. On parlera donc de complexité *asymptotique*.

Définition (**Ordre de complexité**) . Soit $f : \mathbb{R} \longrightarrow \mathbb{R}$ une application. Soit Π un problème. On dit que la complexité temporelle de Π est d'ordre $O(f)$ ssi il existe un algorithme A résulvant Π et $n_0 \in \mathbb{N}, c \in \mathbb{R}$ tels que :

$$\forall x, \quad |x| > n_0 \quad \Longrightarrow \quad Time_A(x) \leqslant cf(|x|) \tag{4.2.1}$$

4.2.2 Classe P

Comme dit précédement, nous souhaitons regrouper des problèmes de même ordre de complexité temporelle. Nous définissons donc le concept de classe.

Définition (**Classes de complexité**) . La classe de complexité $TIME[f]$ est l'ensemble des problèmes de complexité temporelle $O(f)$. On a donc :

$$TIME[n] \subset TIME[n \log n] \subset TIME[n^2] \subset TIME[e^n] \subset TIME[2^n]$$

Définition (**Classe P**) . On définit ainsi la classe P comme l'ensemble des classes de problèmes de complexité polynomiale. D'où :

$$\boxed{P = \bigcup_{k \geqslant 0} TIME[n^k]} \tag{4.2.2}$$

La classe P est indépendante du modèle d'ordinateur considéré (machine de Turing ou à RAM). Elle est aussi indépendante du langage utilisé pour l'algorithme ainsi que des structures de données utilisées pour stocker les variables du problème.

Remarque Un problème est donc dit polynomial si il peut être résolut par un algorithme de complexité temporelle polynomiale. Or pour de très grandes valeurs de k, cela revient à résoudre un problème de complexité exponentielle. En pratique, nous pourrons donc résoudre des problèmes de complexité $O(n^2), O(n^3)$ ou $O(n^4)$.

Rappel (k-SAT) Soit $k \in \mathbb{N}$. Un problème de k-SAT est un problème de *satisfaisabilité booléenne* portant sur des formules booléennes en forme normale conjonctive (CNF).
Une formule est composée :

- de *variables booléennes* $x_1, \ldots, x_n$;
- de *littéraux*, qui sont soit une variable soit sa négation : $l \in \{x_i, \neg x_i\}$;
- de *clauses*, qui sont des disjonctions de k littéraux :

$$C = l_1 \vee l_2 \vee \cdots \vee l_k;$$

- et d'une *formule* CNF, qui est une conjonction de clauses :

$$\varphi = \bigwedge_{j=1}^{m} C_j.$$

Le problème de k-SAT consiste à déterminer s'il existe une assignation de vérité des variables qui rende la formule φ vraie. Si une telle assignation existe, on dit que la formule est *satisfaisable*.

Théorème (**SAT**) . 2-SAT $\in P$.

Définition (**Réduction polynomiale**) . On dit qu'il existe une *réduction polynomiale* entre deux problèmes Π_1 et Π_2 si il existe une fonction f calculable en temps polynomial si, x est une instance positive de Π_1 ssi $f(x)$ est une instance positive de Π_2. On note alors $\Pi_1 \leqslant \Pi_2$.

Théorème (**Réduction**) . 2-COLORATION $\leqslant$ 2-SAT.

4.2.3 Vérifieurs et classe NP

Apprentissage Automatique

Chapitre 1

Fondements, Réseaux Denses et Optimisation

Contents

1.1 Généralités 97
1.1.1 Un peu d'histoire... 97
1.1.2 Définition et terminologie 98
1.1.3 L'apprentissage 99
1.2 Du Perceptron au MLP 99
1.2.1 Concept 99
1.2.2 Apprentissage 100
1.2.3 Perceptron Multi-Couches (MLP) 100
1.3 Fonctions d'Activation et Classification Multiclasse 101
1.3.1 Fonctions d'activation usuelles 101
1.3.2 Classification multiclasse et fonction Softmax 102
1.4 Fonctions de Perte (*Loss Functions*) 102
1.4.1 Entropie croisée binaire (BCE) 102
1.5 Rétropropagation du Gradient et Dérivation Automatique 102
1.5.1 Rétropropagation sur un neurone formel 103
1.5.2 Rétropropagation dans un MLP (*Backpropagation*) 103
1.5.3 Graphes de calcul et implémentation PyTorch 103
1.5.4 Optimisation par descente de gradient avec inertie (Momentum) . . . 104

1.1 Généralités

1.1.1 Un peu d'histoire...

Nous sommes en été 1955 aux États-Unis. La naissance de l'informatique pousse les mathématiciens John McCarthy, Marvin Minsky, Nathaniel Rochester et l'ingénieur Claude Shannon à proposer un projet de recherche sur l'intelligence articielle au Dartmouth college "A proposal for the Dartmouth summer research project on artificial intelligence". Ils obtiennent ainsi 7500 $ de la fonction Rockfeller pour organiser un atelier de travail sur les machines pensantes durant deux mois. Cependant, l'intérêt pour ce domaine de recherche est encore très faible parmi la

communauté scientifique. Les faibles performances des premières machines constituent aussi un important frein au développement de prototypes.
En 1957, Frank Rosenblatt présente le premier réseau de neurones tel que nous les connaissons aujourd'hui appelés *Perceptron*. En 1986, la notion de Perceptron multicouches est explorée et en 1955 Yann LeCun et Yoshua Bengio développe le premier réseau de neurones à convolution à la base de la vision par ordinateur.
La victoire de DeepBlue sur Garry Kasparov aux échecs en 1997 démontre la puissance et le potentiel de ces nouvelles technologies. L'affluence de données disponibles ces 30 dernières années et l'évolution des capacités de calcul des ordinateurs a ouvert la porte à des systèmes de plus en plus performants. Enfin, 2015 marque l'apogée des modèles d'IA dits "classiques" avec la victoire d'AlphaGo sur les champions d'Europe et du monde.

1.1.2 Définition et terminologie

Selon Wikipédia, l'intelligence artificielle se définit comme suit :

> *"L'intelligence artificielle (IA) est l'ensemble des systèmes informatiques capables d'effectuer des tâches typiquement associées à l'intelligence, telles que l'apprentissage, le raisonnement, la résolution de problèmes, la perception ou la prise de décision."*

Le but de l'IA est donc de reproduire numériquement le mode de pensée humaine. Ces concepts ont étés, et sont toujours, à la base des réseaux de neurones actuels puisqu'ils sont contitués de neurones organisés en couches permettant de traiter l'information. C'est donc une discipline très vaste dans lequel il est facile de confondre les notions.

L'*apprentissage automatique* en est un sous-domaine qui consiste en la création d'algorithmes capables d'apprendre à partir de données précédement collectées. Il faut donc distinguer plusieurs phases dans le processus de développement d'un tel modèle :

1. La collecte des données d'entraînement.
2. L'entraînement du modèle.
3. La phase de test des performances du modèle.
4. La mise sur le marché pour une utilisation publique.

Nous traiterons ici les phases 1 et 2 de ce processus. Les autres appartiennent à des domaines différents de l'IA. Pour le développement d'un tel modèle, nous devons donc avoir accès à des **observations** consituées d'exemples ou de données collectés. Ces données sont consituées de deux éléments : les **attributs**, appelés *features*, consituent des réprésentations numériques des observations et les **étiquettes**, ou *labels*, qui associent à chaque élément une catégorie ou une valeur. L'enjeu est donc d'entraîner un modèle à distinguer les différents attributs pour chaque observation et d'être capable de répondre correctement à la question : "Étant donné une observation, à quelle catégorie appartient-elle ?". Pour cela, le jeu de données initial, appelée *dataset*, sera séparé en deux, l'un pour l'entraînement, et l'autre pour le test.

Plus formellement, pour un ensemble $E = \{(x_i, y_i) \mid i \in [\![1, N]\!]\}$ de données consitué d'observation X et de données Y, on cherche à apprendre une fonction f telle que :

$$f : \begin{cases} X \longrightarrow Y \\ x_i \longmapsto y_j = f(x_i) \end{cases}$$

et l'on souhaite que $i = j$ pour le plus de cas possibles. C'est-à-dire que notre modèle ne se trompe pas dans la prédiction y_i de la classe de x_i. En fonction de la nature de l'ensemble Y, on distinguera plusieurs types de problèmes. Si Y est un ensemble fini, on parlera de *problème de classification*, si $Y = \{vrai, faux\}$, on parlera de *prédiction* et si $Y = \mathbb{R}$, on parlera de régression.

1.1.3 L'apprentissage

L'apprentissage constitue le cœur de ce que nous allons voir ici. Cette phase permet de définir "les bonnes réponses" au modèle. On distingue ainsi différents types d'apprentissages :

- **L'apprentissage supervisé** : on dispose ici de données étiquetées et le modèle doit apprendre à distinguer les classes des observations.
- **L'apprentissage non supervisé** : ici le modèle n'a accès qu'aux observations et doit déterminer lui-même leur classes.
- **L'apprentissage par renforcement** : le modèle est ici très différent, il se représente sous la forme d'un agent devant apprendre une série d'actions en fonction de la configuration de son environnement. Pour cela, à chaque essai, on lui attribue une récompense positive ou négative.

Nous nous intéressons ici à l'apprentissage supervisé.

1.2 Du Perceptron au MLP

Le *Perceptron* peut être vu comme le premier réseau de neurones moderne, bien que sa conception date de 1956. À cette époque, Frank Rosenblatt pense à une "solution à tout" basée sur une représentation simplifiée du cerveau humain. L'idée est de recréer un ensemble de neurones connectés les uns aux autres capables d'"apprendre".

1.2.1 Concept

On se pose donc un problème à deux classes (c_-, c_+) avec m exemples :

$$\mathcal{D} = \{(x_i, y_i) \in \mathbb{R}^p \times \{-1, 1\} \mid j \in [\![1, m]\!]\}$$

Pour chaque exemple, x_i représente l'observation et y_i la classe correspondante. On veut développer un modèle capable de déterminer la classe d'une observation brute. On effectue donc une hypothèse forte qui est que les deux classes c_- et c_+ sont séparables par un hyperplan de $\mathbb{R}^p$. Le problème revient donc à construire une famille $(w_1, \dots, w_p)$ de *poids* qui définiront cet hyperplan. Formellement, on veut "apprendre" une fonction y telle que :

$$\forall i \in [\![1, m]\!], \quad y(x) = \begin{cases} 1 \text{ si } \sum_{j=1}^{p} w_j x_j > T \\ -1 \text{ sinon} \end{cases} .$$

On appellera cette fonction y, notée $\hat{y}$ la *prédiction*. Ici notre réseau de neurones a déjà pris forme. Il se représente encore plutôt sous la forme d'un neurone :

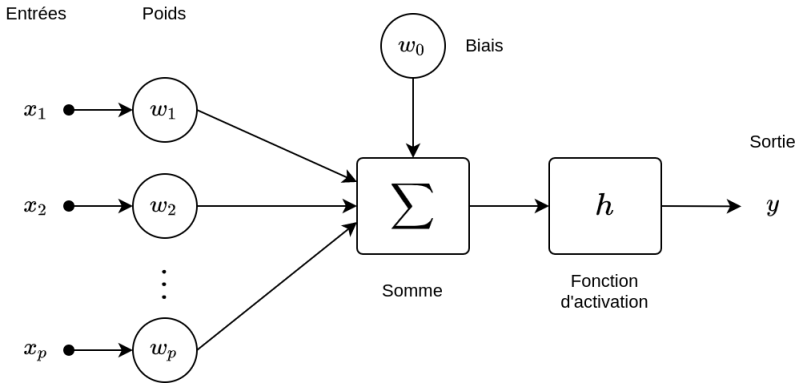


FIGURE 1.1 – Schéma formel d'un neurone

On utilisera aussi une *fonction d'activation* représentée par le seuil T. Plus tard, nous utiliserons des fonctions plus complexes pour donner une valeur de "probabilité" à la sortie d'un neurone.

1.2.2 Apprentissage

Une fois la structure posée, nous devons définir un algorithme pour apprendre les poids $(w_1, \dots, w_p)$ dans le but d'avoir une séparation optimale des deux classes dans $\mathbb{R}^p$. L'idée est donc de faire passer des valeurs x dans le perceptron et d'actualiser les poids w_j si la prédiction $\hat{y}$ est différente de la classe y. Une règle simple d'actualisation est de modifier w_j en fonction de la valeur x_j de l'entrée qu'il pondère. On aura donc la règle suivante :

$$w = \begin{cases} w + x \text{ si } y = 1 \text{ et } \hat{y} = -1 \\ w - x \text{ si } y = -1 \text{ et } \hat{y} = 1 \end{cases} \iff w = w + yx \text{ si } x \text{ est mal classé.}$$

Remarque (Variantes) Notre définition de l'actualisation des poids les modifie à chaque itération lors de l'apprentissage. Cela permet une convergence rapide de l'apprentissage, mais peut conduire à une instabilité si le nombre de poids à traiter est trop grand. Une solution est donc d'effectuer un *apprentissage par batch*. Cela consiste à faire passer successivement plusieurs vecteurs x, on calcule ensuite l'erreur moyenne commise et on met à jour les poids en fonction de cette erreur moyenne. Un autre possibilité est d'utiliser un *learning rate* ν qui bride l'apprentissage. On utilise alors la règle suivante pour actualier w :

$$w = \eta(y - \hat{y})x.$$

Les perceptrons permettent donc de simuler des fonctions booléennes comme illustré ci-dessous :

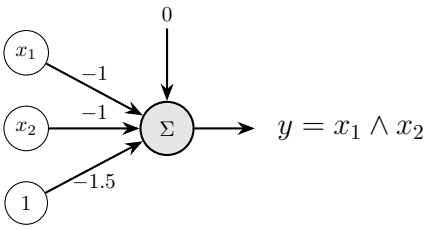


FIGURE 1.2 – Fonction AND avec un perceptron

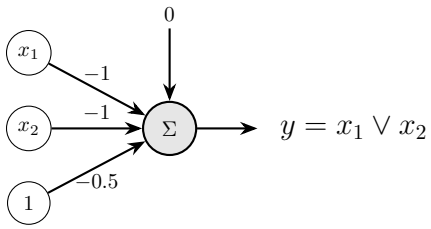


FIGURE 1.3 – Fonction OR avec un perceptron

Après réflexion, on se rend compte qu'il est impossible de simuler la fonction XOR avec un seul neurone. Ce sera donc l'objectif des *MLP (Multi Layer Perceptron)* composés de plusieurs couches de neurones interconnectés.

1.2.3 Perceptron Multi-Couches (MLP)

Comme énoncé précédemment, le perceptron simple de Rosenblatt présente une limitation fondamentale mise en lumière par Marvin Minsky et Seymour Papert en 1969 : l'impossibilité géométrique de séparer des classes qui ne sont pas linéairement séparables dans l'espace d'entrée $\mathbb{R}^d$, à l'instar de la fonction logique XOR (ou exclusif).
Pour contourner ce problème, l'approche naturelle consiste à empiler plusieurs couches de neurones formels. Un *Perceptron Multi-Couches* (Multi-Layer Perceptron, noté MLP) est composé :

1. D'une couche d'entrée (vecteur $x \in \mathbb{R}^d$) ;
2. D'une suite de $L \geqslant 1$ **couches cachées** (hidden layers), qui apprennent des représentations hiérarchiques distribuées ;
3. D'une couche de sortie produisant la prédiction $\hat{y}$.

Définition (**Formalisation de la passe avant d'un MLP**) . Soit un réseau à L couches cachées. Notons $h^{(0)} = x \in \mathbb{R}^{d_0}$ le vecteur d'entrée. Pour chaque couche $k \in [\![1, L]\!]$ de dimension d_k, le vecteur d'activation est régi par les relations de récurrence :

$$a^{(k)} = W^{(k)} h^{(k-1)} + b^{(k)} \in \mathbb{R}^{d_k} \qquad \text{(pré-activation)} \qquad (1.2.1)$$

$$h^{(k)} = g^{(k)}\left(a^{(k)}\right) \in \mathbb{R}^{d_k} \qquad \text{(activation)} \qquad (1.2.2)$$

où $W^{(k)} \in \mathbb{R}^{d_k \times d_{k-1}}$ désigne la matrice de poids de la couche k, $b^{(k)} \in \mathbb{R}^{d_k}$ le vecteur de biais associé, et $g^{(k)} : \mathbb{R} \to \mathbb{R}$ une fonction d'activation non linéaire appliquée composante par composante. Pour la couche de sortie $k = L + 1$, on applique une fonction terminale o :

$$\hat{y} = h^{(L+1)} = o\left(W^{(L+1)} h^{(L)} + b^{(L+1)}\right). \qquad (1.2.3)$$

Remarque (Interprétation géométrique des frontières de décision) Chaque neurone d'une première couche cachée partitionne l'espace $\mathbb{R}^d$ par un demi-espace délimité par un hyperplan affine. Les neurones de la deuxième couche cachée effectuent une intersection booléenne (opérateur ET) de ces demi-espaces, formant ainsi des polyèdres convexes. Enfin, les couches suivantes peuvent réaliser l'union (opérateur OU) de régions convexes disjointes, rendant le réseau théoriquement capable d'approximer n'importe quelle frontière de décision fermée ou ouverte arbitrairement complexe.

1.3 Fonctions d'Activation et Classification Multiclasse

1.3.1 Fonctions d'activation usuelles

Le choix d'une fonction d'activation non linéaire g est indispensable : sans elle, la composition successive d'opérations affines $W^{(L)}(\dots(W^{(1)}x+b^{(1)})\dots)+b^{(L)}$ resterait strictement équivalente à une unique transformation affine globale $\tilde{W}x + \tilde{b}$, ruinant tout l'intérêt de la profondeur.

Définition (**Fonctions d'activation courantes**) . On définit ainsi différentes fonctions d'activations communément utilisées en sortie d'un neuronne. Pour tout réel $z \in \mathbb{R}$ on a :

- **Sigmoïde (logistique)** :
$$\sigma(z) = \frac{1}{1 + e^{-z}}. \qquad (1.3.1)$$
Elle projette $\mathbb{R}$ sur $]0, 1[$ et vérifie l'identité remarquable $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.
- **Tangente hyperbolique** :
$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in]-1, 1[, \qquad (1.3.2)$$
centrée en zéro, de dérivée $\tanh'(z) = 1 - \tanh^2(z)$.
- **Rectified Linear Unit (ReLU)** :
$$g(z) = \max(0, z). \qquad (1.3.3)$$
Sa dérivée vaut 1 pour $z > 0$ et 0 pour $z < 0$. Elle évite la saturation des gradients pour des entrées positives.
- **Leaky ReLU** : pour $\alpha \in]0, 1[$ (couramment $\alpha = 0{,}01$), $g(z) = \max(\alpha z, z)$. Elle évite le problème des "neurones morts" lorsque $z < 0$.

- **Softplus** :
$$g(z) = \log(1 + e^z), \tag{1.3.4}$$
approximation analytique différentiable de la ReLU.

1.3.2 Classification multiclasse et fonction Softmax

Considérons un problème de classification à $K > 2$ classes mutuellement exclusives. La couche finale produit un vecteur de scores réels non normalisés $a = W^{(L+1)}h^{(L)} + b^{(L+1)} \in \mathbb{R}^K$, appelés *logits*.

Définition (**Opérateur Softmax**) . L'opérateur Softmax normalise un vecteur de logits $a = (a_1, \dots, a_K)^\top$ en une distribution discrète de probabilités $p = (p_1, \dots, p_K)^\top \in [0, 1]^K$ définie par :
$$\forall c \in [\![1, K]\!], \quad p_c = \text{Softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^K e^{a_j}}. \tag{1.3.5}$$
On a immédiatement $\forall c,\ p_c > 0$ et $\sum_{c=1}^K p_c = 1$. La classe prédite est $\hat{y} = \arg\max_c p_c$.

1.4 Fonctions de Perte (*Loss Functions*)

En apprentissage supervisé, puisque nous connaissons les *labels* associés à chaque prédiction, nous sommes capables d'évaluer le modèle en temps réel sur chacune de ses prédiction sur les données d'entraînement. Ainsi, l'évaluation de la discordance entre la cible réelle y et la sortie estimée $\hat{y}$ s'effectue au moyen d'une fonction de coût différentiable $\mathcal{L}(y, \hat{y})$ que l'on appellera *fonction de perte* ou *loss*.

1.4.1 Entropie croisée binaire (BCE)

Remarque (Limites de la MSE) Pour une classification binaire avec $y \in \{0, 1\}$ et une prédiction de probabilité $\hat{y} = \sigma(z) \in]0, 1[$, l'erreur quadratique usuelle MSE s'avère sous-optimale du fait de zones de saturation où le gradient s'annule pour des erreurs maximales. On introduit la perte d'entropie croisée binaire.

Définition (**Binary Cross-Entropy**) . Pour $y \in \{0, 1\}$ et $\hat{y} \in]0, 1[$:
$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \tag{1.4.1}$$

Proposition (Stabilité numérique de la BCE) En injectant l'activation logistique $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ directement dans la perte, on évite les dépassements de capacité flottante (*overflow*) via l'expression analytique exacte :
$$\mathcal{L}_{\text{BCE}}(y, \sigma(z)) = \max(z, 0) - z \cdot y + \log\left(1 + e^{-|z|}\right). \tag{1.4.2}$$

1.5 Rétropropagation du Gradient et Dérivation Automatique

Comme vu au début de ce chapitre, l'apprentissage se fait par une recherche des "meilleurs" poids possibles pour le modèle dans l'objectif de minimiser l'erreur commise. Maintenant que

nous avons introduit la fonciton de perte *loss*, nous cherchons donc à faire une minimisation de cette fonction *loss*.
La principale problématique dans ce processus est le calcul de son *gradient*. En effet, pour trouver des candidats minimiseurs, la condition d'optimalité d'ordre 1 nous demande d'annuler ce gradient. Nous devons donc être capable de calculer les dérivées partielles de la *loss* en fonction des poids du modèle. On utilise pour cela l'algorithme de *rétropropagation* à l'aide de *graphes computationnels*.

1.5.1 Rétropropagation sur un neurone formel

Considérons un unique neurone formel doté d'une entrée $x \in \mathbb{R}^d$, d'une pré-activation $a = w^\top x + b$, d'une activation $\hat{y} = \sigma(a)$, et d'une perte quadratique $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$. Par application directe de la règle de dérivation en chaîne (*Chain Rule*) :

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{\partial a}{\partial w_i} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot x_i. \tag{1.5.1}$$

On retrouve le terme d'erreur $(y-\hat{y})$, pondéré par la pente de la sigmoïde $\hat{y}(1-\hat{y})$ et la projection d'entrée x_i. Si l'on remplace la perte quadratique par la $\mathcal{L}_{\text{BCE}}$, le terme $\hat{y}(1 - \hat{y})$ se simplifie élégamment au dénominateur :

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a} = \hat{y} - y \implies \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial w} = (\hat{y} - y)x. \tag{1.5.2}$$

1.5.2 Rétropropagation dans un MLP (*Backpropagation*)

Dans un graphe multicouches, les erreurs locales sont propagées de l'aval vers l'amont :

Définition (**Rétropropagation multicouche**) . Posons $\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}$ le vecteur de sensibilité de la couche k.

$$\text{Couche de sortie } (L+1): \quad \delta^{(L+1)} = \nabla_{a^{(L+1)}} \mathcal{L} \tag{1.5.3}$$

$$\text{Couches cachées } (k = L, \dots, 1): \quad \delta^{(k)} = \left((W^{(k+1)})^\top \delta^{(k+1)}\right) \odot g'^{(k)}\left(a^{(k)}\right) \tag{1.5.4}$$

où $\odot$ désigne le produit d'Hadamard (terme à terme). Les gradients s'en déduisent immédiatement :

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \left(h^{(k-1)}\right)^\top \in \mathbb{R}^{d_k \times d_{k-1}}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)} \in \mathbb{R}^{d_k}. \tag{1.5.5}$$

1.5.3 Graphes de calcul et implémentation PyTorch

En pratique, le calcul symbolique manuel des dérivées est remplacé par l'auto-différenciation en mode inverse (*reverse-mode automatic differentiation*) matérialisée par un graphe acyclique orienté (DAG).

```python
import torch
import torch.nn as nn

class MultiLayerPerceptron(nn.Module):
    def __init__(self, in_features: int, hidden_dim: int, out_classes: int):
        super(MultiLayerPerceptron, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(in_features, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
```

```
            nn.ReLU(),
            nn.Linear(hidden_dim, out_classes)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Retourne les logits bruts (non normalises)
        return self.network(x)

# Instanciation et passe avant/arriere
model = MultiLayerPerceptron(in_features=784, hidden_dim=128, out_classes=10)
criterion = nn.CrossEntropyLoss() # Integre log-softmax et NLL
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

x_dummy = torch.randn(32, 784) # Batch de 32 observations
y_target = torch.randint(0, 10, (32,))

optimizer.zero_grad()
logits = model(x_dummy)
loss = criterion(logits, y_target)
loss.backward() # Calcul automatique des gradients via Autograd
optimizer.step()
```

Listing 1.1 – Implémentation d'un MLP complet sous PyTorch

1.5.4 Optimisation par descente de gradient avec inertie (Momentum)

La descente de gradient stochastique standard (SGD) présente de fortes oscillations transverses dans les vallées mal conditionnées (matrice hessienne à fort conditionnement). La méthode avec moment régularise la trajectoire en accumulant une moyenne exponentielle glissante des vitesses passées :

$$v_t = \beta v_{t-1} + \alpha \nabla_W \mathcal{L}_t \quad \text{avec } \beta \in [0, 1[\tag{1.5.6}$$

$$W_{t+1} = W_t - v_t \tag{1.5.7}$$

où $\alpha > 0$ représente le pas d'apprentissage (*learning rate*).

Chapitre 2

Réseaux Convolutifs

Contents

2.1 L'Opérateur de Convolution en Vision Artificielle 105
2.1.1 Formulation mathématique 106
2.1.2 Dimensions, Stride, Padding et Dilatation 106
2.2 Sous-Échantillonnage, Champ Réceptif et Calcul 106
2.2.1 Couches de Pooling 107
2.2.2 Notion de champ réceptif (*Receptive Field*) 107
2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM 107
2.3 Techniques de Normalisation et Réseaux Résiduels 107
2.3.1 Normalisation par Lots (*Batch Normalization*) 107
2.3.2 Réseaux Résiduels (ResNet) 108
2.3.3 Implémentation d'un bloc résiduel sous PyTorch 108

Dans le cadre du traitement d'images numériques, l'utilisation de couches entièrement connectées (MLP) devient vite impraticable :

1. **Explosion du nombre de paramètres** : pour une image couleur haute définition de taille $1000 \times 1000 \times 3$, un premier neurone connecté à chaque pixel requiert 3×10^6 poids, induisant un coût mémoire prohibitif et un sur-apprentissage critique.
2. **Perte de corrélation spatiale** : vectoriser une image 2D/3D détruit la proximité géométrique locale entre pixels adjacents.
3. **Absence d'invariance par translation** : si un motif d'intérêt (un œil, une roue) se déplace dans l'image, un MLP doit réapprendre ce motif sur chacun des emplacements distincts.

La solution réside dans l'exploitation de la structure spatiale via deux principes directeurs : la *connectivité locale* (chaque neurone n'observe qu'un sous-voisinage) et le *partage des paramètres* (weight sharing).

2.1 L'Opérateur de Convolution en Vision Artificielle

L'objectif est donc d'être capable d'identifier des schémas caractéristiques dans une image, indépendamment de sa position dans celle-ci. Pour cela, on utilisera un opérateur de *convolution*. L'idée est de balayer la surface de l'image par une matrice plus petite appelée *noyau*. À chaque déplacement, on effectuera le produit de la partie de l'image couverte par le noyau. L'image ainsi obtenue sera plus petite et contiendra la même information mais sous forme condensée.

En effectuant cette étape de façon successive, on sera capable de réduire la taille de l'image d'origine à celle d'un vecteur qui pourra être facilement interprété par un MLP.

2.1.1 Formulation mathématique

En traitement du signal numérique, l'opération discrète bidimensionnelle appliquée entre une entrée matricielle I et un noyau (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ est définie usuellement sous la forme d'une corrélation croisée :

$$S(i,j) = (I * K)(i,j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m,\, j+n)\, K(m,n). \tag{2.1.1}$$

Dans une couche de convolution standard d'un CNN, l'entrée est un tenseur à trois dimensions $\mathcal{X} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ où C_{in} est le nombre de canaux (ex : Rouge, Vert, Bleu). La couche est paramétrée par C_{out} filtres tridimensionnels $W_k \in \mathbb{R}^{C_{\text{in}} \times k_h \times k_w}$ et un vecteur de biais $b \in \mathbb{R}^{C_{\text{out}}}$. La k-ième carte de caractéristiques de sortie s'exprime par :

$$S_k = b_k + \sum_{c=1}^{C_{\text{in}}} \mathcal{X}_c * W_{k,c}. \tag{2.1.2}$$

2.1.2 Dimensions, Stride, Padding et Dilatation

Propriété (Régression des dimensions spatiales) . Soit une entrée de taille $(H_{\text{in}}, W_{\text{in}})$, convoluée avec un noyau de taille (K_h, K_w), un rembourrage (*padding*) P, un pas (*stride*) S, et un taux de dilatation (*dilation*) D. La taille de la carte de sortie $(H_{\text{out}}, W_{\text{out}})$ vérifie :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \tag{2.1.3}$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor. \tag{2.1.4}$$

Proposition (Nombre de paramètres d'une couche convolutive) Pour une couche convolutive associant C_{in} canaux d'entrée à C_{out} canaux de sortie au travers de noyaux $K_h \times K_w$, le nombre de paramètres scalaires libres vaut :

$$N_{\text{params}} = C_{\text{out}} \times (C_{\text{in}} \times K_h \times K_w + 1) \tag{2.1.5}$$

où le terme $+1$ comptabilise le biais scalaire associé à chaque carte de sortie. Ce nombre est strictement indépendant de la résolution spatiale $(H_{\text{in}}, W_{\text{in}})$ de l'image.

Remarque Les couches de convolutions servent donc à *extraire* des schémas (*pattern*) et les caractéristiques (*features*) de l'image et de les compresser en vecteurs. Ensuite, un MLP est chargé *d'interpréter* ces *features* en prédiction (voir chapitre précédent). L'utilisation des convolutions permet de réduire drastiquement le nombre de paramètres et de couches à utiliser pour l'extraction de ces *features*.

2.2 Sous-Échantillonnage, Champ Réceptif et Calcul

L'opération de convolution est très simple à mettre en place algorithmiquement. En fonction de l'opération effectuée avec son résultat et de la dimension de son noyau, on peut observer plusieurs propriétés remarquables. Elles permettent ensuite de définir de nouvelles couches de convolutions ayant différents cas d'usages.

2.2.1 Couches de Pooling

Les couches de regroupement (*pooling*) réduisent la résolution spatiale des tenseurs intermédiaires pour accroître l'invariance aux translations locales et diminuer la charge computationnelle :

- **Max-Pooling** : extrait la valeur maximale sur une fenêtre $p_h \times p_w$. Il conserve le signal d'activation dominant tout en rejetant les variations de fond.
- **Average-Pooling** : calcule la moyenne arithmétique sur la fenêtre, agissant comme un filtre passe-bas régularisant.

2.2.2 Notion de champ réceptif (*Receptive Field*)

Le champ réceptif d'un neurone correspond à la zone d'extension dans l'image d'entrée d'origine qui influence directement son activation.

Proposition (Croissance du champ réceptif) Pour une succession de couches convolutives à noyaux de taille k_l et de pas unitaire, le champ réceptif cumulé R_L à la couche L vérifie :

$$R_L = R_{L-1} + (k_L - 1) = 1 + \sum_{l=1}^{L} (k_l - 1). \tag{2.2.1}$$

Ainsi, l'empilement de deux convolutions successives de taille 3×3 possède un champ réceptif équivalent de 5×5, tout en utilisant $2 \times (3^2) = 18$ poids au lieu de $5^2 = 25$ pour un noyau direct, tout en introduisant deux non-linéarités intermédiaires au lieu d'une.

2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM

Les unités graphiques de calcul (GPU) n'exécutent pas les convolutions spatiales par boucles imbriquées, mais projettent l'opération sous forme de multiplication matricielle dense hautement vectorisée (*General Matrix Multiply*, GEMM) :

1. **L'algorithme Im2Col** déplie chaque bloc spatial réceptif $C_{\text{in}} \times K_h \times K_w$ présent dans le tenseur d'entrée en un vecteur colonne, formant une vaste matrice :
$$X_{\text{col}} \in \mathbb{R}^{(C_{\text{in}} \cdot K_h \cdot K_w) \times (H_{\text{out}} \cdot W_{\text{out}})}. \tag{2.2.2}$$
2. Les poids des filtres sont aplatis en lignes pour construire une matrice :
$$W_{\text{row}} \in \mathbb{R}^{C_{\text{out}} \times (C_{\text{in}} \cdot K_h \cdot K_w)}. \tag{2.2.3}$$
3. Le produit matriciel dense calcule l'ensemble des réponses en une passe :
$$Y_{\text{flat}} = W_{\text{row}} \cdot X_{\text{col}} \in \mathbb{R}^{C_{\text{out}} \times (H_{\text{out}} \cdot W_{\text{out}})}. \tag{2.2.4}$$
4. La matrice résultante est réorganisée (*reshapée*) en dimension $(C_{\text{out}}, H_{\text{out}}, W_{\text{out}})$.

2.3 Techniques de Normalisation et Réseaux Résiduels

2.3.1 Normalisation par Lots (*Batch Normalization*)

Pour stabiliser la covariance interne des couches profondes (*Internal Covariate Shift*), on introduit la couche de Batch Normalization.

Définition (**Batch Normalization**) . Pour un mini-lot d'activations scalaires $\mathcal{B} = \{x_1, \ldots, x_m\}$ correspondant à un canal donné :

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \tag{2.3.1}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{2.3.2}$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{avec } \gamma, \beta \text{ paramètres différentiables appris.} \tag{2.3.3}$$

2.3.2 Réseaux Résiduels (ResNet)

Au-delà d'une vingtaine de couches, l'entraînement d'un réseau standard subit une dégradation critique due à l'évanescence des gradients. He et al. (2015) introduisent les connexions résiduelles directes (*skip connections*) :

$$\mathcal{H}(x) = \mathcal{F}(x, \{W_i\}) + x \tag{2.3.4}$$

où $\mathcal{F}$ représente la transformation non linéaire de quelques couches convolutives. Si la solution optimale est l'identité, le réseau n'a besoin que d'annuler les poids $\mathcal{F}(x) \to 0$, ce qui est infiniment plus aisé à apprendre. Lors de la rétropropagation, le gradient transite sans atténuation multiplicative via la dérivation directe :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}}\left(\frac{\partial \mathcal{F}}{\partial x} + I\right). \tag{2.3.5}$$

2.3.3 Implémentation d'un bloc résiduel sous PyTorch

```python
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, channels: int):
        super(ResidualBlock, self).__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(channels)
        )
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = x
        out = self.conv_block(x)
        out += identity  # Connexion residuelle directe
        out = self.relu(out)
        return out
```

Listing 2.1 – Bloc résiduel classique sous PyTorch

Chapitre 3

Réseaux Récurrents

Contents

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN) 109
3.1.1 Architecture et Dépliement Temporel 109
3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies . . . 110
3.2.1 Calcul du gradient BPTT 110
3.2.2 Évanescence et explosion du gradient 110
3.3 Cellules à Portes de Contrôle : LSTM et GRU 110
3.3.1 Long Short-Term Memory (LSTM) 110
3.3.2 Gated Recurrent Unit (GRU) 111
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention 111
3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq) 111
3.4.2 Mécanisme d'attention d'alignement dynamique 111
3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent 111

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)

Contrairement aux images dont les dimensions peuvent être ajustées à des formats fixes, les signaux temporels, acoustiques ou textuels se présentent sous la forme de séquences de longueur variable :

$$X = (x^{(1)}, x^{(2)}, \dots, x^{(T)}), \quad x^{(t)} \in \mathbb{R}^d. \tag{3.1.1}$$

3.1.1 Architecture et Dépliement Temporel

Un réseau récurrent traite séquentiellement chaque élément en maintenant un vecteur d'état interne $h^{(t)} \in \mathbb{R}^m$, constituant la mémoire synthétique du passé.

Définition (**Équations du RNN simple d'Elman**) . Pour chaque pas temporel $t \in [\![1, T]\!]$:

$$h^{(t)} = \tanh\left(Wh^{(t-1)} + Ux^{(t)} + b_h\right) \tag{3.1.2}$$

$$\hat{y}^{(t)} = \text{Softmax}\left(Vh^{(t)} + b_y\right) \tag{3.1.3}$$

où $U \in \mathbb{R}^{m \times d}$ est la matrice de projection des entrées, $W \in \mathbb{R}^{m \times m}$ la matrice de récurrence

état-à-état, et $V \in \mathbb{R}^{K \times m}$ la matrice de décodage vers l'espace de sortie. Les paramètres $\{U, W, V, b_h, b_y\}$ sont **partagés** à travers tous les pas de temps.

3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies

3.2.1 Calcul du gradient BPTT

L'erreur globale commise sur la séquence est la somme des coûts temporels $E = \sum_{t=1}^{T} E_t$. Pour dériver le gradient par rapport à la matrice de récurrence W, on déroule la dépendance temporelle :

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{T} \sum_{k=1}^{t} \frac{\partial E_t}{\partial h^{(t)}} \cdot \left(\prod_{j=k+1}^{t} \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \cdot \frac{\partial h^{(k)}}{\partial W}. \tag{3.2.1}$$

3.2.2 Évanescence et explosion du gradient

Le produit matriciel cumulé $\prod_{j=k+1}^{t} \frac{\partial h^{(j)}}{\partial h^{(j-1)}}$ gouverne la propagation temporelle du signal d'erreur. Si la matrice jacobienne présente des valeurs singulières strictement supérieures à 1, la norme du gradient diverge exponentiellement vers l'infini pour de grandes séquences : c'est l'**explosion du gradient**. On y remédie par la troncature heuristique (*Gradient Clipping*) :

$$g \leftarrow g \cdot \min\left(1, \ \frac{\text{seuil}}{\|g\|_2}\right). \tag{3.2.2}$$

Inversement, si les valeurs propres sont inférieures à 1, le gradient s'évanouit exponentiellement vers zéro au fur et à mesure que $t - k$ croît (**dissipation du gradient**), interdisant l'apprentissage de corrélations à long terme.

3.3 Cellules à Portes de Contrôle : LSTM et GRU

3.3.1 Long Short-Term Memory (LSTM)

Conçue par Hochreiter et Schmidhuber (1997), l'architecture LSTM résout l'évanescence du gradient via un état cellulaire interne C_t mis à jour par des opérations additives protégées par des portes logiques différentiables.

Définition (**Formulation complète d'un bloc LSTM**) . Soit x_t l'entrée au temps t et h_{t-1} l'état caché antérieur. Les portes sont régies par :

$$f_t = \sigma\left(W_f [h_{t-1}, x_t] + b_f\right) \quad \text{(porte d'oubli / Forget Gate)} \tag{3.3.1}$$

$$i_t = \sigma\left(W_i [h_{t-1}, x_t] + b_i\right) \quad \text{(porte d'entrée / Input Gate)} \tag{3.3.2}$$

$$\tilde{C}_t = \tanh\left(W_C [h_{t-1}, x_t] + b_C\right) \quad \text{(état candidat)} \tag{3.3.3}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \textbf{(mise à jour cellulaire additive)} \tag{3.3.4}$$

$$o_t = \sigma\left(W_o [h_{t-1}, x_t] + b_o\right) \quad \text{(porte de sortie / Output Gate)} \tag{3.3.5}$$

$$h_t = o_t \odot \tanh(C_t) \quad \text{(état caché final).} \tag{3.3.6}$$

Grâce à la relation linéaire $C_t = f_t \odot C_{t-1} + \ldots$, si la porte d'oubli est activée à saturation ($f_t \approx 1$), le gradient d'erreur se propage à travers l'axe temporel sans aucune atténuation multiplicative exponentielle.

3.3.2 Gated Recurrent Unit (GRU)

Proposé par Cho et al. (2014), le bloc GRU fusionne l'état cellulaire et l'état caché tout en réduisant le nombre de portes à deux :

$$z_t = \sigma\left(W_z[h_{t-1}, x_t] + b_z\right) \quad \text{(porte de mise à jour / Update Gate)} \tag{3.3.7}$$

$$r_t = \sigma\left(W_r[h_{t-1}, x_t] + b_r\right) \quad \text{(porte de réinitialisation / Reset Gate)} \tag{3.3.8}$$

$$\tilde{h}_t = \tanh\left(W[r_t \odot h_{t-1}, x_t] + b\right) \tag{3.3.9}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \tag{3.3.10}$$

3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention

3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq)

Dans une tâche de traduction ou de résumé, la longueur de la séquence générée est distincte de la séquence source. L'encodeur compresse l'intégralité de la séquence d'entrée en un vecteur contexte unique $c = h^{(T_{\text{in}})}$. Le décodeur génère les sorties mot par mot en conditionnant chaque tirage sur ce vecteur. Ce goulot d'étranglement unique entraîne une forte détérioration des performances lorsque la phrase dépasse une vingtaine de mots.

3.4.2 Mécanisme d'attention d'alignement dynamique

L'attention (Bahdanau et al., 2015 ; Luong et al., 2015) lève cette contrainte en permettant au décodeur de consulter l'ensemble des états intermédiaires de l'encodeur $(s_1, \ldots, s_S)$ à chaque pas de génération t.

Définition (**Calcul du vecteur de contexte par attention**) . Pour un état courant du décodeur h_t et un état source s_k :

$$\text{Score d'alignement :} \quad e_{t,k} = \begin{cases} v_a^\top \tanh\left(W_a h_t + U_a s_k\right) & \text{(Attention additive de Bahdanau)} \\ h_t^\top W_a s_k & \text{(Attention générale de Luong)} \\ h_t^\top s_k & \text{(Attention par produit scalaire direct)} \end{cases} \tag{3.4.1}$$

$$\text{Poids d'attention :} \quad \alpha_{t,k} = \frac{\exp(e_{t,k})}{\sum_{j=1}^{S} \exp(e_{t,j})} \quad \in [0, 1] \tag{3.4.2}$$

$$\text{Vecteur de contexte :} \quad c_t = \sum_{k=1}^{S} \alpha_{t,k} s_k. \tag{3.4.3}$$

3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent

```python
import torch
import torch.nn as nn

class Seq2SeqRNN(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int):
        super(Seq2SeqRNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
    bidirectional=True)
```

```
        # Decodeur prenant en compte la bidirectionnalite (hidden_dim * 2)
        self.decoder = nn.LSTM(embed_dim, hidden_dim * 2, batch_first=True)
        self.fc_out = nn.Linear(hidden_dim * 2, vocab_size)

    def forward(self, src: torch.Tensor, trg: torch.Tensor) -> torch.Tensor:
        # Encodage
        embedded_src = self.embedding(src)
        enc_outputs, (h_n, c_n) = self.encoder(embedded_src)

        # Concatener les etats terminaux des deux directions de l'encodeur
        h_dec = torch.cat((h_n[0:1], h_n[1:2]), dim=2)
        c_dec = torch.cat((c_n[0:1], c_n[1:2]), dim=2)

        # Decodage autoregressif
        embedded_trg = self.embedding(trg)
        dec_outputs, _ = self.decoder(embedded_trg, (h_dec, c_dec))
        logits = self.fc_out(dec_outputs)
        return logits
```

Listing 3.1 – Encodeur LSTM et Décodeur avec projection sous PyTorch

Chapitre 4

Modèles de Markov Caché

Contents

4.1 Modèles de Langage et Hypothèses Markoviennes 113
4.1.1 Modélisation séquentielle par règle de la chaîne 113
4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m 114
4.1.3 Modèles à classes de Brown 115
4.2 Formalisme des Modèles de Markov Cachés (HMM) 115
4.2.1 Définition mathématique 115
4.2.2 Les trois problèmes fondamentaux de Rabiner (1989) 116
4.3 Résolution Algorithmique des Trois Problèmes de Rabiner 116
4.3.1 Problème 1 : Évaluation et Algorithme Forward-Backward 116
4.3.2 Problème 2 : Décodage et Inférence 118
4.3.3 Stabilité Numérique et Domaine Logarithmique 119
4.3.4 Problème 3 : Apprentissage des Paramètres 120

Ce chapitre traite des *Modèles de Markov Cachés* (Hidden Markov Models, HMM). Historiquement développés pour le traitement automatique de la parole (*Automatic Speech Recognition*, Rabiner, 1989) et le traitement du langage naturel (étiquetage morpho-syntaxique *POS tagging*, modèles de langage, alignement bilingue), les HMM offrent un cadre probabiliste rigoureux pour les séquences discrètes. Bien que les modèles de fondation neuronaux (Transformers) dominent aujourd'hui ces applications, les algorithmes sous-jacents aux HMM (Forward-Backward, Viterbi, Baum-Welch/EM) constituent les bases fondamentales des modèles graphiques probabilistes et de la programmation dynamique.

4.1 Modèles de Langage et Hypothèses Markoviennes

Dans ce cours, on introduit les HMM en tant que modèles de langage. On comment ici par formaliser mathématiquement ces modèles en utilisant les chaînes de Markov.

4.1.1 Modélisation séquentielle par règle de la chaîne

Soit $\mathcal{V}$ un vocabulaire fini de symboles (mots ou *tokens*). On cherche à définir une loi de probabilité conjointe sur des séquences de longueur arbitraire bordées par deux balises déterministes start et stop :

$$O = (\text{start}, O_1, O_2, \ldots, O_n, \text{stop}) \in \mathcal{V}^*. \tag{4.1.1}$$

Par application exacte de la règle de décomposition en chaîne des probabilités conditionnelles, toute distribution sur une séquence s'écrit de manière causale (contexte gauche) :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{start}, O_1, \ldots, O_{i-1}) \tag{4.1.2}$$

en posant conventionnellement $O_{n+1} = \text{stop}$.

Remarque (Intuition des dés de Noah Smith et explosion combinatoire) On peut interpréter ce processus génératif à l'aide d'une collection de dés dont les faces représentent les mots de $\mathcal{V}$. À chaque étape, l'historique complet $h = (\text{start}, O_1, \ldots, O_{i-1})$ détermine le dé à lancer, dont la distribution sur les faces fournit le mot suivant O_i. Bien que d'une expressivité totale, ce modèle présente deux écueils rédhibitoires :

1. **Explosion paramétrique** : pour un historique de longueur au plus n et un vocabulaire de taille $V = |\mathcal{V}|$, le nombre d'historiques possibles vaut $\sum_{k=0}^{n-1} V^k = \frac{V^n - 1}{V - 1}$. Pour $V = 10^5$ et $n = 10$, cela exigerait de stocker les paramètres d'environ 10^{50} distributions catégorielles, ce qui est matériellement impossible.
2. **Défaut de généralisation** : toute séquence contenant un préfixe non observé lors de l'apprentissage se voit affecter une probabilité nulle.

4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m

Pour rendre l'estimation calculable, on réduit l'historique conditionnel via des hypothèses d'indépendance conditionnelle.

Définition (**Modèle Sac de Mots / Unigramme**) . L'hypothèse la plus forte pose l'indépendance mutuelle totale de chaque observation vis-à-vis du contexte passé :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n} \mathbb{P}(O_i). \tag{4.1.3}$$

Ce modèle ne requiert que $V - 1$ paramètres libres. En revanche, il détruit toute syntaxe : la probabilité d'une séquence est invariante par permutation de ses termes.

Le compromis canonique consiste à borner la mémoire du processus aux m derniers symboles émis.

Définition (**Chaîne de Markov d'ordre m / n-grammes**) . Un modèle de Markov stationnaire d'ordre $m \geqslant 1$ pose que la loi conditionnelle du token courant ne dépend que des m tokens immédiatement antérieurs :

$$\mathbb{P}(O_i \mid \text{start}, O_1, \ldots, O_{i-1}) = \mathbb{P}(O_i \mid O_{i-m}, \ldots, O_{i-1}). \tag{4.1.4}$$

La probabilité conjointe de la séquence s'écrit alors :

$$\mathbb{P}(\text{start}, O_1, \ldots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid O_{i-m}^{i-1}) \tag{4.1.5}$$

où $O_{i-m}^{i-1} = (O_{i-m}, \ldots, O_{i-1})$.

Remarque (Taxonomie et complexité des n-grammes) Dans la terminologie du traitement automatique du langage :

- $m = 0$ correspond au modèle *unigramme* ($V - 1$ paramètres) ;
- $m = 1$ correspond au modèle *bigramme* ($V(V - 1)$ paramètres de transition) ;
- $m = 2$ correspond au modèle *trigramme* ($V^2(V - 1)$ paramètres).

Au-delà de $m = 3$ ou 4, la matrice des comptes devient extrêmement creuse (*sparsity*), rendant indispensables des techniques de lissage statistique (Good-Turing, Kneser-Ney).

4.1.3 Modèles à classes de Brown

Pour injecter de la régularité syntaxique sans subir l'explosion combinatoire des n-grammes lexicaux, Brown et al. (1992) partitionnent le vocabulaire $\mathcal{V}$ en $C \ll V$ classes syntaxiques disjointes via une application déterministe $\mathrm{cl} : \mathcal{V} \to [\![1, C]\!]$ obtenue par partitionnement statistique :

Définition (**Modèle de séquence fondé sur des classes**) . La génération d'une observation O_i est scindée en deux lois : une transition d'état à état entre classes, suivie de l'émission du mot conditionnellement à sa propre classe :

$$\mathbb{P}(\mathrm{start}, O_1, \ldots, O_n, \mathrm{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \mathrm{cl}(O_i)) \cdot \mathbb{P}(\mathrm{cl}(O_i) \mid \mathrm{cl}(O_{i-1})). \tag{4.1.6}$$

En notant $A \in \mathbb{R}^{C \times C}$ la matrice de transition inter-classes et $B \in \mathbb{R}^{C \times V}$ la matrice d'émission des mots sachant la classe, ce modèle ne requiert que $C^2 + C \times V$ paramètres.
Cependant, ce modèle suppose que la fonction de projection cl est déterministe et connue à l'avance. Or, dans la langue naturelle, un mot est fondamentalement polysémique : le mot « *flies* » peut être un verbe ou un nom selon le contexte. L'état syntaxique sous-jacent est donc par essence *non observé* : c'est l'acte de naissance des *Modèles de Markov Cachés*.

4.2 Formalisme des Modèles de Markov Cachés (HMM)

4.2.1 Définition mathématique

Un modèle de Markov caché de premier ordre modélise un processus bivarié $(S_t, O_t)_{t \geq 1}$ où la suite des états (S_t) forme une chaîne de Markov non observable (latente), et (O_t) est la suite des observations visibles émises à chaque instant conditionnellement à l'état courant.

Définition (**Paramètres canoniques d'un HMM**) . Un HMM à états discrets est formellement caractérisé par le triplet $\lambda = (\pi, A, B)$:

1. Un ensemble fini d'états cachés $\mathcal{S} = \{s_1, \ldots, s_N\}$.
2. Un alphabet fini d'observations $\mathcal{V} = \{v_1, \ldots, v_M\}$.
3. La distribution initiale des états $\pi = (\pi_i)_{i=1}^N$ avec $\pi_i = \mathbb{P}(S_1 = s_i)$.
4. La matrice de transition $A = (a_{ij}) \in \mathbb{R}^{N \times N}$ régie par l'hypothèse markovienne :

$$a_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i), \quad \sum_{j=1}^{N} a_{ij} = 1. \tag{4.2.1}$$

5. La matrice d'émission $B = (b_j(k)) \in \mathbb{R}^{N \times M}$ vérifiant l'hypothèse d'indépendance

conditionnelle :

$$b_j(k) = \mathbb{P}(O_t = v_k \mid S_t = s_j), \quad \sum_{k=1}^{M} b_j(k) = 1. \tag{4.2.2}$$

Pour une séquence d'états $S = (S_1, \ldots, S_T)$ et d'observations $O = (O_1, \ldots, O_T)$, la loi conjointe se factorise immédiatement grâce aux indépendances conditionnelles du graphe :

$$\mathbb{P}(O, S \mid \lambda) = \pi_{S_1} b_{S_1}(O_1) \prod_{t=2}^{T} a_{S_{t-1}, S_t} b_{S_t}(O_t). \tag{4.2.3}$$

4.2.2 Les trois problèmes fondamentaux de Rabiner (1989)

L'utilisation des modèles HMM induit trois problèmes fondamentaux appelés des *problèmes de Rabiner*, en l'honneur de la personne qui les a formalisés la première fois en 1989.

Proposition (Problèmes de Rabiner - 1989) Les 3 problèmes présentés par Rabiner sont :

1. **L'Évaluation** : À partir d'une séquence d'observation donnée $O = (O_1, \ldots, O_m)$ comment calculer la probabilité qu'elle soit générée par le modèle ? L'approche naïve serait de dénombrer toutes les séquences possibles de probabilité non nulle, mais c'est un problème de complexité exponentielle par rapport à la taille de la séquence. En pratique, on utilisera l'algorithme de programmation dynnamique *Forward*.
2. **Inférence/Décodage** : À partir d'une observation $O = (O_1, \ldots, O_n)$ quelle est la séquence d'états cachés $S = (S_1, \ldots, S_n)$ qui aurait généré ces observations la plus probable ? On pourrait aussi calculer des fréquences sur toutes les séquences d'états possibles pouvant générer cette observation mais c'est aussi de complexité exponentiell. On utilisera l'algorithme de *Viterbi*.
3. **L'Apprentissage** : À partir d'un jeu de données d'observations, comment estimer les matrices de transition A et B pour construire le modèle ? On distingue ici deux cas :
   - **Apprentissage Supervisé** : On dispose ici des séquences d'observations et les états cachés correspondant : il suffit alors de calculer des fréquences dessus pour estimer les deux matrices.
   - **Apprentissage Non-Supervisé** : On ne dispose que des séquences d'observations. On utilisera pour cela l'algorithme *Expectation-Maximization (EM)* développé par Dempster en 1977. Cet algorithme se base sur le calcul d'un maximum de vraissemblance sur les états possibles.

     On peut aussi citer des approches par clustering sur les observations pour définir des états cachés ou des k-means.

4.3 Résolution Algorithmique des Trois Problèmes de Rabiner

4.3.1 Problème 1 : Évaluation et Algorithme Forward-Backward

Soit un modèle HMM fixé $\lambda = (\pi, A, B)$ et une séquence d'observations $O = (O_1, O_2, \ldots, O_T) \in \mathcal{V}^T$. Le problème d'évaluation consiste à déterminer la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$. Commençons par étudier la complexité de l'approche naïve pour se convaincre de l'utilisation d'algorithmes plus performants.

Proposition (Approche naïve) Par la loi des probabilités totales, la vraisemblance marginale s'obtient en sommant la loi conjointe sur l'ensemble $\mathcal{S}^T$ des trajectoires d'états latents possibles :

$$\mathbb{P}(O \mid \lambda) = \sum_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda) = \sum_{s_{i_1}=1}^{N} \cdots \sum_{s_{i_T}=1}^{N} \pi_{s_{i_1}} b_{s_{i_1}}(O_1) \prod_{t=2}^{T} a_{s_{i_{t-1}}, s_{i_t}} b_{s_{i_t}}(O_t). \tag{4.3.1}$$

L'espace $\mathcal{S}^T$ possède un cardinal de N^T. Chaque terme nécessitant $2T-1$ multiplications, la complexité temporelle brute est en $\mathcal{O}(T \cdot N^T)$. Ce calcul devient physiquement irréalisable dès que T dépasse quelques dizaines d'instants (pour $N = 10$ et $T = 50$, $N^T = 10^{50}$).

On préférera donc l'algorithme Forward ou Backward définis ci-dessous. L'algorithme Forward résout ce problème par *programmation dynamique* en factorisant les calculs sur un treillis d'états (*trellis*).

Proposition (Algorithme Forward) On introduit la variable intermédiaire $\alpha_t(i)$, définie comme la probabilité conjointe du préfixe d'observations jusqu'à l'instant t et de l'état latent au temps t :

$$\alpha_t(i) = \mathbb{P}(O_1, \ldots, O_t, S_t = s_i \mid \lambda), \quad \forall i \in [\![1, N]\!],\, t \in [\![1, T]\!]. \tag{4.3.2}$$

Algorithme 2 : Algorithme Forward

---

1. **Initialisation** ($t = 1$) :

$$\alpha_1(i) = \pi_i \, b_i(O_1), \quad \forall i \in [\![1, N]\!]. \tag{4.3.3}$$

2. **Induction / Récurrence** ($t = 1, \ldots, T-1$) :

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) \, a_{ij} \right] b_j(O_{t+1}), \quad \forall j \in [\![1, N]\!]. \tag{4.3.4}$$

3. **Terminaison** :

$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^{N} \alpha_T(i). \tag{4.3.5}$$

Remarque (Complexité) Le treillis compte $N \times T$ nœuds. À chaque transition temporelle $t \to t{+}1$, le calcul de $\alpha_{t+1}(j)$ agrège les contributions des N états antérieurs via N multiplications et additions. La complexité temporelle chute ainsi à $\mathcal{O}(N^2 T)$, avec une complexité spatiale en mémoire de $\mathcal{O}(NT)$ (ou $\mathcal{O}(N)$ si l'on ne conserve que le vecteur de l'instant précédent).

Proposition (Algorithme Backward) De manière duale, on définit la variable Backward $\beta_t(i)$ comme la probabilité conditionnelle du suffixe d'observations allant de $t+1$ à T, conditionnellement à l'état latent présent $S_t = s_i$:

$$\beta_t(i) = \mathbb{P}(O_{t+1}, \ldots, O_T \mid S_t = s_i, \lambda), \quad \forall i \in [\![1, N]\!],\, t \in [\![1, T]\!]. \tag{4.3.6}$$

Algorithme 3 : Algorithme Backward

1. **Initialisation** $(t = T)$:
$$\beta_T(i) = 1, \quad \forall i \in [\![1, N]\!]. \tag{4.3.7}$$
2. **Induction régressive** $(t = T-1, \dots, 1)$:
$$\beta_t(i) = \sum_{j=1}^{N} a_{ij}\, b_j(O_{t+1})\, \beta_{t+1}(j), \quad \forall i \in [\![1, N]\!]. \tag{4.3.8}$$
3. **Terminaison** :
$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^{N} \pi_i\, b_i(O_1)\, \beta_1(i). \tag{4.3.9}$$

4.3.2 Problème 2 : Décodage et Inférence

Étant donnée une séquence observée O, le décodage consiste à inférer la séquence d'états latents sous-jacente. Selon l'objectif visé, deux formulations distinctes sont envisagées. Le critère le plus classique recherche la trajectoire complète $S^* = (S_1^*, \dots, S_T^*)$ qui maximise la loi conjointe (et donc la loi a posteriori globale $\mathbb{P}(S \mid O, \lambda)$) :

$$S^* = \arg\max_{S \in \mathcal{S}^T} \mathbb{P}(S \mid O, \lambda) = \arg\max_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda). \tag{4.3.10}$$

On introduit la variable de Viterbi $\delta_t(i)$, représentant la probabilité maximale d'une séquence partielle d'états se terminant en s_i au temps t, ainsi qu'une matrice de pointeurs $\psi_t(i)$ (*back-pointers*) stockant l'antécédent optimal :

$$\delta_t(i) \triangleq \max_{S_1, \dots, S_{t-1}} \mathbb{P}(O_1, \dots, O_t, S_1, \dots, S_{t-1}, S_t = s_i \mid \lambda). \tag{4.3.11}$$

Algorithme 4 : Algorithme de Viterbi

1. **Initialisation** $(t = 1)$:
$$\delta_1(i) = \pi_i\, b_i(O_1), \quad \forall i \in [\![1, N]\!], \quad \psi_1(i) = 0. \tag{4.3.12}$$
2. **Récurrence** $(t = 2, \dots, T)$:
$$\delta_t(j) = \left[\max_{1 \le i \le N} \big(\delta_{t-1}(i)\, a_{ij}\big)\right] b_j(O_t), \quad \forall j \in [\![1, N]\!], \tag{4.3.13}$$
$$\psi_t(j) = \arg\max_{1 \le i \le N} \big(\delta_{t-1}(i)\, a_{ij}\big), \quad \forall j \in [\![1, N]\!]. \tag{4.3.14}$$
3. **Terminaison** :
$$P^* = \max_{1 \le i \le N} \delta_T(i), \tag{4.3.15}$$
$$S_T^* = \arg\max_{1 \le i \le N} \delta_T(i). \tag{4.3.16}$$
4. **Reconstruction du chemin (*Backtracking*)** $(t = T-1, \dots, 1)$:
$$S_t^* = \psi_{t+1}(S_{t+1}^*). \tag{4.3.17}$$

Proposition (*Posterior Decoding*) Une alternative consiste à déterminer, pour chaque pas de temps t pris isolément, l'état qui maximise la probabilité a posteriori conditionnelle $\mathbb{P}(S_t = s_i \mid O, \lambda)$:

$$S_*^t = \arg\max_{1 \le i \le N} \mathbb{P}(S_t = s_i \mid O, \lambda). \tag{4.3.18}$$

Grâce aux indépendances conditionnelles du modèle, la probabilité conjointe de l'état $S_t = s_i$ et de l'ensemble de la séquence d'observations O se factorise par le produit des variables Forward et Backward :

$$\mathbb{P}(S_t = s_i, O \mid \lambda) = \alpha_t(i)\,\beta_t(i). \tag{4.3.19}$$

Par la formule de Bayes :

$$\mathbb{P}(S_t = s_i \mid O, \lambda) = \frac{\mathbb{P}(S_t = s_i, O \mid \lambda)}{\mathbb{P}(O \mid \lambda)} = \frac{\alpha_t(i)\,\beta_t(i)}{\sum_{j=1}^{N} \alpha_t(j)\,\beta_t(j)}. \tag{4.3.20}$$

Le décodage a posteriori revient donc à évaluer :

$$S_*^t = \arg\max_{1 \le i \le N} \big(\alpha_t(i)\,\beta_t(i)\big). \tag{4.3.21}$$

Remarque (Comparaison Viterbi vs Décodage a posteriori)

- **Viterbi** garantit la validité structurelle globale du chemin prédit : aucune transition de probabilité nulle ($a_{ij} = 0$) ne peut être empruntée.
- **Le décodage a posteriori** minimise le taux d'erreur moyen par position temporelle. Cependant, en maximisant chaque état indépendamment de ses voisins, il peut produire deux états consécutifs S_*^t et S_*^{t+1} pour lesquels la transition est impossible ($a_{S_*^t, S_*^{t+1}} = 0$). Dans ce cas, la probabilité globale de la séquence d'états résultante est strictement nulle.

Les algorithmes Forward et Viterbi partagent la même topologie de programmation dynamique sur le treillis. Ils correspondent à une instanciation formelle du même algorithme abstrait sur deux *semi-anneaux* distincts $(K, \oplus, \otimes, \bar{0}, \bar{1})$:

Caractéristique	**Semi-anneau des Réels (Forward)**	**Semi-anneau Tropical (Viterbi)**
Support K	$\mathbb{R}_+$	$\mathbb{R}_+$
Élément nul $\bar{0}$	0	0
Élément unité $\bar{1}$	1	1
Addition $\oplus$	$+$ (somme des alternatives)	max (sélection du meilleur chemin)
Multiplication $\otimes$	$\times$ (accumulation causale)	$\times$ (accumulation causale)

TABLE 4.1 – Correspondance algébrique entre Forward et Viterbi via les semi-anneaux.

4.3.3 Stabilité Numérique et Domaine Logarithmique

Sur de longues séquences, la multiplication itérative de probabilités inférieures à 1 entraîne rapidement un dépassement de capacité par le bas (*arithmetic underflow*) sur les flottants standard. Pour éviter l'écrasement à zéro, on manipule les logarithmes des probabilités. Pour deux log-probabilités $u = \ln p$ et $v = \ln q$:

- La multiplication devient une addition : $\ln(p \times q) = u + v$.
- Dans l'algorithme de Viterbi, la monotonie du logarithme commute avec le maximum :

$$\ln \delta_t(j) = \ln b_j(O_t) + \max_{1 \le i \le N} \big[\ln \delta_{t-1}(i) + \ln a_{ij}\big]. \tag{4.3.22}$$

Pour l'algorithme Forward, l'addition de probabilités dans le domaine log nécessite le calcul de $\ln(p+q) = \ln(\mathrm{e}^u + \mathrm{e}^v)$. De manière générale, pour un vecteur $l = (l_1, \ldots, l_n) \in \mathbb{R}^n$, on définit l'opérateur *logsumexp* :

$$\mathrm{LSE}(l) \triangleq \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i}\right). \tag{4.3.23}$$

Si certains composants de l sont très grands ($l_i \gg 0$), le calcul brut de e^{l_i} provoque un dépassement par le haut (*overflow*).

Proposition (Factorisation stable de Log-Sum-Exp) Pour tout scalaire $a \in \mathbb{R}$:

$$\mathrm{LSE}(l) = a + \mathrm{LSE}(l-a) = a + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right). \tag{4.3.24}$$

Démonstration En factorisant par e^a dans la somme :

$$\begin{aligned}
\mathrm{LSE}(l) &= \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i}\right) = \ln\left(\sum_{i=1}^{n} \mathrm{e}^{a} \cdot \mathrm{e}^{l_i - a}\right) \\
&= \ln\left(\mathrm{e}^{a} \sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right) = \ln(\mathrm{e}^a) + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right) \\
&= a + \ln\left(\sum_{i=1}^{n} \mathrm{e}^{l_i - a}\right).
\end{aligned} \tag{4.3.25}$$

En pratique, on pose $a = \max_{1 \le i \le n} l_i$. Ainsi, pour tout i, $l_i - a \le 0$, ce qui borne $\mathrm{e}^{l_i - a} \in]0, 1]$ et neutralise tout risque d'overflow.

4.3.4 Problème 3 : Apprentissage des Paramètres

L'objectif de l'apprentissage est d'estimer les paramètres du modèle $\lambda = (\pi, A, B)$ à partir d'un ensemble de séquences d'entraînement.

Proposition (Cas Supervisé) Lorsque les séquences d'observations et les séquences d'états associées sont intégralement observées (par exemple en étiquetage POS annoté manuellement), l'estimation des paramètres par *Maximum de Vraisemblance* (MLE) se réduit à des ratios de fréquences empiriques :

$$\hat{\pi}_i = \frac{\mathrm{Count}(S_1 = s_i)}{\sum_{j=1}^{N} \mathrm{Count}(S_1 = s_j)}, \tag{4.3.26}$$

$$\hat{a}_{ij} = \frac{\mathrm{Count}(S_{t-1} = s_i, S_t = s_j)}{\sum_{k=1}^{N} \mathrm{Count}(S_{t-1} = s_i, S_t = s_k)}, \tag{4.3.27}$$

$$\hat{b}_j(k) = \frac{\mathrm{Count}(S_t = s_j, O_t = v_k)}{\sum_{m=1}^{M} \mathrm{Count}(S_t = s_j, O_t = v_m)}. \tag{4.3.28}$$

Proposition (Cas Non-Supervisé : Algorithme EM) Lorsque les états latents ne sont pas observés, l'optimisation directe de la log-vraisemblance marginale $\ln \mathbb{P}(O \mid \lambda) = \ln \sum_S \mathbb{P}(O, S \mid \lambda)$ est non convexe. On applique l'algorithme d'*Espérance-Maximisation* (EM), qui prend le nom d'algorithme de *Baum-Welch* dans le cadre des HMM.
L'algorithme itère deux étapes jusqu'à convergence :

1. **Étape E (Espérance)** : À l'aide des paramètres courants $\lambda^{(k)}$ et via les algorithmes Forward et Backward, on calcule les distributions a posteriori sur les variables latentes :

- Probabilité d'être dans l'état s_i au temps t :

$$\gamma_t(i) = \mathbb{P}(S_t = s_i \mid O, \lambda^{(k)}) = \frac{\alpha_t(i)\,\beta_t(i)}{\sum_{j=1}^{N} \alpha_t(j)\,\beta_t(j)}. \tag{4.3.29}$$

- Probabilité de transiter de s_i à s_j entre les instants t et $t+1$:

$$\xi_t(i,j) = \mathbb{P}(S_t = s_i, S_{t+1} = s_j \mid O, \lambda^{(k)}) = \frac{\alpha_t(i)\,a_{ij}\,b_j(O_{t+1})\,\beta_{t+1}(j)}{\mathbb{P}(O \mid \lambda^{(k)})}. \tag{4.3.30}$$

2. **Étape M (Maximisation)** : On met à jour les paramètres $\lambda^{(k+1)}$ en maximisant l'espérance de la log-vraisemblance des données complètes :

$$\pi_i^{(k+1)} = \gamma_1(i), \tag{4.3.31}$$

$$a_{ij}^{(k+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \tag{4.3.32}$$

$$b_j(m)^{(k+1)} = \frac{\sum_{t=1}^{T} \gamma_t(j) \cdot \mathbb{I}(O_t = v_m)}{\sum_{t=1}^{T} \gamma_t(j)}. \tag{4.3.33}$$

L'algorithme garantit la croissance monotone de la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$ à chaque itération et converge vers un extremum local du paysage d'optimisation.

Mathématiques du Machine Learning

Chapitre 1

Introduction à l'Apprentissage Supervisé

Contents

1.1 Des données et des prédictions 123
1.2 Formalisation mathématique 124
1.2.1 Fonction de Perte 124
1.2.2 Les Risques 124
1.3 Risque et prédicteur de Bayes 125
1.3.1 Forme explicite selon la tâche 126
1.4 Paradigmes classiques d'apprentissage 126
1.4.1 Minimisation du Risque Empirique (ERM) 126
1.4.2 Méthodes probabilistes et Maximum de Vraisemblance 127
1.4.3 Méthodes par moyennage local (k-plus proches voisins) 128
1.4.4 Méthodes génératives 128
1.4.5 Théorème No Free Lunch 128

L'objectif de ce cours est de formaliser le processus d'apprentissage d'un modèle dans le cas d'un apprentissage supervisé. On commencera par définit ce qu'est ce type d'apprentissage puis ce que l'on entend par "apprendre". Différentes métriques seront introduites pour le quantifier.

1.1 Des données et des prédictions

L'ensemble du processus d'apprentissage s'appuie sur les concepts de *données* et de *prédictions*.

Définition (**Observations**) . Formellement, on définira les *données* d'un problème de machine learning par un ensemble de couples $(x, y) \in \mathcal{X} \times \mathcal{Y}$ que l'on appellera *données d'entraînement* ou *observations*. On notera cet ensemble D qui contiendra N couples.

Ces entrées et sorties attendues peuvent être de différents types en fonction du problème considéré. Les entrées peuvent être des images, des vidéos, du son ou un texte. On les modélisera en général par des vecteurs de $\mathbb{R}^d$ lorsque c'est possible. On appellera donc $d \in \mathbb{N}$ la dimension des données à l'entrée du modèle.
Les sorties peuvent être de deux types différents en apprentissage supervisé :

- Un ensemble d'entiers $\{1, \dots, k\}$. On parlera alors de problème de *classification*.

- Un vecteur de $\mathbb{R}^{d'}$. On parlera alors d'un problème de *régression*.

L'objectif est d'être capable, à partir d'une nouvelle donnée $x \in \mathcal{X}$ d'inférer/prédire le y qui lui correspond le mieux. Plus formellement, on cherche à *apprendre* une fonction f à partir des données d'entraînement telle que :

$$y \simeq f(x) \tag{1.1.1}$$

1.2 Formalisation mathématique

Pour apprendre une telle fonction, nous devons admettre différentes hypothèses sur le jeu de données de départ D. La première est que ce jeu de données suit une certaine loi de probabilité $\mathbb{P}$. La seconde est que chaque échantillon $(x_i, y_i), i \in [\![1, N]\!]$ est indépendant des autres (D est un ensemble de réalisation i.i.d de la loi $\mathbb{P}$).

1.2.1 Fonction de Perte

Pour effectuer cet apprentissage, nous devons être capable de savoir à quel point notre modèle se trompe sur une prédiction donnée par rapport à la vraie valeur issue du jeu de données. Nous définissons pour cela une *fonction de perte*.

Définition (**Fonction de perte**) . La *fonction de perte*, notée l, calcule l'erreur induite par la prédiction de $z \in \mathcal{Y}$ alors que la valeur attendue était $y \in \mathcal{Y}$ pour une entrée $x \in \mathcal{X}$. Elle vérifie donc :

$$l : \mathcal{Y} \times \mathcal{Y} \longrightarrow \mathbb{R}. \tag{1.2.1}$$

Exemple Dans le cas d'une classification binaire (i.e $\mathcal{Y} = \{0, 1\}$), on peut simplement utiliser la *Zero/One Loss* définie par $l(y, z) = \mathbb{1}_{z \neq y}$.
Dans le cas de la régression, on utilisera plutôt les L^1 Loss ou L^2 Loss définies par :

$$l(y, z) = \|y - z\|^2, \quad \text{et} \quad l(y, z) = \|y - z\|$$

1.2.2 Les Risques

Ces différentes fonctions de pertes, aussi bien définies soient-elles, ne nous donnent d'un apperçu des erreurs du modèle en temps réel. Nous avons aussi besoin d'avoir une idée plus fine des performances du modèle vis-à-vis de la distribution de probabilité $\mathbb{P}$ du jeu de données D. On définit pour cela les *risques*.

Définition (**Risque Moyen**) . On définit le *risque moyen*, noté $\mathcal{R}(f)$, d'une fonction f sur l'espace $\mathcal{X} \times \mathcal{Y}$ par l'espérance de la perte de f sur ce même jeu de données.

$$\boxed{\mathcal{R}(f) := \int_{\mathcal{X} \times \mathcal{Y}} l(y, f(x))\, dp(x, y)\, dx\, dy = \mathbb{E}\left(l(y, f(x))\right)} \tag{1.2.2}$$

où p est la densité de la probabilité $\mathbb{P}$.

Exemple Dans le cas de la classification binaire, le risque moyen se simplfie facilement :

$$\begin{aligned}
\mathcal{R}(f) &= \int_{\mathcal{X} \times \mathcal{Y}} \mathbb{1}_{y \neq f(x)}\, d\mathbb{P}(x, y) \\
&= 0 \times \mathbb{P}(f(x) = y) + 1 \times \mathbb{P}(f(x) \neq y) \\
&= \mathbb{P}(f(x) \neq y)
\end{aligned}$$

À noter que ce risque est calculé sur l'espace entier des données possibles. En pratique, si celui-ci est infini, il est incalculable. On préférera donc utiliser le *risque empirique*.

Définition (**Risque Empirique**) . Le risque empirique, noté $\hat{\mathcal{R}}(f)$ est donné par la moyenne empirique des pertes sur un jeu de données D.

$$\boxed{\hat{\mathcal{R}}(f) := \frac{1}{N}\sum_{i=1}^{N} l(y_i, f(x_i)).} \tag{1.2.3}$$

Proposition (Convergence) Grâce aux hypothèses précédentes, on peut invoquer des théorèmes de convergence sur les suites de variables aléatoires pour caractériser la limite de ce risque empirique. Ainsi, si $f \in L^1$, d'après la *Loi Forte des Grands Nombres*, on a que :

$$\hat{\mathcal{R}}(f) \xrightarrow[n\to\infty]{\text{p.s}} \mathcal{R}(f). \tag{1.2.4}$$

Si, de plus $f \in L^2$, alors on a une idée de la vitesse de convergence de ce risque empirique :

$$\sqrt{n}(\hat{\mathcal{R}}(f) - \mathcal{R}(f)) \xrightarrow[n\to\infty]{\text{Loi}} \mathcal{N}(0, \mathbb{V}(l(y, f(x)))). \tag{1.2.5}$$

Remarque (Indépendance) En pratique, pour toute fonction apprise f, on aura $\hat{\mathcal{R}}(f) \simeq \mathcal{R}(f)$. Or, lorsque l'on calcule f sur des données empiriques (ex : jeu de données), on casse l'indépendance de la suite de variables aléatoires et donc le TCL. Formellement, en prenant l'estimateur :

$$\frac{1}{n}\sum_{i=1}^{n} l(x_i, \hat{f}(x_i)),$$

on remarque que le $\hat{f}$ est calculé à partir des $(x_1, y_1), \ldots, (x_n, y_n)$. On peut solutionner ce problème en séparant le jeu de données en deux : *entraînement* et *test* (et éventuellement *validation*). Puisque le modèle ne voit pas les données de test et de validation, la fonction apprise ne dépend pas de ces données, ce qui permet de conserver l'indépendance des variables. On peut ensuite utiliser le TCL pour estimer le risque $\mathcal{R}$ à partir du risque empirique $\hat{\mathcal{R}}$.

1.3 Risque et prédicteur de Bayes

L'objectif théorique de l'apprentissage statistique est de trouver une fonction f qui minimise le risque moyen $\mathcal{R}(f)$ parmi l'ensemble $\mathcal{F}_{\text{all}}$ de toutes les fonctions mesurables de $\mathcal{X}$ dans $\mathcal{Y}$.

Définition (**Prédicteur et Risque de Bayes**) . On appelle *prédicteur de Bayes*, noté f^*, la fonction optimale (si elle existe) qui minimise le risque moyen sur l'espace de toutes les fonctions mesurables :

$$\boxed{f^* \in \arg\min_{f\in\mathcal{F}_{\text{all}}} \mathcal{R}(f)} \tag{1.3.1}$$

Le risque associé, noté $\mathcal{R}^* = \mathcal{R}(f^*)$, est appelé le *risque de Bayes* (ou erreur irréductible). Pour toute fonction mesurable f, on a trivialement :

$$\mathcal{R}(f) \geqslant \mathcal{R}^* \tag{1.3.2}$$

Pour calculer explicitement f^*, on utilise la loi des espérances itérées (conditionnement par rapport à X) :

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y)}\left[l(Y, f(X))\right] = \mathbb{E}_X\left[\mathbb{E}_{Y|X}\left[l(Y, f(X)) \mid X\right]\right] \tag{1.3.3}$$

Minimiser $\mathcal{R}(f)$ revient donc à minimiser l'espérance conditionnelle point par point pour chaque $x \in \mathcal{X}$:

$$f^*(x) \in \arg\min_{z\in\mathcal{Y}} \mathbb{E}_{Y|X=x}\left[l(Y, z) \mid X = x\right] \tag{1.3.4}$$

1.3.1 Forme explicite selon la tâche

Proposition (Cas de la régression quadratique - L^2) Pour la perte quadratique $l(y,z) = (y-z)^2$, le prédicteur de Bayes est l'espérance conditionnelle de Y sachant X :

$$\boxed{f^*(x) = \mathbb{E}[Y \mid X = x]} \tag{1.3.5}$$

Démonstration Fixons $x \in \mathcal{X}$. On cherche $z \in \mathbb{R}$ minimisant $\phi(z) = \mathbb{E}\left[(Y-z)^2 \mid X = x\right]$.

$$\phi(z) = \mathbb{E}\left[Y^2 \mid X = x\right] - 2z\mathbb{E}\left[Y \mid X = x\right] + z^2$$

ϕ est une fonction convexe et dérivable en z. En annulant la dérivée :

$$\phi'(z) = -2\mathbb{E}[Y \mid X = x] + 2z = 0 \iff z = \mathbb{E}[Y \mid X = x]$$

Proposition (Cas de la classification binaire - perte 0/1) Pour $\mathcal{Y} = \{0,1\}$ et la perte 0/1 $l(y,z) = \mathbb{1}_{y \neq z}$, le prédicteur de Bayes attribue la classe la plus probable a posteriori :

$$\boxed{f^*(x) = \mathbb{1}_{\mathbb{P}(Y=1|X=x) \geqslant 1/2}} \tag{1.3.6}$$

Démonstration Pour un $x \in \mathcal{X}$ donné et pour $z \in \{0,1\}$:

$$\begin{aligned}\mathbb{E}\left[\mathbb{1}_{Y \neq z} \mid X = x\right] &= \mathbb{P}(Y \neq z \mid X = x) \\ &= 1 - \mathbb{P}(Y = z \mid X = x)\end{aligned}$$

Minimiser cette quantité revient à maximiser $\mathbb{P}(Y = z \mid X = x)$, ce qui donne la règle du maximum a posteriori (MAP).

Pour résumer, dans le cas d'une *classification*, le prédicteur de Bayes associe la classe la plus probable selon la distribution de probabilité simulée. Dans le cas d'une *régression*, ce prédicteur retournera la moyenne de la distribution de probabilité générée sur les y_i.

1.4 Paradigmes classiques d'apprentissage

Le prédicteur de Bayes f^* étant inconnu car il dépend de la distribution $\mathbb{P}$, différentes stratégies existent pour l'approcher à partir du jeu de données $S = ((x_1,y_1),\ldots,(x_n,y_n))$.

1.4.1 Minimisation du Risque Empirique (ERM)

Définition (**Estimateur ERM**) . On restreint la recherche à une classe paramétrique de fonctions $\mathcal{F}_\Theta = \{f_\theta : \mathcal{X} \to \mathcal{Y},\ \theta \in \Theta\}$. L'estimateur par *minimisation du risque empirique* (ERM) est défini par :

$$\boxed{\hat{\theta}_n \in \arg\min_{\theta\in\Theta} \hat{\mathcal{R}}_S(f_\theta) = \arg\min_{\theta\in\Theta} \frac{1}{n}\sum_{i=1}^{n} l(y_i, f_\theta(x_i))} \tag{1.4.1}$$

Exemple (Régression linéaire et polynomiale) Pour $f_\theta(x) = \theta^\top \varphi(x)$ (où φ est un morphisme d'extraction de caractéristiques / *feature map*) et la perte quadratique L^2, le problème s'écrit :

$$\hat{\theta}_n \in \arg\min_{\theta\in\Theta} \frac{1}{n}\sum_{i=1}^{n} (y_i - \theta^\top \varphi(x_i))^2 \tag{1.4.2}$$

Si $\varphi(x) = (1, x, x^2, \ldots, x^k)^\top$, on retrouve la régression polynomiale de degré k.

Proposition (Décomposition de l'erreur d'excès de risque) Pour tout estimateur $\hat{\theta}$, l'excès de risque par rapport au risque de Bayes se décompose canoniquement en :

$$\boxed{\mathcal{R}(f_{\hat{\theta}}) - \mathcal{R}^* = \underbrace{\left(\mathcal{R}(f_{\hat{\theta}}) - \inf_{\theta\in\Theta} \mathcal{R}(f_\theta)\right)}_{\text{Erreur d'estimation}} + \underbrace{\left(\inf_{\theta\in\Theta} \mathcal{R}(f_\theta) - \mathcal{R}^*\right)}_{\text{Erreur d'approximation}}} \tag{1.4.3}$$

- **Erreur d'approximation** : Déterministe et indépendante de n, elle mesure la perte due au choix de la classe restreinte Θ (diminue si la richesse de Θ augmente).
- **Erreur d'estimation** : Aléatoire (dépend de S), elle mesure le coût lié au nombre fini d'échantillons n (décroît avec n, croît avec la richesse/complexité de Θ).

Remarque (Contrôle de l'erreur d'estimation) En notant $\theta^* \in \arg\min_{\theta\in\Theta} \mathcal{R}(f_\theta)$, on majore l'erreur d'estimation par :

$$\mathcal{R}(f_{\hat{\theta}}) - \mathcal{R}(f_{\theta^*}) \quad \leqslant \quad 2\sup_{\theta\in\Theta} |\hat{\mathcal{R}}_S(f_\theta) - \mathcal{R}(f_\theta)| + \sup_{\theta\in\Theta}(\hat{\mathcal{R}}_S(f_{\hat{\theta}}) - \hat{\mathcal{R}}_S(f_\theta)) \tag{1.4.4}$$

Le premier terme est une déviation uniforme (objet central de la théorie de l'apprentissage statistique), et le second représente l'erreur d'optimisation numérique (nulle si le minimum empirique est calculé exactement).

La décomposition du risque comme présentée ci-dessus peut être observée sur l'apprentissage du modèle. On parlera alors de sur-apprentissage ou de sous-apprentissage en fonction de la complexité du modèle.

- **Sous-apprentissage (*Underfitting*)** : Modèle trop simple (faible complexité de Θ), dominé par l'erreur d'approximation.
- **Sur-apprentissage (*Overfitting*)** : Modèle trop expressif (par exemple interpolation polynomiale de Lagrange de degré $n-1$), l'erreur empirique est quasi-nulle mais l'erreur d'estimation explose sur les nouvelles données.

Pour contrôler ce phénomène, on introduit une pénalisation de complexité $\Omega(\theta)$:

$$\hat{\theta}_{\text{reg}} \in \arg\min_{\theta\in\Theta} \left\{\hat{\mathcal{R}}_S(f_\theta) + \lambda\Omega(\theta)\right\} \tag{1.4.5}$$

où $\lambda > 0$ est un hyperparamètre calibré sur le jeu de validation (exemples : Ridge avec $\Omega(\theta) = \|\theta\|_2^2$, Lasso avec $\Omega(\theta) = \|\theta\|_1$).

1.4.2 Méthodes probabilistes et Maximum de Vraisemblance

L'idée consiste à postuler une famille paramétrique de densités conditionnelles $\{p_\theta(y \mid x) : \theta \in \Theta\}$ et à choisir le paramètre qui maximise la log-vraisemblance sur l'échantillon i.i.d :

$$\hat{\theta}_{\text{MLE}} \in \arg\max_{\theta\in\Theta} \frac{1}{n}\sum_{i=1}^{n} \ln p_\theta(x_i, y_i) \tag{1.4.6}$$

Exemple (Régression Logistique) Pour des labels $y \in \{-1, 1\}$, en posant la fonction sigmoïde $\sigma(z) = \frac{1}{1+e^{-z}}$, le modèle postule :

$$p_\theta(y \mid x) = \sigma(y\,\theta^\top x) = \frac{1}{1 + e^{-y\,\theta^\top x}} \tag{1.4.7}$$

Maximiser la vraisemblance revient exactement à un problème d'ERM :

$$\hat{\theta} \in \arg\min_{\theta} \frac{1}{n}\sum_{i=1}^{n} \ln\left(1 + e^{-y_i\theta^\top x_i}\right) \tag{1.4.8}$$

avec la perte logistique convexe $l(y, z) = \ln(1 + e^{-yz})$.

Remarque (Pont entre Maximum de Vraisemblance et ERM) En posant $l(y, \hat{y}) = -\ln p_\theta(y \mid x)$, toute maximisation de vraisemblance s'interprète comme une minimisation de risque empirique :

- Bruit gaussien $\varepsilon \sim \mathcal{N}(0, \sigma^2) \implies$ Perte quadratique (moindres carrés).
- Loi de Bernoulli $\implies$ Perte logistique.

En classification, on utilise ces fonctions de perte convexes (surrogates) car minimiser directement la perte 0/1 est un problème NP-difficile et non dérivable (gradient nul presque partout).

1.4.3 Méthodes par moyennage local (k-plus proches voisins)

Ces méthodes non paramétriques exploitent une hypothèse de régularité locale : des points proches dans $\mathcal{X}$ partagent des distributions conditionnelles $\mathbb{P}_{y|x}$ similaires.

Définition (k-**Nearest Neighbours** – k-**NN**) . Pour une entrée x', on considère l'ensemble $\mathcal{V}_k(x')$ de ses k plus proches voisins dans le jeu d'entraînement :

- **En classification** : Règle du vote majoritaire :

$$\hat{f}(x') = \arg\max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{V}_k(x')} \mathbb{1}_{y_i = c} \tag{1.4.9}$$

- **En régression** : Moyenne empirique locale :

$$\hat{f}(x') = \frac{1}{k} \sum_{i \in \mathcal{V}_k(x')} y_i \tag{1.4.10}$$

Remarque (Compromis biais-variance pour k-NN) Pour k-NN, l'axe de complexité est inversé :

- k petit (ex. $k = 1$) $\implies$ Modèle très flexible, risque de sur-apprentissage (faible approximation, forte estimation).
- k grand $\implies$ Modèle rigide/lissé, risque de sous-apprentissage (forte approximation, faible estimation).

1.4.4 Méthodes génératives

Au lieu de modéliser directement $f(x)$ ou $\mathbb{P}(Y \mid X)$, les modèles génératifs estiment la loi conjointe en modélisant :

1. La loi a priori des classes $p(y)$.
2. Les densités conditionnelles de chaque classe $p(x \mid y)$.

On en déduit ensuite la loi a posteriori par la règle de Bayes :

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{\sum_c p(x \mid y = c)p(y = c)} \propto p(x \mid y)p(y) \tag{1.4.11}$$

La règle de décision optimale est donnée par le maximum a posteriori (MAP) : $\hat{f}(x') = \arg\max_{\hat{y}} p(x' \mid \hat{y})p(\hat{y})$.

1.4.5 Théorème No Free Lunch

Il n'existe pas d'algorithme d'apprentissage universel optimal sur toutes les distributions possibles sans hypothèse préalable sur les données.

Théorème (**No Free Lunch - Devroye, Györfi, Lugosi**) . Soit le problème de classification binaire avec la perte 0/1 et $\mathcal{X}$ un ensemble infini. Soit $\mathcal{P}$ l'ensemble de toutes les lois de probabilité sur $\mathcal{X} \times \{0, 1\}$. Pour tout entier n et tout algorithme d'apprentissage $\mathcal{A}$, on a :

$$\sup_{\mathbb{P} \in \mathcal{P}} \{\mathbb{E}_{S \sim \mathbb{P}^{\otimes n}} [\mathcal{R}_{\mathbb{P}}(\mathcal{A}(S))] - \mathcal{R}^*_{\mathbb{P}}\} \geqslant \frac{1}{2} \tag{1.4.12}$$

Ce résultat implique que pour n'importe quel algorithme fixé, il existe une distribution sous-jacente défavorable pour laquelle l'algorithme ne fait pas mieux qu'un choix aléatoire (même si le risque de Bayes $\mathcal{R}^* = 0$). Il est donc mathématiquement indispensable de poser des hypothèses régularité, marge, séparabilité ou compacité pour garantir des résultats d'apprentissage.

Chapitre 2

Régression Linéaire et Régression Ridge

Contents

2.1 Moindres Carrés 130
2.2 Analyse statistique – design fixe 131
2.3 Régression Ridge – design fixe 133

On pose ici le contexte d'une *régression linéaire*. On cherche donc à apprendre une fonction pour décrire un jeu de données $(x_1, y_1), \dots, (x_n, y_n) \in \mathcal{X} \times \mathcal{Y}$. Dans le cas du modèle linéaire, on fera l'hypothèse forte suivante :

$$(H) : f_\theta = \varphi(x)^t \theta, \quad \theta \in \mathbb{R}^d. \tag{2.0.1}$$

Pour construire cette fonction, on utilisera l'estimateur du risque empirique (ERM) définit par :

$$\hat{R}(\theta) \in \arg\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f_\theta(x_i)\right)^2. \tag{2.0.2}$$

Exemple La forme de l'application φ caractérisera la *feature map* (i.e l'ensemble des fonctions possibles pour interpoler le nuage de points).

- si $\varphi(x) = x$, alors on aura une estimation linéaire en x,
- si $\varphi(x) = \begin{pmatrix} x \\ 1 \end{pmatrix}$ alors l'estimation sera *affine* en x,
- si $\varphi(x) \in \mathbb{R}^d[x]$, alors l'estimation sera *polynômiale* en x. On pourra aller jusqu'au degré d pour utiliser des polynômes de Lagrange.

Notation On notera $y = \begin{pmatrix} y_1 \\ \dots \\ y_n \end{pmatrix}$ le vecteur des étiquettes du jeu de données et $\Phi = \begin{pmatrix} \varphi(x_1)^t \\ \dots \\ \varphi(x_n)^t \end{pmatrix} \in \mathbb{R}^{n \times d}$ la matrice contenant toutes les prédictions données par la *feature map*. On aura donc : $\hat{R}(\theta) = \frac{1}{n}\|y - \Phi\theta\|_2^2$.

2.1 Moindres Carrés

Nous sommes en présence d'un problème de régression. Grâces aux notations matricielles précédement posées, nous sommes capable de calculer une solution explicite à la main.

On cherche à calculer explicitement $\hat{\theta} \in \arg\min_{\theta \in \mathbb{R}^d} \hat{R}(\theta)$. Pour cela, on utilise la condition nécessaire d'optimalité d'ordre 1 : $\nabla_\theta \hat{R}(\hat{\theta}) = 0$. Commençons donc par calculer le gradient :

$$\hat{R}(\theta) = \frac{1}{n}\|y - \Phi\theta\|_2^2 \tag{2.1.1}$$

$$\nabla_\theta \hat{R}(\theta) = \frac{2}{n}\Phi^\top(\Phi\theta - y) \tag{2.1.2}$$

En résolvant l'équation $\nabla_\theta \hat{R}(\hat{\theta}) = 0$, on obtient les *équations normales* :

$$(*) : \Phi^\top \Phi\hat{\theta} = \Phi^\top y \tag{2.1.3}$$

En supposant que Φ est de rang plein ($rg(\Phi) = d$, soit des colonnes linéairement indépendantes), la matrice de Gram $\Phi^\top\Phi \in \mathbb{R}^{d\times d}$ est inversible, ce qui donne :

$$(*) \iff \boxed{\hat{\theta} = \left(\Phi^\top\Phi\right)^{-1}\Phi^\top y} \tag{2.1.4}$$

La fonction $\hat{R}$ étant convexe, ce point stationnaire est un minimum global. On calcule ainsi explicitement l'estimateur des moindres carrés (OLS), qui fournit le prédicteur linéaire optimal sur l'échantillon d'entraînement.

Proposition Géométriquement, les prédictions $\hat{y} = \Phi\hat{\theta}$ correspondent à la projection orthogonale du vecteur d'observations $y \in \mathbb{R}^n$ sur le sous-espace vectoriel $\mathrm{Im}(\Phi) \subset \mathbb{R}^n$ engendré par les colonnes de Φ :

$$\hat{\theta} \in \arg\min_{\theta\in\mathbb{R}^d} \frac{1}{n}\|y - \Phi\theta\|_2^2 \quad \iff \quad \hat{y} \in \arg\min_{z\in \mathrm{Im}(\Phi)} \frac{1}{n}\|y - z\|_2^2 \tag{2.1.5}$$

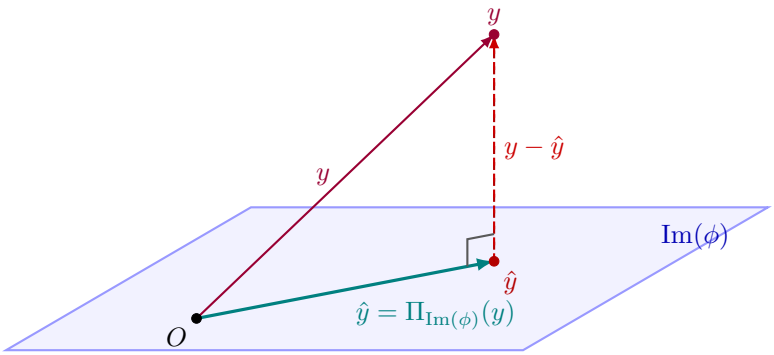


FIGURE 2.1 – Projection orthogonale du vecteur d'observations y sur le sous-espace affine $\mathrm{Im}(\phi)$ et décomposition de l'erreur résiduelle $y - \hat{y} \perp \mathrm{Im}(\phi)$.

2.2 Analyse statistique – design fixe

Dans cette section, on se place sous l'hypothèse du *design fixe* : les points $x_i \in \mathcal{X}$ (et donc la matrice $\Phi \in \mathbb{R}^{n\times d}$) sont déterministes et fixés. Seules les étiquettes y_i sont aléatoires.

Définition (**Modèle à design fixe**) . Le modèle probabiliste d'observation s'écrit :

1. $y = \Phi\theta_* + \varepsilon$, avec $\theta_* \in \mathbb{R}^d$ le paramètre vrai inconnu.
2. Le vecteur de bruit $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^\top \in \mathbb{R}^n$ vérifie $\mathbb{E}(\varepsilon) = 0$ et $\mathrm{Var}(\varepsilon) = \mathbb{E}(\varepsilon\varepsilon^\top) = \sigma^2 I_n$.

Le risque moyen sous design fixe est défini par :

$$R(\theta) = \frac{1}{n}\mathbb{E}_y\left[\|y - \Phi\theta\|_2^2\right]. \tag{2.2.1}$$

On introduit la matrice de covariance empirique $\hat{\Sigma} := \frac{1}{n}\Phi^\top\Phi \in \mathbb{R}^{d\times d}$, associée à la semi-norme $\|u\|_{\hat{\Sigma}}^2 := u^\top\hat{\Sigma}u = \frac{1}{n}\|\Phi u\|_2^2$.

Proposition (Décomposition Biais-Variance) Pour tout vecteur fixe $\theta \in \mathbb{R}^d$:

$$\begin{aligned} R(\theta) &= \frac{1}{n}\mathbb{E}_\varepsilon\left[\|\Phi(\theta_* - \theta) + \varepsilon\|_2^2\right] \\ &= \frac{1}{n}\|\Phi(\theta_* - \theta)\|_2^2 + \frac{1}{n}\mathbb{E}\left[\|\varepsilon\|_2^2\right] + \frac{2}{n}(\theta_* - \theta)^\top\Phi^\top\mathbb{E}[\varepsilon] \\ &= \|\theta - \theta_*\|_{\hat{\Sigma}}^2 + \sigma^2. \end{aligned}$$

Le risque de Bayes est atteint en θ_* : $R^* = R(\theta_*) = \sigma^2$. Par conséquent, pour un estimateur aléatoire $\hat{\theta}$ dépendant de y (donc de ε) :

$$\boxed{\mathbb{E}_{\hat{\theta}}\left[R(\hat{\theta})\right] - R^* = \underbrace{\left\|\mathbb{E}(\hat{\theta}) - \theta_*\right\|_{\hat{\Sigma}}^2}_{\text{Biais}^2} + \underbrace{\mathbb{E}_{\hat{\theta}}\left[\left\|\hat{\theta} - \mathbb{E}(\hat{\theta})\right\|_{\hat{\Sigma}}^2\right]}_{\text{Variance}}} \tag{2.2.2}$$

Théorème (**Biais et Variance des Moindres Carrés**) . Dans le cas de l'estimateur des moindres carrés $\hat{\theta} = (\Phi^\top\Phi)^{-1}\Phi^\top y$:

$$\text{Biais}^2 = 0 \quad \text{et} \quad \mathbb{E}\left[R(\hat{\theta})\right] - R^* = \frac{\sigma^2 d}{n}. \tag{2.2.3}$$

Démonstration

- *Calcul du biais* :

$$\begin{aligned} \mathbb{E}(\hat{\theta}) &= \mathbb{E}\left[(\Phi^\top\Phi)^{-1}\Phi^\top(\Phi\theta_* + \varepsilon)\right] \\ &= \theta_* + (\Phi^\top\Phi)^{-1}\Phi^\top\mathbb{E}[\varepsilon] = \theta_*. \end{aligned}$$

  L'estimateur est sans biais, d'où $\text{Biais}^2 = 0$.

- *Calcul de la variance* : On exprime la variance sous forme de trace :

$$\begin{aligned} \text{Variance} &= \mathbb{E}\left[(\hat{\theta} - \theta_*)^\top\hat{\Sigma}(\hat{\theta} - \theta_*)\right] \\ &= \mathbb{E}\left[\mathrm{tr}\left(\hat{\Sigma}(\hat{\theta} - \theta_*)(\hat{\theta} - \theta_*)^\top\right)\right] \\ &= \mathrm{tr}\left(\hat{\Sigma}\,\mathbb{E}\left[(\hat{\theta} - \theta_*)(\hat{\theta} - \theta_*)^\top\right]\right). \end{aligned}$$

Or, $\hat{\theta} - \theta_* = (\Phi^\top \Phi)^{-1} \Phi^\top \varepsilon$, donc :

$$\begin{aligned}\mathbb{E}\left[(\hat{\theta} - \theta_*)(\hat{\theta} - \theta_*)^\top\right] &= (\Phi^\top \Phi)^{-1} \Phi^\top \mathbb{E}[\varepsilon \varepsilon^\top] \Phi (\Phi^\top \Phi)^{-1} \\ &= \sigma^2 (\Phi^\top \Phi)^{-1} \Phi^\top I_n \Phi (\Phi^\top \Phi)^{-1} \\ &= \sigma^2 (\Phi^\top \Phi)^{-1} = \frac{\sigma^2}{n} \hat{\Sigma}^{-1}.\end{aligned}$$

En réinjectant dans l'expression de la trace :

$$\text{Variance} = \operatorname{tr}\left(\hat{\Sigma} \cdot \frac{\sigma^2}{n} \hat{\Sigma}^{-1}\right) = \frac{\sigma^2}{n} \operatorname{tr}(I_d) = \frac{\sigma^2 d}{n}.$$

2.3 Régression Ridge – design fixe

Dans le modèle linéaire usuel, l'erreur de variance vaut $\frac{\sigma^2 d}{n}$. Lorsque les covariables sont fortement corrélées (multicolinéarité), la matrice $\hat{\Sigma}$ possède des valeurs propres très proches de 0, ce qui rend $(\Phi^\top \Phi)^{-1}$ instable et fait exploser la norme du paramètre appris.
La régression Ridge (pénalisation de Tikhonov / régularisation L^2) remédie à ce problème en contractant les coefficients vers zéro via une pénalisation quadratique. On accepte d'introduire un léger biais en échange d'une réduction drastique de la variance.

Définition (**Estimateur Ridge**) . Pour un paramètre de régularisation $\lambda > 0$, l'estimateur Ridge est défini par :

$$\hat{\theta}_\lambda \in \underset{\theta \in \mathbb{R}^d}{\arg\min} \frac{1}{n} \|y - \Phi\theta\|_2^2 + \lambda \|\theta\|_2^2. \tag{2.3.1}$$

En annulant le gradient, la solution explicite est unique et donnée par :

$$\boxed{\hat{\theta}_\lambda = \left(\hat{\Sigma} + \lambda I_d\right)^{-1} \frac{1}{n} \Phi^\top y = \left(\Phi^\top \Phi + n\lambda I_d\right)^{-1} \Phi^\top y} \tag{2.3.2}$$

où $\hat{\Sigma} = \frac{1}{n}\Phi^\top \Phi$. La matrice $\hat{\Sigma} + \lambda I_d$ est symétrique définie positive pour tout $\lambda > 0$, donc inversible même si $\Phi^\top \Phi$ ne l'est pas.

Proposition (Décomposition Biais-Variance de Ridge) Sous le modèle à design fixe $y = \Phi\theta_* + \varepsilon$:

- *Biais* : L'espérance de l'estimateur vaut $\mathbb{E}(\hat{\theta}_\lambda) = (\hat{\Sigma} + \lambda I_d)^{-1} \hat{\Sigma} \theta_*$. Le terme de biais s'écrit donc :

$$\text{Biais}^2 = \left\|\mathbb{E}(\hat{\theta}_\lambda) - \theta_*\right\|_{\hat{\Sigma}}^2 = \lambda^2 \theta_*^\top \left(\hat{\Sigma} + \lambda I_d\right)^{-1} \hat{\Sigma} \left(\hat{\Sigma} + \lambda I_d\right)^{-1} \theta_*. \tag{2.3.3}$$

- *Variance* : En utilisant $\operatorname{Var}(\hat{\theta}_\lambda) = \frac{\sigma^2}{n} (\hat{\Sigma} + \lambda I_d)^{-1} \hat{\Sigma} (\hat{\Sigma} + \lambda I_d)^{-1}$, on obtient :

$$\text{Variance} = \frac{\sigma^2}{n} \operatorname{tr}\left(\hat{\Sigma}^2 \left(\hat{\Sigma} + \lambda I_d\right)^{-2}\right). \tag{2.3.4}$$

Le risque d'excès s'écrit donc :

$$\mathbb{E}\left[R(\hat{\theta}_\lambda)\right] - R^* = \lambda^2 \theta_*^\top \left(\hat{\Sigma} + \lambda I_d\right)^{-2} \hat{\Sigma}\, \theta_* + \frac{\sigma^2}{n} \operatorname{tr}\left(\hat{\Sigma}^2 \left(\hat{\Sigma} + \lambda I_d\right)^{-2}\right). \tag{2.3.5}$$

Remarque (Degrés de liberté effectifs) La quantité $df(\lambda) = \operatorname{tr}\left(\hat{\Sigma}(\hat{\Sigma} + \lambda I_d)^{-1}\right) = \sum_{j=1}^{d} \frac{\mu_j}{\mu_j + \lambda}$ mesure les *degrés de liberté effectifs* du modèle. Pour $\lambda = 0$, on retrouve $df(0) = d$ et la variance des moindres carrés $\frac{\sigma^2 d}{n}$. Lorsque $\lambda \to \infty$, $df(\lambda) \to 0$.

Proposition (Contrôle spectral et borne d'erreur) Soit $\hat{\Sigma} = U \operatorname{diag}(\mu_1, \ldots, \mu_d) U^\top$ la décomposition spectrale de $\hat{\Sigma}$ (avec $\mu_j \geq 0$).

- Pour le biais : pour tout $\mu \geq 0$ et $\lambda > 0$, on a $\frac{\lambda^2 \mu}{(\mu+\lambda)^2} \leq \frac{\lambda}{4}$ (car $(\mu + \lambda)^2 \geq 4\lambda\mu$), d'où :

$$\text{Biais}^2 \leq \frac{\lambda}{4} \|\theta_*\|_2^2. \tag{2.3.6}$$

- Pour la variance : comme $\frac{\mu_j^2}{(\mu_j+\lambda)^2} \leq \frac{\mu_j}{\lambda}$, on a :

$$\text{Variance} \leq \frac{\sigma^2}{n\lambda} \operatorname{tr}(\hat{\Sigma}). \tag{2.3.7}$$

En équilibrant les deux bornes avec le choix optimal $\lambda^* \propto \frac{\sigma}{\sqrt{n}}$, on garantit une vitesse de convergence de l'excès de risque en $\mathcal{O}(1/\sqrt{n})$, indépendante du plus petit spectre de $\hat{\Sigma}$.

Chapitre 3

Méthodes à Noyaux

Contents

3.1 De la similarité au Théorème du Représentant 135
3.1.1 Intuition géométrique et plongement 135
3.1.2 Le Théorème du Représentant 136
3.2 Noyaux définis positifs, Théorème de Bochner, et Théorème d'Aronszajn 137
3.2.1 Noyau SDP : Définition et propriétés 137
3.2.2 Construire des noyaux SDP : Théorème de Bochner 138
3.2.3 Théorème d'Aronszajn 139
3.3 Séparateurs à Vaste Marge (SVM) 141
3.3.1 Cas linéairement séparable : Maximisation de la marge 141
3.3.2 Cas non séparable : Variables d'écart et perte charnière 141
3.3.3 Dualité de Wolfe et formulation duale 142
3.3.4 Le Kernel SVM (SVM à noyau) 142

L'objectif des méthodes à noyaux est de transformer une mesure heuristique de similarité entre données en un algorithme d'apprentissage non linéaire. En s'appuyant implicitement sur un espace de Hilbert de dimension arbitraire (potentiellement infinie), on conserve toute la simplicité de l'optimisation linéaire tout en évitant d'avoir à calculer explicitement les plongements.

3.1 De la similarité au Théorème du Représentant

3.1.1 Intuition géométrique et plongement

La plupart des jeux de données ne sont pas linéairement séparables dans leur espace d'entrée d'origine $\mathcal{X}$. L'idée fondamentale consiste à envoyer les données dans un espace de caractéristiques de plus grande dimension (un espace de Hilbert $\mathcal{H}$) via une *feature map* :

$$\varphi : \mathcal{X} \longrightarrow \mathcal{H} \tag{3.1.1}$$

Dans $\mathcal{H}$, un hyperplan linéaire peut alors séparer des classes qui formaient des régions géométriques complexes ou intriquées dans $\mathcal{X}$. On formule alors le problème sous la forme d'une minimisation de risque empirique régularisé sur la totalité de l'espace hilbertien $\mathcal{H}$:

$$\hat{\theta}_S \in \arg\min_{\theta \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} l\left(y_i, \langle \varphi(x_i), \theta \rangle_{\mathcal{H}}\right) + \frac{\lambda}{2} \|\theta\|_{\mathcal{H}}^2 \tag{3.1.2}$$

où $\lambda > 0$ contrôle la régularisation.

Remarque (Obstruction numérique) Si $\mathcal{H}$ est de dimension infinie, on ne peut ni représenter le vecteur de paramètres $\theta \in \mathcal{H}$, ni évaluer explicitement $\varphi(x)$ par descente de gradient standard. Le théorème suivant résout ce paradoxe.

3.1.2 Le Théorème du Représentant

Théorème (**Théorème du Représentant**) . Soit $\mathcal{X}$ un ensemble, $\mathcal{H}$ un espace de Hilbert réel muni du produit scalaire $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ et $\varphi : \mathcal{X} \to \mathcal{H}$. Soit un échantillon $S = (x_1, \dots, x_n)$ et le sous-espace engendré par les observations :

$$\mathcal{H}_D := \text{vect}\{\varphi(x_1), \dots, \varphi(x_n)\} \subset \mathcal{H} \tag{3.1.3}$$

Soit $\Psi : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}$ une fonction croissante par rapport à son second argument. En notant $u(\theta) = (\langle \varphi(x_1), \theta \rangle_{\mathcal{H}}, \dots, \langle \varphi(x_n), \theta \rangle_{\mathcal{H}})^\top \in \mathbb{R}^n$, on a :

$$\inf_{\theta \in \mathcal{H}} \Psi\left(u(\theta), \|\theta\|_{\mathcal{H}}^2\right) = \inf_{\theta \in \mathcal{H}_D} \Psi\left(u(\theta), \|\theta\|_{\mathcal{H}}^2\right) \tag{3.1.4}$$

Si l'infimum est atteint, il l'est en un point $\hat{\theta}_S \in \mathcal{H}_D$. De plus, si Ψ est *strictement croissante* par rapport à son second argument, tout minimiseur appartient nécessairement à $\mathcal{H}_D$.

Grâce à ce théorème, on cherche donc $\hat{\theta}_S \in \mathcal{H}_D$ de dimension finie. Puisque l'on en connaît une famille génératrice, on cherche donc :

$$\alpha = (\alpha_1, \dots, \alpha_n)^\top \in \mathbb{R}^n, \quad \text{tels que} \quad \hat{\theta}_S = \sum_{i=1}^n \alpha_i \varphi(x_i).$$

Ainsi, la dimension du problème d'optimisation ne dépend plus de celle de $\mathcal{H}$ mais seulement du nombre de d'échantillons n. Déterminons maintenant comment trouver ce $\hat{\alpha}_S$.

Proposition (Kernel Trick) Remplaçons θ par sa nouvelle expression dans le problème de départ 3.1.2 :

- La norme devient :

$$\begin{aligned}\|\theta\|_{\mathcal{H}}^2 &= \Big\langle \sum_{i=1}^n \alpha_i \varphi(x_i), \sum_{j=1}^n \alpha_j \varphi(x_j) \Big\rangle_{\mathcal{H}} = \sum_{i,j=1}^n \alpha_i \alpha_j \big\langle \varphi(x_i), \varphi(x_j) \big\rangle_{\mathcal{H}} \\ &= \sum_{i,j=1}^n \alpha_i \alpha_j K_{i,j} \quad \text{avec} \quad K = \left(\big\langle \varphi(x_i), \varphi(x_j) \big\rangle_{\mathcal{H}}\right)_{i,j}\end{aligned}$$

- Les prédictions deviennent :

$$\langle \varphi(x_i), \theta \rangle_{\mathcal{H}} = \sum_{j=1}^n \alpha_j \langle \varphi(x_i), \varphi(x_j) \rangle_{\mathcal{H}} = (K\alpha)_i$$

On a donc le problème réécrit suivant :

$$\hat{\alpha}_S \in \underset{\alpha \in \mathbb{R}^n}{\arg\min} \frac{1}{n} \sum_{i=1}^n l\left(y_i, (K\alpha)_i\right) + \frac{\lambda}{2} \alpha^\top K \alpha. \tag{3.1.5}$$

On remarque que l'on a plus besoin de calculer explicitement φ ni f : tout dépend de ce que l'on appelera le *noyau* K. Notons ici $K = (k(x_i, x_j))_{i,j}$ tel que $k(x, x_j) = \langle \varphi(x), \varphi(x_j) \rangle_{\mathcal{H}}$. Ainsi, pour l'entraînement nous n'avons besoin que de la matrice de Gram K de taille $n \times n$. Pour l'inférence sur un x donné, nous notons $k_x = (k(x, x_1), \dots, k(x, x_n)) \in \mathbb{R}^n$ tel que $f_{\hat{\theta}_S} = \hat{\alpha}_S^\top k_x$.

On peut maintenant remarquer que la *feature map* φ n'est jamais calculée et que les prédictions en x utilisent seulement les n valeurs $\langle \varphi(x), \varphi(x_i) \rangle_{\mathcal{H}}$. Ainsi, au lieu de définir un espace $\mathcal{H}$ dans lequel nous ne pouvons faire aucun calcul, on pourrait directement choisir k telle que $k : \mathcal{X} \times \mathcal{X} \longrightarrow \mathbb{R}$ de la forme $k(x, x') = \langle \varphi(x), \varphi(x') \rangle$. C'est le sujet de la section suivante qui définit les *noyaux* et introduit la théorie nécessaire dessus.

3.2 Noyaux définis positifs, Théorème de Bochner, et Théorème d'Aronszajn

Dans la partie précédente, nous avons reformulé le problème de minimisation du risque empirique à l'aide d'un plongement dans un espace de Hilbert $\mathcal{H}$. Nous avons ensuite remarqué que tous les calculs dans cet espace dépendent en réalité d'une *fonction de similitude* encodée dans un *noyau* K.
Or, en pratique, on raisonne dans le sens inverse : à partir du jeu de données, on définit un noyau K et on souhaite savoir si l'on peut effectivement utiliser ce noyau pour notre minimisation. C'est le théorème d'Aronszajn qui nous l'assure. Mais avant de l'aborder, définissons formellement la notion de noyau ainsi que ses propriétés.

3.2.1 Noyau SDP : Définition et propriétés

Au lieu de définir $\mathcal{H}$ puis φ, on choisit directement une fonction $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ et on cherche à quelles conditions elle correspond à un produit scalaire hilbertien (i.e un noyau semi-défini positif).

Définition (**Noyau semi-défini positif**) . Une application $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ est appelée un *noyau semi-défini positif* (noyau SDP ou PDSK) si elle vérifie :

1. **Symétrie** : $\forall x, x' \in \mathcal{X}, \quad k(x, x') = k(x', x)$.
2. **Positivité** : $\forall n \geqslant 1,\ \forall (x_1, \ldots, x_n) \in \mathcal{X}^n,\ \forall (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n$:

$$\sum_{i,j=1}^{n} \alpha_i \alpha_j k(x_i, x_j) \geqslant 0 \iff K \in \mathcal{S}_n^+(\mathbb{R}) \tag{3.2.1}$$

Propriété (**Caractérisation par matrice de Gram**) . La matrice de Gram $G = \left(\langle u_i, u_j \rangle\right)_{i,j}$ associée à toute famille finie $(u_1, \ldots, u_n)$ est symétrique semi-définie positive.

Proposition (Tout produit scalaire définit un noyau SDP) Si $\mathcal{H}$ est un espace de Hilbert et $\varphi : \mathcal{X} \to \mathcal{H}$, alors $k(x, x') := \langle \varphi(x), \varphi(x') \rangle_{\mathcal{H}}$ est un noyau SDP.

Exemple Voici quelques exemples de noyaux SDP :

- Noyau *linéaire* : $k(x, x') = x^\top x'$ sur $\mathcal{X} = \mathbb{R}^d$.
- Noyau *polynomial* : $k(x, x') = (x^\top x')^q$ pour $q \in \mathbb{N}^*$.
- Noyau *exponentiel* : $k(x, x') = \exp\left(-a\|x - x'\|_2^2\right)$ pour $a > 0$.

Les noyaux SDP peuvent être construit par opérations. Le théorème suivant détaille les opérations possibles entre noyaux SDP.

Théorème (**Opérations**) . L'ensemble des noyaux semi-définis positifs sur un espace $\mathcal{X}$ est stable par :

- multiplication scalaire par un coefficient positif ou nul,
- somme finie,
- produits,
- limite ponctuelle quand elle existe,
- conjugaison par une fonction.

Grâce à ce théorème, nous pouvons montrer que beaucoup d'application k sont des noyaux. Voyons-en quelques unes.

Proposition Soit $q \geqslant 1$ un entier, alors $(x, x') \longmapsto (x^\top x')^q$ et $(x, x') \longmapsto (1 + x^\top x')^q$ sont des noyaux SDP.

Démonstration Pour le premier, $(x, x') \mapsto x^\top x'$ est un produit scalaire donc c'est un noyau. À la puissance q, c'est une multiplication finie. C'est donc un noyau par induction sur q.
Pour le second, on identifie $1 + x^\top x' = k(x, x') + x^\top x'$ avec $k(x, x') = 1$. Puisque $\sum_{i,j} \alpha_i \alpha_j = \left(\sum_i \alpha_i\right)^2 \geqslant 0$, alors c'est un noyau. Par somme et d'après le résultat précédent, $(x, x') \longmapsto (1 + x^\top x')^q$ est aussi un noyau SDP.

Proposition Soit k un noyau SDP, alors $\exp(k) : (x, x') \longmapsto \exp(k(x, x'))$ est un noyau SDP.

Démonstration Si k est un noyau alors, $\forall x, x' \in \mathcal{X}$, on a :

$$\exp(k(x, x')) = \sum_{n=0}^{\infty} \frac{k(x, x')^n}{n!}.$$

Or, pour tout $n \geqslant 0$, $\frac{k(x,x')^n}{n!}$ est un noyau par produit fini et multiplication par un scalaire positif. Par somme, $\sum_{i=1}^{n} \frac{k(x,x')^i}{i!}$ en est un aussi. Enfin, par définition de l'exponentielle, $\exp a$ converge pour tout $a \in \mathbb{R}$, donc par limite de noyaux SDP, $\exp(k)$ est aussi un noyau SDP.

3.2.2 Construire des noyaux SDP : Théorème de Bochner

Montrer qu'un noyau est SDP est simple lorsqu'il dérive de produits scalaires combinés algébriquement. En revanche, si l'on cherche à mesurer la proximité géométrique locale entre deux observations, le calcul direct devient inextricable. Par exemple, pour le noyau gaussien $k(x, x') = \exp\big(-a\|x - x'\|_2^2\big)$, vérifier point par point que :

$$\forall n \geqslant 1, \quad \forall (x_1, \ldots, x_n) \in (\mathbb{R}^d)^n, \quad \forall (\alpha_1, \ldots, \alpha_n) \in \mathbb{R}^n, \quad \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j \exp\big(-a\|x_i - x_j\|_2^2\big) \geqslant 0$$

est difficile sans outil analytique. Le théorème de Bochner permet de lever ce verrou en transposant la question de l'espace spatial à l'espace fréquentiel via la transformée de Fourier.

Théorème (**Théorème de Bochner – Invariance par translation**) . Soit $q : \mathbb{R}^d \to \mathbb{R}$ continue. Le noyau stationnaire $k(x, x') := q(x - x')$ est semi-défini positif si et seulement si q est la transformée de Fourier inverse d'une mesure borélienne positive finie ρ sur $\mathbb{R}^d$:

$$q(z) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{iz^\top \omega} \, \mathrm{d}\rho(\omega). \tag{3.2.2}$$

Démonstration (Preuve de la condition suffisante) Supposons que q s'écrive sous forme d'intégrale de Fourier par rapport à une mesure positive finie ρ. Pour toute famille $(x_1, \dots, x_n)$ et tout vecteur $\alpha \in \mathbb{R}^n$:

$$\begin{aligned}
\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j q(x_i - x_j) &= \sum_{i,j=1}^{n} \alpha_i \alpha_j \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{i(x_i - x_j)^\top \omega} \, \mathrm{d}\rho(\omega) \\
&= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \sum_{i=1}^{n} \alpha_i e^{i x_i^\top \omega} \sum_{j=1}^{n} \alpha_j e^{-i x_j^\top \omega} \, \mathrm{d}\rho(\omega) \\
&= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \left(\sum_{i=1}^{n} \alpha_i e^{i x_i^\top \omega} \right) \overline{\left(\sum_{j=1}^{n} \alpha_j e^{i x_j^\top \omega} \right)} \, \mathrm{d}\rho(\omega) \\
&= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \underbrace{\left| \sum_{i=1}^{n} \alpha_i e^{i x_i^\top \omega} \right|^2}_{\geqslant 0} \underbrace{\mathrm{d}\rho(\omega)}_{\geqslant 0} \geqslant 0.
\end{aligned}$$

La fonction q étant paire dès lors que la mesure est symétrique ($\rho(-\omega) = \rho(\omega)$), le noyau est bien symétrique et semi-défini positif.

Exemple (Cas du noyau Gaussien / RBF) Pour $a > 0$ et $z \in \mathbb{R}^d$, la fonction gaussienne $q(z) = \exp\bigl(-a\|z\|_2^2\bigr)$ admet pour transformée de Fourier inverse :

$$e^{-a\|z\|_2^2} = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{i z^\top \omega} \left(\frac{\pi}{a}\right)^{d/2} e^{-\frac{\|\omega\|_2^2}{4a}} \, \mathrm{d}\omega.$$

La mesure $\mathrm{d}\rho(\omega) = (\pi/a)^{d/2} e^{-\|\omega\|_2^2/(4a)} \, \mathrm{d}\omega$ est une mesure gaussienne positive, finie et invariante par $\omega \mapsto -\omega$. Le théorème de Bochner garantit immédiatement que le noyau gaussien est semi-défini positif.

3.2.3 Théorème d'Aronszajn

Reprenons le raisonnement de ce chapitre. Le théorème du représentant nous a montré que pour notre problème d'optimisation, si on a un espace de Hilbert $\mathcal{H}$ et une application φ, il nous suffit seulement de calculer des produits scalaires $k(x, x') = \langle \varphi(x), \varphi(x') \rangle_{\mathcal{H}}$.
Ensuite, nous avons développé une théorie sur les noyaux SDP pour les construire facilement. Le théorème de Bochner nous permet d'en construire autant que l'on souhaite à partir de mesures positives finies.
Or, nous ne sommes pas assurés que pour un tel noyau construit spécifiquement pour nos données, il existe un espace $\mathcal{H}$ et un plongement φ qui convienne. C'est ici que le théorème d'Aronszajn intervient : il nous assure que tout noyau SDP est le produit scalaire d'un espace unique de fonctions appelé RKHS.

Théorème (**Théorème d'Aronszajn**) . Soit $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ un noyau symétrique semi-défini positif. Alors, il existe un unique espace de Hilbert $\mathcal{H}_k$ constitué de fonctions de $\mathcal{X}$ dans $\mathbb{R}$, et une application canonique $\varphi_k : \mathcal{X} \to \mathcal{H}_k$, tels que :

1. Pour tout $x \in \mathcal{X}$, la fonction $k_x := k(x, \cdot)$ appartient à $\mathcal{H}_k$.
2. **Propriété de reproduction** : pour toute fonction $f \in \mathcal{H}_k$ et tout $x \in \mathcal{X}$,
$$f(x) = \langle f, k_x \rangle_{\mathcal{H}_k}. \tag{3.2.3}$$
3. Pour tous $x, x' \in \mathcal{X}$:
$$k(x, x') = \langle k_x, k_{x'} \rangle_{\mathcal{H}_k} = \langle \varphi_k(x), \varphi_k(x') \rangle_{\mathcal{H}_k} \quad \text{en posant } \varphi_k(x) = k_x. \tag{3.2.4}$$

L'espace $\mathcal{H}_k$ est appelé l'*espace de Hilbert à noyau reproduisant* (RKHS) associé à k.

Démonstration (Esquisse de preuve) La construction s'effectue en trois étapes principales :

1. **Espace pré-hilbertien** : on pose $\mathcal{H}_0 = \text{vect}\{k_x \mid x \in \mathcal{X}\}$. Pour deux combinaisons linéaires $f = \sum_{i=1}^{m} a_i k_{x_i}$ et $g = \sum_{j=1}^{m'} b_j k_{z_j}$, on munit $\mathcal{H}_0$ de la forme bilinéaire :
$$\langle f, g \rangle_{\mathcal{H}_0} := \sum_{i=1}^{m} \sum_{j=1}^{m'} a_i b_j k(x_i, z_j).$$
La symétrie et la positivité de k assurent que cette forme est symétrique et positive. De plus, par définition, $\langle f, k_x \rangle_{\mathcal{H}_0} = \sum_{i=1}^{m} a_i k(x_i, x) = f(x)$.
2. **Caractère défini** : d'après l'inégalité de Cauchy-Schwarz appliquée à cette forme semi-définie positive, pour tout $x \in \mathcal{X}$:
$$|f(x)| = |\langle f, k_x \rangle_{\mathcal{H}_0}| \leqslant \|f\|_{\mathcal{H}_0} \sqrt{k(x, x)}.$$
Ainsi, $\|f\|_{\mathcal{H}_0} = 0 \implies \forall x \in \mathcal{X},\ f(x) = 0$, donc f est identiquement nulle. L'espace $\mathcal{H}_0$ est donc un espace pré-hilbertien séparé.
3. **Complétion fonctionnelle** : on complète $\mathcal{H}_0$ en un espace de Hilbert $\mathcal{H}_k$. Grâce à la majoration ponctuelle issue de Cauchy-Schwarz, toute suite de Cauchy converge ponctuellement vers une fonction bien définie sur $\mathcal{X}$. Les éléments complétés restent donc de véritables fonctions, et la propriété de reproduction s'étend par continuité.

Nous avons donc l'équivalence : les noyaux semi-définis positifs sont exactement des produits scalaires de *feature maps*. Choisir un noyau k ou choisir un espace $(\mathcal{H}, \varphi)$ est donc la même chose, sauf que l'on sait explicitement calculer k. On peut donc complètement oublier $\mathcal{H}$ et φ dans le processus d'apprentissage.

Proposition (La recette) L'algorithme complet d'apprentissage supervisé à noyau se résume donc en 4 étapes :

- *Choix de k* : On définit un noyau k mesurant la "distance" de nos données en fonction de leur forme.
- *Matrice de Gram* : On calcule l'ensemble des paires $K_{i,j} = k(x_i, x_j)$ pour construire la matrice de Gram $K \in \mathbb{R}^{n \times n}$.
- *Optimisation* : On résout un problème d'optimisation pour trouver $\hat{\alpha}_S \in \mathbb{R}^n$:
$$\hat{\alpha}_S \in \underset{\alpha \in \mathbb{R}^n}{\arg\min} \frac{1}{n} \sum_{i=1}^{n} l\left(y_i, (K\alpha)_i\right) + \frac{\lambda}{2} \alpha^\top K \alpha. \tag{3.2.5}$$

- *Inférence* : Une fois trouvé, on put prédire le label de tout nouveau x par :

$$\hat{y} = \sum_{j=1}^{n} \hat{\alpha}_j k(x, x_j). \tag{3.2.6}$$

3.3 Séparateurs à Vaste Marge (SVM)

L'algorithme des Séparateurs à Vaste Marge (*Support Vector Machines*) illustre parfaitement la puissance des méthodes à noyaux. On commence par formuler le problème géométrique dans le cas linéaire élémentaire, avant d'en dériver la version duale et d'y injecter le *Kernel Trick*.

3.3.1 Cas linéairement séparable : Maximisation de la marge

Soit un échantillon d'apprentissage $S = ((x_1, y_1), \dots, (x_n, y_n)) \in (\mathbb{R}^d \times \{-1, 1\})^n$. On suppose dans un premier temps les données linéairement séparables : il existe un hyperplan passant par l'origine $H(\theta) = \{z \in \mathbb{R}^d \mid \theta^\top z = 0\}$ tel que pour tout $i \in [\![1, n]\!]$, $y_i(\theta^\top x_i) > 0$.

Proposition (Distance à l'hyperplan) Pour tout $\theta \neq 0$, la distance orthogonale d'un point $x \in \mathbb{R}^d$ à l'hyperplan $H(\theta)$ est donnée par :

$$\operatorname{dist}(x, H(\theta)) = \frac{|\theta^\top x|}{\|\theta\|_2}. \tag{3.3.1}$$

Sous l'hypothèse de séparabilité, $y_i(\theta^\top x_i) = |\theta^\top x_i|$. L'objectif consiste à trouver l'hyperplan qui maximise la distance minimale aux observations d'entraînement (la marge géométrique) :

$$\max_{\theta \neq 0} \min_{i \in [\![1,n]\!]} \frac{y_i(\theta^\top x_i)}{\|\theta\|_2}. \tag{3.3.2}$$

Le problème étant invariant par changement d'échelle $\theta \mapsto c\theta$ ($c > 0$), on impose la normalisation canonique $\min_i y_i(\theta^\top x_i) = 1$. Maximiser $1/\|\theta\|_2$ sous cette contrainte équivaut à minimiser $\frac{1}{2}\|\theta\|_2^2$. En relâchant l'égalité en une inégalité (qui sera saturée à l'optimum), on obtient le programme quadratique convexe :

$$\min_{\theta \in \mathbb{R}^d} \frac{1}{2}\|\theta\|_2^2 \quad \text{s.c.} \quad \forall i \in [\![1, n]\!], \quad y_i(\theta^\top x_i) \geqslant 1. \tag{3.3.3}$$

3.3.2 Cas non séparable : Variables d'écart et perte charnière

En pratique, les données se chevauchent ou comportent du bruit, rendant la séparation stricte impossible. Pour autoriser des erreurs tout en les contrôlant, on introduit des variables d'écart (*slack variables*) $\xi_i \geqslant 0$ pénalisées par un coefficient $C > 0$:

$$\min_{\theta \in \mathbb{R}^d,\, \xi \in \mathbb{R}^n} \frac{1}{2}\|\theta\|_2^2 + C \sum_{i=1}^{n} \xi_i \quad \text{s.c.} \quad \forall i \in [\![1, n]\!], \quad y_i(\theta^\top x_i) \geqslant 1 - \xi_i \quad \text{et} \quad \xi_i \geqslant 0. \tag{3.3.4}$$

Ce problème admet toujours des solutions admissibles dès que les ξ_i sont choisis suffisamment grands.

Proposition (Pont avec la minimisation du risque empirique) À θ fixé, l'objectif est strictement croissant en chaque variable ξ_i. À l'optimum, ξ_i atteint donc sa plus petite borne autorisée par les contraintes, soit :

$$\xi_i = \max(0, 1 - y_i(\theta^\top x_i)) = \big(1 - y_i(\theta^\top x_i)\big)_+. \tag{3.3.5}$$

En substituant cette expression, en divisant l'objectif par $nC > 0$ et en posant $\lambda := \frac{1}{nC}$, le problème géométrique coïncide rigoureusement avec un problème d'ERM régularisé pour la perte charnière (*hinge loss*) :

$$\min_{\theta \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^{n} \big(1 - y_i(\theta^\top x_i)\big)_+ + \frac{\lambda}{2}\|\theta\|_2^2. \tag{3.3.6}$$

3.3.3 Dualité de Wolfe et formulation duale

Le problème primal étant convexe avec des contraintes affines, les conditions de qualification de Slater sont vérifiées (par exemple pour $\theta = 0$ et $\xi_i = 2$), garantissant la dualité forte.
On introduit les multiplicateurs de Lagrange $\alpha_i \geqslant 0$ (pour les contraintes de marge) et $\beta_i \geqslant 0$ (pour la positivité de ξ_i) :

$$\mathcal{L}(\theta, \xi, \alpha, \beta) = \frac{1}{2}\|\theta\|_2^2 + C\sum_{i=1}^{n} \xi_i - \sum_{i=1}^{n} \alpha_i \left(y_i \theta^\top x_i + \xi_i - 1\right) - \sum_{i=1}^{n} \beta_i \xi_i. \tag{3.3.7}$$

Proposition Minimisons le Lagrangien :

- **En ξ :** Le Lagrangien s'écrit $\sum_{i=1}^{n}(C - \alpha_i - \beta_i)\xi_i + \dots$ L'infimum par rapport à $\xi_i \geqslant 0$ vaut $-\infty$ sauf si $C - \alpha_i - \beta_i = 0$. Cette condition permet d'éliminer $\beta_i \geqslant 0$, ce qui équivaut à la contrainte de boîte :
$$0 \leqslant \alpha_i \leqslant C \quad \forall i \in [\![1, n]\!]. \tag{3.3.8}$$
- **En θ :** La condition d'annulation du gradient $\nabla_\theta \mathcal{L} = \theta - \sum_{i=1}^{n} \alpha_i y_i x_i = 0$ impose :
$$\theta^* = \sum_{i=1}^{n} \alpha_i y_i x_i. \tag{3.3.9}$$

En réinjectant ces relations dans $\mathcal{L}$, le problème dual de Wolfe s'écrit sous forme d'un programme quadratique concave :

$$\max_{\alpha \in \mathbb{R}^n} \sum_{i=1}^{n} \alpha_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j (x_i^\top x_j) \quad \text{s.c.} \quad \forall i \in [\![1, n]\!], \quad 0 \leqslant \alpha_i \leqslant C. \tag{3.3.10}$$

Remarque Les données d'entrée x_i n'interviennent dans ce problème dual que par leurs produits scalaires $x_i^\top x_j$.

3.3.4 Le Kernel SVM (SVM à noyau)

Soit k un noyau semi-défini positif sur $\mathcal{X}$. En vertu du théorème d'Aronszajn, k définit implicitement un RKHS $\mathcal{H}_k$ et une application canonique φ_k tels que $k(x, x') = \langle \varphi_k(x), \varphi_k(x') \rangle_{\mathcal{H}_k}$.
On formule le problème primal directement dans $\mathcal{H}_k$:

$$\min_{\theta \in \mathcal{H}_k} \frac{1}{n} \sum_{i=1}^{n} \left(1 - y_i \langle \theta, \varphi_k(x_i) \rangle_{\mathcal{H}_k}\right)_+ + \frac{\lambda}{2}\|\theta\|_{\mathcal{H}_k}^2. \tag{3.3.11}$$

Grâce au *Kernel Trick*, on substitue directement chaque produit scalaire par l'évaluation du noyau $k(x_i, x_j) = K_{i,j}$ dans la formulation duale. En posant le changement de variable $\mu_i = \lambda \alpha_i$ (avec $\lambda = \frac{1}{nC}$, ce qui donne $0 \leqslant \mu_i \leqslant \frac{1}{n}$), le problème dual s'écrit sous forme matricielle :

$$\boxed{\max_{\mu \in \mathbb{R}^n} \sum_{i=1}^{n} \mu_i - \frac{1}{2\lambda}\sum_{i=1}^{n}\sum_{j=1}^{n} y_i y_j \mu_i \mu_j k(x_i, x_j) \quad \text{s.c.} \quad \forall i \in [\![1, n]\!], \quad 0 \leqslant \mu_i \leqslant \frac{1}{n}.} \tag{3.3.12}$$

Il s'agit d'un problème d'optimisation quadratique à n variables, dont la résolution numérique s'effectue uniquement à l'aide de la matrice de Gram $K \in \mathcal{S}_n^+(\mathbb{R})$.

Proposition (Règle de décision et vecteurs de support) D'après l'expression de θ^* et la propriété de reproduction dans le RKHS, la fonction de décision optimale pour classer un nouveau point $x \in \mathcal{X}$ est :

$$f(x) = \text{sign}\left(\sum_{i=1}^{n} \hat{\alpha}_i y_i k(x_i, x)\right). \tag{3.3.13}$$

D'après les conditions de complémentarité de Karush-Kuhn-Tucker (KKT), les coefficients $\hat{\alpha}_i$ sont strictement nuls pour tous les points situés en dehors de la marge. La décision ne dépend donc que des points vérifiant $\hat{\alpha}_i > 0$, appelés *vecteurs de support* :

$$f(x) = \text{sign}\left(\sum_{i \in \text{Support}} \hat{\alpha}_i y_i k(x_i, x)\right). \tag{3.3.14}$$

Ce résultat garantit la parcimonie du modèle et rend l'évaluation en phase d'inférence très rapide.

Chapitre 4

Théorie de l'Apprentissage et Minimisation du Risque Empirique

Contents

4.1 Convexification du risque et substituts convexes 145
4.1.1 Reparamétrisation et identité de marge 145
4.1.2 Risque de substitution et théorème de calibration 146
4.2 Décomposition du Risque et Classes Finies 146
4.2.1 Décomposition canonique de l'excès de risque 146
4.2.2 Majoration de l'erreur du risque – cas fini 147
4.2.3 Majoration de l'erreur du risque – cas infini 149
4.3 Complexité de Rademacher 150
4.3.1 Le procédé de symétrisation 151
4.3.2 Définitions et propriétés 151
4.3.3 Le retour du cas fini 152
4.3.4 Principe de contraction 154
4.3.5 Modèles linéaires contraints et dualité de normes 154
4.4 Théorèmes d'excès de risque pour les modèles linéaires 156
4.4.1 Borne en espérance 156
4.4.2 Borne à grande probabilité 157

Dans les chapitres précédents, nous avons présenté différentes méthodes d'apprentissage basées sur la minimisation du risque empirique (ERM, *Empirical Risk Minimization*). Or la seule analyse de la convergence de ces méthodes que nous ayons faite est la décomposition biais-variance du risque ainsi que la décomposition de son erreur en deux : l'erreur *d'approximation* et l'erreur *d'estimation*.

Dans la seconde partie de ce chapitre, nous nous attardons justement sur l'erreur faite sur le risque lors de l'apprentissage. Pour cela, nous développons une nouvelle théorie basée sur la complexité de l'espace de recherche $\mathcal{F}$.

Mais avant, nous développons le concept de *convexification du risque* dans la première partie. En effet, pour certaines fonction de pertes (notamment la *0-1 loss*) nous ne disposons pas de "bonnes" propriétés analytiques pour utiliser des algorithmes d'optimisation. Nous présentons ainsi une méthode permettant d'y remédier.

4.1 Convexification du risque et substituts convexes

4.1.1 Reparamétrisation et identité de marge

En classification binaire, on cherche à prédire une étiquette parmi deux classes $\mathcal{Y} = \{-1, 1\}$ via une fonction $f : \mathcal{X} \to \mathcal{Y}$. La mesure naturelle d'erreur est la perte 0-1 (*0-1 loss*) définie par :

$$l(y, \hat{y}) = \mathbb{1}_{y \neq \hat{y}}. \tag{4.1.1}$$

La minimisation exacte du risque empirique sous la perte 0-1 sur une classe d'hypothèses générale est un problème combinatoire NP-difficile. De plus, le risque empirique associé est une fonction constante par morceaux de ses paramètres : son gradient est nul presque partout, ce qui rend inapplicables les méthodes d'optimisation par descente de gradient.
Pour lever ce verrou, on reparamétrise le classifieur discret à l'aide d'une fonction à valeurs réelles.

Définition (**Prédicteur par fonction signe randomisée**) . Soit $g : \mathcal{X} \to \mathbb{R}$ une fonction réelle mesurable. On définit le classifieur binaire $f : \mathcal{X} \to \{-1, 1\}$ par $f(x) = \operatorname{sgn}(g(x))$ avec la convention :

$$\operatorname{sgn}(u) = \begin{cases} 1 & \text{si } u > 0, \\ -1 & \text{si } u < 0, \\ U \sim \operatorname{Unif}(\{-1, 1\}) & \text{si } u = 0, \end{cases} \tag{4.1.2}$$

où U est une variable aléatoire auxiliaire indépendante du couple d'observations $(X, Y) \sim \mathbb{P}$.

Cette reparamétrisation permet de découpler la *décision brute* (donnée par $\operatorname{sgn}(g(x))$) de la *confiance / distance à la frontière* (quantifiée par l'amplitude $|g(x)|$). On introduit alors la *marge fonctionnelle* $u = Yg(X)$:

- $u > 0$: la prédiction est correcte ($\operatorname{sgn}(g(X)) = Y$).
- $u < 0$: la prédiction est erronée ($\operatorname{sgn}(g(X)) \neq Y$).
- $u = 0$: le point se situe exactement sur la frontière de décision.

Proposition (Identité de marge pour la perte 0-1) Le risque théorique de classification binaire associé à $f = \operatorname{sgn} \circ g$ s'exprime comme l'espérance d'une fonction scalaire de la marge fonctionnelle $Yg(X)$:

$$R(f) = \mathbb{E}_{(X,Y)\sim\mathbb{P}} \left[\Phi_{0-1}(Yg(X)) \right], \tag{4.1.3}$$

avec la fonction de saut :

$$\Phi_{0-1}(u) = \mathbb{1}_{u<0} + \frac{1}{2}\mathbb{1}_{u=0}. \tag{4.1.4}$$

Démonstration En explicitant l'espérance conjointe sur (X, Y) et sur la variable de randomisation U :

$$\begin{aligned} R(f) &= \mathbb{E}_{(X,Y),U} \left[\mathbb{1}_{f(X)\neq Y} \right] \\ &= \mathbb{E} \left[\mathbb{1}_{g(X)\neq 0} \, \mathbb{1}_{f(X)\neq Y} \right] + \mathbb{E} \left[\mathbb{1}_{g(X)=0} \, \mathbb{1}_{f(X)\neq Y} \right] \\ &= \mathbb{P}(Yg(X) < 0) + \mathbb{E} \left[\mathbb{1}_{g(X)=0} \, \mathbb{1}_{U\neq Y} \right]. \end{aligned}$$

Comme U est indépendante de (X, Y) et uniformément distribuée sur $\{-1, 1\}$, $\mathbb{P}(U \neq Y \mid g(X) = 0) = \frac{1}{2}$. D'où :

$$R(f) = \mathbb{P}(Yg(X) < 0) + \frac{1}{2}\mathbb{P}(g(X) = 0) = \mathbb{E}_{(X,Y)\sim\mathbb{P}} \left[\Phi_{0-1}(Yg(X)) \right].$$

4.1.2 Risque de substitution et théorème de calibration

Dans le cas général, lorsque l'on fait face à une fonction de coût l non continue ou non convexe, on définit une fonction convexe Φ et une fonction g pour définir un risque de substitution ou *surrogate risk* $R_\Phi(g)$.

Définition (**Risque de substitution – Surrogate Risk**) . Soit $\Phi : \mathbb{R} \to [0, \infty[$ une fonction convexe. Le risque de substitution associé à Φ pour une fonction $g : \mathcal{X} \to \mathbb{R}$ est défini par :

$$R_\Phi(g) := \mathbb{E}_{(X,Y)\sim\mathbb{P}} \left[\Phi(Yg(X))\right]. \tag{4.1.5}$$

Exemples classiques :

- Perte charnière (Hinge) : $\Phi(u) = (1-u)_+$,
- Perte quadratique : $\Phi(u) = (1-u)^2$,
- Perte logistique : $\Phi(u) = \ln(1+e^{-u})$.

Or, nous ne sommes pas assurés qu'optimiser sur $R_\Phi(g)$ nous donne une solution optimale $\hat{f}$ pour notre risque initial $R(f)$. Le théorème de calibration ci-dessous nous donne une estimation de l'erreur commise.

Théorème (**Calibration – Zhang, Bartlett-Jordan-McAuliffe**) . Soit $\Phi : \mathbb{R} \to [0, \infty[$ une fonction convexe, dérivable en 0 avec $\Phi'(0) < 0$. Il existe une fonction convexe, non décroissante et continue $\psi : [0,1] \to [0, \infty[$ vérifiant $\psi(0) = 0$ telle que pour toute fonction mesurable $g : \mathcal{X} \to \mathbb{R}$:

$$\psi\left(R(\mathrm{sgn} \circ g) - R^*\right) \leqslant R_\Phi(g) - \inf_{g'} R_\Phi(g'). \tag{4.1.6}$$

En particulier :

- Pour la perte charnière : $\psi(t) = t$,
- Pour la perte quadratique : $\psi(t) = t^2$,
- Pour la perte logistique : $\psi(t) \geqslant \frac{t^2}{2}$.

4.2 Décomposition du Risque et Classes Finies

4.2.1 Décomposition canonique de l'excès de risque

Dans les chapitres précédents, nous avons décomposé le risque empirique en une erreur *d'approximation* (liée au choix de l'espace de fonction à optimiser $\mathcal{F}$) et une erreur *d'estimation* (liée à l'échantillon restreint S de taille n que l'on considère, purement statistique) :

$$R(\hat{f}_S) - R^* = \underbrace{\left(R(\hat{f}_S) - \inf_{f\in\mathcal{F}} R(f)\right)}_{\text{estimation}} + \underbrace{\left(\inf_{f\in\mathcal{F}} R(f) + R^*\right)}_{\text{approximation}}, \tag{4.2.1}$$

avec $\hat{f}_S$ le minimiseur du risque empirique sur $\mathcal{F}$. Soit $f^\dagger \in \mathcal{F}$ le meilleur prédicteur théorique possible dans $\mathcal{F}$. On a donc :

$$R(\hat{f}_S) - R^* = R(\hat{f}_S) - R(f^\dagger) + R(f^\dagger) - R^*. \tag{4.2.2}$$

Or ici, $\hat{f}_S$ est calculé sur un échantillon fixé S, on ne peut donc pas appliquer la LGN sur la taille de l'échantillon pour obtenir une convergence p.s vers $f^\dagger$. Pour séparer ce qui dépend des

données et ce qui dépend de l'estimation dans ce risque, on introduit les risques empiriques $\hat{R}_S(\hat{f}_S)$ et $\hat{R}_S(f^\dagger)$:

$$R(\hat{f}_S) - R(f^\dagger) = \left[R(\hat{f}_S) - \hat{R}_S(\hat{f}_S)\right] + \left[\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)\right] + \left[\hat{R}_S(f^\dagger) - R(f^\dagger)\right]. \tag{4.2.3}$$

En regardant chaque élément :

- $R(\hat{f}_S) - \hat{R}_S(\hat{f}_S)$ correspond à l'erreur commise sur le calcul du risque de l'estimateur restreint à l'échantillon S. Comme $\hat{f}_S \in \mathcal{F}$, on peut le majorer brutalement par le pire estimateur de la classe $\mathcal{F}$:

$$R(\hat{f}_S) - \hat{R}_S(\hat{f}_S) \leqslant \sup_{f\in\mathcal{F}} \left|R(f) - \hat{R}_S(f)\right|. \tag{4.2.4}$$

- $\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)$ correspond à l'erreur empirique du risque entre l'optimum calculé et le meilleur théorique,
- $\hat{R}_S(f^\dagger) - R(f^\dagger)$ correspond à l'erreur commise sur le calcul du risque sur l'échantillon S du meilleur estimateur théorique $f^\dagger$. De la même façon que pour le premier, on peut le majorer par le pire estimateur de $\mathcal{F}$:

$$\hat{R}_S(\hat{f}_S) - R(f^\dagger) \leqslant \sup_{f\in\mathcal{F}} \left|\hat{R}_S(f) - R(f)\right|. \tag{4.2.5}$$

Le terme du milieu $\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)$ correspond à l'erreur d'optimisation. Cette erreur est simplement due à la performance de notre algorithme ou à la puissance de notre unité de calcul. On peut donc la majorer arbitrairement par une constante ε_{opt} (qui dépend par exemple de la puissance de notre GPU, du temps de calcul, etc...).

Proposition (Décomposition canonique de l'excès de risque) Soit $\mathcal{F}$ une classe d'hypothèses, $\hat{f}_S \in \arg\min_{f\in\mathcal{F}} \hat{R}_S(f)$ le minimiseur empirique et $f^\dagger \in \arg\min_{f\in\mathcal{F}} R(f)$ le meilleur estimateur possible dans $\mathcal{F}$. L'erreur d'estimation se majore par :

$$\boxed{R(\hat{f}_S) - R(f^\dagger) \leqslant 2\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)| + \varepsilon_{\text{opt}}} \tag{4.2.6}$$

où $\varepsilon_{\text{opt}} = \hat{R}_S(\hat{f}_S) - \inf_{f\in\mathcal{F}} \hat{R}_S(f)$ représente l'erreur d'optimisation numérique (nulle si la minimisation est exacte).

On souhaite maintenant estimer (ou borner) l'erreur maximale que l'on peut faire dans $\mathcal{F}$ en estimant le risque $\hat{R}_S$ sur l'échantillon S : $\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)|$. On va pour cela distinguer deux cas : si $\mathcal{F}$ est une classe finie ou non.

4.2.2 Majoration de l'erreur du risque – cas fini

On suppose ici que $\mathcal{F}$ dispose d'un nombre fini d'éléments $|\mathcal{F}| = M < \infty$ et que les pertes sont bornées par l_∞. Pour majorer cette quantité, on utilise l'inégalité de Hoeffding.

Théorème (**Inégalité de Hoeffding**) . Soit $(X_k)_{1\leqslant k\leqslant n}$ une suite de variables aléatoires réelles *indépendantes* telles que pour deux suites $(a_k)_{1\leqslant k\leqslant n}$ et $(b_k)_{1\leqslant k\leqslant n}$, on ait :

$$\forall k \in [\![1, n]\!], \quad \mathbb{P}(a_k \leqslant X_k \leqslant b_k) = 1. \tag{4.2.7}$$

Alors, pour tout $t \geqslant 0$, on peut minorer la concentration de $S_n = X_1 + \cdots + X_n$ autour de sa moyenne en probabilité :

$$\mathbb{P}(|S_n - \mathbb{E}(S_n)| \geqslant t) \leqslant 2\exp\left(-\frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2}\right). \tag{4.2.8}$$

Proposition Or ici, pour toute fonction $f \in \mathcal{F}$ fixée indépendante de l'échantillon S, la perte sur chaque observation $Z_i = l(y_i, f(x_i))$ est une variable aléatoire réelle telle que :

1. Les $(Z_i)_{1\leqslant i\leqslant n}$ sont i.i.d.
2. Pour tout $i \in [\![1, n]\!]$, on a $Z_i \in [0, l_\infty]$.
3. La moyenne empirique est : $\hat{R}_S(f) = \frac{1}{n}\sum_{i=1}^n Z_i$.
4. L'espérance est : $\mathbb{E}[Z_i] = R(f)$.

On peut donc appliquer l'inégalité de Hoeffding sur la moyenne empirique $\hat{R}_S(f)$ avec $a_k = 0$ et $b_k = l_\infty/n$ pour tout $k \in [\![1, n]\!]$. D'où :

$$\forall t \geqslant 0, \quad \mathbb{P}(|R(f) - \hat{R}_S(f)| \geqslant t) \leqslant 2\exp\left(-\frac{2nt^2}{l_\infty^2}\right). \tag{4.2.9}$$

Sachant contrôler chaque estimateur de la classe $\mathcal{F}$ indépendamment, on souhaite maintenant les contrôler tous [1]. Pour contrôler simultanément tous les estimateurs de la classe, on applique l'inégalité de Boole (*Union Bound*) [2] sur les $M = |\mathcal{F}|$ éléments :

$$\begin{aligned}\mathbb{P}\left(\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)| \geqslant t\right) &\leqslant \sum_{f\in\mathcal{F}} \mathbb{P}\left(|\hat{R}_S(f) - R(f)| \geqslant t\right) \\ &\leqslant 2|\mathcal{F}|\exp\left(-\frac{2nt^2}{l_\infty^2}\right).\end{aligned}$$

En posant cette probabilité d'erreur égale à $\delta \in]0, 1[$, on obtient le seuil d'écart $t = l_\infty\sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}}$.

Théorème (**Borne uniforme pour les classes finies**) . Sous les hypothèses $l(y, f(x)) \in [0, l_\infty]$ et $|\mathcal{F}| < \infty$, pour tout $\delta \in]0, 1[$, avec probabilité au moins $1 - \delta$ sur le tirage de $S \sim \mathbb{P}^{\otimes n}$:

$$\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)| \leqslant l_\infty\sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}}. \tag{4.2.10}$$

Par conséquent, pour le minimiseur empirique exact $\hat{f}_S$, on a avec probabilité au moins $1 - \delta$:

$$R(\hat{f}_S) \leqslant \inf_{f\in\mathcal{F}} R(f) + 2l_\infty\sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}} + \varepsilon_{\text{opt}}. \tag{4.2.11}$$

Remarque Le résultat est immédiat ici. Si les hypothèses sont vérifiées, nous sommes capables de borner le risque. En revanche, si $|\mathcal{F}| = \infty$, les bornes de celui-ci explosent. Nous devons donc décrire une nouvelle théorie pour traiter ce cas.

1. "Un anneau unique pour les contrôler tous".
2. Cette inégalité découle des propriétés des mesures. Elle affirme que pour une famille au plus dénombrable de partie mesurables $A_1, A_2, A_3, \ldots$, pour toute mesure μ, on a : $\mu\left(\bigcup_n A_n\right) \leqslant \sum_n \mu(A_n)$.

4.2.3 Majoration de l'erreur du risque – cas infini

Analysons le résultat précédent : on calcule la probabilité que $\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)| \geqslant t$ par une somme finie d'unions d'événements sur $\mathcal{F}$. C'est donc de là que la borne explose si $\mathcal{F}$ est infini.
La solution est donc de traiter la classe entière de fonctions $\mathcal{F}$ en considérant une application g définie sur l'échantillon S qui lui associe l'erreur la plus importante possible sur la classe de fonctions $\mathcal{F}$:

$$g(S) := \sup_{f\in\mathcal{F}} \left(\hat{R}_S(f) - R(f)\right). \tag{4.2.12}$$

C'est donc maintenant l'échantillon qui définit la variation d'erreur. On souhaite toujours la borner : on introduit donc la *méthode des différences bornées*.

Définition (**Différences Bornées**) . Soit $f : Z_1 \times \cdots \times Z_n \longrightarrow \mathbb{R}$ une application. On dit que f vérifie la *condition des différences bornées* pour les constantes $(c_1, \ldots, c_n) \in \mathbb{R}^n$ si pour tout $i \in [\![1, n]\!]$ on a :

$$\sup_{z_1,\ldots,z_i,\ldots,z_n,z_i'} |f(z_1, \ldots, z_i, \ldots, z_n) - f(z_1, \ldots, z_i', \ldots, z_n)| \leqslant c_i. \tag{4.2.13}$$

Autrement dit, si l'on change la ième entrée z_i de f en z_i', sa sortie ne bouge pas plus de c_i, et cela pour tout i.

Proposition (Différences bornées sur g) Appliquons les différences bornées sur l'application g. Puisque pour tout $f \in \mathcal{F}$, le risque $R(f)$ ne dépend pas des données S, on a :

$$\hat{R}_S(f) - \hat{R}_{S'}(f) = \frac{1}{n}\left(l(z_i, f) - l(z_i', f)\right). \tag{4.2.14}$$

Or, par hypothèse, la perte directe est contenue dans $[0, l_\infty]$. Nous devons passer au sup pour conclure sur g.

Lemme (Passage au sup) Soient $u, v : \mathcal{F} \longrightarrow \mathbb{R}$ deux applications. Pour tout $f \in \mathcal{F}$, on a l'implication suivante :

$$|u(f) - v(f)| \leqslant c \longrightarrow \left|\sup_{f\in\mathcal{F}} u(f) - \sup_{f\in\mathcal{F}} v(f)\right| \leqslant c. \tag{4.2.15}$$

En appliquant le lemme précédent avec $c = \frac{l_\infty}{n}$, on en déduit que pour tout $i \in [\![1, n]\!]$:

$$|g(S) - g(S')| = \left|\sup_{f\in\mathcal{F}}(\hat{R}_S(f) - R(f)) - \sup_{f\in\mathcal{F}}(\hat{R}_{S'}(f) - R(f))\right| \leqslant \frac{l_\infty}{n}. \tag{4.2.16}$$

La fonction g vérifie donc la propriété des différences bornées avec les constantes coordonnées :

$$c_i = \frac{l_\infty}{n} \quad \text{pour tout } i \in [\![1, n]\!]. \tag{4.2.17}$$

Théorème (**Inégalité de McDiarmid**) . Soient $Z_1, \dots, Z_n$ des variables aléatoires indépendantes à valeurs dans un espace mesurable $\mathcal{Z}$. Soit $\phi : \mathcal{Z}^n \to \mathbb{R}$ une fonction vérifiant l'hypothèse des différences bornées pour des constantes $(c_1, \dots, c_n) \in \mathbb{R}_+^n$. Alors, pour tout $t \geqslant 0$:

$$\mathbb{P}\left(\phi(S) - \mathbb{E}_S[\phi(S)] \geqslant t\right) \leqslant \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right), \tag{4.2.18}$$

$$\mathbb{P}\left(\mathbb{E}_S[\phi(S)] - \phi(S) \geqslant t\right) \leqslant \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right), \tag{4.2.19}$$

$$\mathbb{P}\left(|\phi(S) - \mathbb{E}_S[\phi(S)]| \geqslant t\right) \leqslant 2\exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right). \tag{4.2.20}$$

On décompose la déviation uniforme bilatérale en deux déviations unilatérales :

$$\sup_{f\in\mathcal{F}} |\hat{R}_S(f) - R(f)| \leqslant \sup_{f\in\mathcal{F}} (\hat{R}_S(f) - R(f)) + \sup_{f\in\mathcal{F}} (R(f) - \hat{R}_S(f)). \tag{4.2.21}$$

Puisque chacune de ces deux applications admet les constantes de variations coordonnées $c_i = \frac{l_\infty}{n}$, la somme des carrés s'évalue explicitement par :

$$\sum_{i=1}^n c_i^2 = \sum_{i=1}^n \left(\frac{l_\infty}{n}\right)^2 = \frac{l_\infty^2}{n}. \tag{4.2.22}$$

L'application de la forme unilatérale de McDiarmid fournit alors le contrôle de concentration suivant :

Proposition (Concentration de la déviation uniforme unilatérale) Sous l'hypothèse d'une perte bornée $l(y, f(x)) \in [0, l_\infty]$, pour tout $\delta \in]0, 1[$, chacune des inégalités suivantes est vérifiée avec probabilité au moins $1 - \delta$ sur le tirage de $S \sim \mathbb{P}^{\otimes n}$:

$$\sup_{f\in\mathcal{F}} (\hat{R}_S(f) - R(f)) \leqslant \mathbb{E}_S\left[\sup_{f\in\mathcal{F}} (\hat{R}_S(f) - R(f))\right] + l_\infty \sqrt{\frac{\ln(1/\delta)}{2n}}, \tag{4.2.23}$$

$$\sup_{f\in\mathcal{F}} (R(f) - \hat{R}_S(f)) \leqslant \mathbb{E}_S\left[\sup_{f\in\mathcal{F}} (R(f) - \hat{R}_S(f))\right] + l_\infty \sqrt{\frac{\ln(1/\delta)}{2n}}. \tag{4.2.24}$$

Remarque Ce résultat marque une avancée théorique majeure : la déviation uniforme se concentre à une vitesse en $\mathcal{O}(1/\sqrt{n})$ autour de son espérance, *sans aucune hypothèse de finitude ou de structure* sur la classe $\mathcal{F}$.
Toutefois, le problème n'est pas encore résolu : la déviation se concentre autour d'une espérance théorique inconnue $\mathbb{E}_S\left[\sup_{f\in\mathcal{F}}(\hat{R}_S(f) - R(f))\right]$. Si la classe $\mathcal{F}$ est arbitrairement riche (par exemple l'ensemble de toutes les fonctions mesurables), cette espérance vaut identiquement 1 pour tout n. L'objectif de la section suivante consiste donc à borner cette espérance résiduelle à l'aide de la *symétrisation* et de la *complexité de Rademacher*.

4.3 Complexité de Rademacher

Notation Pour alléger les calculs, on note $z = (x, y)$ et $z_i = (x_i, y_i)$ tirés selon la loi $\mathbb{P}$. Pour tout prédicteur $f \in \mathcal{F}$ et toute fonction de perte $l : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$, on associe la fonction de perte induite $h_f : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ définie par $h_f(z) = l(y, f(x))$. On étudie ainsi la classe de fonctions :

$$\mathcal{H} = l \circ \mathcal{F} = \{z \mapsto l(y, f(x)) \mid f \in \mathcal{F}\}. \tag{4.3.1}$$

Le risque empirique devient $\hat{R}_S(f) = \frac{1}{n}\sum_{i=1}^n h(z_i)$ et le risque théorique $R(f) = \mathbb{E}_z[h(z)]$.

La section précédente a établi que la déviation uniforme se concentre autour de son espérance théorique :

$$\mathbb{E}_S\left[\sup_{f\in\mathcal{F}}(R(f)-\hat{R}_S(f))\right]=\mathbb{E}_S\left[\sup_{h\in\mathcal{H}}\left(\mathbb{E}_z[h(z)]-\frac{1}{n}\sum_{i=1}^{n}h(z_i)\right)\right]. \tag{4.3.2}$$

Le processus de *symétrisation* permet de majorer cette espérance par une quantité calculable à partir de perturbations aléatoires de l'échantillon.

4.3.1 Le procédé de symétrisation

Proposition (Symétrisation par échantillon fantôme) Soit $S'=(z'_1,\dots,z'_n)\sim\mathbb{P}^{\otimes n}$ un second échantillon (dit *échantillon fantôme*) indépendant de S et de même loi. Par linéarité de l'espérance :

$$\mathbb{E}_z[h(z)]=\mathbb{E}_{S'}\left[\frac{1}{n}\sum_{i=1}^{n}h(z'_i)\right]. \tag{4.3.3}$$

Le premier échantillon S étant indépendant de S', on regroupe les termes :

$$\mathbb{E}_z[h(z)]-\frac{1}{n}\sum_{i=1}^{n}h(z_i)=\mathbb{E}_{S'}\left[\frac{1}{n}\sum_{i=1}^{n}\big(h(z'_i)-h(z_i)\big)\right]. \tag{4.3.4}$$

Par convexité de la fonction supremum (inégalité de Jensen, $\sup\mathbb{E}\leqslant\mathbb{E}\sup$), puis en intégrant par rapport à S :

$$\mathbb{E}_S\left[\sup_{h\in\mathcal{H}}\left(\mathbb{E}_z[h(z)]-\frac{1}{n}\sum_{i=1}^{n}h(z_i)\right)\right]\leqslant\mathbb{E}_{S,S'}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}\big(h(z'_i)-h(z_i)\big)\right]. \tag{4.3.5}$$

Proposition (Injection des variables de Rademacher) Posons $d_i=h(z'_i)-h(z_i)$. Comme z_i et z'_i sont i.i.d., $d_i\overset{\mathcal{L}}{=}-d_i$. Soient $\varepsilon_1,\dots,\varepsilon_n$ des variables de Rademacher indépendantes de S,S' telles que $\mathbb{P}(\varepsilon_i=1)=\mathbb{P}(\varepsilon_i=-1)=\frac{1}{2}$. Alors $(d_1,\dots,d_n)\overset{\mathcal{L}}{=}(\varepsilon_1d_1,\dots,\varepsilon_nd_n)$ pour tout $h\in\mathcal{H}$ conjointement :

$$\begin{aligned}\mathbb{E}_{S,S'}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}d_i\right]&=\mathbb{E}_{S,S',\varepsilon}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}\varepsilon_i\big(h(z'_i)-h(z_i)\big)\right]\\&\leqslant\mathbb{E}_{S',\varepsilon}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}\varepsilon_ih(z'_i)\right]+\mathbb{E}_{S,\varepsilon}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}(-\varepsilon_i)h(z_i)\right].\end{aligned}$$

Comme $-\varepsilon_i\overset{\mathcal{L}}{=}\varepsilon_i$, les deux termes sont égaux et l'échantillon fantôme s'élimine :

$$\mathbb{E}_S\left[\sup_{h\in\mathcal{H}}\left(\mathbb{E}_z[h(z)]-\frac{1}{n}\sum_{i=1}^{n}h(z_i)\right)\right]\leqslant2\,\mathbb{E}_{S,\varepsilon}\left[\sup_{h\in\mathcal{H}}\frac{1}{n}\sum_{i=1}^{n}\varepsilon_ih(z_i)\right]. \tag{4.3.6}$$

4.3.2 Définitions et propriétés

Les calculs précédents nous ont introduit une nouvelle grandeur (voir Équation 4.3.6) définissant la *complexité de Rademacher*.

Définition (**Complexité de Rademacher**) . Soit $\mathcal{H}$ une classe de fonctions à valeurs réelles et $S = (z_1, \dots, z_n)$. On définit la *complexité de Rademacher empirique* de l'espace $\mathcal{H}$ sous S par :

$$\mathfrak{R}_S(\mathcal{H}) = \mathbb{E}_\varepsilon \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i h(z_i) \right]. \tag{4.3.7}$$

Plus généralement, lorsque $\mathcal{H}$ dépend d'une loi de probabilité $\mathbb{P}$, on définit la complexité de Rademacher par :

$$\mathfrak{R}_n(\mathcal{H}) = \mathbb{E}_{S \sim \mathbb{P}^{\otimes n}} \left[\mathfrak{R}_S(\mathcal{H}) \right] \tag{4.3.8}$$

où les $\varepsilon_i \in \{-1, 1\}$.

Cette complexité traduit la capacité de l'espace de fonction $\mathcal{H}$ à approximer données S bruitées par les ε_i. Une classe pour laquelle c'est le cas a une complexité de l'ordre de $\sup_h \sup_i |h(z_i)|$, dans le cas contraire, sa complexité de Rademacher tends vers 0 à mesure que n augmente. Nous pouvons donc conclure notre procédé de symétrisation par le théorème suivant.

Théorème (**Symétrisation**) . Soit $\mathcal{H}$ une classe de fonctions, une famille de valeurs $z_1, \dots, z_n$ et une famille de coefficients $\varepsilon_i \in \{-1, 1\}^n$. On a le résultat suivant :

$$\mathbb{E}_S \left[\sup_{h \in \mathcal{H}} \left(\frac{1}{n} \sum_{i \leqslant n} h(z_i) - \mathbb{E}_z[h(z)] \right) \right] \leqslant 2\mathfrak{R}_n(\mathcal{H}) \tag{4.3.9}$$

$$\mathbb{E}_S \left[\sup_{h \in \mathcal{H}} \left(\mathbb{E}_z[h(z)] - \frac{1}{n} \sum_{i \leqslant n} h(z_i) \right) \right] \leqslant 2\mathfrak{R}_n(\mathcal{H}) \tag{4.3.10}$$

Remarque Le second résultat est issu du même processus de symétrisation appliqué à la classe de fonctions $-\mathcal{H} = \{-h \mid h \in \mathcal{H}\}$.

Définissons maintenant quelques propiétés calculatoires de la complexité de Rademacher.

Propriété (**Complexité de Rademacher**) . Soient $\mathcal{H}$ une classe de fonctions, $h_0 \in \mathcal{H}$ et $c \in \mathbb{R}$, on a :

- *Homogénéité* : $\mathfrak{R}_S(c\mathcal{H}) = |c|\mathfrak{R}_S(\mathcal{H})$.
- *Translation* : $\mathfrak{R}_S(\mathcal{H} + h_0) = \mathfrak{R}_S(\mathcal{H})$.
- *Enveloppe convexe* : $\mathfrak{R}_S(\operatorname{conv} \mathcal{H}) = \mathfrak{R}_S(\mathcal{H})$.

Démonstration

1. Par linéarité de l'opérateur d'application, de la somme, du sup et de l'espérance. Le cas $c < 0$ est vérifié par le fait que l'application de la fonction $\varepsilon \longmapsto -\varepsilon$ ne change pas la loi des variables aléatoires et donc la valeur de l'espérance.
2. On observe que $\sup_h \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i (h + h_0)(z_i) = \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i h_0(z_i) + \sup_h \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i h(z_i)$. Or la moyenne sur le premier terme s'annule du fait de la distribution uniforme des ε_i sur $\{-1, 1\}$.
3. Pour ε fixé, l'application $h \longmapsto \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i h(z_i)$ est linéaire et donc maximisée sur un point extrême de l'enveloppe convexe de $\mathcal{H}$. Point qui est déjà inclut dans le sup.

4.3.3 Le retour du cas fini

On souhaite ici appliquer les résultats théoriques précédents sur la complexité de Rademacher au cas fini vu en début de chapitre. Pour cela, on défini pour chaque fonction $h \in \mathcal{H}$ un vecteur

a_h tel que :

$$a_h = (h(z_1), \dots, h(z_n))^\top \in \mathbb{R}^n. \tag{4.3.11}$$

Si $\mathcal{H}$ est fini, cet ensemble de vecteurs forme un sous-ensemble fini A de $\mathbb{R}^n$ tel que$A = \{a_h \mid h \in \mathcal{H}\} \subset \mathbb{R}^n$. On peut donc observer l'égalité entre la complexité de Rademacher $\mathfrak{R}_S(\mathcal{H})$ de $\mathcal{H}$ et le maximum de la moyenne des a_i pondérée par les ε_i pour tout $a \in A$:

$$\sup_{h\in\mathcal{H}} \frac{1}{n} \sum_{i\leqslant n} \varepsilon_i h(z_i) = \max_{a\in A} \frac{1}{n} \sum_{i\leqslant n} \varepsilon_i a_i. \tag{4.3.12}$$

Développons un peu de théorie pour borner le terme de gauche à l'aide du lemme de Massart.

Lemme (Massart) Soit $A \subset \mathbb{R}^n$ fini, non vide et $\rho = \max_{a\in A} \|a\|_2$. Alors on a :

$$\mathbb{E}_\varepsilon \left[\max_{a\in A} \frac{1}{n} \sum_{i\leqslant n} \varepsilon_i a_i \right] \leqslant \frac{\rho\sqrt{2\ln|A|}}{n}. \tag{4.3.13}$$

Pour la démonstration de ce lemme, on s'appuie sur le lemme de Hoeffding ci-dessous.

Lemme (Hoeffding) Pour toute variable aléatoire réelle X centrée (i.e $\mathbb{E}[X] = 0$) et bornée $X \in [u, v]$ p.s, alors :

$$\forall \lambda \in \mathbb{R}, \quad \mathbb{E}\left[e^{\lambda X}\right] \leqslant \exp\left(\frac{\lambda^2 (v-u)^2}{8}\right). \tag{4.3.14}$$

On dit alors que X est *sous-gaussienne* de paramètre $\sigma^2 = \frac{(v-u)^2}{4}$. On notera alors $X \sim \mathcal{G}(\sigma^2)$.

Démonstration Soit $A \subset \mathbb{R}^n$ borné par $\rho = \max\limits_{a\in A} \|a\|_2$. Pour tout $a = (a_1, \dots, a_n)^\top \in A$, on pose $X_i = \varepsilon_i a_i$ où les ε_i sont les variables de Rademacher.
On remarque que :

$$\mathbb{E}(X_i) = a_i \mathbb{E}(\varepsilon_i) = 0, \quad \text{et} \quad X_i \in [-|a_i|, |a_i|]. \tag{4.3.15}$$

D'après le lemme de Hoeffding, on a donc que $X_i \sim \mathcal{G}(\sigma^2)$ avec $\sigma^2 = \frac{4a_i^2}{4} = a_i^2$, d'où :

$$\forall \lambda \in \mathbb{R}, \quad \mathbb{E}_\varepsilon\left[e^{\lambda \varepsilon_i a_i}\right] \leqslant \exp\left(\frac{\lambda^2 a_i^2}{2}\right). \tag{4.3.16}$$

Or les ε_i sont deux à deux indépendants, donc en posant $W_a = \sum_{i\leqslant n} \varepsilon_i a_i$, on a :

$$\begin{aligned}
\mathbb{E}_\varepsilon\left[e^{\lambda W_a}\right] = \mathbb{E}_\varepsilon\left[\prod_{i\leqslant n} e^{\lambda \varepsilon_i a_i}\right] &= \prod_{i\leqslant n} \mathbb{E}_\varepsilon\left[e^{\lambda \varepsilon_i a_i}\right] \\
&\leqslant \prod_{i\leqslant n} \exp\left(\frac{\lambda^2 a_i^2}{2}\right) = \exp\left(\frac{\lambda^2 \|a\|_2^2}{2}\right).
\end{aligned}$$

Or, par construction, $\|a\|_2^2 \leqslant \rho^2$, donc $W_a \sim \mathcal{G}(\rho^2)$. On peut donc appliquer successivement l'inégalité de Jensen et poser $\lambda = \sqrt{2\ln|A|}/\rho$:

$$\begin{aligned}
e^{\lambda \mathbb{E}[\max_{a\in A} W_a]} \leqslant \mathbb{E}\left[e^{\lambda \max_{a\in A} W_a}\right] &\leqslant \sum_{a\in A} \mathbb{E}\left[e^{\lambda W_a}\right] \leqslant |A| \exp\left(\frac{\lambda^2\rho^2}{2}\right) \\
&\iff \mathbb{E}\left[\max_{a\in A} W_a\right] \leqslant \frac{\ln|A|}{\lambda} + \frac{\lambda\rho^2}{2} \\
&\iff \mathbb{E}_\varepsilon\left[\max_{a\in A} \frac{1}{n} \sum_{i\leqslant n} \varepsilon_i a_i\right] \leqslant \frac{\rho\sqrt{2\ln|A|}}{n}.
\end{aligned}$$

Proposition En appliquant le lemme de Massart sur le contexte présenté en début de sous-section, on obtient l'inégalité suivante :

$$\mathbb{E}_\varepsilon \left[\max_{a \in A} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i a_i \right] \leqslant \frac{\rho \sqrt{2 \ln |A|}}{n}. \tag{4.3.17}$$

Pour faire le lien avec la complexité de Rademacher, il suffit de prendre l'espérance sur 4.3.12 et on retrouve le résultat énoncé pour le cas fini en 4.2.2 :

$$\mathfrak{R}_S(\mathcal{H}) = \mathbb{E}_\varepsilon \left[\max_{a \in A} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i a_i \right] = \mathbb{E}_\varepsilon \left[\max_{a \in A} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i a_i \right] \leqslant \frac{\rho \sqrt{2 \ln |A|}}{n}. \tag{4.3.18}$$

4.3.4 Principe de contraction

Les résultats présentés jusqu'à maintenant sont très forts pour donner une borne supérieure de l'erreur d'approximation que l'on fait en choisissant une classe de fonction $\mathcal{F}$. Or, si l'on regarde bien l'expression de la complexité de Rademacher, on s'apperçoit que celle-ci dépend de la fonction de *loss* utilisée :

$$\mathfrak{R}_n(\mathcal{H}) = \mathfrak{R}_n(l \circ \mathcal{F}) = \mathbb{E}_{S,\varepsilon} \left[\sup_{f \in \mathcal{F}} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i l(y_i, f(x_i)) \right]. \tag{4.3.19}$$

Or, on souhaiterait être capable de calculer cette complexité indépendamment du processus d'apprentissage, et donc de la fonction de perte l choisie. C'est le principe de contraction qui l'assure.

Proposition (Principe de contraction) Soit $A \subset \mathbb{R}^n$ et $g_1, \ldots, g_n : \mathbb{R} \longrightarrow \mathbb{R}$ des applications G-lipschitziennes. On a alors :

$$\mathbb{E}_\varepsilon \left[\sup_{a \in A} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i g_i(a_i) \right] \leqslant G \cdot \mathbb{E}_\varepsilon \left[\sup_{a \in A} \frac{1}{n} \sum_{i \leqslant n} \varepsilon_i (a_i) \right]. \tag{4.3.20}$$

Si l'on fait l'hypothèse simple que la fonction de perte utilisée $l(y_i, \cdot)$ est G-lipschitzienne, on a alors que :

$$\mathfrak{R}_S(l \circ \mathcal{F}) \leqslant G \cdot \mathfrak{R}_S(\mathcal{F}), \quad \text{donc} \quad \mathfrak{R}_n(l \circ \mathcal{F}) \leqslant G \cdot \mathfrak{R}_n(\mathcal{F}) \tag{4.3.21}$$

pour un domaine $A = \{(f(x_1), \ldots, f(x_n)) \mid f \in \mathcal{F}\}$.

Remarque Ce principe de contraction permet donc de séparer complètement les erreurs d'apprentissage dues à l'échantillon de celles dues à l'approximation faite par le choix de la classe $\mathcal{F}$. Ainsi, grâce à la contraction, la borne d'excès de risque sépare les aspects du problème en trois parties : l'optimisation numérique et la régularité du critère sont prises en charge par la constante G et la *loss* l, la géométrie de l'architecture d'apprentissage est entièrement contenue dans $\mathfrak{R}_n(\mathcal{F})$, et les fluctuations statistiques de l'échantillon sont, quant à elles, contenues dans le terme de McDiarmid en $\mathcal{O}(1/\sqrt{n})$.

4.3.5 Modèles linéaires contraints et dualité de normes

On applique les résultats précédents à des prédicteurs linéaires sous contrainte de régularisation :

$$\mathcal{F} = \{f_\theta : x \longmapsto \theta^\top \varphi(x) \mid \Omega(\theta) \leqslant D\}, \tag{4.3.22}$$

où $\varphi : \mathcal{X} \to \mathbb{R}^d$ est une *feature map*, Ω une norme sur $\mathbb{R}^d$ et $D > 0$ le rayon de la boule de contrainte. On rappelle que la *norme duale* Ω^* de Ω est définie par :

$$\Omega^*(u) = \sup_{\Omega(\theta) \leqslant 1} \langle \theta, u \rangle. \tag{4.3.23}$$

La complexité de Rademacher de cette classe se réduit à l'espérance de la norme duale d'une somme de vecteurs symétriques :

$$\begin{aligned}
\mathfrak{R}_n(\mathcal{F}) &= \mathbb{E}_{S,\varepsilon}\left[\sup_{\Omega(\theta)\leqslant D} \frac{1}{n}\sum_{i\leqslant n} \varepsilon_i \theta^\top \varphi(x_i)\right] \\
&= \mathbb{E}_{S,\varepsilon}\left[\sup_{\Omega(\theta)\leqslant D} \frac{1}{n}\theta^\top (\Phi^\top \varepsilon)\right] \\
&= \frac{D}{n}\mathbb{E}_{S,\varepsilon}\left[\Omega^*(\Phi^\top \varepsilon)\right],
\end{aligned}$$

où $\Phi \in \mathbb{R}^{n\times d}$ est la matrice dont la i-ème ligne est $\varphi(x_i)^\top$, et $\Phi^\top \varepsilon = \sum_{i\leqslant n} \varepsilon_i \varphi(x_i) \in \mathbb{R}^d$.

Proposition (Cas euclidien ℓ_2) On pose $\Omega = \|\cdot\|_2$, auto-duale ($\Omega^* = \|\cdot\|_2$). Par concavité de la racine carrée (inégalité de Jensen) et cyclicité de la trace :

$$\begin{aligned}
\mathfrak{R}_n(\mathcal{F}) &= \frac{D}{n}\mathbb{E}\left[\|\Phi^\top \varepsilon\|_2\right] \\
&\leqslant \frac{D}{n}\sqrt{\mathbb{E}\left[\|\Phi^\top \varepsilon\|_2^2\right]} \\
&= \frac{D}{n}\sqrt{\mathbb{E}\left[\operatorname{tr}\left(\Phi\Phi^\top \varepsilon\varepsilon^\top\right)\right]} \\
&= \frac{D}{n}\sqrt{\mathbb{E}_S\left[\operatorname{tr}(\Phi\Phi^\top)\right]} \qquad \text{car } \mathbb{E}_\varepsilon[\varepsilon\varepsilon^\top] = I_n \text{ et } \varepsilon \perp S \\
&= \frac{D}{n}\sqrt{\sum_{i=1}^n \mathbb{E}_x\left[\|\varphi(x)\|_2^2\right]} \\
&= \frac{D}{\sqrt{n}}\sqrt{\mathbb{E}_x\left[\|\varphi(x)\|_2^2\right]}.
\end{aligned}$$

La dimension ambiante d a disparu : seule compte l'énergie moyenne des descripteurs $\mathbb{E}[\|\varphi(x)\|_2^2]$ et le rayon D. Le résultat reste donc valide en dimension infinie ($d = \infty$), notamment pour les méthodes à noyaux (RKHS).

Proposition (Cas parcimonieux ℓ_1 – Lasso) On pose $\Omega = \|\cdot\|_1$, dont la norme duale est $\Omega^* = \|\cdot\|_\infty$. On suppose que les descripteurs sont bornés composante par composante : $|\varphi_j(x)| \leqslant B_\infty$ pour tout $j \in [\![1, d]\!]$ et tout $x \in \mathcal{X}$. On a :

$$\mathfrak{R}_n(\mathcal{F}) = \frac{D}{n}\mathbb{E}_{S,\varepsilon}\left[\|\Phi^\top \varepsilon\|_\infty\right] = \frac{D}{n}\mathbb{E}_{S,\varepsilon}\left[\max_{j\leqslant d}\left|\sum_{i\leqslant n} \varepsilon_i \varphi_j(x_i)\right|\right]. \tag{4.3.24}$$

Puisque $|u| = \max\{u, -u\}$, on ramène ce terme au maximum sur l'ensemble fini $A \subset \mathbb{R}^n$ des $2d$ vecteurs colonnes orientés :

$$A = \left\{\pm\left(\varphi_j(x_1), \ldots, \varphi_j(x_n)\right)^\top \mid j \leqslant d\right\} \subset \mathbb{R}^n. \tag{4.3.25}$$

Ainsi, $\max_{j\leqslant d}|\sum_{i\leqslant n}\varepsilon_i\varphi_j(x_i)| = \max_{a\in A}\sum_{i\leqslant n}\varepsilon_i a_i$. On a $|A|\leqslant 2d$, et pour tout $a\in A$:

$$\|a\|_2^2 = \sum_{i\leqslant n}\varphi_j(x_i)^2 \leqslant nB_\infty^2 \implies \rho = \max_{a\in A}\|a\|_2 \leqslant B_\infty\sqrt{n}. \tag{4.3.26}$$

En appliquant le lemme de Massart conditionnellement à S :

$$\frac{1}{n}\mathbb{E}_\varepsilon\left[\max_{a\in A}\sum_{i\leqslant n}\varepsilon_i a_i\right] \leqslant \frac{\rho\sqrt{2\ln|A|}}{n} \leqslant B_\infty\sqrt{\frac{2\ln(2d)}{n}}. \tag{4.3.27}$$

Comme cette borne est indépendante de S, elle survit à l'espérance $\mathbb{E}_S$:

$$\mathfrak{R}_n(\mathcal{F}) \leqslant DB_\infty\sqrt{\frac{2\ln(2d)}{n}}. \tag{4.3.28}$$

Remarque (Comparaison géométrique ℓ_1 versus ℓ_2) En dimension finie d, si chaque composante vérifie $|\varphi_j(x)|\leqslant B_\infty$, l'énergie euclidienne est de l'ordre de $\mathbb{E}[\|\varphi(x)\|_2^2]\approx dB_\infty^2$, ce qui donne une complexité ℓ_2 en $\mathcal{O}\left(DB_\infty\sqrt{d/n}\right)$.

Contraindre la norme ℓ_1 remplace le facteur $\sqrt{d}$ par $\sqrt{\ln(2d)}$. C'est la justification quantitative fondamentale du Lasso : en très grande dimension ($d\gg n$), la régularisation ℓ_1 brise le fléau de la dimension dès lors que le vecteur cible admet une représentation parcimonieuse.

4.4 Théorèmes d'excès de risque pour les modèles linéaires

Nous pouvons désormais assembler l'ensemble des résultats établis au cours de ces deux sections pour borner l'erreur d'estimation des modèles linéaires.

4.4.1 Borne en espérance

Théorème (Excès de risque en espérance) . Soit $\mathcal{F} = \{f_\theta : \|\theta\|_2\leqslant D\}$ et supposons que pour tout $y\in\mathcal{Y}$, la perte $u\mapsto l(y,u)$ est G-lipschitzienne. Soit $\hat{f}_S$ un minimiseur exact du risque empirique sur $\mathcal{F}$. Alors :

$$\mathbb{E}_S[R(\hat{f}_S)] \leqslant \inf_{\|\theta\|_2\leqslant D} R(f_\theta) + \frac{2GD}{\sqrt{n}}\sqrt{\mathbb{E}_x\left[\|\varphi(x)\|_2^2\right]}. \tag{4.4.1}$$

Démonstration Soit $f^\dagger\in\arg\min_{f\in\mathcal{F}}R(f)$. La décomposition canonique donne :

$$\mathbb{E}_S[R(\hat{f}_S)] - R(f^\dagger) = \mathbb{E}_S[R(\hat{f}_S) - \hat{R}_S(\hat{f}_S)] + \mathbb{E}_S[\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)] + \mathbb{E}_S[\hat{R}_S(f^\dagger) - R(f^\dagger)].$$

Par définition du minimiseur empirique exact, $\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)\leqslant 0$. De plus, $f^\dagger$ étant indépendant de S, $\mathbb{E}_S[\hat{R}_S(f^\dagger)] = R(f^\dagger)$, donc le dernier terme est nul. Il reste :

$$\begin{aligned}
\mathbb{E}_S[R(\hat{f}_S)] - R(f^\dagger) &\leqslant \mathbb{E}_S\left[\sup_{f\in\mathcal{F}}(R(f) - \hat{R}_S(f))\right] \\
&\leqslant 2\,\mathfrak{R}_n(l\circ\mathcal{F}) \quad \text{(par symétrisation)} \\
&\leqslant 2G\,\mathfrak{R}_n(\mathcal{F}) \quad \text{(par contraction de Ledoux-Talagrand)} \\
&\leqslant \frac{2GD}{\sqrt{n}}\sqrt{\mathbb{E}_x\left[\|\varphi(x)\|_2^2\right]} \quad \text{(par la borne euclidienne).}
\end{aligned}$$

4.4.2 Borne à grande probabilité

Théorème (**Excès de risque à grande probabilité**) . On suppose de plus que la fonction de perte est bornée : $l(y, f(x)) \in [0, l_\infty]$ presque sûrement. Pour tout $\delta \in]0, 1[$, avec probabilité au moins $1 - \delta$ sur le tirage de $S \sim \mathbb{P}^{\otimes n}$:

$$R(\hat{f}_S) \leqslant \inf_{\|\theta\|_2 \leqslant D} R(f_\theta) + \frac{2GD}{\sqrt{n}} \sqrt{\mathbb{E}_x \left[\|\varphi(x)\|_2^2\right]} + 2l_\infty \sqrt{\frac{\ln(2/\delta)}{2n}}. \tag{4.4.2}$$

Démonstration On reprend la décomposition trajectorielle :

$$R(\hat{f}_S) - R(f^\dagger) \leqslant \sup_{f \in \mathcal{F}} (R(f) - \hat{R}_S(f)) + \big(\hat{R}_S(f^\dagger) - R(f^\dagger)\big).$$

1. D'après McDiarmid, $\sup_{f\in\mathcal{F}}(R(f) - \hat{R}_S(f)) \leqslant \mathbb{E}_S\left[\sup_{f\in\mathcal{F}}(R(f) - \hat{R}_S(f))\right] + l_\infty\sqrt{\frac{\ln(2/\delta)}{2n}}$ avec probabilité au moins $1 - \delta/2$. L'espérance est bornée par $\frac{2GD}{\sqrt{n}}\sqrt{\mathbb{E}[\|\varphi(x)\|_2^2]}$ comme démontré précédemment.
2. Pour le prédicteur fixé $f^\dagger$, l'inégalité de Hoeffding unilatérale assure que $\hat{R}_S(f^\dagger) - R(f^\dagger) \leqslant l_\infty\sqrt{\frac{\ln(2/\delta)}{2n}}$ avec probabilité au moins $1 - \delta/2$.

Un *Union Bound* sur les deux événements garantit que les deux inégalités ont lieu simultanément avec probabilité au moins $1 - \delta$.

Remarque (Bilan structurel : le compromis régularisation/généralisation) Cette borne synthétise tout le problème de l'apprentissage statistique dans une seule formule :

$$R(\hat{f}_S) \leqslant \underbrace{\inf_{\|\theta\|_2 \leqslant D} R(f_\theta)}_{\text{Erreur d'approximation}} + \underbrace{\frac{2GD}{\sqrt{n}} \sqrt{\mathbb{E}_x \left[\|\varphi(x)\|_2^2\right]} + 2l_\infty \sqrt{\frac{\ln(2/\delta)}{2n}}}_{\text{Erreur d'estimation}}.$$

- *Rôle du rayon D* : augmenter D enrichit la classe d'hypothèses, ce qui fait décroître l'erreur d'approximation mais croître linéairement l'erreur d'estimation. Le réglage de l'hyperparamètre de régularisation (le choix de D) matérialise l'arbitrage exact entre sous-apprentissage et sur-apprentissage.
- *Vitesse universelle en $n^{-1/2}$* : que la classe soit finie ou infinie, la vitesse de convergence reste universellement en $\mathcal{O}(1/\sqrt{n})$.

Annexe A

Inégalités de concentration et régime sous-gaussien

Contents

A.1 La méthode exponentielle de Cramér-Chernoff 158
A.1.1 Inégalité de Markov 158
A.1.2 La méthode de Cramér-Chernoff 159
A.1.3 Transformée de Cramér-Legendre et exactitude asymptotique 160
A.2 Régime Sous-Gaussien 161
A.2.1 Généralités 161
A.2.2 Propriétés algébriques 162
A.2.3 Des sommes aux fonctionnelles 163
A.2.4 Au-delà des sommes indépendantes : l'inégalité de McDiarmid 164
A.3 Bilan : la division du travail en apprentissage statistique 166

Cette annexe rassemble les fondements probabilistes des inégalités de concentration mobilisées en théorie de l'apprentissage statistique. L'ensemble de la théorie repose sur une démarche unifiée : appliquer l'inégalité de Markov à une fonctionnelle exponentielle bien choisie.

A.1 La méthode exponentielle de Cramér-Chernoff

A.1.1 Inégalité de Markov

Le socle de toutes les inégalités de déviation est le lemme de Markov.

Théorème (**Inégalité de Markov**) . Soit Z une variable aléatoire réelle presque sûrement positive ($Z \geqslant 0$ p.s.) et $t > 0$. Alors :

$$\mathbb{P}(Z \geqslant t) \leqslant \frac{\mathbb{E}[Z]}{t}. \tag{A.1.1}$$

Démonstration Comme $Z \geqslant 0$, on a l'inégalité ponctuelle $t\,\mathbb{1}_{\{Z \geqslant t\}} \leqslant Z$. En prenant l'espérance par monotonie de l'intégrale :

$$t\,\mathbb{P}(Z \geqslant t) = \mathbb{E}\left[t\,\mathbb{1}_{\{Z \geqslant t\}}\right] \leqslant \mathbb{E}[Z].$$

Cette inégalité est à la base de toutes les autres. Nous l'améliorons en l'appliquant à des fonctions ϕ bien choisies. Pour cela on utilise le résultat de la proposition suivante.

Proposition (Génération des inégalités de déviation) Soit $\phi : \mathbb{R} \to [0, \infty[$ une fonction strictement croissante sur son domaine. Pour tout t, l'inclusion d'événements $\{Z \geqslant t\} \subseteq \{\phi(Z) \geqslant \phi(t)\}$ assure que :

$$\mathbb{P}(Z \geqslant t) \leqslant \frac{\mathbb{E}[\phi(Z)]}{\phi(t)}. \tag{A.1.2}$$

Remarque (Bienaymé-Tchebychev) L'application de cette proposition à la fonction quadratique $\phi(z) = z^2$ sur la variable centrée $|X - \mathbb{E}[X]|$ donne l'inégalité de Bienaymé-Tchebychev :

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant t) \leqslant \frac{\mathrm{Var}(X)}{t^2}. \tag{A.1.3}$$

Considérons une somme de n variables i.i.d. centrées $S_n = \sum_{i=1}^n X_i$ de variance σ^2. Comme $\mathrm{Var}(S_n) = n\sigma^2$, Tchebychev fournit pour une déviation à l'échelle centrale $t = u\sigma\sqrt{n}$:

$$\mathbb{P}(|S_n| \geqslant u\sigma\sqrt{n}) \leqslant \frac{n\sigma^2}{u^2 n\sigma^2} = \frac{1}{u^2}. \tag{A.1.4}$$

Cette décroissance est purement polynomiale en u^{-2}. Or, d'après le théorème central limite, la loi limite de $S_n/(\sigma\sqrt{n})$ est gaussienne et sa queue décroît en réalité comme $\exp\big(-u^2/2\big)$. Contrôler seulement deux moments est donc insuffisant pour capturer les déviations exponentielles : il est nécessaire de contrôler des moments exponentiels, c'est la méthode de Cramér-Chernoff.

A.1.2 La méthode de Cramér-Chernoff

Pour obtenir une vitesse exponentielle, on applique la proposition A.1.2 à la famille de fonctions $\phi(z) = e^{\lambda z}$ pour $\lambda > 0$. Mais avant, on définit la transformée de Log-Laplace possédant de bonnes propriétés analytiques et calculatoires.

Définition (**Transformée log-Laplace**) . Soit X une variable aléatoire réelle. On définit sa *transformée log-Laplace* (ou *log-moment generating function*) $\psi_X : \mathbb{R} \longrightarrow]-\infty, +\infty]$ par :

$$\boxed{\psi_X(\lambda) = \ln \mathbb{E}\left[e^{\lambda X}\right]}, \tag{A.1.5}$$

définie sur le domaine $D_X = \{\lambda \in \mathbb{R} \mid \psi_X(\lambda) < \infty\}$.

En appliquant l'inégalité de Markov à la tranformée de Log-Laplace d'une variable aléatoire, on est capable de contrôler la queue de sa loi plus finement. D'où le théorème de Chernoff suivant.

Théorème (**Borne de Chernoff**) . Soit X une variable aléatoire réelle de transformée de log-Laplace ψ_X. Pour tout seuil $t > 0$ et pour tout $\lambda > 0$ tel que $\lambda \in D_X$, on a :

$$\mathbb{P}(X \geqslant t) \leqslant e^{-\lambda t} \cdot \mathbb{E}\left[e^{\lambda X}\right] = \exp\big(-(\lambda t - \psi_X(\lambda))\big). \tag{A.1.6}$$

Comme le membre de gauche ne dépend pas du paramètre libre λ, on optimise en prenant le supremum sur $\lambda > 0$, et donc :

$$\mathbb{P}(X \geqslant t) \leqslant \exp\left(-\sup_{\lambda > 0}\{\lambda t - \psi_X(\lambda)\}\right). \tag{A.1.7}$$

L'évaluation de la borne de Chernoff repose sur quatre propriétés structurelles de ψ_X.

Propriété (**Log-Laplace** ψ_X) . Supposons X centrée ($\mathbb{E}[X] = 0$). Sur l'intérieur de son domaine de définition D_X, on a les propriétés suivantes :

1. *Convexité et ancrage :* $\psi_X(0) = 0$ et ψ_X est convexe.
2. *Décalage exponentiel (Exponential tilting) :* Pour $\lambda \in \mathring{D}_X$, on définit la mesure de probabilité décalée $\mathbb{P}_\lambda$ par sa densité $d\mathbb{P}_\lambda = e^{\lambda x - \psi_X(\lambda)} d\mathbb{P}$. Alors :
$$\psi'_X(\lambda) = \mathbb{E}_{X\sim\mathbb{P}_\lambda}[X] \quad \text{et} \quad \psi''_X(\lambda) = \mathrm{Var}_{X\sim\mathbb{P}_\lambda}(X). \tag{A.1.8}$$
3. *Comportement local au voisinage de* 0 *:* On a $\psi'_X(0) = 0$, $\psi''_X(0) = \mathrm{Var}(X) = \sigma^2$, d'où le développement de Taylor en 0 :
$$\psi_X(\lambda) = \frac{\sigma^2\lambda^2}{2} + o(\lambda^2). \tag{A.1.9}$$
4. *Tensorisation :* Si $X_1, \dots, X_n$ sont des variables indépendantes, alors pour $S_n = \sum_{i=1}^n X_i$:
$$\psi_{S_n}(\lambda) = \sum_{i=1}^n \psi_{X_i}(\lambda). \tag{A.1.10}$$

Démonstration 1. $\psi_X(0) = \ln \mathbb{E}[1] = 0$. Pour la convexité, soient $\lambda_1, \lambda_2 \in D_X$ et $\alpha \in [0,1]$. Posons $p = 1/\alpha$ et $q = 1/(1-\alpha)$ de sorte que $1/p + 1/q = 1$. Par l'inégalité de Hölder :
$$\mathbb{E}\left[e^{(\alpha\lambda_1 + (1-\alpha)\lambda_2)X}\right] \leqslant \left(\mathbb{E}\left[e^{\lambda_1 X}\right]\right)^{\alpha} \left(\mathbb{E}\left[e^{\lambda_2 X}\right]\right)^{1-\alpha}.$$
En prenant le logarithme, la concavité est exclue et on obtient la convexité de ψ_X.

2. La dérivation sous l'intégrale est licite sur l'intérieur du domaine :
$$\psi'_X(\lambda) = \frac{\mathbb{E}[Xe^{\lambda X}]}{\mathbb{E}[e^{\lambda X}]} = \int x\, e^{\lambda x - \psi_X(\lambda)} d\mathbb{P}(x) = \mathbb{E}_{X\sim\mathbb{P}_\lambda}[X].$$
En dérivant une seconde fois, la dérivée du quotient donne classiquement $\mathbb{E}_{\mathbb{P}_\lambda}[X^2] - (\mathbb{E}_{\mathbb{P}_\lambda}[X])^2 = \mathrm{Var}_{\mathbb{P}_\lambda}(X)$.

3. À $\lambda = 0$, la mesure $\mathbb{P}_0$ coïncide avec $\mathbb{P}$. Donc $\psi'_X(0) = \mathbb{E}[X] = 0$ et $\psi''_X(0) = \mathrm{Var}(X) = \sigma^2$. La formule de Taylor-Young en 0 fournit l'équivalent local.

4. Par indépendance mutuelle, l'espérance du produit se factorise :
$$\mathbb{E}\left[e^{\lambda S_n}\right] = \mathbb{E}\left[\prod_{i=1}^n e^{\lambda X_i}\right] = \prod_{i=1}^n \mathbb{E}\left[e^{\lambda X_i}\right] \implies \ln \mathbb{E}\left[e^{\lambda S_n}\right] = \sum_{i=1}^n \ln \mathbb{E}\left[e^{\lambda X_i}\right].$$

Remarque Dans le théorème précédent, on arrive à borner la queue de la variable aléatoire X grâce à sa transformée de log-Laplace. Il reste cependant un paramètre $\lambda > 0$ sur lequel on peut jouer pour améliorer cette borne, c'est pourquoi on propose une optimisation en λ pour obtenir :
$$\mathbb{P}(X \geqslant t) \leqslant \exp\left(-\sup_{\lambda>0}\{\lambda t - \psi_X(\lambda)\}\right). \tag{A.1.11}$$
L'idée de la partie suivante est de développer la théorie sur ce sup et de prouver que cette optimisation produit effectivement la meilleure borne. On définit pour cela l'exposant exponentiel comme la fonction ψ_X^*.

A.1.3 Transformée de Cramér-Legendre et exactitude asymptotique

Définition (**Transformée de Cramér-Legendre**) . On appelle *transformée de Cramér* de la variable X la fonction $\psi_X^* : \mathbb{R}_+ \longrightarrow [0, \infty]$ définie par :

$$\psi_X^*(t) = \sup_{\lambda>0}\{\lambda t - \psi_X(\lambda)\}. \tag{A.1.12}$$

La borne de Chernoff A.1.2 s'écrit alors de manière compacte : $\mathbb{P}(X \geqslant t) \leqslant e^{-\psi_X^*(t)}$.

Propriété (ψ_X^* **et moyenne empirique**) . La fonction ψ_X^* est convexe, positive, nulle en 0, et non décroissante sur $\mathbb{R}_+$. Au voisinage de 0, le développement dual donne :

$$\psi_X^*(t) = \frac{t^2}{2\sigma^2} + o(t^2). \tag{A.1.13}$$

De plus, pour des variables $(X_i)_{1\leqslant i\leqslant n}$ i.i.d., la propriété de tensorisation entraîne $\psi_{S_n/n}^*(t) = n\psi_X^*(t)$, de sorte que la moyenne empirique vérifie :

$$\mathbb{P}\left(\frac{1}{n}S_n \geqslant t\right) \leqslant e^{-n\psi_X^*(t)}. \tag{A.1.14}$$

Remarque (Théorème de Cramér, 1938 : l'optimalité de l'exposant) Le théorème de Cramér en théorie des grandes déviations stipule que sous l'hypothèse où ψ_X est finie au voisinage de 0, on a pour tout $t > 0$:

$$\lim_{n\to\infty} \frac{1}{n} \ln \mathbb{P}\left(\frac{1}{n}S_n \geqslant t\right) = -\psi_X^*(t). \tag{A.1.15}$$

La méthode de Chernoff est donc *asymptotiquement exacte* à l'échelle exponentielle : aucune information n'est perdue en passant par la transformée de Laplace. Les approximations qui interviendront par la suite (comme l'hypothèse sous-gaussienne) découleront uniquement de majorations analytiques de $\psi_X(\lambda)$, et non d'une faiblesse intrinsèque de la méthode de Chernoff.

A.2 Régime Sous-Gaussien

En probabilité, nous travaillons souvent avec des variables aléatoires Gaussiennes qui proviennent généralement de résultats de convergence tels que le TCL. Leur expression simple ainsi que leur bonnes propriétés calculatoires en font des outils pratiques à utiliser. On souhaite ici contrôler les queues d'une variable aléatoire à partir de variables gaussiennes. On les appellera des *variables aléatoires sous-gausiennes*.

A.2.1 Généralités

Définition (**Variable aléatoire sous-gausienne**) . Soit X une variable aléatoire réelle, on dira que X est une variable aléatoire *sous-gaussienne* de *variance proxy* σ^2, noté $X \sim \mathcal{G}(\sigma^2)$, si :

$$\boxed{\forall \lambda \in \mathbb{R}, \quad \psi_X(\lambda) \leqslant \frac{\sigma^2\lambda^2}{2}.} \tag{A.2.1}$$

Proposition Soit $X \sim \mathcal{G}(\sigma^2)$, en appliquant la borne de Chernoff à $-X$, on obtient la relation suivante :

$$\forall \lambda > 0, \quad \mathbb{P}_X(X \geqslant t) \leqslant e^{-t^2/(2\sigma^2)}, \quad \mathbb{P}_X(|X| \geqslant t) \leqslant 2e^{-t^2/(2\sigma^2)}. \tag{A.2.2}$$

Le théorème suivant propose d'autres écritures à cette conséquence.

Théorème (**Équivalences**) . Soit X une variable aléatoire centrée. Les assertions ci-dessous sont équivalence à constante positives K_i près :

1. $\forall t > 0, \quad \mathbb{P}_X(|X| \geqslant t) \leqslant 2e^{-t^2/K_1^2}$.
2. $\forall p \geqslant 1, \quad (\mathbb{E}_X|X|^p)^{1/p} \leqslant K_2\sqrt{p}$.
3. $\mathbb{E}_X\left(e^{X^2/K_3^2}\right) \leqslant 2$.
4. $\forall \lambda \in \mathbb{R}, \quad \psi_X(\lambda) \leqslant K_4^2 \times \dfrac{\lambda^2}{2}$.

A.2.2 Propriétés algébriques

Décrivons maintenant quelques opérations utiles sur les variables sous-gaussiennes. Commençons par définir la *norme d'Orlicz*.

Définition (**Norme d'Orlicz**) . Soit X une variable aléatoire, on définit sa *norme d'Orlicz* comme :

$$\|X\|_{\psi_2} = \inf\left\{K > 0 \mid \mathbb{E}_X\left[e^{X^2/K^2} \leqslant 2\right]\right\}. \tag{A.2.3}$$

Proposition (Scaling) Soit $X \sim \mathcal{G}(\sigma^2)$, puisque $\psi_X(x, \lambda) = \psi_{cX}(\lambda)$, alors $cX \sim \mathcal{G}(c^2\sigma^2)$.

Proposition (Somme) Soient $X_1, \ldots, X_n \in \mathcal{G}(\sigma_i^2)$, puisque :

$$\psi_{\sum_i X_i}(\lambda) = \sum_i \psi_{X_i}(\lambda) \leqslant \left(\sum_i \sigma_i^2\right)\frac{\lambda^2}{2},$$

alors :

$$\sum_i X_i \sim \mathcal{G}\left(\sum_i \sigma_i^2\right). \tag{A.2.4}$$

Proposition Sans indépendance, on a $\|X + Y\|_{\psi_2} \leqslant \|X\|_{\psi_2} + \|Y\|_{\psi_2}$.

Proposition (Cas gaussien) Soit $X \sim \mathcal{N}(0, \sigma^2)$, alors $\psi_X(\lambda) = \sigma^2\lambda^2/2$, donc il existe bien des variables sous-gaussiennes.

Proposition (Maximum) Soient $X_1, \ldots, X_n$ des variables gaussiennes $\mathcal{G}(\sigma^2)$, pas nécessairement indépendantes. Alors on a :

$$\mathbb{E}_S\left[\max_{i\leqslant n} X_i\right] \leqslant \sigma\sqrt{2\log n}. \tag{A.2.5}$$

Démonstration Soit $\lambda > 0$, alors :

$$\begin{aligned}
\exp\left(\lambda\mathbb{E}_S\left[\max_{i\leqslant n} X_i\right]\right) &\leqslant \mathbb{E}_S\left[e^{\lambda \max_{i\leqslant n} X_i}\right], && \text{Jensen}\\
&\leqslant \sum_{i\leqslant n}\mathbb{E}_{X_i}\left[e^{\lambda \max_{i\leqslant n} X_i}\right], && \text{car } \max \leqslant \text{sum}\\
&\leqslant ne^{\lambda^2\sigma^2/2}.
\end{aligned}$$

En appliquant $t \longmapsto \frac{\log t}{\lambda}$, et en posant $\lambda = \sqrt{2\log n}/\sigma$, on obtient :

$$\mathbb{E}_S[\max_{i\leqslant n} X_i] \leqslant \frac{\log n}{\lambda} + \frac{\lambda\sigma^2}{n} = \sigma\sqrt{2\log n}.$$

A.2.3 Des sommes aux fonctionnelles

Jusqu'ici, le régime sous-gaussien suppose une majoration a priori sur $\psi_X(\lambda)$. Le lemme suivant établit un résultat fondamental : toute variable aléatoire bornée presque sûrement est intrinsèquement sous-gaussienne, sans aucune hypothèse distributionnelle supplémentaire.

Lemme (Hoeffding) Soit X une variable aléatoire centrée telle que $X \in [a, b]$ p.s. Alors pour tout $\lambda \in \mathbb{R}$, on a :

$$X \in \mathcal{G}\left(\frac{(b-a)^2}{4}\right) \quad \text{et} \quad \psi_X(\lambda) \leqslant \lambda^2 \frac{(b-a)^2}{8}. \tag{A.2.6}$$

Démonstration La preuve se déroule en trois temps : on commence par borner la variable de X en l'écrivant comme le minimiseur de l'écart quadratique moyen, ensuite on paramètre la densité de X par λ, on finit par appliquer l'étape 1 au développement de Taylor à l'ordre 2 de ψ_X.

1. *Variance* : Soit Y une variable aléatoire et $c \in \mathbb{R}$. La variance minimise l'écart quadratique moyen, d'où :
$$\mathbb{V}(Y) = \inf_{c \in \mathbb{R}} \mathbb{E}\left((Y-c)^2\right) \leqslant \mathbb{E}\left[\left(Y - \frac{a+b}{2}\right)^2\right]$$
car $(a+b)/2 \in \mathbb{R}$. Or on sait que $Y \in [a, b]$ donc on majore par l'écart maximal, puis on applique la fonction carré et on prend l'espérance :
$$\begin{aligned} & \left|Y - \frac{a+b}{2}\right| \leqslant \frac{b-a}{2}, \\ \implies \quad & \left(Y - \frac{a+b}{2}\right)^2 \leqslant \frac{(b-a)^2}{4}, && t \mapsto t^2 \\ \implies \quad & \mathbb{V}(Y) \leqslant \mathbb{E}\left[\frac{(b-a)^2}{4}\right] = \frac{(b-a)^2}{4}. && \mathbb{E}(\cdot) \end{aligned}$$

2. *Densité* : On peut paramétrer la densité de X en définissant :
$$\mathrm{d}\mathbb{P}_\lambda(x) = e^{\lambda x - \psi_X(\lambda)} \mathrm{d}\mathbb{P}(x).$$
donc si $X \in [a, b]$ sous $\mathbb{P}$, alors $X \in [a, b]$ sous $\mathbb{P}_\lambda$ aussi. En appliquant l'étape 1) à $\mathbb{P}_\lambda$, on obtient :
$$\forall \lambda \in \mathbb{R}, \quad \psi_X''(\lambda) = \mathbb{V}_{\mathbb{P}_\lambda}(X) \leqslant \frac{(b-a)^2}{4}.$$

3. *Accroissements finis* : En appliquant la formule des accroissements finis à l'ordre 2 à ψ_x entre 0 et λ, on a :
$$\psi_X(\lambda) = \psi_X(0) + \lambda \psi_X'(0) + \frac{\lambda^2}{2} \psi_X''(\theta), \quad \theta \in [0, \lambda].$$
Or X est centrée donc $\psi_X(0) = 0$ et $\psi_X'(0) = \mathbb{E}(X) = 0$, d'où :
$$\psi_X(\lambda) = \frac{\lambda^2}{2} \psi_X''(\theta) \leqslant \frac{\lambda^2}{2} \frac{(b-a)^2}{4} = \frac{\lambda^2 (b-a)^2}{8}.$$

Par définition, $X \in \mathcal{G}\left(\frac{(b-a)^2}{4}\right)$.

Si le lemme de Hoeffding résout le cas des sommes de variables indépendantes par simple tensorisation, il reste impuissant face à des fonctionnelles non linéaires comme les suprema d'écarts empiriques. L'inégalité de McDiarmid lève cette restriction en adaptant le contrôle sous-gaussien aux fonctions stables par coordonnées.

A.2.4 Au-delà des sommes indépendantes : l'inégalité de McDiarmid

L'inégalité de Hoeffding classique permet de contrôler finement la concentration d'une somme de variables aléatoires indépendantes $S_n = \sum_{i=1}^n X_i$. Toutefois, en théorie de l'apprentissage statistique, l'objet fondamental que l'on cherche à borner est l'écart uniforme sur une classe d'hypothèses $\mathcal{F}$:

$$\Phi(X_1, \dots, X_n) = \sup_{f \in \mathcal{F}} \big(R(f) - \hat{R}_S(f) \big). \tag{A.2.7}$$

En raison de la présence de l'opérateur supremum, la fonctionnelle Φ n'est pas séparable en une somme de termes indépendants, ce qui rend l'inégalité de Hoeffding directement inapplicable. Pour étendre les propriétés de concentration sous-gaussiennes à de telles fonctionnelles arbitraires, la clé réside dans une hypothèse de stabilité par perturbation locale : si la modification d'une seule observation n'altère que marginalement la valeur globale de la fonction, alors cette dernière se concentre exponentiellement autour de son espérance.

Définition (**Propriété des différences bornées**) . Soit $\mathcal{X}$ un espace mesurable. Une fonction $f : \mathcal{X}^n \to \mathbb{R}$ satisfait la *propriété des différences bornées* pour des constantes $c_1, \dots, c_n \geqslant 0$ si, pour tout indice $i \in \{1, \dots, n\}$ et pour tous $x_1, \dots, x_n, x_i' \in \mathcal{X}$:

$$|f(x_1, \dots, x_i, \dots, x_n) - f(x_1, \dots, x_i', \dots, x_n)| \leqslant c_i. \tag{A.2.8}$$

Autrement dit, le remplacement de la i-ème coordonnée ne peut pas modifier la sortie de f de plus de c_i, quelles que soient les valeurs fixées sur les autres composantes.

Théorème (**Inégalité de McDiarmid - Bounded Differences**) . Soient $X_1, \dots, X_n$ des variables aléatoires indépendantes à valeurs dans $\mathcal{X}$. Si la fonction $f : \mathcal{X}^n \to \mathbb{R}$ vérifie la propriété des différences bornées associée aux constantes $(c_1, \dots, c_n)$, alors la variable centrée $g(S) = f(S) - \mathbb{E}[f(S)]$ est sous-gaussienne de paramètre de variance :

$$\sigma^2 = \frac{1}{4} \sum_{i=1}^n c_i^2. \tag{A.2.9}$$

En particulier, pour tout seuil $t > 0$, on a les déviations unilatérales et bilatérales suivantes :

$$\mathbb{P}\big(f(S) - \mathbb{E}[f(S)] \geqslant t \big) \leqslant \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2} \right), \tag{A.2.10}$$

$$\mathbb{P}\big(\mathbb{E}[f(S)] - f(S) \geqslant t \big) \leqslant \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2} \right), \tag{A.2.11}$$

$$\mathbb{P}\big(|f(S) - \mathbb{E}[f(S)]| \geqslant t \big) \leqslant 2 \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2} \right). \tag{A.2.12}$$

La démonstration repose sur une idée féconde due à Doob : décomposer l'écart global $f(S) - \mathbb{E}[f(S)]$ en une somme télescopique d'accroissements de martingale $V_i = D_i - D_{i-1}$, conditionner pas à pas pour appliquer le lemme de Hoeffding unitaire à chaque coordonnée, puis effeuiller le produit par récurrence.

Démonstration Sous les hypothèses du théorème, on veut montrer que $g(S) = f(S) - \mathbb{E}_S(f(S))$ vérifie :

$$\psi_{g(S)} \leqslant \frac{\lambda^2}{8} \sum_{i \leqslant n} c_i^2.$$

Conditionnons $\mathbb{E}(f(S))$ par une suite de filtrations $\mathcal{F}_i = \sigma(X_1, \dots, X_i)$. On pose alors :

$$D_i = \mathbb{E}[f(S) \mid \mathcal{F}_i] \quad \text{et} \quad \begin{cases} D_0 = \mathbb{E}(f(S)) \\ D_n = f(S) \end{cases}$$

On a alors, par télescopage :

$$f(S) - \mathbb{E}(f(S)) = D_n - D_0 = \sum_{i=1}^{n} \underbrace{D_i - D_{i-1}}_{V_i} = \sum_{i=1}^{n} V_i.$$

Calculons :

$$\begin{aligned} \mathbb{E}[V_i \mid \mathcal{F}_{i-1}] &= \mathbb{E}(D_i - D_{i-1} \mid \mathcal{F}_{i-1}) \\ &= \mathbb{E}(D_i \mid \mathcal{F}_{i-1}) - \mathbb{E}(D_{i-1} \mid \mathcal{F}_{i-1}) \\ &= \mathbb{E}\left[\mathbb{E}(f(S) \mid \mathcal{F}_i) \mid \mathcal{F}_{i-1}\right] - \mathbb{E}(D_{i-1} \mid \mathcal{F}_{i-1}) \\ &= \mathbb{E}(f(S) \mid \mathcal{F}_{i-1}) - \mathbb{E}(D_{i-1} \mid \mathcal{F}_{i-1}) \\ &= D_{i-1} - D_{i-1} = 0. \end{aligned}$$

Essayons maintenant de borner les V_i par les c_i à partir de la propriété des différences bornés. Posons :

$$D_i = h_i(X_1, \dots, X_i) \quad \text{avec} \quad h_i(x_1, \dots, x_i) = \mathbb{E}[f(x_1, \dots, x_i, X_{i+1}, \dots, X_n)]$$

On a alors :

$$\begin{aligned} D_{i-1} &= \mathbb{E}\left[f(X_1, \dots, X_n) \mid \mathcal{F}_{i-1}\right] \\ &= \mathbb{E}\left[D_i \mid \mathcal{F}_{i-1}\right] \\ &= \mathbb{E}\left[h_i(X_1, \dots, X_{i-1}, X_i) \mid \mathcal{F}_{i-1}\right] \\ &= \mathbb{E}_{X_i'}\left[h_i(X_1, \dots, X_{i-1}, X_i')\right]. \end{aligned}$$

D'où la différence :

$$V_i = D_i - D_{i-1} = h_i(X_1, ;X_i) - \mathbb{E}_{X_i'}\left[h_i(X_1, \dots, X_{i-1}, X_i')\right].$$

Conditionnellement à $\mathcal{F}_{i-1}$, les $X_1, \dots, X_{i-1}$ sont constants (i.e déjà connus). Quand X_i varie, par différences bornées sur h_i, la quantité $h_i(X_1, \dots, X_i)$ ne varie pas de plus de c_i. Il existe donc $k \in \mathbb{R}$ tel que $V_i \in [k - c_i, k + c_i]$, on peut donc appliquer l'ingalité de Hoeffding conditionnellement à $\mathcal{F}_{i-1}$:

$$\mathbb{E}\left[e^{\lambda V_i} \mid \mathcal{F}_{i-1}\right] \leqslant \exp\left(\frac{\lambda^2 c_i^2}{8}\right).$$

On finit la preuve en sortant les $e^{\frac{\lambda^2 c_n^2}{8}}$ itérativement de l'espérance :

$$\begin{aligned} \mathbb{E}\left[e^{\lambda \sum_{i \leqslant n} V_i}\right] &= \mathbb{E}\left[e^{\lambda \sum_{i \leqslant n-1} V_i} \cdot e^{\lambda V_n}\right] \\ &= \mathbb{E}\left[e^{\lambda \sum_{i \leqslant n-1} V_i} \cdot \mathbb{E}\left[e^{\lambda V_n} \mid \mathcal{F}_{n-1}\right]\right] \\ &\leqslant e^{\frac{\lambda^2 c_n^2}{8}} \mathbb{E}\left[e^{\lambda \sum_{i=1}^{n-1} V_i}\right]. \end{aligned}$$

On a donc au final :

$$\mathbb{E}\left[e^{\lambda g(S)}\right] \leqslant \prod_{i=1}^{n} e^{\frac{\lambda^2 c_i^2}{8}} = \exp\left(\frac{\lambda^2}{8} \sum_{i \leqslant n} c_i^2\right).$$

A.3 Bilan : la division du travail en apprentissage statistique

Les outils probabilistes développés dans cette annexe fournissent le fondement rigoureux de l'analyse d'excès de risque menée au Chapitre 4. En apprentissage statistique, l'objectif central est de contrôler la déviation uniforme :

$$\Phi(S) = \sup_{f \in \mathcal{F}} \big(R(f) - \hat{R}_S(f)\big). \tag{A.3.1}$$

Grâce aux résultats établis, l'analyse théorique se découple naturellement en deux étapes hermétiques :

1. **La concentration (Inégalité de McDiarmid) :** Sous l'hypothèse de pertes bornées dans $[0, l_\infty]$, la fonctionnelle Φ vérifie la propriété des différences bornées avec $c_i = l_\infty/n$. L'inégalité de McDiarmid assure alors que, pour tout $\delta \in]0,1[$, avec probabilité au moins $1-\delta$:

$$\sup_{f \in \mathcal{F}} \big(R(f) - \hat{R}_S(f)\big) \leqslant \mathbb{E}_S \left[\sup_{f \in \mathcal{F}} \big(R(f) - \hat{R}_S(f)\big) \right] + l_\infty \sqrt{\frac{\ln(1/\delta)}{2n}}. \tag{A.3.2}$$

   Les fluctuations stochastiques d'échantillonnage sont ainsi universellement contrôlées à une vitesse en $\mathcal{O}(1/\sqrt{n})$, sans qu'aucune hypothèse de finitude ou de compacité ne soit requise sur la classe de prédicteurs $\mathcal{F}$.

2. **La complexité géométrique (Symétrisation et Rademacher) :** Le problème est ainsi ramené à l'évaluation d'une seule quantité scalaire déterministe, l'espérance résiduelle. C'est à ce stade précis que s'articulent les outils développés au cours :
   - Le procédé de symétrisation majore cette espérance par la complexité de Rademacher $2\mathfrak{R}_n(\ell \circ \mathcal{F})$.
   - Le lemme de contraction sépare le critère de perte de la géométrie intrinsèque de $\mathcal{F}$.
   - Enfin, pour borner cette complexité, la proposition sur le maximum de variables sous-gaussiennes fournit directement le lemme de Massart pour les classes finies, tandis que la dualité de normes s'applique aux modèles linéaires sous contrainte de régularisation (L^1 et L^2).

Cette division du travail illustre la complémentarité parfaite entre l'analyse de concentration sous-gaussienne (qui gère l'aléa) et l'analyse de complexité fonctionnelle (qui capture le pouvoir expressif de la classe d'hypothèses).

Informatique Graphique

Chapitre A

Modélisation Géométrique

Contents

A.1 Cadre mathématique et prérequis géométriques 168
A.1.1 Espaces affines et arithmétique des points 169
A.1.2 Combinaisons affines et barycentres 169
A.1.3 Courbes paramétrées 170
A.2 Représentations polynomiales 170
A.2.1 Interpolation vs Approximation 170
A.2.2 Courbes de Bézier et Polynômes de Bernstein 171
A.2.3 B-Splines et Vecteur Nodal 173

L'informatique graphique est une discipline définissant différents modèles et algorithmes pour modéliser numériquement des scènes et des objets. Pour cela, elle se base sur des notions de Mathématiques Appliquées, Géométrie Différentielle et d'Optimisation. L'ojectif est de proposer à des graphistes, des dessinateurs, ou des ingénieurs des algorithmes et logiciels capables de modéliser et visualiser tout types de formes, d'un personnage d'animation à une pièce à usiner. Historiquement, l'informatique graphique se regroupe sur trois sous-disciplines transverses. La première se concentre sur *l'animation* : on cherche ici à développer des modèles pour animer des pièces déjà créées. Par exemple on peut chercher à définir des rotations à appliquer à des objets pour décrire le mouvement d'un personnage. La seconde est le *rendu en temps réel*. On cherche ici à donner de la texture à une animation en temps réel pour donner à un utilisateur une vision acceptable de sa scène. Par exemple, on peut s'intéresser au rendu temps réel dans un jeu vidéo où l'utilisateur doit avoir sous les yeux des images qui rentrent dans la charte graphique de son jeu tout en gardant une fluidité suffisante d'affichage à l'écran. Enfin, le *rendu physiquement réaliste* cherche à rendre une image ou une scène la plus réaliste possible. Ici, on cherchera à simuler au plus près les jeux de lumière sur la matière pour donner une vision la plus réaliste possible à l'utilisateur.
L'objectif de ce cours et de proposer des méthodes mathématiques et algorithmiques permettant la création et la modélisation d'environnements en 3D et le traitement d'une géométrie à partir d'une capture.

A.1 Cadre mathématique et prérequis géométriques

La modélisation géométrique vise à représenter et manipuler des formes numériques par des outils d'analyse et d'algèbre linéaire. Contrairement à une description discrète (maillages polygonaux), on privilégie ici une modélisation paramétrique continue à haut niveau.

Pour construire cette modélisation paramétrique, nous nous basons sur des *points de contrôles*. Ces points seront définis par l'utilisateur et serviront de base pour générer une *courbe approximante* de la forme qu'ils décrivent.

A.1.1 Espaces affines et arithmétique des points

En synthèse d'images et en modélisation, il est donc fondamental de distinguer les notions d'*emplacements* (points) et de *déplacements* (vecteurs). Pour les premiers, nous utilisons la notions *d'espace affine* pour les définir formellement.

Définition (**Espace affine**) . Soit E un $\mathbb{R}$-espace vectoriel. Un *espace affine* attaché à E est un ensemble non vide $\mathcal{E}$ muni d'une action libre et transitive du groupe additif $(E, +)$ sur $\mathcal{E}$:

$$\begin{aligned} \Phi : \mathcal{E} \times E &\longrightarrow \mathcal{E} \\ (A, \vec{v}) &\longmapsto A + \vec{v} \end{aligned} \tag{A.1.1}$$

qui vérifie :

1. $\forall A \in \mathcal{E},\ A + \vec{0} = A$;
2. $\forall A \in \mathcal{E},\ \forall \vec{u}, \vec{v} \in E,\ (A + \vec{u}) + \vec{v} = A + (\vec{u} + \vec{v})$;
3. $\forall A, B \in \mathcal{E}$, il existe un unique vecteur noté $\vec{AB} \in E$ tel que $B = A + \vec{AB}$.

La dimension de $\mathcal{E}$ est par définition la dimension de E. Dans la suite, on considérera $\mathcal{E} \simeq \mathbb{R}^2$ ou $\mathbb{R}^3$.

On définira donc l'espace des points comme un *espace affine* dans lequel nous ne pourrons effectuer que ces opérations. À noter que les opérations sur les points ainsi que leur définition sont *indépendantes* du repère utilisé.

Remarque (Opérations licites) L'espace affine n'est pas muni d'une addition interne naturelle sur ses éléments. Ainsi :

- La somme de deux points $A + B$ n'a *aucun sens géométrique* intrinsèque (elle dépendrait de l'origine choisie).
- La différence de deux points $B - A$ est un vecteur bien défini ($\vec{AB} \in E$).
- La translation d'un point par un vecteur $A + \vec{v}$ est un point bien défini de $\mathcal{E}$.

A.1.2 Combinaisons affines et barycentres

Pour interpoler ou moyenner des positions de points de contrôle, nous avons besoin d'une opération linéaire adaptée à l'espace affine.

Définition (**Combinaison affine et Barycentre**) . Soit $(P_0, \ldots, P_n) \in \mathcal{E}^{n+1}$ une famille finie de points et $(\alpha_0, \ldots, \alpha_n) \in \mathbb{R}^{n+1}$ une famille de scalaires.
L'expression formelle $\sum_{i=0}^{n} \alpha_i P_i$ n'est indépendante de tout repère d'origine $O \in \mathcal{E}$ si et seulement si la somme des poids vérifie la *condition de partition de l'unité* :

$$\sum_{i=0}^{n} \alpha_i = 1 \tag{A.1.2}$$

Dans ce cas, le point $G \in \mathcal{E}$ défini de manière unique par :

$$\sum_{i=0}^{n} \alpha_i \vec{GP_i} = \vec{0} \iff G = O + \sum_{i=0}^{n} \alpha_i \vec{OP_i} \quad (\forall O \in \mathcal{E}) \tag{A.1.3}$$

est appelé le *barycentre* du système pondéré $\{(P_i, \alpha_i)\}_{i=0}^{n}$, noté par abus $G = \sum_{i=0}^{n} \alpha_i P_i$.

Définition (**Enveloppe convexe**) . Une combinaison affine est dite *convexe* si de plus tous les coefficients sont positifs ou nuls :

$$\forall i \in [\![0, n]\!],\ \alpha_i \geqslant 0 \quad \text{avec} \quad \sum_{i=0}^{n} \alpha_i = 1 \tag{A.1.4}$$

L'*enveloppe convexe* d'un ensemble de points $\mathcal{P} = \{P_0, \ldots, P_n\}$, notée $\mathrm{Conv}(\mathcal{P})$, est l'ensemble de toutes leurs combinaisons convexes :

$$\boxed{\mathrm{Conv}(\mathcal{P}) := \left\{ \sum_{i=0}^{n} \alpha_i P_i \;\middle|\; \alpha_i \geqslant 0,\ \sum_{i=0}^{n} \alpha_i = 1 \right\}} \tag{A.1.5}$$

En topologie affine, $\mathrm{Conv}(\mathcal{P})$ correspond au plus petit ensemble convexe fermé contenant $\mathcal{P}$. C'est une propriété stabilisatrice clé : toute courbe construite par mélange convexe reste confinée dans cette zone sans divergence numérique.

A.1.3 Courbes paramétrées

Une fois nos *points de contrôles* définis, nous allons définir une courbe approximante de ces points. Nous utilisons pour cela les *courbes paramtriques*.

Définition (**Courbe paramétrée**) . Une *courbe paramétrée* C dans $\mathcal{E}$ de classe $\mathcal{C}^k$ est la donnée d'une application $f : I \subset \mathbb{R} \longrightarrow \mathcal{E}$ de classe $\mathcal{C}^k$. On posera $I = [u_{\min}, u_{\max}]$ un intervalle fermé de $\mathbb{R}$. On posera donc :

$$C = f(I) = \{f(u) \mid u \in I\} \subset \mathcal{E}. \tag{A.1.6}$$

A.2 Représentations polynomiales

L'objectif est de modéliser des courbes ou des surfaces par des représentations paramétriques continues. Contrairement aux maillages discrets, cette approche dite de *haut niveau* offre une empreinte mémoire réduite et garantit un tracé parfaitement lisse.
Pour manipuler ces formes, on introduit une famille de *points de contrôle* $(P_0, \ldots, P_n)$. Ces points servent d'interface intuitive à l'utilisateur pour contraindre la géométrie globale. Idéalement, la courbe générée doit satisfaire trois propriétés géométriques fondamentales :

1. **Diminution de la variation :** la courbe ne doit pas osciller davantage que son polygone de contrôle.
2. **Confinement convexe :** la courbe doit être entièrement contenue dans l'enveloppe convexe formée par ses points de contrôle.
3. **Invariance par symétrie :** le tracé ne doit pas dépendre du sens de parcours des indices des points.

A.2.1 Interpolation vs Approximation

La première approche, naturelle, consiste à chercher une courbe qui *interpole* strictement tous les points de contrôle. Cependant, l'interpolation polynomiale globale (comme celle de Lagrange) devient rapidement instable à mesure que le nombre de points augmente. Comme l'illustre la

– Ajouter figure –

FIGURE A.1

Figure A.1 avec le phénomène de Runge, le polynôme produit de fortes oscillations parasites aux extrémités et s'échappe largement de l'enveloppe convexe des données.
Face à ces limites, on privilégie l'approche par *approximation*. Plutôt que d'imposer le passage par chaque point intermédiaire, on utilise les points de contrôle comme des "attracteurs". On garantit ainsi le lissage, la stabilité numérique et le respect strict des contraintes géométriques énoncées ci-dessus.

A.2.2 Courbes de Bézier et Polynômes de Bernstein

Une première approche d'approximation incontournable est celle des courbes de Bézier. Celles-ci reposent sur une pondération barycentrique via les *polynômes de Bernstein*.

Définition (**Polynômes de Bernstein**) . Pour tout entier $n \geqslant 0$, on définit la famille des $n+1$ *polynômes de Bernstein* de degré n sur $[0,1]$ par :

$$B_i^n(u) = \binom{n}{i} u^i (1-u)^{n-i}, \quad \forall i \in [\![0, n]\!]. \tag{A.2.1}$$

Propriété (**Propriétés des polynômes de Bernstein**) . Pour tout $u \in [0,1]$, la famille $\{B_i^n(u)\}_{i=0}^n$ satisfait les propriétés fondamentales suivantes :

1. *Partition de l'unité :*
$$\sum_{i=0}^{n} B_i^n(u) = 1 \tag{A.2.2}$$
2. *Positivité :*
$$\forall i \in [\![0, n]\!], \quad B_i^n(u) \geq 0 \tag{A.2.3}$$
3. *Symétrie :*
$$\forall i \in [\![0, n]\!], \quad B_i^n(u) = B_{n-i}^n(1-u) \tag{A.2.4}$$
4. *Valeurs aux bornes :*
$$B_i^n(0) = \delta_{i,0} \quad \text{et} \quad B_i^n(1) = \delta_{i,n} \tag{A.2.5}$$
où $\delta_{i,j}$ désigne le symbole de Kronecker.

Remarque (Origine algébrique via la formule du binôme) Ces polynômes s'obtiennent naturellement en développant l'identité scalaire $1^n = 1$ pour tout $u \in [0,1]$:

$$1 = \big(u + (1-u)\big)^n = \sum_{i=0}^{n} \binom{n}{i} u^i (1-u)^{n-i} = \sum_{i=0}^{n} B_i^n(u) \tag{A.2.6}$$

Ce développement démontre immédiatement la partition de l'unité ainsi que la positivité sur $[0,1]$ (car $u \geqslant 0$ et $1-u \geqslant 0$).

On utilisera donc les polynômes de Bernstein comme premier approximant de nos points de contrôle. Pour chaque point de contrôle, on définira un polynôme de Bernstein associé. La somme de tous les polynômes pondérés par les points définira une *courbe paramétrique* dans l'enveloppe convexe de ces derniers.

Définition (**Courbe de Bézier**) . Soit $\mathcal{E}$ un espace affine de dimension 2 ou 3 et $(P_0, \ldots, P_n) \in \mathcal{E}^{n+1}$ une famille ordonnée de $n+1$ *points de contrôle*. La *courbe de Bézier* associée est l'application paramétrique définie par l'application $p : [0,1] \longrightarrow \mathcal{E}$:

$$\boxed{p(u) = \sum_{i=0}^{n} B_i^n(u) P_i, \quad \forall u \in [0,1]}. \tag{A.2.7}$$

Puisque $\sum_{i=0}^{n} B_i^n(u) = 1$ et $B_i^n(u) \geqslant 0$, le point $p(u)$ est une combinaison convexe bien définie dans $\mathcal{E}$ pour tout $u \in [0,1]$. Pour $m = n+1$ points de contrôle, on définira le degré de la courbe par $d = n = m-1$.

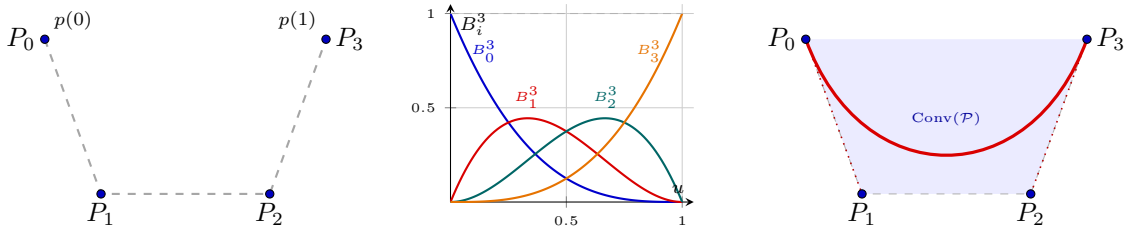
Propriété (**Courbes de Bézier**) . Une courbe de Bézier B_i^n définie sur un ensemble de points de contrôle $(P_0, \ldots, P_n)$ possède différentes propriétés :

- Elle interpole son premier et son dernier point P_0 et P_n.
- Elle est *tangente* au premier segment du polygône de contrôle en P_0 et au dernier segment en P_n.
- Elle approxime tous les autres points $\forall u \in]0,1[$.

Exemple (Tracé d'une approximation de Bézier) Soient quatre points de contrôle (P_0, P_1, P_2, P_3) placés dans un repère quelconque. On souhaite construire la courbe de Bézier cubique associée (Figure A.2). Le procédé se décompose en trois étapes :

1. Définition du polygone de contrôle formé par les quatre sommets (Figure A.2a).
2. Évaluation de la base des polynômes de Bernstein de degré 3 correspondants (Figure A.2b).
3. Combinaison barycentrique $p(u) = \sum_{i=0}^{3} B_i^3(u) P_i$ formant la trajectoire lissée (Figure A.2c).

On observe bien l'interpolation stricte en P_0 et P_3, la tangence aux premier et dernier segments, ainsi que le confinement rigoureux dans l'enveloppe convexe.



(a) Polygône de contrôle (b) Base de Bernstein (c) Courbe résultante

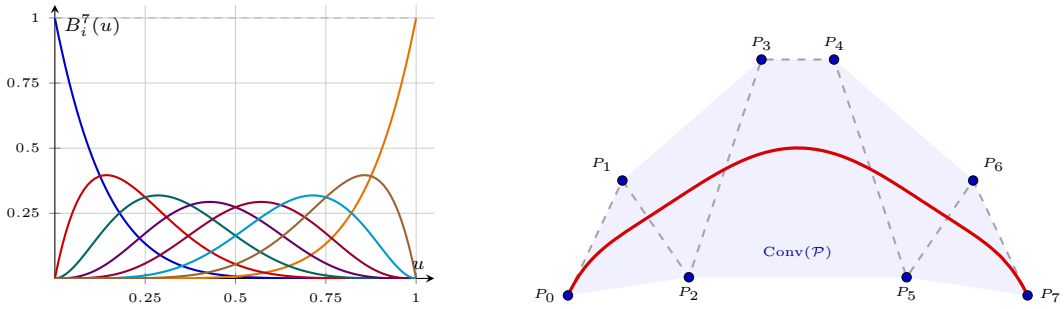
FIGURE A.2 – Construction d'une approximation de Bézier cubique : du polygône de contrôle au tracé final borné par l'enveloppe convexe.

Remarque En fonction du nombre de points de contrôle, la courbe obtenue aura différentes propriétés :

- Segment Linéaire : 2 points de contrôle,
- Parabole quardatique : 3 points de contrôle,
- Parabole cubique : 4 points de contrôle.

Après plusieurs tracés, on identifie rapidement plusieurs limitations à l'utilisation des courbes de Bézier pour tracer des formes plus complexes. La première est un phénomène d'écrasement

lorsque l'on dépasse 5 à 6 points de contrôles. En effet, par définition, la famille des polynômes de Bernstein à utiliser doit sommer à 1, ils doivent donc mécaniquement s'applitir pour respecter cette conrtainte (visible dans la Figure A.3a). Cela produit donc une courbe paramétrique plus applatie (voir Figure A.3).



(a) Base de Bernstein de degré 7 ($B_0^7, \ldots, B_7^7$) (b) Aplatissement de la courbe paramétrique

FIGURE A.3 – Phénomène d'aplatissement pour un degré élevé ($n = 7$) : l'écrasement des polynômes intermédiaires limite la portée locale des points de contrôle centraux.

A.2.3 B-Splines et Vecteur Nodal

Les B-Splines permettent de pallier le principal défaut des courbes de Bézier (l'absence de contrôle local et l'aplatissement) en découplant le degré polynomial du nombre de points de contrôle.

Définition (**Vecteur nodal et fonctions de base de Cox-de Boor**) . Soit $k \in \mathbb{N}^*$ l'*ordre* de la B-Spline (correspondant à un degré polynomial $p = k - 1$). Soit $U = (u_0, u_1, \ldots, u_m)$ une suite finie non décroissante de réels appelée *vecteur nodal*.
Pour tout $i \in [\![0, m-k]\!]$, les fonctions de base N_i^k sont définies par la récurrence de *Cox-de Boor* :

- **Initialisation (ordre 1, degré 0) :**

$$N_i^1(u) = \begin{cases} 1 & \text{si } u \in [u_i, u_{i+1}[\\ 0 & \text{sinon} \end{cases} \tag{A.2.8}$$

- **Hérédité (ordre $k \geqslant 2$, degré $k-1$) :**

$$N_i^k(u) = \frac{u - u_i}{u_{i+k-1} - u_i} N_i^{k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} N_{i+1}^{k-1}(u) \tag{A.2.9}$$

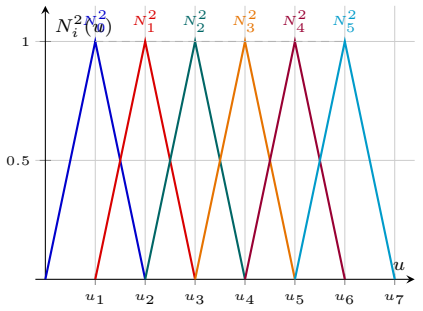
Dans le cas où deux nœuds consécutifs sont confondus, on applique par convention $0/0 = 0$.

Définition (**Courbe B-Spline**) . Soit une famille de $n+1$ points de contrôle $(P_0, \ldots, P_n)$ et un ordre k. Le vecteur nodal associé doit nécessairement comporter $m + 1 = n + k + 1$ nœuds.
La *courbe B-Spline* associée est définie sur l'intervalle valide $[u_{k-1}, u_{n+1}]$ par :

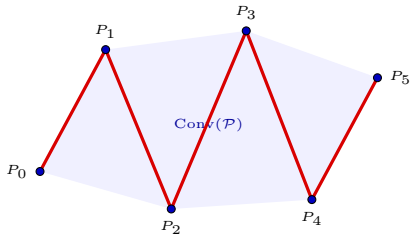
$$\forall u \in [u_{k-1}, u_{n+1}], \quad P(u) = \sum_{i=0}^{n} N_i^k(u) P_i \tag{A.2.10}$$

Sur cet intervalle, la partition de l'unité $\sum_{i=0}^{n} N_i^k(u) = 1$ est vérifiée, garantissant le confinement dans l'enveloppe convexe locale.

Exemple (Degré 1 — Ordre $k = 2$) Soient $n + 1 = 6$ points de contrôle $(P_0, \ldots, P_5)$. On construit la B-Spline de degré 1 (ordre $k = 2$).
Le vecteur nodal uniforme nécessite $m+1 = (n+1)+k = 6+2 = 8$ nœuds : $U = (0, 1, 2, 3, 4, 5, 6, 7)$. Les fonctions de base $N_i^2(u)$ sont linéaires par morceaux (« dents de scie ») à support compact sur $[u_i, u_{i+2}]$ (Figure A.4a). La courbe résultante $P(u) = \sum_{i=0}^{5} N_i^2(u) P_i$ est définie sur $[u_1, u_6] = [1, 6]$ et réalise l'interpolation linéaire directe des sommets (Figure A.4b).



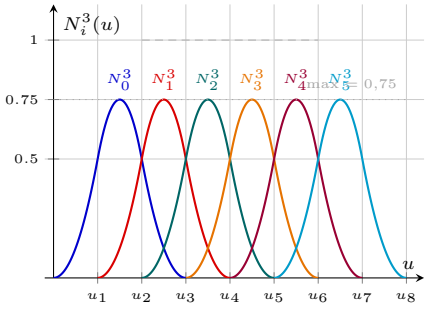
(a) Fonctions de base $N_i^2(u)$ en dents de scie



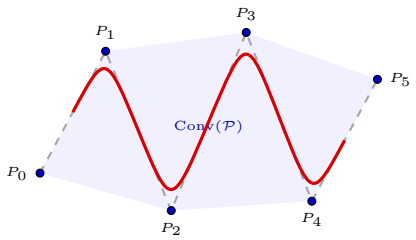
(b) Trajectoire linéaire par morceaux

FIGURE A.4 – B-Spline de degré 1 sur 6 points de contrôle : support compact à deux intervalles $[u_i, u_{i+2}]$ et interpolation linéaire continue C^0 épousant le profil en vague sans aplatissement.

Exemple (Degré 2 — Ordre $k = 3$) En conservant les 6 mêmes points de contrôle, on choisit cette fois un degré 2 (ordre $k = 3$).
Le vecteur nodal requiert désormais $m + 1 = 6 + 3 = 9$ nœuds : $U = (0, 1, 2, 3, 4, 5, 6, 7, 8)$. Les fonctions de base $N_i^3(u)$ sont quadratiques par morceaux de classe $\mathcal{C}^1$ sur $[u_i, u_{i+3}]$ (Figure A.5a). La courbe $P(u) = \sum_{i=0}^{5} N_i^3(u) P_i$ est évaluée sur $[u_2, u_6] = [2, 6]$ et produit une trajectoire lissée sans angle vif (Figure A.5b).



(a) Fonctions de base $N_i^3(u)$ de degré 2 ($\mathcal{C}^1$)



(b) Courbe lissée $\mathcal{C}^1$ par morceaux

FIGURE A.5 – B-Spline uniforme de degré 2 sur 6 points de contrôle.

Chapitre B

Fondements physiques et modélisation du transport radiatif

Contents

B.1 Grandeurs radiométriques fondamentales 175
B.1.1 Principe du rendu 176
B.1.2 De l'angle 2D à l'angle solide 176
B.1.3 Grandeurs physiques 177
B.1.4 Distribution spectrale et émission thermique : le corps noir 178
B.2 Équation des transferts radiatif & Équation du rendu 179
B.2.1 Modélisation du capteur et de la caméra 179
B.2.2 Formulation de l'ETR 180
B.2.3 Résolution analytique : forme intégrale de l'ETR 182
B.2.4 De l'ETR à l'équation du rendu de Kajiya 183
B.3 Modélisation de la réflexion 185
B.3.1 L'intuition de la corde : transversalité et polarisation 185
B.3.2 Réflexion & Réfraction 186
B.3.3 Fonction de distribution de la réflectance bidirectionnelle - BRDF . . 188

On s'intéresse dans ce chapitre et les suivants à la modélisation de l'éclairage sur une scène déjà modélisée numériquement. L'intérêt est de distribuer de façon réaliste une partie du spectre électromagnétique généré par une source de lumière sur les objets pour générer des images réalistes. On se base pour cela sur les propriétés physiques des matériaux représentés pour estimer numériquement leurs interactions avec la lumière. Théoriquement, on se basera sur des équations aux dérivées partielles décrivant ces interactions. Nous les résoudrons ensuite via des méthodes numériques, notament par des méthodes de Monte-Carlo.

B.1 Grandeurs radiométriques fondamentales

Avant de parler d'équation du transfert radiatif, commençons par définir correctement les grandeurs physiques utilisées pour la description des interactions du spectre électromagnétique avec la matière.

B.1.1 Principe du rendu

On modélise ces interactions simplement par trois éléments : la source, la matière et le détecteur représentés en Figure B.1. Un signal $\mathcal{E}$ est émis par une source lumineuse (soleil, ampoule) et est transporté sous la forme de rayonnements jusqu'aux objets de la scène. On définit alors un *opérateur de transport* $\mathcal{T}$ décrivant l'interaction de ce signal avec la matière (absorption, réflexion). Enfin un capteur (caméra, oeil) acquiert le signal reçu $\mathcal{S} = \mathcal{T} \cdot \mathcal{E}$. En fonction de sa sensibilité à chaque longueur d'onde $\mathcal{V}$, celui le capte différement, on obtient donc l'équation suivante :

$$\mathcal{R} = \int \mathcal{V}(\lambda) \cdot \mathcal{S}(\lambda) \mathrm{d}\lambda. \tag{B.1.1}$$

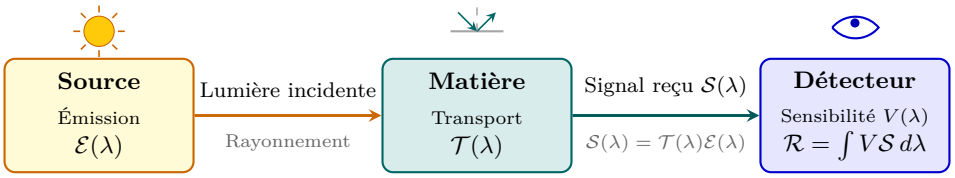


FIGURE B.1 – Chaîne fondamentale de formation de l'image en synthèse d'images : de l'émission à la réponse du capteur.

L'objectif du rendu est de modéliser l'ensemble de ces notions physiques. La principale difficulté est dans l'estimation du transport $\mathcal{T}$. On considèrera pour cela les interactions *microscopiques* (entre les photons et la structure atomique du matériaux) et les interactions *macroscopiques*. Pour cela, on développera des modèles statistiques macroscopiques basés sur les propriétés microscopiques des matériaux : on parlera de *modèles hybrides*.

B.1.2 De l'angle 2D à l'angle solide

Nous posons ici les définitions fondamentales de la radiométrie. Intuitivement, on part de la puissance d'un rayonnement émis et on essaye de décrire ses interactions avec la matière en fonction de son angle incident ainsi que la quantité de surface (aire) interceptée.

Définition (**Angle solide**) . Pour quantifier la partition de l'espace visuel sous-tendue par un objet vu depuis un point quelconque, on défini la notion *d'angle solide*. Pour cela, on projette l'objet sur une sphère de rayon r centrée sur le point de vue, on obtient alors l'expression de la surface de la sphère interceptée à partir de l'aire couverte par le rayonnement et son rayon :

$$\mathrm{d}\omega = \frac{\mathrm{d}A_{\text{sphère}}}{r^2} \tag{B.1.2}$$

exprimé en stéradians (sr). En utilisant les coordonnées sphériques, pour une petite variation $\mathrm{d}\theta$ de la colatitude θ et une variation $\mathrm{d}\varphi$ de la longitude φ, on a la formule suivante :

$$\mathrm{d}\omega = \sin\theta \mathrm{d}\theta \mathrm{d}\varphi. \tag{B.1.3}$$

Dans le cas d'une surface dont la normale fait un angle θ avec la direction d'observation, alors l'aire induite n'est que de $\mathrm{d}A\cos\theta$, on peut alors approximer l'angle solide par :

$$\mathrm{d}\omega \simeq \frac{\mathrm{d}A\cos\theta}{r^2}. \tag{B.1.4}$$

Proposition

B.1.3 Grandeurs physiques

Les angles solides permettent ensuite de définir de nouvelles notions et grandeurs physiques permettant de décrire le comportement du spectre électromagnétique. Ici, on commence donc par proposer quelques métriques permettant de quantifier l'énergie émise par une source (flux et intensité rayonnante) puis on présente des métriques quantifiant le rayonnement reçu ou réfléchit par des surfaces.

Définition (**Puissance Rayonnante - Flux**) . On appelle *puissance rayonnante* ou *flux* P (ou Φ) l'énergie totale émise, transportée ou reçue par unité de temps. Elle s'exprime en Joules par secondes dont l'unité internationnale est le Watts.

Définition (**Intensité rayonnante**) . On considère une source ponctuelle émettant de l'énergie par rayonnement d'une puissance P. On s'intéresse à la quantité d'énergie qu'elle distribue dans la direction $\mathrm{d}\omega$ d'un angle solide. Cette dernière est appelée *intensité rayonnante* et est définie par :

$$I = \frac{\mathrm{d}P}{\mathrm{d}\omega} \quad [\mathrm{W} \cdot \mathrm{sr}^{-1}] \tag{B.1.5}$$

Elle s'exprime en Watts par stéradians. Si la source émet la même quantité d'énergie dans toutes les directions, on dit qu'elle est *isotrope* et on a $I = \frac{P}{4\pi}\ \mathrm{W} \cdot \mathrm{sr}^{-1}$.

On peut maintenant s'intéresser à la quantité d'énergie acquise ou émise par une surface quelconque. On défini pour cela l'exitance et l'irradiance.

Définition (**Exitance – Irradiance**) . Soit une surface $\mathrm{d}A$ émettant du rayonnement de flux $P_{\text{émis}}$ et recevant un rayonnement de flux $P_{\text{reçu}}$. On peut alors définir la densité surfacique de rayonnement qui quitte la surface, appelée *exitance* :

$$M = \frac{P_{\text{émis}}}{\mathrm{d}A} \quad [\mathrm{W} \cdot \mathrm{m}^{-2}]. \tag{B.1.6}$$

De même, on définit la densité surfacique de rayonnement qui arrive sur la surface par *l'irradiance* :

$$E = \frac{P_{\text{reçu}}}{\mathrm{d}A} \quad [\mathrm{W} \cdot \mathrm{m}^{-2}]. \tag{B.1.7}$$

Ces trois dernières grandeurs nous permettent de quantifier une quantité de rayonnement par rapport à une surface précise (exitance/irradiance) ou une direction donnée (intensité rayonnante). On souhaiterait maintenant combiner les deux : quantifier la puissance émise/reçu par une surface donnée dans une direction donnée, c'est la *radiance*.

Définition (**Radiance/Luminance**) . Soit un objet de surface $\mathrm{d}A$, une direction donnée par l'angle solide $\mathrm{d}\omega$ et une intensité I. On défini la *radiance* (ou luminance) de cette surface dans la direction $\mathrm{d}\omega$ par :

$$L = \frac{\mathrm{d}^2 P}{\mathrm{d}A \cos\theta \mathrm{d}\omega} \quad [\mathrm{W} \cdot \mathrm{m}^{-2} \cdot \mathrm{sr}^{-1}] \tag{B.1.8}$$

Remarque La radiance est invariante dans le vide en position. En effet pour un observateur d'une surface $\mathrm{d}A$, la quantité de rayonnement (intensité) qui part dans sa direction $\mathrm{d}\omega$ depuis la surface est égale à celle qu'il reçoit. D'autre part, la réponse du capteur est proportionnelle à la radiance du corps qu'il observe.

Proposition (Transfert radiatif) Soient deux éléments de surfaces dont une source $\mathrm{d}A_1$ et un récepteur $\mathrm{d}A_2$ séparés par une distance r représentés dans la Figure B.2. On suppose que la

source émet de la radiance L sous un angle θ_1 par rapport à la direction (en bleu). Le récepteur $\mathrm{d}A_2$ sous-tend depuis $\mathrm{d}A_1$ un angle solide $\mathrm{d}\omega_1$ tel que :

$$\mathrm{d}\omega_1 = \frac{\mathrm{d}A_2 \cos\theta_2}{r^2}. \tag{B.1.9}$$

En réinjectant $\mathrm{d}\omega_1$ dans la formule de la radiance $\mathrm{d}^2P = L{\cdot}\mathrm{d}A_1{\cdot}\cos\theta_1\mathrm{d}\omega_1$, on obtient l'expression du transfert élémentaire d'énergie entre les deux surfaces :

$$\mathrm{d}^2P = L\frac{\cos\theta_1\cos\theta_2}{r^2}\mathrm{d}A_1\mathrm{d}A_2 = L\cdot G(x_1,x2)\cdot\mathrm{d}A_1\mathrm{d}A_2. \tag{B.1.10}$$

Où l'on note $G(x_1,x_2) = \frac{\cos\theta_1\cos\theta 2}{\|x_1-x_2\|^2}$ le *terme géométrique*.

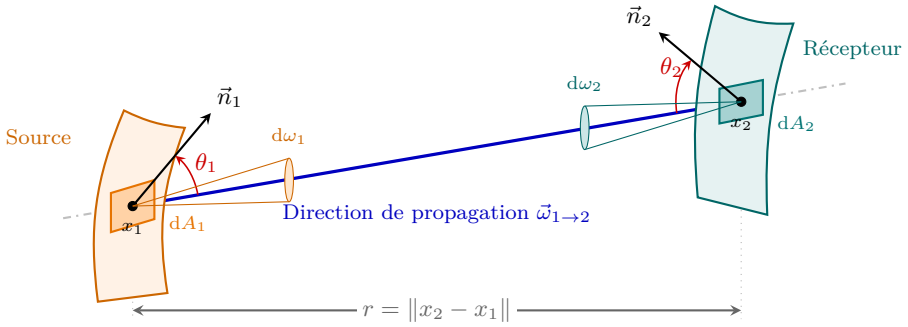


FIGURE B.2 – Transport radiatif différentiel entre deux éléments de surface orientés $\mathrm{d}A_1$ et $\mathrm{d}A_2$ séparés par une distance r.

B.1.4 Distribution spectrale et émission thermique : le corps noir

Les grandeurs précédement définies ne tenaient pas compte de la longueur d'onde d'émission/réception des objets ni de leur température. Or l'énergie transportée par un rayonnement en dépend fortement. L'introduction du principe du *corps noir* permet justement de formaliser cette différence.
En physique, un corps noir est un objet idéal qui absorbe tout le rayonnement reçu. Pour rester en équilibre thermodynnamique à une température T, il réémet cet excès d'énergie sous la forme de rayonnement spectral isotrope.

Définition (**Exitance du corps noir**) . La loi de Planck donne l'exitance monochromatique M^0_λ (pour une longueur d'onde donnée) du corps noir :

$$M^0_\lambda = \frac{2\pi hc^2}{\lambda^5\left(\exp\left(\frac{hc}{k\lambda T}\right)-1\right)} \quad [\mathrm{W}\cdot\mathrm{m}^{-2}\cdot\mathrm{nm}^{-1}]. \tag{B.1.11}$$

Où h est la constante de Planck, c la célérité de la lumière dans le vide, k la constante de Boltzman et T la température du corps en kelvin.

Cette équation défini la quantité de rayonnement émise par un corps noir à température constante pour une longueur d'onde donnée par unité de surface.

Proposition (Radiance Spectrique) Comme le corps noir diffuse son surplus d'énergie de manière isotrope, on peut calculer sa radiance spectrique (radiance pour une longueur d'onde donnée) à l'aide de la relation $M^0_\lambda = \pi L_{\lambda,b}$. On a donc :

$$L_{\lambda,b} = \frac{2hc^2}{\lambda^5\left(\exp\left(\frac{hc}{k\lambda T}\right)-1\right)} \quad [\mathrm{W}\cdot\mathrm{m}^{-2}\cdot\mathrm{sr}^{-1}\cdot\mathrm{nm}^{-1}]. \tag{B.1.12}$$

Dans une scène réelle, les matériaux ne se comportent pas comme des corps noirs parfaits : ils n'absorbent qu'une fraction du rayonnement incident et n'émettent qu'une partie du flux théorique prédit par la loi de Planck. On introduit alors des coefficients spectraux adimensionnels compris dans $[0, 1]$ pour caractériser leur comportement radiatif.

Définition (**Émissivité, absorptivité et albédo**) . Pour un matériau donné à la température T et pour une longueur d'onde λ :

- L'*émissivité* $\epsilon_\lambda \in [0, 1]$ est le rapport entre l'exitance réelle du matériau M_λ et celle du corps noir équivalent M^0_λ à la même température :

$$\epsilon_\lambda = \frac{M_\lambda}{M^0_\lambda} \in [0, 1]. \tag{B.1.13}$$

- L'*absorptivité* $\alpha_\lambda \in [0, 1]$ correspond à la fraction du flux énergétique incident absorbée par le matériau à la longueur d'onde λ.
- L'*albédo* spectral (ou réflectance hémisphérique) $\rho_\lambda \in [0, 1]$ est la proportion du flux incident réfléchie par la surface.

Proposition (Loi de Kirchhoff et conservation de l'énergie) À l'équilibre thermodynamique local, la *loi du rayonnement de Kirchhoff* établit que l'émissivité d'un corps est égale à son absorptivité pour chaque longueur d'onde :

$$\epsilon_\lambda = \alpha_\lambda. \tag{B.1.14}$$

Dès lors, pour un matériau opaque (pour lequel aucune transmission interne n'a lieu, i.e. transmittance nulle), le principe fondamental de conservation de l'énergie impose $\rho_\lambda + \alpha_\lambda = 1$. On en déduit la relation liant la réflectance à l'émissivité :

$$\rho_\lambda = 1 - \alpha_\lambda = 1 - \epsilon_\lambda. \tag{B.1.15}$$

Cette formulation permet de paramétrer directement les termes sources et les propriétés de surface : un corps noir correspond au cas limite $\epsilon_\lambda = \alpha_\lambda = 1$ et $\rho_\lambda = 0$, tandis qu'un réflecteur idéal satisfait $\rho_\lambda = 1$ et $\epsilon_\lambda = 0$.

B.2 Équation des transferts radiatif & Équation du rendu

Maintenant que nous avons posé le contexte de la modélisation de l'éclairage ainsi que les métriques nécessaires à sa description, nous pouvons nous intéresser à sa modélisation mathématique.

B.2.1 Modélisation du capteur et de la caméra

Au début de ce chapitre, nous avons fait l'hypothèse que le signal reçu par un capteur est de la forme :

$$\mathcal{R} = \int \mathcal{V}(\lambda) \cdot \mathcal{T}(\lambda) \cdot \mathcal{E}(\lambda) \mathrm{d}\lambda \tag{B.2.1}$$

où $\mathcal{T}$ est l'opérateur de transport du rayonnement, $\mathcal{E}$ représente l'émission, $\mathcal{V}$ la sensibilité du capteur et $\mathcal{R}$ le signal reçu par ce dernier.

Proposition Soit $S(x, y)$ le signal reçu par un pixel (x, y) du capteur de la caméra. Pour que celui-ci reçoit de la lumière de la scène 3d, le signal doit traverser la lentille du capteur de surface $\mathcal{L}$ paramétrée par (u, v) (ex : focale, largeur, etc...). Ainsi la quantité de lumière perçue par le

capteur est le cumul de tous les rayons collectés sur l'étendue de la lentille $\mathrm{d}A(u,v)$ dans toutes les directions incidentes possibles $\mathrm{d}\omega_l$ pendant toute la durée de l'exposition $\mathrm{d}t$. On peut donc modéliser ces notions par des applications :

- $W(x,y,x_l,\omega_l,t)$ la fonction représentant le transfert optique de la lentille,
- $L_i(x,y,\omega_l,t)$ est la radiance arrivant sur la lentille au point x_l, depuis la direction ω_l, à l'instant t.

On peut alors exprimer le signal reçu par le pixel (x,y) du capteur :

$$S(x,y) = \int_T \int_{\mathcal{L}} \int_{\Omega} W(x,y,x_l,\omega_l,t) L_i(x,y,\omega_l,t) \mathrm{d}\omega_l \mathrm{d}A(u,v) \mathrm{d}t. \tag{B.2.2}$$

Or un pixel P sur un écran n'est pas un point mathématique sans dimension : il correspond à une petite zone bidimensionnelle centrée en (x_P, y_P). En réalité, le capteur physique intègre toute la lumière qui tombe sur sa zone sensible. De plus, en traitement du signal graphique, on applique un filtre de reconstruction noté h_P. L'intégrale du pixel s'écrit donc :

$$I_p = \int_{s(h_P)} h_P(\Delta x, \Delta y)\, S(x_P + \Delta x,\, y_P + \Delta y)\, \mathrm{d}\mathcal{A}(\Delta x, \Delta y) \tag{B.2.3}$$

où :

- $(\Delta x, \Delta y)$ est le décalage local par rapport au centre du pixel (x_P, y_P).
- $s(h_P)$: le support spatial du filtre (par exemple, un carré de 1×1 pixel pour un filtre boîte élémentaire, ou une zone de 2×2 pixels pour une gaussienne ou un filtre de Mitchell-Netravali).
- $h_P(\Delta x, \Delta y)$: la fonction de pondération (le noyau du filtre). Elle donne plus d'importance aux rayons qui frappent le centre du pixel qu'à ceux qui touchent la périphérie. Pour conserver l'énergie, on a toujours $\int h_P = 1$.
- $\mathrm{d}\mathcal{A}(\Delta x, \Delta y)$: l'élément d'aire infinitésimal sur la surface du pixel ($\mathrm{d}x\,\mathrm{d}y$).

On peut donc maintenant se demander comment exprimer la radiance arrivant sur la lentille au point x_i. Celle-ci dépend directement de la géométrie de la scène et des propriétés de son éclairage.

B.2.2 Formulation de l'ETR

La section précédente s'est concentrée sur la modélisation physique de la réception d'un rayon par un capteur. Ici, on s'intéresse aux interactions de ce rayon dans l'environnement de la scène (air, eau, poussière). On appellera cette équation *l'équation du transfert radiatif*.
Soit un rayon lumineux $\vec{u}$ traversant un milieu (voir Figure B.3), on note x un point de ce rayon et on s'intéresse à la variation de luminance de ce rayon pour une petite variation d'abscisse $\mathrm{d}l$ de x. On peut alors observer différentes interactions avec l'environnement :

- **Absorption** : Une partie des photons composant le rayon sont absorbés par le milieu, notament lors de percussions avec des atomes, qui augmente le niveau d'énergie. C'est de l'énergie perdu pour le rayon $\vec{u}$, elle s'exprime de la façon suivante :

$$\left(\frac{\partial L_\lambda}{\partial l}\right)_{\text{abs}} = -\kappa_\lambda L_\lambda(x \to \vec{u}). \tag{B.2.4}$$

  où κ_λ est le coefficient d'absorption du milieu en m^{-1}.

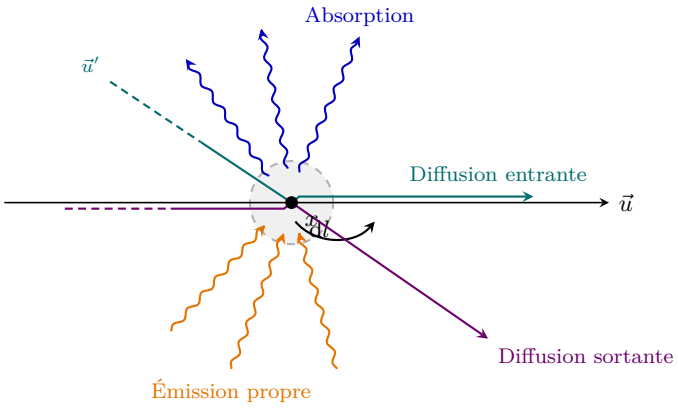


FIGURE B.3 – Bilan des interactions locales de l'ETR en un point matériel x sur l'élément de longueur dl le long de la direction $\vec{u}$.

- **Diffusion sortante** : Des photons peuvent se propager dans le milieu dû à l'énergie environnante. Lorsqu'ils entrent en collision avec le rayon, certains photons de ce dernier peuvent être déviés et perdus dans le milieu. On exprime cette perte par :

$$\left(\frac{\partial L_\lambda}{\partial l}\right)_{\text{diff, out}} = -\sigma_\lambda L_\lambda(x \to \vec{u}) \tag{B.2.5}$$

  avec σ_λ le coefficient de diffusion du milieu en m^{-1}.
- **Émission propre** : Le milieu n'étant pas vide et à une certaine température, ce surplus d'énergie est transféré sous la forme de rayonnement dont une partie est transmise au rayon $\vec{u}$:

$$\left(\frac{\partial L_\lambda}{\partial l}\right)_{\text{emis}} = \kappa_\lambda L_{\lambda,b}(x) \tag{B.2.6}$$

  avec $L_{\lambda,b}(x)$ l'émission du corps noir.
- **Diffusion entrante** : À l'inverse, des photons du milieu percutant le rayon $\vec{u}$ peuvent être déviés dans le direction de ce rayon, c'est donc de l'énergie gagnée. On a :

$$\left(\frac{\partial L_\lambda}{\partial l}\right)_{\text{diff, in}} = \frac{\sigma_\lambda}{4\pi}\int_{4\pi} L_\lambda(x \leftarrow \vec{u'})\Phi(\lambda, \vec{u}, \vec{u'})\mathrm{d}\omega(\vec{u'}) \tag{B.2.7}$$

  où $\Phi(\lambda, \vec{u}, \vec{u'})$ est la fonction de phase (BRDF volumique) : elle normalise la probabilité qu'un photon venant d'un autre rayon $\vec{u'}$ soit dévié par $\vec{u}$. On intègre cette probabilité sur toute la sphère autour de x pour prendre en compte tous les rayons possibles $\vec{u'}$ du milieu.

L'équation du transfert radiatif s'écrit alors simplement comme la somme de toutes ces variations.

Théorème (**ETR – Chandrasekhar, 1960**) . L'équation du transfert radiatif dans le contexte posé précédement fut proposée par Chandrasekhar en 1960. Elle s'exprime sous la forme :

$$\frac{\partial}{\partial l}L_\lambda(x \to \vec{u}) = -(\kappa_\lambda + \sigma_\lambda)L_\lambda(x \to \vec{u}) + \kappa_\lambda L_{\lambda,b}(x) \tag{B.2.8}$$

$$+ \frac{\sigma_\lambda}{4\pi}\int_{4\pi} L_\lambda(x \to \vec{u'})\Phi(\lambda, \vec{u}, \vec{u'})\mathrm{d}\omega(\vec{u'}). \tag{B.2.9}$$

On peut poser $k(s) = \kappa(s) + \sigma(s)$ représentant la perte par unité de longueur dl.

On peut en proposer une version plus condensé pour obtenir sa formulation canonique.

Définition (**Coefficient d'extinction et terme source volumique**) . Afin de condenser l'écriture différentielle, on regroupe les mécanismes selon leur nature (pertes proportionnelles au flux incident vs gains locaux) :

- Le *coefficient d'extinction* $k_\lambda(x)$ quantifie l'atténuation totale subie par le faisceau par unité de longueur :

$$k_\lambda(x) = \kappa_\lambda(x) + \sigma_\lambda(x) \quad [\mathrm{m}^{-1}]. \tag{B.2.10}$$

- Le *terme source volumique* $L_{i,\lambda}(x \to \vec{u})$ regroupe l'ensemble de l'énergie thermique créée et de l'énergie ambiante rabattue vers la direction $\vec{u}$ par le milieu :

$$L_{i,\lambda}(x \to \vec{u}) = \kappa_\lambda(x)L_{\lambda,b}(x) + \frac{\sigma_\lambda(x)}{4\pi}\int_{4\pi} L_\lambda(x \leftarrow \vec{u}')\,\Phi(\lambda, \vec{u}, \vec{u}')\,\mathrm{d}\omega(\vec{u}'). \tag{B.2.11}$$

Proposition (Forme canonique de l'ETR) En paramétrant le rayon lumineux par son abscisse curviligne s le long de la direction $\vec{u}$, l'ETR se réécrit comme une équation différentielle linéaire du premier ordre à coefficients non constants :

$$\boxed{\frac{\mathrm{d}L_\lambda(s)}{\mathrm{d}s} + k_\lambda(s)L_\lambda(s) = L_{i,\lambda}(s).} \tag{B.2.12}$$

B.2.3 Résolution analytique : forme intégrale de l'ETR

L'équation différentielle linéaire du premier ordre établie précédemment décrit le comportement local du transport lumineux. Pour déterminer la radiance atteignant un observateur en un point x, on intègre cette équation le long du rayon lumineux en partant du premier obstacle matériel rencontré en x_0 jusqu'à x.

Théorème (**Forme intégrale de l'ETR**) . Soit un rayon paramétré par son abscisse curviligne $s \in [0, d]$ où $s = 0$ correspond à la surface frontière x_0 et $s = d$ au point de réception x. La radiance perçue en x s'exprime par :

$$L(x \leftarrow \vec{\omega}) = \tau(x_0, x)\,L_r(x_0 \to \vec{\omega}) + \int_{x_0}^{x} \tau(s, x)\,L_i(s \leftarrow \vec{\omega})\,\mathrm{d}s \tag{B.2.13}$$

où $\tau(s_1, s_2)$ est la transmittance volumique entre deux points du trajet.

Démonstration On résout l'équation différentielle scalaire non homogène :

$$\frac{\mathrm{d}L(s)}{\mathrm{d}s} + k(s)L(s) = L_i(s). \tag{B.2.14}$$

On introduit le facteur intégrant $\mu(s) = \exp\left(\int_0^s k(t)\,\mathrm{d}t\right)$. Sa dérivée vaut :

$$\mu'(s) = k(s)\exp\left(\int_0^s k(t)\,\mathrm{d}t\right) = k(s)\mu(s). \tag{B.2.15}$$

En multipliant l'équation différentielle par $\mu(s)$, le membre de gauche s'identifie à la dérivée d'un produit :

$$\frac{\mathrm{d}}{\mathrm{d}s}\left[L(s)\mu(s)\right] = \left(\frac{\mathrm{d}L(s)}{\mathrm{d}s} + k(s)L(s)\right)\mu(s) = L_i(s)\mu(s). \tag{B.2.16}$$

On intègre cette égalité par rapport à la variable muette s' entre l'origine sur la frontière ($s' = 0$) et la position d'évaluation ($s' = d$) :

$$\left[L(s')\mu(s')\right]_0^d = \int_0^d L_i(s')\exp\left(\int_0^{s'} k(t)\,\mathrm{d}t\right)\mathrm{d}s'. \tag{B.2.17}$$

Comme $\mu(0) = \exp(0) = 1$, le membre de gauche se développe en :

$$L(d)\mu(d) - L(0) = \int_0^d L_i(s')\exp\left(\int_0^{s'} k(t)\,\mathrm{d}t\right)\mathrm{d}s'. \tag{B.2.18}$$

En isolant $L(d)$ et en multipliant par $\mu(d)^{-1} = \exp\left(-\int_0^d k(t)\,\mathrm{d}t\right)$:

$$L(d) = L(0)\exp\left(-\int_0^d k(t)\,\mathrm{d}t\right) + \int_0^d L_i(s')\exp\left(\int_0^{s'} k(t)\,\mathrm{d}t - \int_0^d k(t)\,\mathrm{d}t\right)\mathrm{d}s'. \tag{B.2.19}$$

Par la relation de Chasles, $\int_0^{s'} k(t)\,\mathrm{d}t - \int_0^d k(t)\,\mathrm{d}t = -\int_{s'}^d k(t)\,\mathrm{d}t$. On définit alors la *transmittance volumique* (loi de Beer-Lambert) entre deux abscisses quelconques $a \leqslant b$:

$$\tau(a,b) = \exp\left(-\int_a^b k(t)\,\mathrm{d}t\right) \in [0,1]. \tag{B.2.20}$$

On identifie directement $\tau(0,d) = \tau(x_0,x)$ pour l'atténuation du rayonnement de surface $L(0) = L_r(x_0 \to \vec{\omega})$, et $\tau(s',d) = \tau(s',x)$ pour l'atténuation du terme source volumique émis en s'. L'évaluation finale en x donne la formule intégrale recherchée.

B.2.4 De l'ETR à l'équation du rendu de Kajiya

La forme intégrale de l'ETR fait apparaître une condition aux limites : la radiance $L_r(x_0 \to \vec{\omega})$ réfléchie par la surface d'un objet matériel situé en x_0.
Dans la majorité des scènes de synthèse classiques, on fait l'hypothèse d'un *milieu non participant* (air clair, vide). L'absence d'absorption et de diffusion dans le milieu implique $k(s) = 0$ et $L_i(s) = 0$. La transmittance est alors totale ($\tau(x_0,x) = 1$) et la radiance se conserve le long du rayon :

$$L(x \leftarrow \vec{\omega}) = L_r(x_0 \to \vec{\omega}). \tag{B.2.21}$$

Le problème du rendu se ramène dès lors entièrement à l'évaluation de cette condition aux limites sur les surfaces de la scène.

Proposition (Formulation directionnelle – hémisphérique) Considérons un point x sur une surface opaque orientée par sa normale unitaire sortante $\vec{n}$. On note $\Omega^+ = \{\vec{\omega} \in \mathbb{S}^2 \mid \langle\vec{\omega},\vec{n}\rangle \geqslant 0\}$ l'hémisphère supérieur visible depuis x. La radiance sortante dans une direction d'observation ω_r résulte de deux contributions énergétiques :

- L'émission propre de la surface $L_e(x \to \omega_r)$ (non nulle si la surface est une source de lumière).
- La réflexion de la radiance incidente provenant de l'ensemble des directions $\omega_i \in \Omega^+$.

Chaque direction incidente ω_i apporte une irradiance élémentaire $\mathrm{d}E_i = L_i(x \leftarrow \omega_i)\cos\theta_i \mathrm{d}\omega_i$. En intégrant ces contributions pondérées par la BRDF du matériau f_r, on obtient la formulation hémisphérique suivante.

Théorème (Équation du rendu – Kajiya, 1986) . La radiance sortante en un point de surface x dans la direction ω_r s'exprime par :

$$L_r(x \to \omega_r) = L_e(x \to \omega_r) + \int_{\Omega^+} f_r(x, \omega_i, \omega_r)\, L_i(x \leftarrow \omega_i) \cos\theta_i \,\mathrm{d}\omega_i. \tag{B.2.22}$$

Pour résoudre cette équation par des méthodes globales (radiosité, algorithmes de Monte-Carlo), il est commode de transformer l'intégrale directionnelle sur le dôme Ω^+ en une intégrale spatiale sur l'ensemble des surfaces visibles de la scène, noté $\mathcal{M}$.
Soit un point $x' \in \mathcal{M}$ visible depuis x le long de la direction incidente ω_i. L'élément d'angle solide $\mathrm{d}\omega_i$ sous-tendu par la surface élémentaire $\mathrm{d}A(x')$ s'écrit :

$$\mathrm{d}\omega_i = \frac{\cos\theta_{x'}}{\|x - x'\|^2}\,\mathrm{d}A(x') \tag{B.2.23}$$

où $\theta_{x'}$ est l'angle entre la normale en x' et la direction du rayon vers x. En injectant cette relation dans le terme sous l'intégrale et en notant θ_x l'angle d'incidence en x, on obtient le terme différentiel de transport :

$$\cos\theta_x \mathrm{d}\omega_i = \frac{\cos\theta_x \cos\theta_{x'}}{\|x - x'\|^2}\,\mathrm{d}A(x'). \tag{B.2.24}$$

Pour étendre l'intégration à l'ensemble des surfaces $\mathcal{M}$, on introduit :

- La fonction de visibilité binaire $V(x, x') \in \{0, 1\}$, qui vaut 1 si le segment reliant x et x' n'est intercepté par aucun obstacle, et 0 sinon.
- Le *terme géométrique* symétrique $G(x \leftrightarrow x')$:

$$G(x \leftrightarrow x') = V(x, x')\frac{\cos\theta_x \cos\theta_{x'}}{\|x - x'\|^2}. \tag{B.2.25}$$

En exploitant l'invariance de la radiance le long du rayon dans le vide ($L_i(x \leftarrow \omega_i) = L(x' \to x)$), l'équation du rendu prend sa forme surfacique intégrale de Fredholm de seconde espèce :

Proposition (Formulation surfacique de l'équation du rendu) Pour trois points consécutifs $x', x, x'' \in \mathcal{M}$:

$$L(x \to x'') = L_e(x \to x'') + \int_{\mathcal{M}} f_r(x' \to x \to x'')\, L(x' \to x)\, G(x \leftrightarrow x')\,\mathrm{d}A(x'). \tag{B.2.26}$$

L'équation surfacique ci-dessus explicite le bilan énergétique via une chaîne de trois points $x' \to x \to x''$ sur la variété $\mathcal{M}$:

- x' : un point émetteur ou réflecteur quelconque de la scène ;
- x : le point matériel d'interaction en cours d'évaluation ;
- x'' : le point de destination (capteur ou surface suivante).

L'intégrale cumule la contribution de chaque élément d'aire infinitésimal de la scène. Sous l'intégrale, le produit se décompose en trois termes physiques :

1. *Radiance émise ou réfléchie en amont* $L(x' \to x)$: la quantité de lumière quittant x' en direction de x.
2. *Noyau géométrique* $G(x \leftrightarrow x')$: il couple spatialement les deux éléments de surface selon :

$$G(x \leftrightarrow x') = V(x, x') \frac{\cos\theta_x \cos\theta_{x'}}{\|x - x'\|^2}. \tag{B.2.27}$$

   La visibilité binaire $V(x, x')$ gère l'occlusion (ombres portées), le dénominateur $\|x - x'\|^2$ traduit l'atténuation en carré inverse de la distance, et les cosinus projettent les aires selon l'inclinaison relative des normales. Le produit $L(x' \to x)G(x \leftrightarrow x')\mathrm{d}A(x')$ quantifie ainsi l'irradiance différentielle $\mathrm{d}E(x)$ reçue en x depuis la pastille $\mathrm{d}A(x')$.
3. *Fonction de diffusion bidirectionnelle* $f_r(x' \to x \to x'')$: elle pondère quelle proportion de cette énergie incidente en x est renvoyée précisément vers l'observateur en x''.

B.3 Modélisation de la réflexion

Dans le développement précédent ayant permis l'énoncé de l'équation du transfert radiatif et l'équation du rendu, nous avons utilisé une application f_r exprimant la proportion de l'énergie incidente sur une surface renvoyé vers la scène. En supposant que la radiance du rayon lumineux $\vec{u}$ est invariante (i.e $(L_i(x \to \omega_i) = L(x' \to x))$), alors c'est ce terme f_r qui dicte quelle partie de l'énergie électromagnétique est perdue par rebonds successifs sur les objets de la scène.
Dans cette section, nous développons donc les outils physiques et mathématiques permettant de l'exprimer en décrivant les phénomènes de base de la réflexion jusqu'aux méthodes d'approximation utilisées en pratique.

B.3.1 L'intuition de la corde : transversalité et polarisation

Un faisceau lumineux se déplace en ligne droite dans un milieu homogène tant qu'il ne rencontre pas d'interface entre deux milieux aux propriétés optiques différentes. Pour comprendre la nature de son onde sans recourir aux équations de Maxwell, on peut se le représenter comme une corde tendue perpendiculairement à un mur en face de soi.

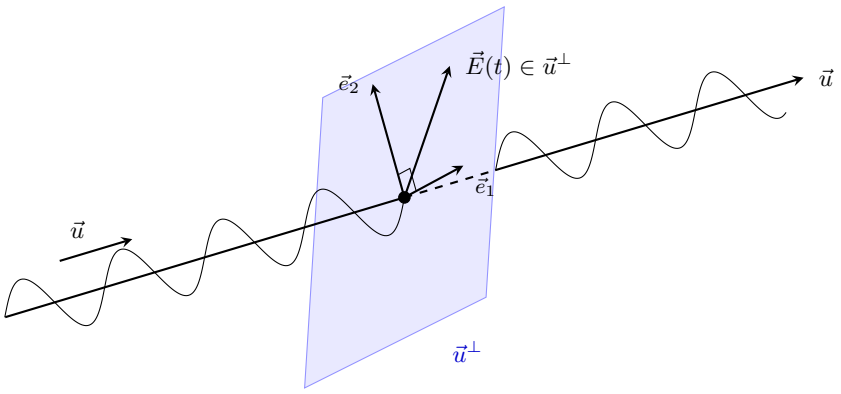


FIGURE B.4 – Propagation libre le long de $\vec{u}$ et confinement de l'oscillation transverse $\vec{E}(t)$ dans le plan orthogonal $\vec{u}^\perp = \mathrm{Vect}(\vec{e}_1, \vec{e}_2)$.

Si l'on secoue la corde, la perturbation ondulatoire se propage vers l'avant, mais le déplacement de la corde s'effectue toujours de manière transverse (perpendiculaire à l'axe de visée). On peut

secouer la corde de gauche à droite ou de bas en haut : la trajectoire du mouvement se projette alors sur le mur dans un plan orthogonal à la corde. Ce plan vectoriel de dimension 2 admet une base orthonormée $(\vec{e}_1, \vec{e}_2)$ qui permet de décomposer toute oscillation transverse arbitraire. En transposant cette analogie à la lumière, la corde matérialise la direction de propagation $\vec{u}$. En chaque point du trajet, le champ électrique $\vec{E}(t)$ est strictement confiné dans le plan orthogonal $\vec{u}^{\perp}$. Les deux directions de base $(\vec{e}_1, \vec{e}_2)$ représentent les deux états de polarisation linéaire indépendants de l'onde. La fréquence temporelle d'oscillation fixe la longueur d'onde λ (la couleur spectrale), tandis que l'amplitude du vecteur $\vec{E}(t)$ régit l'énergie transportée.

B.3.2 Réflexion & Réfraction

Les principes de réflexion et réfraction décrivent le comportement d'un faisceau traversant une discontinuité entre deux milieux aux propriétés optiques distinctes. Pour quantifier la réponse de ces milieux vis-à-vis de la lumière, on introduit la notion *d'indice de réfraction.*

Définition (**Indice de réfraction**) . Soit un milieu homogène et isotrope, on appelle *indice de réfraction* n de ce milieu le rapport adimensionnel entre la vitesse de la lumière dans le vide c et la vitesse de phase v de l'onde dans ce milieu :

$$n = \frac{c}{v}. \tag{B.3.1}$$

Puisque la vitesse dans la matière vérifie $v \leqslant c$, on a généralement $n \geqslant 1$ dans le domaine optique. Plus l'indice n est élevé, plus le milieu est dit *réfringent* et plus la propagation de l'onde y est ralentie.

Soient deux milieux d'indices respectifs n_1 (milieu incident) et n_2 (milieu de transmission). On étudie le comportement d'un rayon lumineux frappant une interface plane séparant ces deux milieux (voir Figure B.5).

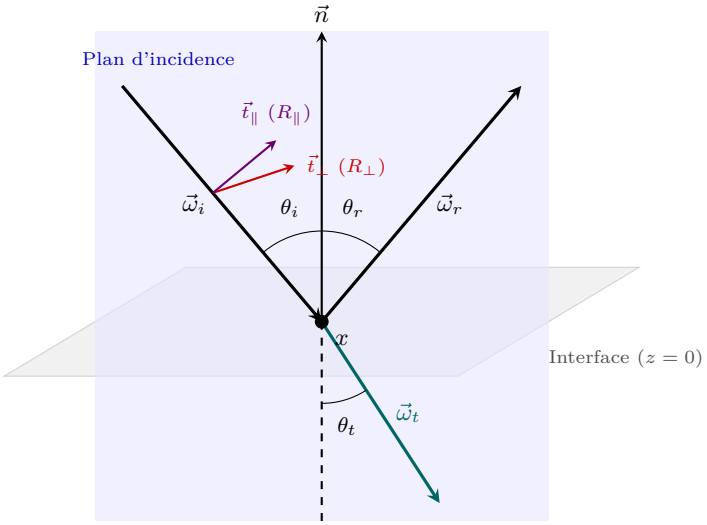


FIGURE B.5 – Décomposition des composantes de polarisation orthogonale $(R_{\perp})$ et parallèle $(R_{\parallel})$ par rapport au plan d'incidence.

Le rayon incident est modélisé par sa direction unitaire $\vec{\omega}_i$, faisant un angle $\theta_i \in [0, \frac{\pi}{2}]$ avec la normale sortante $\vec{n}$ à l'interface. L'interaction génère deux composantes :

- un rayon réfléchi $\vec{\omega}_r$, dont l'angle vérifie la loi de réflexion spéculaire $\theta_r = \theta_i$;

- un rayon réfracté (ou transmis) $\vec{\omega}_t$, dont l'angle θ_t dans le second milieu est dicté par la loi de Snell-Descartes :
$$n_1 \sin\theta_i = n_2 \sin\theta_t. \tag{B.3.2}$$

Remarque (Réflexion totale interne) Lorsque le faisceau se propage d'un milieu plus réfringent vers un milieu moins réfringent ($n_1 > n_2$), la quantité $\frac{n_1}{n_2}\sin\theta_i$ peut excéder 1. Il existe alors un angle critique $\theta_c = \arcsin\left(\frac{n_2}{n_1}\right)$ au-delà duquel aucun rayon ne peut être transmis dans le second milieu : toute l'énergie est réfléchie dans le premier milieu (*réflexion totale interne*).

Proposition (Décomposition géométrique de la polarisation) Pour analyser l'oscillation du champ électrique transverse $\vec{E} \in \vec{\omega}_i^{\perp}$, la présence de la surface brise l'isotropie spatiale et impose un plan de référence. Deux vecteurs non colinéaires $\vec{\omega}_i$ et $\vec{n}$ définissent le *plan d'incidence* $\mathcal{P} = \mathrm{Vect}(\vec{\omega}_i, \vec{n})$.
Le plan des oscillations permises $\vec{\omega}_i^{\perp}$ admet alors une décomposition canonique en deux directions orthogonales :

1. **Direction orthogonale $\vec{t}_\perp$ (polarisation s)** : perpendiculaire au plan d'incidence $\mathcal{P}$, obtenue par produit vectoriel :
$$\vec{t}_\perp = \frac{\vec{\omega}_i \times \vec{n}}{\|\vec{\omega}_i \times \vec{n}\|}. \tag{B.3.3}$$
Ce vecteur est par construction orthogonal à $\vec{n}$: il oscille parallèlement à l'interface matérielle.
2. **Direction parallèle $\vec{t}_\parallel$ (polarisation p)** : contenue dans le plan d'incidence $\mathcal{P}$ et orthogonale à la direction de propagation $\vec{\omega}_i$:
$$\vec{t}_\parallel = \vec{\omega}_i \times \vec{t}_\perp. \tag{B.3.4}$$

Toute onde incidente se projette de façon unique sur cette base : $\vec{E}_i = E_\perp \vec{t}_\perp + E_\parallel \vec{t}_\parallel$.

Proposition (Équations de Fresnel et bilan énergétique) La conservation de l'énergie impose que la somme de la réflectance R et de la transmittance T vaille 1. Les équations de Fresnel fournissent les coefficients de réflexion énergétique exacts pour chacune des deux polarisations :

$$R_\perp(\theta_i) = \left(\frac{n_1\cos\theta_i - n_2\cos\theta_t}{n_1\cos\theta_i + n_2\cos\theta_t}\right)^2 = \left(\frac{\sin(\theta_i-\theta_t)}{\sin(\theta_i+\theta_t)}\right)^2, \tag{B.3.5}$$

$$R_\parallel(\theta_i) = \left(\frac{n_2\cos\theta_i - n_1\cos\theta_t}{n_2\cos\theta_i + n_1\cos\theta_t}\right)^2 = \left(\frac{\tan(\theta_i-\theta_t)}{\tan(\theta_i+\theta_t)}\right)^2. \tag{B.3.6}$$

En synthèse d'images, la lumière naturelle et les sources d'éclairage sont modélisées comme non polarisées : les directions d'oscillation dans $\vec{\omega}_i^{\perp}$ sont réparties de manière isotrope. Le coefficient de Fresnel effectif $F(\theta_i)$ est la moyenne équipondérée des deux modes ($s = p = 0{,}5$) :

$$F(\theta_i) = \frac{R_\perp(\theta_i) + R_\parallel(\theta_i)}{2}. \tag{B.3.7}$$

Proposition (Approximation de Schlick) Pour éviter l'évaluation trigonométrique coûteuse des équations (B.3.5) et (B.3.6) lors du tracé de rayons, on utilise l'approximation polynomiale de Schlick :
$$F(\theta_i) \approx R_0 + (1 - R_0)(1 - \cos\theta_i)^5 \tag{B.3.8}$$
où R_0 représente la réflectance mesurée à incidence normale ($\theta_i = 0$, soit $\cos\theta_i = 1$) :
$$R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2. \tag{B.3.9}$$

Cette approximation reproduit fidèlement deux propriétés physiques majeures :

- À incidence normale ($\theta_i \approx 0$), la réflectance vaut R_0 (typiquement de l'ordre de 0,04 pour des matériaux tels que verre et l'eau) ;
- À incidence rasante ($\theta_i \to \frac{\pi}{2}$, $\cos\theta_i \to 0$), le terme $(1-\cos\theta_i)^5 \to 1$, d'où $F(\theta_i) \to 1$: toute surface devient un réflecteur parfait aux angles rasants.

B.3.3 Fonction de distribution de la réflectance bidirectionnelle - BRDF

Les modèles vus précédement caractérisent la réponse locale d'une interface plane et optiquement lisse, où la réflexion est exclusivement spéculaire. Pour modéliser des matériaux macroscopiques réels présentant de la rugosité ou des phénomènes de diffusion interne, il est nécessaire d'introduire un formalisme reliant l'irradiance incidente à la radiance réémise dans chaque direction de l'espace. Ici, nous commençons par présenter le modèle général puis proposons une simplification utilisant des lois de probabilités.
Dans le cas le plus général, le transport de lumière à une frontière matérielle est un phénomène continu à 12 dimensions :

$$S(x_i, \vec{\omega}_i, t_i, \lambda_i \ \to \ x_r, \vec{\omega}_r, t_r, \lambda_r) \tag{B.3.10}$$

où x_i, x_r sont les positions spatiales d'entrée et de sortie, $\vec{\omega}_i, \vec{\omega}_r$ les directions sphériques correspondantes, t_i, t_r les instants d'interaction, et λ_i, λ_r les longueurs d'onde associées.

Proposition (Simplification) En synthèse d'images usuelle, trois hypothèses simplificatrices sont admises :

- **Indépendance temporelle** ($t_i = t_r$) : la vitesse de propagation est considérée infinie à l'échelle de la scène, excluant les effets de phosphorescence.
- **Cohérence spectrale** ($\lambda_i = \lambda_r$) : la lumière réfléchie conserve sa longueur d'onde incidente, excluant la fluorescence.
- **Hypothèse de localité spatiale** ($x_i \approx x_r = x$) : pour les matériaux opaques, la distance de diffusion interne sous la surface est négligeable devant les échelles macroscopiques. La fonction de diffusion sous-surfacique (BSSRDF, en dimension 8) se réduit alors à une fonction locale définie en un unique point matériel x.

Cette réduction dimensionnelle aboutit à la *Bidirectional Reflectance Distribution Function* (BRDF), fonction à quatre paramètres angulaires $(\theta_i, \phi_i, \theta_r, \phi_r)$ paramétrée par la longueur d'onde λ.

Définition (**BRDF**) . Soit un point x sur une surface, la BRDF, notée f_r, est le rapport différentiel entre la radiance réfléchie $\mathrm{d}L_r$ dans la direction $\vec{\omega}_r$ et l'irradiance incidente différentielle $\mathrm{d}E_i$ arrivant depuis le secteur solide élémentaire $\mathrm{d}\omega_i$ autour de $\vec{\omega}_i$:

$$f_r(x, \vec{\omega}_i, \vec{\omega}_r, \lambda) = \frac{\mathrm{d}L_r(x \to \vec{\omega}_r, \lambda)}{\mathrm{d}E_i(x \leftarrow \vec{\omega}_i, \lambda)} = \frac{\mathrm{d}L_r(x \to \vec{\omega}_r, \lambda)}{L_i(x \leftarrow \vec{\omega}_i, \lambda)\cos\theta_i \,\mathrm{d}\omega_i}. \tag{B.3.11}$$

Elle s'exprime en stéradians inverses (sr^{-1}).

Propriété (**Fondamentales**) . Pour être physiquement admissible, tout modèle mathématique de BRDF doit impérativement respecter deux contraintes thermodynamiques :

1. **Réciprocité d'Helmholtz** : la fonction de transfert optique est invariante par inversion du sens de parcours de la lumière :

$$f_r(x, \vec{\omega}_i, \vec{\omega}_r, \lambda) = f_r(x, \vec{\omega}_r, \vec{\omega}_i, \lambda). \tag{B.3.12}$$

Cette propriété garantit l'équivalence entre les algorithmes traçant les rayons depuis l'observateur (*path tracing*) et ceux partant des sources de lumière (*ligh-tracing*).

2. **Conservation de l'énergie** : une surface passive ne pouvant amplifier l'énergie incidente, la puissance totale réémise sur l'hémisphère supérieur Ω^+ ne peut excéder la puissance reçue. Pour toute direction incidente $\vec{\omega}_i$:

$$\int_{\Omega^+} f_r(x, \vec{\omega}_i, \vec{\omega}_r, \lambda) \cos\theta_r \, \mathrm{d}\omega_r \leqslant 1. \tag{B.3.13}$$

Traitement du signal

=======

Planification

Chapitre 1

Formalisation de la Planification Classique

Contents

1.1 Le cadre classique 192
1.1.1 États et Actions 192
1.1.2 Plan d'action et problème de planification 194
1.1.3 Propriétés et hypothèses du cadre classique 194
1.2 Algorithmes et Langages de représentation 195
1.2.1 Typologie des approches algorithmiques 195
1.2.2 Formalisme STRIPS et résolution du problème du décor 196
1.2.3 Extension ADL – *Action Description Language* et PDDL 197

La planification est un outil essentiel dans beaucoup de domaines aujourd'hui et est utilisée par tout le monde : que ce soit pour prévoir des tâches à effectuer, suivre une recette de cuisine ou anticiper et prévoir l'évolution d'un programme industriel.
Toutes ces applications peuvent être formalisées et généralisées au sein de la théorie de la planification. Celle-ci cherche à synthétiser et résoudre de manière automatique un *problème de planification* par la production d'un *plan*. Ce dernier peut se voir comme une collection d'actions à effectuer pour modifier l'état courant d'une tâche dans le but d'atteindre un *état cible* appelé *but*. L'exécution de ce plan est donc la réalisation de ces différentes étapes dans le monde pour lequel a été défini le problème.
Nous allons ici formaliser mathématiquement ces concepts pour l'instant abstrait et brièvement étudier la notion de complexité de résolution de tels problèmes.

1.1 Le cadre classique

Commençons donc par définir formellement ce qu'est un problème de planification et les différents objets utilisés pour le résoudre.

1.1.1 États et Actions

Définition (**État – Fluent**) . On appelle *état* un ensemble fini de formules atomiques sans symbole de variable pour un monde défini. Cet ensemble représente un état possible

du problème considéré.
Une formule atomique (suite finie d'éléments d'un alphabet ne contenant aucun symbole de variable) est appelée un *fluent*.

Pour passer d'un état à un autre, on définira des *actions*.

Définition (**Opérateur – Action**) . On appellera *opérateur* o un modèle d'action. Il est défini par deux éléments :

1. son nom,
2. un triplet d'ensembles $\langle \text{pr}, \text{add}, \text{de} \rangle$ finis de formules atomiques représentant respectivement les *préconditions*, les *ajouts* et les *suppression* de fluents opérés par l'opérateur sur l'état courant.

On appellera *action*, notée a, une instance d'un opérateur o (i.e toutes les variables de o sont instanciées à partir des variables/constantes du problème). On notera $\text{Pre}(o), \text{Add}(o)$ et $\text{Del}(o)$ les ensembles pr, add et de de l'opérateur o (idem pour a).

Définition (**Action Applicable**) . Soit E un état et a une action d'un problème de planification. On dira que a est applicable à E si les préconditions de a son vérifiées dans E, formellement, cela revient à dire que :

$$\text{Pre}(a) \subseteq E. \tag{1.1.1}$$

On définira alors l'état résultant E' de l'action de a sur E par :

$$\boxed{E' := E \uparrow a = (E \setminus \text{Del}(a)) \cup \text{Add}(a).} \tag{1.1.2}$$

Exemple Considérons la préparation d'un gâteau nécessitant de mélanger des ingrédients puis de les cuire. Définissons un opérateur permettant de cuire un ingrédient ou une préparation désignée par la variable $?x$:

$$\begin{aligned} \text{Cuire}(?x) = \big\langle\, & \text{Pre} = \{\text{melange}(?x), \text{cru}(?x), \text{four_chaud}\}, \\ & \text{Add} = \{\text{cuit}(?x)\}, \\ & \text{Del} = \{\text{cru}(?x)\}\big\rangle \end{aligned}$$

En instanciant la variable $?x$ avec la constante pate, on obtient l'action close :

$$\begin{aligned} a = \text{Cuire}(\text{pate}) = \big\langle\, & \text{Pre}(a) = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}\}, \\ & \text{Add}(a) = \{\text{cuit}(\text{pate})\}, \\ & \text{Del}(a) = \{\text{cru}(\text{pate})\}\big\rangle \end{aligned}$$

Soit l'état courant :

$$E = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule})\}$$

Puisque $\text{Pre}(a) \subseteq E$, l'action a est applicable. L'état résultant $E' = E \uparrow a$ est :

$$\begin{aligned} E' &= \big(E \setminus \text{Del}(a)\big) \cup \text{Add}(a) \\ &= \{\text{melange}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule}), \text{cuit}(\text{pate})\} \end{aligned}$$

On note que le fait propre(moule) reste vrai dans E' sans qu'il ait été nécessaire de mentionner son invariance (résolution du problème du décor).

1.1.2 Plan d'action et problème de planification

La résolution d'un problème se fait par l'application séquentielle d'actions applicables sur un état initial E : on appelera cela un *plan séquentiel*.

Définition (**Plan Séquentiel & Application**) . On définit un *plan séquentiel*, P, comme une séquence finie d'actions $\langle a_1, a_2, \ldots, a_n \rangle$. L'application d'un plan P à un état E, notée $E\mathcal{A}P$, est définie par récurrence sur la longueur de P :

- Si $E = \bot$, alors $\bot\mathcal{A}P := \bot$.
- Pour le plan vide : $E\mathcal{A}\langle\rangle := E$.
- Pour $P = \langle a \rangle \cdot P'$ (où a est une action et P' une séquence d'actions) :

$$E\mathcal{A}(\langle a \rangle \cdot P') := \begin{cases} (E \uparrow a)\mathcal{A}P' & \text{si } \mathrm{Prec}(a) \subseteq E, \\ \bot & \text{sinon.} \end{cases} \tag{1.1.3}$$

On peut alors définir le concept de *problème de planification*, noté $\langle \mathcal{O}, I, B \rangle$ tel que :

- $\mathcal{O}$ désigne l'ensemble des opérateurs utilisables dans le domaine considéré. A désigne l'ensemble des actions que l'on peut instancier à partir des opérateurs de $\mathcal{O}$.
- I est l'état initial du problème composé d'un ensemble de fluents.
- B est le but du problème (un état), aussi composé de fluents.

L'objectif de la planification étant de développer un plan séquentiel permettant de résoudre le problème $\langle \mathcal{O}, I, B \rangle$, on appellera un tel plan un *plan solution*.

Définition (**Plan-solution**) . Soit $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ un problème de planification. Un plan séquentiel P est un *plan-solution* (ou *plan correct*) si et seulement si :

$$I\mathcal{A}P \neq \bot \quad \text{et} \quad B \subseteq (I\mathcal{A}P) \tag{1.1.4}$$

Remarque (Flexibilité) La *flexibilité* d'un plan-solution désigne sa capacité à subir des modifications (modification de l'ordonnancement d'actions non contraintes, instanciation retardée de variables libres) sans perdre sa correction.

1.1.3 Propriétés et hypothèses du cadre classique

La production d'un *plan-solution* sera réalisée par un algorithme de planification, noté Planif. On pourra définir plusieurs propriétés recherchées pour cet algorithme.

Propriété (**Planificateur**) . Soit un algorithme de planification Planif opérant sur un formalisme donné.

- **Sain (ou correct) :** Planif est sain si tout plan qu'il produit pour une instance $\mathcal{P}$ est un plan-solution.
- **Complet :** Planif est complet si, dès lors qu'il existe au moins un plan-solution pour $\mathcal{P}$, il en produit un en temps fini.
- **Systématique :** Planif est systématique si deux feuilles distinctes quelconques de son arbre de recherche correspondent à des plans-solutions distincts (absence de redondance dans l'exploration).
- **Optimal :** Planif est optimal relativement à un critère de coût donné (nombre d'actions, temps d'exécution, parallélisme) si tout plan-solution produit minimise ce critère.

Toute la complexité de la planification réside donc dans la recherche d'un tel algorithme.

Proposition (Hypothèses du cadre classique) Le cadre classique de la planification restreint l'environnement et l'espace d'actions selon six hypothèses :

1. **Fini :** L'ensemble des objets et des fluents est fini.
2. **Statique :** Le monde n'évolue que sous l'effet des actions de l'agent (pas d'événements exogènes).
3. **Observabilité totale :** L'état courant est parfaitement et entièrement connu à chaque étape.
4. **Déterministe :** L'application d'une action a sur un état E produit un unique état résultant prévisible.
5. **Discret :** Les transitions d'états s'effectuent par sauts discrets.
6. **Instantané :** Les actions n'ont pas de durée propre (durée nulle) et ne consomment pas de ressources continues.

La résolution d'un problème de planification par la recherche d'un plan-solution est assez difficile car elle nécessite de définir l'ensemble des objets pour modéliser le problème (états, fluents, actions, etc...). Il faut définir des actions pertinentes en fonction des états choisis et être capable de les décrire au mieux pour permettre un ordonnancement exécutable des plans d'actions générés. Le théorème ci-dessous donne un ordre d'idée de la complexité de cette recherche.

Théorème (**Complexité en cadre classique**) . Pour un problème de planification classique $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ représenté en logique propositionnelle / STRIPS :

- Le problème de décision de l'*existence d'un plan-solution* (PLANEXIST) est **PSPACE-complet**.
- Le problème de décision de l'*existence d'un plan-solution de longueur inférieure ou égale à* $k \in \mathbb{N}$ (avec k encodé en unaire) est **NP-complet**.

1.2 Algorithmes et Langages de représentation

Cette section introduit les principaux paradygmes de résolution d'un problème de planification. On présentera les différentes méthodes canoniques avant d'introduire les langages de représentation (PDDL, STRIPS) et nous finirons par une introduction à GRAPHLAN avant de le traiter plus en détail dans le chapitre suivant.

1.2.1 Typologie des approches algorithmiques

La résolution effective d'un problème de planification $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ se heurte à l'explosion combinatoire de l'espace de recherche. Les algorithmes du cadre classique se divisent en deux grandes familles fondamentales.
La première se concentre sur une recherche directe dans les espaces de recherche. Cette approche explore explicitement un graphe orienté où chaque transition correspond à une décision de planification. On distingue deux espaces fondamentaux :

- **Espace des états :** Les nœuds du graphe sont des états complets du monde (voir Figure 1.1). On peut alors trouver un plan-solution par *progression (chaînage avant)* en explorant l'espace depuis l'état initial I et en appliquant des actions applicables jusqu'à atteindre un état contenant B. Ou alors, par *régression (chaînage arrière)* : on part de la spécification du but B et on remonte vers l'état initial I par calcul de préconditions inverses.

- **Espace des plans partiels (POCL – *Partial Order Causal Link*) :** Les nœuds sont des plans incomplets (actions partiellement ordonnées, variables non totalement instanciées et liens causaux). La recherche consiste à éliminer les défauts du plan (préconditions non satisfaites ou conflits causaux) jusqu'à obtenir un plan valide.

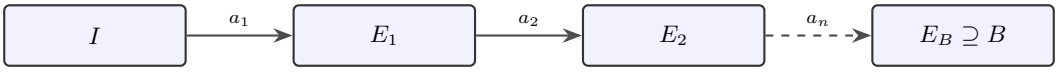


FIGURE 1.1 – Schéma d'un espace d'états d'un problème de planification. On représente les états dans un graphe orienté où chaque arrête traduit l'effet d'une action applicable entre deux états.

Plutôt que d'explorer l'espace d'états de manière séquentielle, les approches par *compilation de plans* (voir Figure 1.2) traduisent le problème de planification de taille ou d'horizon fixé k en une instance d'un problème cible bien caractérisé, bénéficiant ainsi de l'efficacité des solveurs génériques modernes.

- **SATPLAN / BLACKBOX (Compilation propositionnelle) :** Le problème de planification borné à k pas de temps est codé sous la forme d'une formule propositionnelle en forme normale conjonctive (CNF). Un solveur SAT (DPLL, CDCL) recherche une affectation satisfaisable dont le modèle est ensuite décodé en un plan-solution.
- **GP-CSP (Compilation en CSP) :** Les interactions entre fluents et actions sont modélisées sous la forme d'un problème de satisfaction de contraintes (CSP) où les variables représentent les supports d'actions et les contraintes modélisent les exclusions et préconditions.
- **GRAPHPLAN :** Approche intermédiaire qui commence par construire en temps polynomial un graphe compact alternant couches de fluents et d'actions, annoté d'exclusions mutuelles (*mutex*), avant d'y extraire une solution par retour arrière ou compilation.

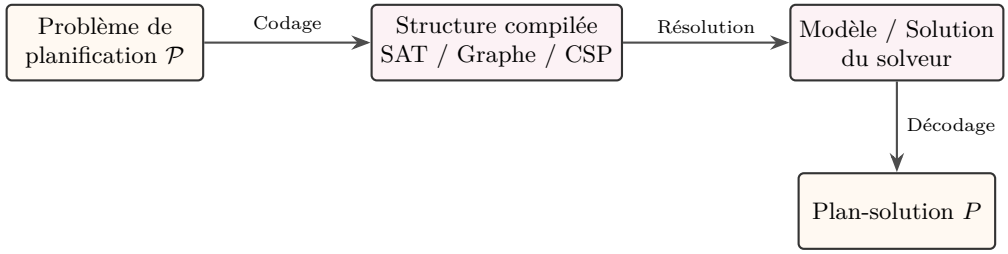


FIGURE 1.2 – Schéma représentant les étapes de construction d'un plan-solution à l'aide d'une méthode par compilation de plans.

La formalisation effective d'un univers de planification impose un compromis fondamental entre le pouvoir expressif du langage de modélisation et la complexité algorithmique du calcul des transitions d'états. L'évolution des formalismes historiques traduit la recherche de cet équilibre.

1.2.2 Formalisme STRIPS et résolution du problème du décor

Le formalisme STRIPS (*Stanford Research Institute Problem Solver*) constitue le noyau minimaliste de la planification classique.

Définition (**Modélisation STRIPS**) . Dans le formalisme STRIPS, le langage est restreint à la logique propositionnelle sans négation explicite :

- Un état $E \subseteq \mathcal{F}$ est une conjonction finie de formules atomiques closes (fluents). Les faits absents de E sont réputés faux en vertu de l'*hypothèse du monde clos*.
- Un opérateur o est défini par le triplet $\langle \text{Prec}(o), \text{Add}(o), \text{Del}(o) \rangle$, où chaque ensemble ne contient que des littéraux atomiques positifs.

Proposition (Résolution du problème du décor – *Frame Problem*) Soit une action a applicable sur un état E (i.e $\text{Prec}(a) \subseteq E$). STRIPS résout le problème du décor par le principe de persistance implicite :

$$\forall f \in E, \quad f \notin \text{Del}(a) \implies f \in (E \uparrow a) \tag{1.2.1}$$

Aucun axiome d'invariance n'est nécessaire : les fluents non mentionnés dans $\text{Del}(a)$ sont conservés par défaut après l'application de a sur l'état E.

Remarque (Verrous expressifs de STRIPS) La simplicité de STRIPS garantit un calcul de transition $E \uparrow a$ en temps linéaire, mais restreint la modélisation. En effet, il devient impossible de formuler des préconditions négatives ou disjonctives. L'impact d'une action sur un état est aussi statique et indépendant de faits périphériques. Enfin, il est impossible de quantifier sur des ensembles : modifier n entités impose de concevoir et planifier n actions unitaires distinctes.

1.2.3 Extension ADL – *Action Description Language* et PDDL

Introduit pour lever la rigidité de STRIPS, ADL enrichit la description des opérateurs en intégrant des fragments de la logique du premier ordre.

Définition (**Caractéristiques du formalisme ADL**) . Dans ADL, un opérateur est caractérisé par un couple $\langle \text{Prec}(o), \text{Eff}(o) \rangle$:

- **Préconditions du premier ordre :** $\text{Prec}(o)$ autorise les connecteurs booléens usuels $(\wedge, \vee, \neg)$, les prédicats d'égalité $(=)$ et les quantificateurs $(\forall, \exists)$.
- **Effets conditionnels :** $\text{Eff}(o)$ accepte des clauses d'implication de la forme :

  $$c \longrightarrow e \tag{1.2.2}$$

  Le littéral e (positif ou négatif) n'est appliqué à l'état que si la condition c était satisfaite avant le déclenchement de l'action.
- **Effets universels :** Des expressions de type $\forall x \, (P(x) \rightarrow e(x))$ permettent de mettre à jour un ensemble arbitraire d'objets en une seule transition.

PDDL - *Planning Domain Description Language* sépare formellement la physique du système de l'instance concrète à résoudre afin de rendre les solveurs indépendants des domaines d'application.

Définition (**Architecture PDDL**) . Une instance PDDL dissocie la modélisation en deux composantes :

1. **Fichier Domaine (`domain.pddl`) :** Spécifie les types d'objets, la liste des prédicats relationnels admis et les schémas d'actions paramétrés.
2. **Fichier Problème (`problem.pddl`) :** Déclare les constantes réelles (objets), l'état initial exhaustif I et la formule logique du but B.

Chapitre 2

Le Planificateur GRAPHPLAN

Contents

2.1 Définitions et fonctionnement 198
2.1.1 Interactions entre actions et Indépendance 198
2.1.2 Exclusions mutuelles (*Mutex*) 200
2.1.3 Algorithme GRAPHPLAN 201
2.2 Améliorations de Graphplan 205
2.2.1 LCGP – Relations d'Autorisation 205
2.2.2 Codages SAT du graphe 206
2.2.3 Codages CSP du graphe 207

2.1 Définitions et fonctionnement

L'algorithme de planification GRAPHPLAN sépare le processus de planification en deux étapes différentes.
La première consiste en la construction d'un graphe de planification représentant les interdépendances entre les états au travers des actions possibles. Cette construction se fait avec une complexité polynômiale en temps et en espace par rapport à la taille du problème posé.
La seconde étape est la recherche d'un solution (s'il en existe) à partir d'un sous-graphe du précédent. Elle produit alors un plan-solution au problème initialement posé.

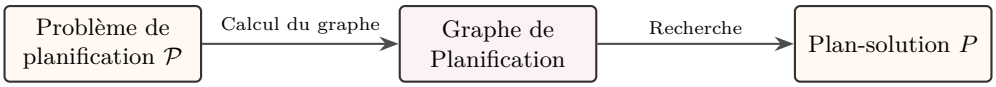


FIGURE 2.1 – Schéma représentant les étapes de l'algorithme GRAPHPLAN pour la recherche d'un plan solution à partir d'un problème de planification énoncé dans le langage STRIPS.

Nous allons ici commencer par poser un formalisme supplémentaire pour traduire les relations d'inter-dépendances dans un tel graphe d'état.

2.1.1 Interactions entre actions et Indépendance

Pour de gros problèmes de planification, tels que des programmes de production industriels ou des projets collaboratifs, il est intéressant de pouvoir exécuter plusieurs actions simultanément dans le graphe de planification. On construit pour cela des ensemble *d'actions indépendantes* Q.

Définition (**Interactions Positives/Négatives**) . Soient deux actions closes a_1 et a_2, on définit deux types d'interactions entre ces deux actions. On dira qu'elles sont en *coopération* ou qu'elles interagissent *positivement* si elles produisent les mêmes effets utiles :

$$\mathrm{Add}(a_1) \cap \mathrm{Add}(a_2) = \emptyset \tag{2.1.1}$$

ou si l'une produit une condition requise par l'autre :

$$\mathrm{Add}(a_1) \cap \mathrm{Prec}(a_2) = \emptyset. \tag{2.1.2}$$

En revanche, on dira qu'elles sont en *conflit* si l'une supprime ce que l'autre ajoute (effet antagoniste) :

$$\mathrm{Add}(a_1) \cap \mathrm{Del}(a_2) \neq \emptyset \quad \text{ou} \quad \mathrm{Add}(a_2) \cap \mathrm{Del}(a_1) \neq \emptyset \tag{2.1.3}$$

ou si l'une supprime une condition nécessaire au déclenchement de l'autre (effet croisé) :

$$\mathrm{Del}(a_1) \cap \mathrm{Prec}(a_2) \neq \emptyset \quad \text{ou} \quad \mathrm{Del}(a_2) \cap \mathrm{Prec}(a_1) \neq \emptyset. \tag{2.1.4}$$

L'objectif sera de limiter au plus les effets conflits entre actions pour favoriser les interactions positives en vue de construire un plan.

Définition (**Indépendance**) . On dira que deux actions a_1 et a_2 sont *indépendantes*, noté $a_1 \# a_2$, si elles n'ont pas d'interactions négatives (conflits) :

$$\mathrm{Del}(a_1) \cap (\mathrm{Prec}(a_2) \cup \mathrm{Add}(a_2)) = \emptyset \quad \text{et} \quad \mathrm{Del}(a_2) \cap (\mathrm{Prec}(a_1) \cup \mathrm{Add}(a_1)) = \emptyset. \tag{2.1.5}$$

On dira que Q est un ensemble d'actions indépendantes si toutes ses actions sont deux à deux indépendantes.

Propriété (**Indépendance**) . Soient deux actions a_1 et a_2 applicables sur un état E. Si a_1 et a_2 sont indépendantes, alors leur ordre d'application sur E n'a pas d'importance :

$$(E \uparrow a_1) \uparrow a_2 = (E \uparrow a_2) \uparrow a_1 = E \uparrow \{a_1, a_2\}. \tag{2.1.6}$$

L'intérêt de travailler avec des ensembles d'opérations indépendantes est de pouvoir toutes les exécuter à la fois dans uen couche du graphe.

Définition (**Action de maintien – *NoOp***) . Soit $f \in \mathcal{F}$ un fluent atomique. On appelle *action de maintien* (ou *NoOp*, pour *No Operation*), notée N_f, l'action triviale définie par :

$$N_f = \big\langle \mathrm{Prec}(N_f) = \{f\}, \quad \mathrm{Add}(N_f) = \{f\}, \quad \mathrm{Del}(N_f) = \emptyset \big\rangle \tag{2.1.7}$$

Dans le graphe de planification, les NoOps assurent deux fonctions fondamentales :

1. *Transport d'invariance :* modéliser explicitement la persistance d'un fait d'une couche F_{i-1} vers la couche suivante F_i sans altérer l'état du monde.
2. *Propagation des conflits :* soumettre la conservation d'un fait aux mêmes règles d'indépendance et d'exclusion mutuelle que les actions standard (notamment avec toute action qui supprime ce fait).

Proposition Soit Q un ensemble d'actions indépendantes et E un état. Alors Q est *applicable* sur E ssi :

$$\bigcup_{a_i \in Q} \mathrm{Prec}(a_i) \subseteq E. \tag{2.1.8}$$

L'état résultant de l'application de tous les $a_i \in Q$ sur E sera caractérisé par :

$$E \uparrow Q = \left(E \setminus \bigcup_{a_i \in Q} \mathrm{Del}(a_i) \right) \cup \left(\bigcup_{a_i \in Q} \mathrm{Add}(a_i) \right). \tag{2.1.9}$$

On peut alors appliquer un ensemble d'actions indépendantes à un état donné en une fois. On définira alors des *plans d'ensembles d'actions indépendantes* $P = \langle Q_1, Q_2, \ldots, Q_n \rangle$ où les Q_i sont des actions indépendantes. Leur application se fera donc successivement sur un état itinial E comme illustré en Figure 2.2.

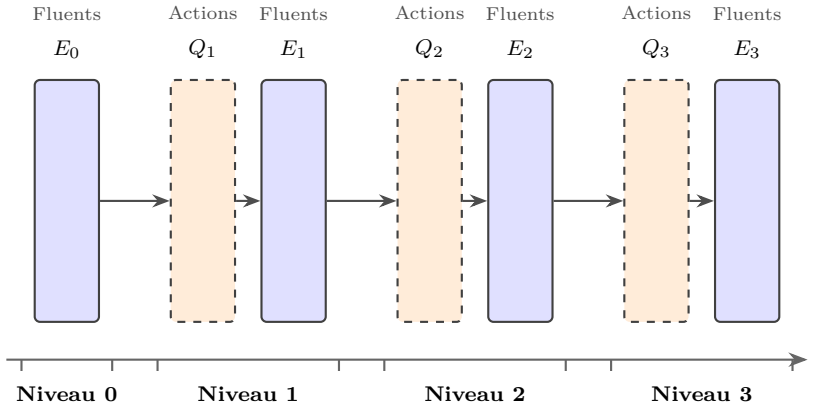


FIGURE 2.2 – Structure alternée par niveaux dans un plan d'ensembles d'actions. Chaque niveau $i \geqslant 1$ applique simultanément un ensemble d'actions indépendantes Q_i sur l'état courant pour produire le nouvel état de fluents E_i.

2.1.2 Exclusions mutuelles (*Mutex*)

Le graphe de planification sur-approximant l'ensemble des états atteignables, des relations d'exclusion mutuelle (*mutex*) sont introduites à chaque niveau pour contraindre la recherche et élaguer les combinaisons physiquement irréalisables. La relation est symétrique et binaire et est définie aussi bien sur les actions que sur les fluents.

Définition (**Exclusion mutuelle entre actions**) . Soient deux actions a_1, a_2 appartenant au même niveau d'actions i. Les actions a_1 et a_2 sont dites *mutuellement exclusives* au niveau i, noté $\mathrm{Mutex}(a_1, a_2)$, si et seulement si l'une des deux conditions suivantes est vérifiée :

1. **Non-indépendance (conflit direct) :** Les actions a_1 et a_2 ne sont pas indépendantes ($a_1 \# a_2$), c'est-à-dire :

$$\mathrm{Del}(a_1) \cap \big(\mathrm{Prec}(a_2) \cup \mathrm{Add}(a_2) \big) \neq \emptyset \quad \text{ou} \quad \mathrm{Del}(a_2) \cap \big(\mathrm{Prec}(a_1) \cup \mathrm{Add}(a_1) \big) \neq \emptyset \tag{2.1.10}$$

2. **Besoins concurrents (conflit hérité) :** Il existe au moins une paire de préconditions mutuellement exclusives au niveau de fluents précédent F_{i-1} :

$$\exists (p, q) \in \mathrm{Prec}(a_1) \times \mathrm{Prec}(a_2) \quad \text{tel que} \quad \mathrm{Mutex}(p, q) \text{ dans } F_{i-1} \tag{2.1.11}$$

Définition (**Exclusion mutuelle entre fluents**) . Soient deux fluents $p, q \in F_i$ au niveau de fluents i. Les fluents p et q sont dits *mutuellement exclusifs* au niveau i, noté Mutex(p, q), si et seulement si **toutes** les paires d'actions susceptibles de les produire simultanément à ce niveau sont en exclusion mutuelle :

$$\forall (a_1, a_2) \in A_i \times A_i, \quad \big(p \in \text{Add}(a_1) \wedge q \in \text{Add}(a_2)\big) \implies \text{Mutex}(a_1, a_2) \tag{2.1.12}$$

Autrement dit, p et q ne sont pas mutex dans F_i dès lors qu'il existe au moins une paire d'actions (a_1, a_2) non mutex à l'étape i telle que $p \in \text{Add}(a_1)$ et $q \in \text{Add}(a_2)$.

On pourra aussi définir deux types de mutex :

- *Les mutex structurels* : produits lorsqu'une action supprime des préconditions ou des ajouts d'une autre action. Elles seront alors toujours en mutex dans chaque couche.
- *Les mutex contextuels* : lorsque deux actions ont des mutex comme préconditions.

Remarque (Rôle des actions de persistance) Une action de maintien $N_f = \langle \{f\}, \{f\}, \emptyset \rangle$ est traitée comme une action usuelle. Par conséquent, si une action $a \in A_i$ supprime le fluent f ($f \in \text{Del}(a)$), elle entre directement en conflit de non-indépendance avec N_f car $\text{Del}(a) \cap \text{Add}(N_f) = \{f\} \neq \emptyset$. Cela garantit qu'un fluent f et sa négation deviennent automatiquement mutex au niveau suivant.

2.1.3 Algorithme GRAPHPLAN

Proposé par Blum et Furst (1995), GRAPHPLAN résout le problème de planification en découplant la recherche en deux phases complémentaires :

1. **Phase d'extension :** construction alternée de couches de fluents F_i et de couches d'actions A_i annotées de relations d'exclusion mutuelle. Cette phase est polynomiale en temps et en espace.
2. **Phase d'extraction :** recherche arrière par retour-arrière (*backtracking*) dans le sous-graphe pour identifier un plan valide. La complexité de cette phase est NP-Complète.

Le détail de l'algorithme est donné en Algorithme 5. Mais pour son application pratique à la main, la proposition suivante en détaille les étapes.

Proposition (Algorithme GRAPHPLAN) Le tracé manuel d'un graphe de planification s'effectue par colonnes alternées de gauche à droite. On commence par construire le graphe en développant les couches successivement et en calculant les mutex entre chaque couches. Une fois l'état final découvert, on peut tenter l'extraction d'un plan solution.

1. **Construction du graphe** :
   (a) **Initialisation du niveau 0 (F_0)** :
      - Lister verticalement tous les fluents de l'état initial : $F_0 = I$.
      - *Règle d'or :* Il n'y a *jamais* de relation d'exclusion mutuelle dans la couche initiale F_0.

   (b) **Construction de la couche d'actions (A_k depuis F_{k-1}) :**
      - *Actions de maintien (NoOps) :* Dessiner immédiatement une action N_f pour chaque fait $f \in F_{k-1}$.
      - *Actions du domaine :* Instancier une action $a \in \mathcal{A}$ si et seulement si tous ses fluents préconditions sont présents dans F_{k-1} et qu'aucune paire $(p, q) \subseteq \text{Prec}(a)$ n'est marquée par un arc mutex dans F_{k-1}.
      - Tracer les arcs orientés reliant chaque précondition dans F_{k-1} à son action dans A_k.

(c) **Audit des mutex d'actions dans A_k :** Pour chaque paire non ordonnée d'actions distinctes $\{a_1, a_2\} \subseteq A_k$, vérifier les deux conditions suivantes :

- *Conflit direct (non-indépendance) :* Vérifier si $\text{Del}(a_1) \cap (\text{Prec}(a_2) \cup \text{Add}(a_2)) \neq \emptyset$ ou symétriquement pour a_2. (Dès qu'une action a supprime un fluent f ($f \in \text{Del}(a)$), elle est *immédiatement* mutex avec le NoOp N_f car $\text{Del}(a) \cap \text{Add}(N_f) = \{f\} \neq \emptyset$.)
- *Besoins concurrents (conflit hérité) :* Vérifier s'il existe une paire $(p, q) \in \text{Prec}(a_1) \times \text{Prec}(a_2)$ telle que $\text{Mutex}(p, q)$ dans F_{k-1}.
- Si au moins une condition est satisfaite, tracer un lien d'exclusion mutuelle entre a_1 et a_2.

(d) **Construction de la couche de fluents (F_k depuis A_k) :**

- Reporter dans F_k tous les fluents produits par les effets positifs (Add) des actions et des NoOps de A_k.
- Relier chaque action de A_k à ses effets positifs dans F_k (et optionnellement marquer les effets de suppression Del en pointillés).

(e) **Audit systématique des mutex de fluents dans F_k :** Pour chaque paire de fluents distincts $\{p, q\} \subseteq F_k$:

- Identifier l'ensemble de tous les couples d'actions $(a_1, a_2) \in A_k \times A_k$ susceptibles de les générer conjointement ($p \in \text{Add}(a_1)$ et $q \in \text{Add}(a_2)$).
- Si *toutes* les paires de générateurs sont reliées par un mutex dans A_k, alors p et q sont déclarés mutuellement exclusifs dans F_k.
- Dès qu'il existe *au moins un couple* non mutex d'actions productrices, le conflit saute : p et q ne sont pas mutex.

2. **Phase d'extraction pas-à-pas** : Dès que $B \subseteq F_k$ et qu'aucun couple de buts n'est mutex dans F_k, basculer en recherche arrière.

(a) Poser le sous-but initial au niveau final : $G_k = B$.

(b) Pour chaque niveau i allant de k à 1 :

- Pour chaque fluent $g \in G_i$, sélectionner *une* action productrice $a \in A_i$ (en favorisant les NoOps en premier lieu pour limiter les effets de bord).
- Vérifier que l'ensemble d'actions sélectionné Q_i ne contient *aucune paire en exclusion mutuelle*. Si un conflit survient, faire un retour-arrière (*backtrack*) et tester une combinaison alternative d'actions productrices.
- Calculer le nouveau sous-but à satisfaire au niveau précédent :

$$G_{i-1} = \bigcup_{a \in Q_i} \text{Prec}(a) \tag{2.1.13}$$

(c) Si le retour-arrière atteint le niveau 0 avec $G_0 \subseteq I$, la séquence $\langle Q_1, Q_2, \ldots, Q_k \rangle$ constitue le plan-solution. En supprimant les NoOps de chaque ensemble Q_i, on obtient les actions exécutables du plan.

Exemple (Exemple canonique) Considérons le domaine classique composé des trois actions suivantes et des NoOps associés sur $\{a, b, c, d\}$:

$$A : a \longrightarrow +b, \qquad B : a \longrightarrow +c - a, \qquad C : b \wedge c \longrightarrow +d$$

Soit l'état initial $I = \{a\}$ et le but $G = \{d\}$. Le déroulement met en évidence les mécanismes clés :

- Au niveau A_1, l'action B supprime le fluent a ($\text{Del}(B) = \{a\}$) tandis que N_a l'ajoute ($\text{Add}(N_a) = \{a\}$). Ces actions sont donc en conflit direct : $\text{Mutex}(B, N_a)$.

- Dans F_1, les fluents a et c ne peuvent être produits conjointement que par le couple (N_a, B). Ce couple étant mutex, les fluents héritent de l'exclusion : Mutex(a, c) dans F_1.
- L'action C requiert b et c. Elle ne peut pas être déclenchée valablement au niveau 2 si b et c proviennent d'actions incompatibles, mais peut être satisfaite dès que les paires de supports deviennent mutuellement indépendantes.
- *Plan extrait :* l'extraction arrière produit le plan d'ensembles d'actions :

$$P = \langle Q_1 = \{A, N_a\}, \quad Q_2 = \{N_b, B\}, \quad Q_3 = \{C\} \rangle \tag{2.1.14}$$

Ce plan, une fois post-traité et débarrassé des actions de maintien triviales, fournit la séquence exécutable minimale :

$$\langle A, B, C \rangle \tag{2.1.15}$$

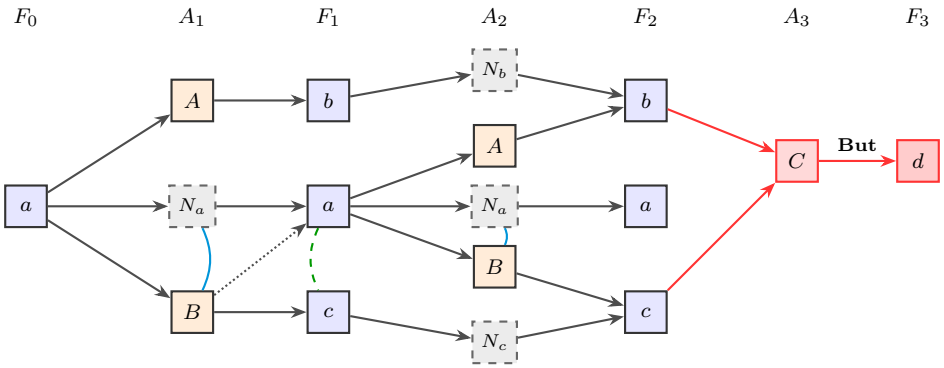


FIGURE 2.3 – Représentation du graphe de planification pour le domaine canonique. Les traits pleins bleus représentent les mutex d'actions, les pointillés verts les mutex de fluents. Le but d est atteint et extrait au niveau 3.

Une caractéristique fondamentale de GRAPHPLAN est que le calcul des exclusions mutuelles est strictement restreint à une *arité 2* (incompatibilités par paires) afin de préserver la complexité polynomiale de la phase d'extension.

Remarque (Sur-approximation et pannes de consistance globale) L'absence de mutex binaire entre tous les éléments d'un sous-ensemble n'implique pas la réalisabilité physique conjointe de ce sous-ensemble. Soient trois actions ou trois fluents mutuellement compatibles par paires :

$$\neg\,\text{Mutex}(x_1, x_2), \quad \neg\,\text{Mutex}(x_2, x_3), \quad \neg\,\text{Mutex}(x_1, x_3) \tag{2.1.16}$$

Bien qu'aucun mutex binaire n'interdise leur sélection conjointe, il peut être structurellement impossible de satisfaire simultanément la conjonction $x_1 \wedge x_2 \wedge x_3$ au sein d'un même état valide. Le graphe de planification constitue ainsi une *sur-approximation optimiste* de l'espace d'atteignabilité. C'est cette insuffisance intrinsèque des contraintes binaires qui justifie le caractère NP-complet de la phase d'extraction arrière, celle-ci devant effectuer un retour-arrière complet pour vérifier la consistance globale du plan.

Algorithme 5 : Algorithme GRAPHPLAN

Entrées : Problème de planification $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, I, B \rangle$
Sortie : Un plan d'ensembles d'actions $P = \langle Q_1, \ldots, Q_k \rangle$, ou **échec**

`// Initialisation du niveau 0`
$k \leftarrow 0$
$F_0 \leftarrow I$
$\text{Mutex}(F_0) \leftarrow \emptyset$
$\mathcal{M} \leftarrow \emptyset$ `// Table de mémorisation (nogoods)`
while ***vrai*** **do**
  `// Condition de déclenchement de la recherche arrière`
  if $B \subseteq F_k$ ***et*** $\forall (g_1, g_2) \in B \times B,\ \neg \text{Mutex}(g_1, g_2 \mid F_k)$ **then**
    $P \leftarrow$ `ExtrairePlan`$(B, k, \mathcal{M})$
    if $P \neq$ ***échec*** **then**
      return P
  `// Test de stabilisation (incomplétude prouvée)`
  if $k > 0$ ***et*** $F_k = F_{k-1}$ ***et*** $\text{Mutex}(F_k) = \text{Mutex}(F_{k-1})$ **then**
    return échec `// Le graphe a stationné sans solution`
  `// Phase d'extension vers le niveau` $k+1$
  $k \leftarrow k + 1$
  $A_k \leftarrow \{N_f \mid f \in F_{k-1}\} \cup \{a \in \mathcal{A} \mid \text{Prec}(a) \subseteq F_{k-1} \wedge \forall p, q \in \text{Prec}(a), \neg \text{Mutex}(p, q \mid F_{k-1})\}$
  Calculer $\text{Mutex}(A_k)$ via conflits directs et préconditions concurrentes
  $F_k \leftarrow \bigcup_{a \in A_k} \text{Add}(a)$
  Calculer $\text{Mutex}(F_k)$ via paires d'actions productrices exclusives
Procédure `ExtrairePlan`$(G, i, \mathcal{M})$ **:**
  if $i = 0$ **then**
    return $\langle\rangle$ `// Succès : le niveau initial est atteint`
  if $(i, G) \in \mathcal{M}$ **then**
    return échec `// Sous-but déjà exploré et prouvé stérile`
  foreach *ensemble non mutex* $Q \subseteq A_i$ *tel que* $G \subseteq \bigcup_{a \in Q} \text{Add}(a)$ **do**
    $G' \leftarrow \bigcup_{a \in Q} \text{Prec}(a)$
    $P' \leftarrow$ `ExtrairePlan`$(G', i-1, \mathcal{M})$
    if $P' \neq$ ***échec*** **then**
      return $P' \oplus \langle Q \rangle$
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{(i, G)\}$ `// Mémorisation du conflit (nogood)`
  return échec

2.2 Améliorations de Graphplan

Bien que la phase d'extension du graphe soit de complexité polynomiale, l'extraction de solution reste un problème NP-complet. Deux axes majeurs d'optimisation ont été développés dans la littérature. La première consiste en une optimisation structurelle du graphe et des exclusions mutuelles via une relaxation de la condition d'indépendance grâce à la *relation d'autorisation* (LCGP). La seconde approche utilise plutôt des traductions du problème de planification en problème de satisfiabilité booléenne (SAT) ou de réseau de contraintes (CSP) afin de tirer parti des algorithmes de résolution modernes.

2.2.1 Lcgp – Relations d'Autorisation

Développé par Cayrol, Régnier et Vidal (2000), le planificateur LCGP (*Linearization of Chains in Graphplan*) repose sur un constat : l'indépendance stricte $a_1 \# a_2$ exigée par GRAPHPLAN pour exécuter deux actions dans une même couche est trop conservative. Deux actions n'ont pas besoin d'être symétriquement indépendantes dès lors qu'il existe un ordre d'exécution séquentiel valide au sein de l'étape sans détruire leurs effets conjoints.

Définition (**Relation d'autorisation**) . Soient deux actions a_1 et a_2. On dit que a_1 *autorise* a_2, noté $a_1 \angle a_2$, si et seulement si l'exécution préalable de a_1 ne détruit ni les préconditions de a_2, ni ses effets d'ajout :

$$a_1 \angle a_2 \iff \begin{cases} \mathrm{Del}(a_1) \cap \mathrm{Prec}(a_2) = \emptyset \\ \mathrm{Del}(a_2) \cap \mathrm{Add}(a_1) = \emptyset \end{cases} \tag{2.2.1}$$

Deux actions sont dites *autorisées* si $a_1 \angle a_2$ ou $a_2 \angle a_1$.

Remarque (Comparaison formelle avec l'indépendance) La relation d'indépendance classique correspond à la conjonction symétrique des deux autorisations :

$$a_1 \# a_2 \iff (a_1 \angle a_2) \wedge (a_2 \angle a_1) \tag{2.2.2}$$

La relation d'autorisation est donc une relaxation stricte de l'indépendance.

Définition (**Ensemble d'actions autorisées**) . Un ensemble d'actions Q est un *ensemble autorisé* s'il existe une permutation (linéarisation) $(a_{\sigma(1)}, a_{\sigma(2)}, \ldots, a_{\sigma(m)})$ de ses éléments telle que chaque action autorise les suivantes :

$$\forall j < k, \quad a_{\sigma(j)} \angle a_{\sigma(k)} \tag{2.2.3}$$

Un tel ensemble Q est applicable sur un état E dès lors que $\bigcup_{a \in Q} \mathrm{Prec}(a) \subseteq E$, produisant l'état cumulé identique $E \uparrow Q$.

La relaxation induite par LCGP modifie directement la détection des mutex au niveau des actions :

- Dans GRAPHPLAN : a_1 et a_2 sont mutex si et seulement si elles ne sont pas indépendantes, c'est-à-dire : $\neg(a_1 \angle a_2) \vee \neg(a_2 \angle a_1)$.
- Dans LCGP : a_1 et a_2 sont mutex si et seulement si *aucune n'autorise l'autre* :

$$\mathrm{Mutex}_{\mathrm{LCGP}}(a_1, a_2) \iff \neg(a_1 \angle a_2) \wedge \neg(a_2 \angle a_1) \tag{2.2.4}$$

Différentes conséquences pratiques peuvent alors être observées :

1. *Moins de contraintes d'exclusion :* Le nombre d'arcs mutex diminue drastiquement dans chaque couche d'actions, ce qui permet de compresser les étapes et d'atteindre le but en moins de niveaux.
2. *Extraction de la solution :* LCGP extrait une séquence d'ensembles autorisés, puis applique un algorithme polynomial (notamment l'algorithme PRF) pour réordonner les actions et retourner un plan séquentiel valide.
3. *Gains empiriques :* Sur les domaines peu contraints (tels que *Logistics*), LCGP est en moyenne jusqu'à 1800 fois plus rapide que GRAPHPLAN.

2.2.2 Codages SAT du graphe

Plutôt que d'extraire la solution par retour-arrière classique dans le graphe, l'approche initiée par BLACKBOX (Kautz et Selman, 1999) compile le graphe de planification jusqu'au niveau k en une formule propositionnelle en forme normale conjonctive (CNF). Un solveur SAT complet tente alors de trouver une affectation satisfaisable :

- Si un modèle propositionnel est identifié, il est décodé directement en un plan-solution valide.
- Si la formule est insatisfiable (UNSAT), aucun plan de longueur $\leqslant k$ n'existe ; le graphe est alors étendu au niveau $k+1$ et le processus réitéré.

Définition (Règles de codage SAT – Kautz & Selman, 1999) . Soit GP_k le graphe de planification développé jusqu'au niveau k. Le problème de planification est traduit en la conjonction des clauses propositionnelles suivantes :

1. *Assertions d'état initial et de but :*

$$\left(\bigwedge_{n_f \in \text{NoeudsInit}(GP_k)} n_f\right) \wedge \left(\bigwedge_{n_f \in \text{NoeudsBut}(GP_k)} n_f\right) \tag{2.2.5}$$

2. *Implication d'action vers préconditions :* Si une action est vraie à l'étape i, chacune de ses préconditions doit être vraie à l'étape $i-1$:

$$\bigwedge_{(n_f, n_a) \in \text{ArcsPrec}(GP_k)} (n_a \Longrightarrow n_f) \quad \equiv \quad \bigwedge_{(n_f, n_a) \in \text{ArcsPrec}(GP_k)} (\neg n_a \vee n_f) \tag{2.2.6}$$

3. *Axiomes d'explication des fluents (Frame-axiomes causaux) :* Un fluent (hors état initial) ne peut être vrai à l'étape i que si au moins une action (normale ou NoOp) l'a produit à cette étape :

$$\bigwedge_{n_f \in \text{NoeudsF}(GP_k) \setminus \text{NoeudsInit}(GP_k)} \left(n_f \Longrightarrow \bigvee_{(n_a, n_f) \in \text{ArcsAdd}(GP_k)} n_a\right) \tag{2.2.7}$$

4. *Exclusions mutuelles entre actions :* Deux actions en conflit direct ou hérité ne peuvent pas être sélectionnées simultanément au même niveau :

$$\bigwedge_{\{n_a, n_b\} \in \text{MutexA}(GP_k)} (\neg n_a \vee \neg n_b) \tag{2.2.8}$$

Remarque (Rôle des NoOps dans le codage SAT) Les actions de maintien N_f font partie intégrante de ArcsAdd(GP_k). L'implication $n_f \Longrightarrow \bigvee n_a$ encode donc élégamment à la fois la causalité d'une action réelle et la persistance par défaut, évitant l'explosion combinatoire des frame-axiomes classiques.

2.2.3 Codages CSP du graphe

Développée dans des systèmes comme GPCSP (Kambhampati et Nigenda, 2000), la formulation CSP transforme l'extraction arrière en un problème de satisfaction de contraintes à variables discrètes.

Définition (**Variables & Domaines**) . Pour un horizon k donné, le réseau de contraintes est structuré de façon inverse à la logique des fluents :

- Variables du CSP : chaque fluent présent dans le graphe constitue une variable $f \in \{f_1, f_2, \ldots, f_m\}$.
- Domaine des variables : pour chaque fluent f, la valeur qui lui est affectée représente *l'action productrice* choisie pour l'établir :

$$D(f) = \{a \in \mathcal{A} \cup \text{NoOps} \mid f \in \text{Add}(a)\} \cup \{\bot\} \tag{2.2.9}$$

  où la valeur spéciale $\bot$ indique que le fluent f est inactif (non requis) dans le plan extrait.
- Cas particulier du but : Pour tout fluent $g \in B$ au niveau terminal, le domaine est restreint à ses seules actions productrices : $\bot \notin D(g)$.

On définit alors un *réseau de contraintes* entre ces variables.

Proposition (Contraintes) Le CSP impose trois familles de contraintes sur ces variables :

1. **Contraintes d'activité (flux causal) :** Si un fluent f est soutenu par une action a ($f = a$), alors toutes les préconditions $p \in \text{Prec}(a)$ doivent nécessairement être activées au niveau précédent :
$$(f = a) \Longrightarrow \bigwedge_{p \in \text{Prec}(a)} (p \neq \bot) \tag{2.2.10}$$
2. **Contraintes d'actions mutex :** Si deux actions a_1 et a_2 sont en exclusion mutuelle au même niveau, elles ne peuvent pas être sélectionnées simultanément pour soutenir deux fluents distincts f_1 et f_2 :
$$\text{Mutex}(a_1, a_2) \Longrightarrow \neg\big((f_1 = a_1) \wedge (f_2 = a_2)\big) \tag{2.2.11}$$
   ce qui s'écrit sous forme binaire : $(f_1 = a_1) \Longrightarrow (f_2 \neq a_2)$.
3. **Contraintes de fluents mutex :** Deux fluents mutuellement exclusifs ne peuvent pas être simultanément actifs au même pas temporel :
$$\text{Mutex}(f_1, f_2) \Longrightarrow \neg\big((f_1 \neq \bot) \wedge (f_2 \neq \bot)\big) \tag{2.2.12}$$

L'avantage déterminant du codage CSP est qu'il permet d'exploiter les heuristiques génériques d'ordonnancement de variables et de valeurs guidées par la topologie du graphe :

- Choix des variables (ordre d'instanciation des fluents) :
  - *Level-based :* Sélectionner en priorité les fluents apparaissant aux plus hauts niveaux du graphe (les plus proches du but).
  - *Most constrained (Number of mutex) :* En cas d'égalité de niveau, choisir le fluent possédant le plus grand nombre d'exclusions mutuelles.
- Choix des valeurs (ordre de sélection des actions productrices) :
  - *NoOps-first :* Tenter d'abord les actions de persistance afin de minimiser le nombre d'actions effectives du plan.

- *Lowest appearance level & Least mutex :* Privilégier les actions apparues le plus tôt dans le graphe et ayant le moins d'arcs mutex, maximisant ainsi les chances d'extension sans conflit.

L'utilisation conjointe de ces heuristiques permet à GPCSP d'être compétitif avec les meilleurs solveurs SAT (comme BLACKBOX) tout en consommant une empreinte mémoire nettement inférieure.

»»»> subjects/planification